Bernoulli Types

An Algebra of Approximate Computation

Alexander Towell

May 21, 2026

© 2026 Alexander Towell.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (`https://creativecommons.org/licenses/by-nc-sa/4.0/`).

The Python companion library `bernoulli` is released separately under the MIT License.

ORCID: `0000-0001-6443-9897`
Web: `https://metafunctor.com`
Source: `https://github.com/queelius/bernoulli-book`

Contents

Preface **vii**

I The Atom **1**

1 The Noisy Bit **3**
1.1 Three Faces of a Noisy Bit . . . 3
1.2 Quantifying the Noise . . . 6
1.3 A First Look at the Boolean Universe . . . 11
1.4 Sampling and Estimating Rates . . . 14
1.5 Why "Atom"? . . . 16
Bibliographic Notes . . . 20
Exercises . . . 21

2 Binary Channels and Model Order **27**
2.1 The Binary Symmetric Channel . . . 27
2.2 The Asymmetric Binary Channel . . . 33
2.3 Composition of Channels . . . 38
2.4 Model Order . . . 44
Bibliographic Notes . . . 51
Exercises . . . 53

3 Boolean Algebra Under Noise **59**
3.1 AND Under Noise . . . 59
3.2 OR and NOT . . . 66
3.3 The Composition Theorem . . . 72
3.4 Miller-Rabin: A Worked Walkthrough . . . 81
3.5 Composition is the Bridge to Sets . . . 88
Bibliographic Notes . . . 92
Exercises . . . 94

4 Two Axioms and Kronecker Factorization 99
4.1 Why Axioms? . . . 99
4.2 Axiom 1: Element-Wise Independence . . . 102
4.3 Axiom 2: Conditional Independence of Block Error Rates . . . 107
4.4 The Kronecker Factorization Theorem . . . 114
4.5 The Bernoulli[bool] Type, Formally . . . 123
4.6 What Comes Next . . . 129
Bibliographic Notes . . . 131
Exercises . . . 133

II From Bits to Sets 137

5 Approximate Sets and Predicates 139
5.1 From Atom to Set . . . 139
5.2 The Bloom Filter From Scratch . . . 143
5.3 The Predicate Algebra . . . 150
5.4 One-Sided vs Two-Sided Approximate Sets . . . 155
5.5 What Comes Next . . . 158
Bibliographic Notes . . . 161
Exercises . . . 162

6 Composition of Sets 169
6.1 Union of Approximate Sets . . . 169
6.2 Intersection, Complement, Difference . . . 174
6.3 Cancellation and the Loss of One-Sidedness . . . 179
6.4 The Perfect Filter Line . . . 184
6.5 What Comes Next . . . 187
Bibliographic Notes . . . 189
Exercises . . . 190

7 Classification Measures 197

8 Information-Theoretic Bounds 199

III From Sets to Maps 201

9 Approximate Functions and n-ary Channels 203

10 Sketches and Estimators 205

11 Randomized Algorithms **207**

12 Approximate Relations **209**

13 The Algebra of Approximate Types **211**

IV Optimality and Construction **213**

14 Optimal Bernoulli Sets **215**

15 Perfect Hash Filters **217**

V The Boundary and Beyond **219**

16 Approximate but Not Bernoulli **221**

17 Cipher Maps and Trapdoor Computing **223**

18 Encrypted Search as Relaxed Trapdoor **225**

A Probability Refresher **227**

B The Five Core Principles **229**

C Library Walkthrough **231**

D Selected Exercise Solutions **233**

Preface

This is a placeholder preface. The final preface will:

- State the book's thesis and audience.
- Declare prerequisites: undergraduate probability, basic discrete math, comfort with proof, working Python knowledge for the notebook companion.
- Explain the lockstep notebook discipline (some chapters carry notebooks; theory-heavy chapters do not, by design).
- Provide a reading guide for the smart-undergrad-upward reader (chapters that admit a soft landing) and the adjacent-field grad student (the modal reader).

Part I

The Atom

Chapter 1

The Noisy Bit

After this chapter you will be able to:

- Recognize a noisy bit when you see one, whether it arrives as a biased coin, a binary symmetric channel, or a binary classifier's verdict.
- Quantify a noisy bit with its false-positive and false-negative rates, and sample to estimate them.
- Name the noisy bit as the atomic object of the framework, and point at the trajectory that develops its algebra across the remaining chapters of Part I.

1.1 Three Faces of a Noisy Bit

Three systems, drawn from three different corners of applied probability, each produce a single Boolean output that may be wrong. The first is a biased coin that reports an external state. The second is a one-bit communication channel that occasionally corrupts what it transmits. The third is a binary classifier that emits a verdict about each input it sees. We look at each in turn, with no formalism beyond informal arithmetic. The question to hold throughout: what is the abstract structure that all three share?

Face 1: The biased coin

Imagine a coin that is not quite fair, and whose bias depends on how it has been held. When you palm it "in" (cupped, warm side down) and flip, it lands heads with probability τ. When you palm it "out" (cool side

down) and flip, it lands heads with probability ϵ. The numbers τ and ϵ are properties of the coin and how it is being held. They need not be equal, and neither needs to be close to 1/2.

This is a contrived setup, on purpose. Read it as a thought-experiment device that lets us write a single object that has two possible inputs (in versus out), one Boolean output (heads versus tails), and two independent parameters for how reliably the output tracks the input. The same flip mechanism is at work in both cases; what changes is how the coin is being held. In the language we are building toward, "in" is a positive condition and "out" is a negative condition, and the coin reports each with its own characteristic reliability. The biased coin returns as a running example throughout Part I, first in Section 2.2 as an asymmetric channel and again as the worked example of the formal definition in Chapter 4.

Face 2: The binary symmetric channel

A single bit is transmitted across a noisy line: a wire, a wireless link, a chip-to-chip trace, or any one-bit communication path subject to disturbance. The line sends $\top$ or $\bot$ ($\top$ for true, $\bot$ for false, used throughout the book) from sender to receiver, and for each bit independently the line behaves as if a small probabilistic adversary acts on it. With probability ε the line flips the bit; with probability $1 - \varepsilon$ it passes the bit through unchanged. The crossover probability ε is the same regardless of which symbol is being transmitted: a $\top$ flips to $\bot$ with probability ε, and a $\bot$ flips to $\top$ with the same probability ε.

This object is the binary symmetric channel, and it is the simplest model of a noisy one-bit communication line. The word *channel* is borrowed from Shannon's information theory and is the standard name for any single-input, single-output stochastic transmission. The symmetry assumption (one parameter for both flip directions) is a strong one. Many real communication systems are asymmetric: a transistor pulled to ground may be more reliable than one held high; a photon detector may have a small dark-count rate but never an extinguished-photon false negative. Such systems need two parameters, one per direction, and we will treat them in Section 2.2. For now, the BSC is the cleanest starting point because it isolates the phenomenon of bit-flip noise from the question of whether one direction is noisier than the other. The BSC is the most studied noisy channel in information theory, and the chapter 2 development of it (Section 2.1) will give it both a formal definition and a matrix representation.

Face 3: The classifier verdict

A spam filter examines each incoming message and emits a verdict: spam ($\top$) or ham ($\bot$). Some messages are genuinely spam; some are genuinely ham; the filter does its best on each. Like any imperfect classifier, it sometimes flags ham as spam (a false positive) and sometimes misses spam, marking it as ham (a false negative). The rate at which it flags ham as spam is the false-positive rate ε; the rate at which it misses spam is the false-negative rate ω. In general these two numbers are not equal, and the asymmetry matters: a spam filter that drops legitimate mail at 1% is a different product from one that lets 1% of spam through.

Notice that the spam filter, unlike the BSC, needs two parameters by design rather than by happenstance. The two error directions are conceptually distinct: a false positive misroutes legitimate mail, and a false negative leaks spam into the inbox. These are different harms with different costs. A useful classifier therefore comes with two rates, not one, and the user of the classifier wants to know both. The classifier verdict is our most realistic noisy-bit setting, because it forces us to take two-parameter error structure seriously from the start. The vocabulary of false positives and false negatives is the bridge to the contingency-table arithmetic we develop in Section 1.2, and the spam filter itself returns as a worked example throughout the chapter.

The unifying observation

We have three systems: a coin held in or out, a wire transmitting one bit, and a filter emitting one verdict. They live in different worlds. The first is physical, the second is informational, the third is algorithmic. Yet at the level of input-output behavior they look the same. Each takes a Boolean input ($\top$ or $\bot$, by whatever name) and produces a Boolean output. Each output is, with some probability, the correct one, and with the remaining probability the wrong one. The error behavior in every case is characterized by at most two numbers: the rate of reporting $\top$ when the truth is $\bot$, and the rate of reporting $\bot$ when the truth is $\top$. The BSC happens to have a single parameter because its two rates are constrained to be equal; the coin and the spam filter need two. In all three cases, two numbers suffice to describe everything probabilistic at the single-query level.

We will call the abstract structure that all three share *a noisy bit*.

The three sections used three vocabularies, by design: each face is more natural in its own letters. The reconciliation is direct. The biased coin's τ is the true-positive rate $\tau = 1 - \omega$, and its ϵ is the false-positive rate ε. The

classifier's ε, ω are the same quantities under different names. The binary symmetric channel is the special case where these two rates are forced to be equal: a single $\varepsilon = \omega$ suffices.

The naming is deliberate. It captures three commitments we will hold to throughout Part I. First, the object is atomic: a noisy bit has one Boolean input slot and one Boolean output slot, and nothing larger. (We will see in chapters 2 and 4 that everything bigger, multi-bit channels, repeated-trial algorithms like Miller-Rabin, classifier ensembles, is built up from noisy bits.) Second, the structure is purely behavioral: we say nothing about how the noise arises, only about its rates. A biased coin, a thermal photon detector, and a logistic-regression classifier may have totally different mechanisms and identical noisy-bit profiles, and our framework treats them identically. Third, the description is parsimonious: at most two numbers per noisy bit, no more. The rest of Part I unfolds the algebra of how these atoms compose, decompose, and combine.

Where the chapter goes from here

Section 1.2 fixes the vocabulary by introducing the 2×2 contingency table that gives ε and ω a precise meaning, and connects this notation to the classifier vocabulary of precision and recall. Section 1.3 walks the four-cell Boolean universe (two true inputs, two observed outputs) qualitatively, deferring the joint-probability formalism to chapter 4. Section 1.4 asks how we estimate ε and ω from data, hooking the chapter's accompanying notebook. Section 1.5 closes the chapter by saying why this object earns the title "atom" and previews the algebraic development of chapters 2, 3, and 4. The three faces return at named points in each of those chapters; by the end of Part I, each will have been treated at four progressively tighter levels of formality, culminating in a single definition that retroactively justifies everything we say about it now.

1.2 Quantifying the Noise

The closing observation of Section 1.1 was that every noisy bit, biased coin, binary symmetric channel, classifier verdict, is characterized by at most two error rates: the rate of reporting $\top$ when the truth is $\bot$, and the rate of reporting $\bot$ when the truth is $\top$. That phrasing is suggestive but informal. What does "rate" mean precisely, what is the denominator, and how do we measure these numbers from data? Those are the questions of this section.

The path is short. We will introduce the 2×2 contingency table that records the outcome of n observations of a noisy bit, name the four rates one can compute from that table, and connect those rates to the precision and recall that practitioners of binary classification already know. By the end of this section the reader can read off the error rates of any noisy bit from a confusion matrix, and can move freely between (ε, ω) on the one hand and $(\text{precision}, \text{recall})$ on the other.

The contingency table

Suppose we run a noisy bit n times. On each trial we know the true input, either $\top$ or $\bot$, and we observe an output, also either $\top$ or $\bot$. The result of n trials is a list of n ordered pairs $(\text{truth}, \text{output})$, and each pair falls into exactly one of four joint states. A pair where the truth is $\top$ and the output is $\top$ is a *true positive*. A pair where the truth is $\top$ but the output is $\bot$ is a *false negative*, the noisy bit reported a $\bot$ when it should have reported $\top$. A pair where the truth is $\bot$ but the output is $\top$ is a *false positive*. A pair where the truth is $\bot$ and the output is $\bot$ is a *true negative*. A standing source of student confusion is that "false negative" names the *system output*, not the truth: a false negative is a $\bot$ output produced when the truth was $\top$.

Let TP, FN, FP, and TN denote the counts of each joint state across the n trials. By construction $\text{TP} + \text{FN} + \text{FP} + \text{TN} = n$. We arrange these four counts in a 2×2 table indexed by truth (rows) and observed output (columns):

	output $\top$	**output** $\bot$	total
truth $\top$	TP	FN	TP + FN
truth $\bot$	FP	TN	FP + TN
total	TP + FP	FN + TN	n

Table 1.1: The 2×2 contingency table for n observations of a noisy bit. Each of the n trials contributes one count to exactly one cell. The row totals record how many trials had each truth value; the column totals record how many trials had each observed output.

Table 1.1 is the empirical record of one experiment. Two conventions matter for reading it. First, rows are conditioned on truth: the top row holds only those trials whose true input was $\top$, and the bottom row holds only those trials whose true input was $\bot$. Second, the diagonal cells (TP and

TN) record agreement between truth and output, and the off-diagonal cells (FN and FP) record disagreement. A perfectly accurate noisy bit produces all of its mass on the diagonal; a maximally noisy one produces it all off the diagonal.

From counts to rates

The two error rates of a noisy bit are conditional: each is normalized by the row total of the truth on which it is conditioned. The *false-positive rate* is the fraction of $\bot$-inputs that produce a $\top$-output,

$$\varepsilon \;=\; \frac{\mathrm{FP}}{\mathrm{TN}+\mathrm{FP}},$$

and the *false-negative rate* is the fraction of $\top$-inputs that produce a $\bot$-output,

$$\omega \;=\; \frac{\mathrm{FN}}{\mathrm{TP}+\mathrm{FN}}.$$

Each rate divides one off-diagonal cell by its row total, not by n. The denominator FP + TN counts only the trials whose true input was $\bot$, and FN + TP counts only the trials whose true input was $\top$. The conditioning matters: it makes ε and ω properties of the noisy bit itself, not of the particular mixture of inputs we happen to have observed.

The two complementary rates are the *true-negative rate* $\eta = 1-\varepsilon$ and the *true-positive rate* $\tau = 1-\omega$. The true-negative rate is the fraction of $\bot$-inputs reported correctly; the true-positive rate is the fraction of $\top$-inputs reported correctly. The true-positive rate is also called *sensitivity* in clinical and classification settings, and the true-negative rate is called *specificity*.

Notation sidebar

Rate symbols and macros. The four rates used throughout this book have fixed Greek letters and fixed `alex.sty` macros.

- ε (`\fprate`) false-positive rate. The rate at which a $\bot$-input is reported as $\top$.
- ω (`\fnrate`) false-negative rate. The rate at which a $\top$-input is reported as $\bot$.
- τ (`\tprate`) true-positive rate, $= 1-\omega$. Also called *sensitivity* or *recall*.
- η (`\tnrate`) true-negative rate, $= 1-\varepsilon$. Also called *specificity*.

In the biased-coin vocabulary of Section 1.1, τ was written τ and ε was written ϵ; here we adopt names that emphasize what each rate measures.

The two free parameters of a generic noisy bit are ε and ω; the other two are derived. The binary symmetric channel of Section 2.1 is the constrained case in which $\varepsilon = \omega$, and one parameter suffices. A consolidated cheat sheet, including how these symbols connect to the five core principles of the framework, appears in Appendix B.

Cross-walk to precision and recall

A practitioner of binary classification looks at Table 1.1 with a different vocabulary in mind. The standard metrics for a classifier are *precision* and *recall*:

$$\text{precision} \;=\; \frac{\text{TP}}{\text{TP}+\text{FP}}, \qquad \text{recall} \;=\; \frac{\text{TP}}{\text{TP}+\text{FN}}.$$

Read off the table: precision divides TP by the *column* total of trials the noisy bit *called* $\top$, and recall divides TP by the *row* total of trials whose true input was $\top$. Recall therefore coincides with the true-positive rate: $\text{recall} = \tau = 1 - \omega$.

Precision is different. Its denominator is the column total TP+FP, which depends not only on the noisy bit's error rates but also on how many $\top$- and $\bot$-inputs were in the sample. Concretely, if the sample contains $n_\top$ positives and $n_\bot$ negatives with $n_\top + n_\bot = n$, then $\text{TP} \approx n_\top \tau$ and $\text{FP} \approx n_\bot \varepsilon$ (treating τ and ε as the population rates, this is the expected-count decomposition; for the finite-sample version see Section 1.4), so

$$\text{precision} \;\approx\; \frac{n_\top \tau}{n_\top \tau + n_\bot \varepsilon} \;=\; \frac{n_\top (1-\omega)}{n_\top (1-\omega) + n_\bot \varepsilon}.$$

Two facts follow. First, precision is a function not only of (ε, ω) but also of the input mix $n_\top / n_\bot$, the *prevalence* of positives in the sample. Recall depends only on the noisy bit. Second, precision and recall together determine ε and ω only when the prevalence is also known. The pair (ε, ω) is intrinsic to the noisy bit; the pair (precision, recall) is intrinsic to the noisy bit together with a workload.

This is why the framework of this book uses (ε, ω) as the primary description of a noisy bit, and treats precision and the related accuracy as derived measures. The full development of these derived measures lives in

the companion paper on classification measures; here we note only that they are expressible in terms of the four cells of Table 1.1 and the prevalence.

Worked example: a spam-filter run

Suppose a spam filter is run on a labelled mailbox of $n = 100$ messages, of which 40 are spam ($\top$) and 60 are ham ($\bot$). The filter's outputs produce the contingency table

	flagged spam	**flagged ham**	total
true spam	35	5	40
true ham	3	57	60
total	38	62	100

Row totals: $35 + 5 = 40$ spam, and $3 + 57 = 60$ ham. Column totals: $35 + 3 = 38$ flagged, and $5 + 57 = 62$ not flagged. Grand total 100. The table is internally consistent.

The four rates are

$$\varepsilon = \frac{3}{60} = 0.05, \quad \omega = \frac{5}{40} = 0.125, \quad \eta = \frac{57}{60} = 0.95, \quad \tau = \frac{35}{40} = 0.875.$$

By construction $\eta = 1 - \varepsilon$ and $\tau = 1 - \omega$, and both identities check.

The two classifier metrics are

$$\text{precision} = \frac{35}{35 + 3} = \frac{35}{38} \approx 0.921, \qquad \text{recall} = \frac{35}{35 + 5} = \frac{35}{40} = 0.875.$$

Recall matches τ, as the cross-walk predicts. Precision lies between τ and 1, reflecting that the column total 38 is smaller than the row total 40 because the filter misses 5 spam messages. The prevalence of spam in this sample is $40/100 = 0.4$; a sample with a different prevalence would yield a different precision from the same (ε, ω), illustrating the point of the previous subsection.

What we have, what comes next

A noisy bit is characterized by two numbers, ε and ω, each a conditional rate read off the contingency table. The complementary rates τ and η are derived, as are the classifier metrics precision and recall. The symbols $\varepsilon, \omega, \tau, \eta$ recur throughout the book and refer always to the macros listed in the notation sidebar.

Section 1.3 now walks the four-cell Boolean universe of joint truth-output pairs whose conditional rates this section has just defined, treating the cells qualitatively before Chapter 4 assigns each cell a joint probability. Section 1.4 returns to the present section's contingency-table arithmetic from the data-analytic side, asking how reliably ε and ω can be estimated from finite n.

1.3 A First Look at the Boolean Universe

A noisy bit has two possible inputs and two possible outputs. The truth on any given trial is either $\top$ or $\bot$, and the reported value is also either $\top$ or $\bot$. Cross those two binary choices and four input-output combinations exhaust the joint behavior of the device. You have already seen this 2×2 grid in Section 1.2, but as a table of counts; here we look at it as a table of *outcome types*, the four ways a noisy bit's trial can turn out. We arrange the combinations in a 2×2 outcome matrix indexed by the true input (rows) and the observed report (columns):

	report $\top$	**report** $\bot$
truth $\top$	correct positive	false negative
truth $\bot$	false positive	correct negative

The four cells are the universe of joint behavior. Anything the noisy bit can do on a single trial is one of these four outcomes; there are no others. The chapter from here on is about the structure of these four cells, what populates each one, how the populations move when the device is composed with another, and how the rates of Section 1.2 fit on top of them.

The four cells, narratively

Take the cells one at a time.

Truth $\top$*, report* $\top$*.* The input was $\top$ and the device reported $\top$. Call this a *correct positive*: the noisy bit got it right on a $\top$-input. In the spam-filter vocabulary of Section 1.1 this is a spam message correctly flagged. In the biased-coin face this is the in-handheld trial that came up heads as it usually does. In the binary-channel face this is the bit that traveled the wire without flipping.

Truth $\top$*, report* $\bot$*.* The input was $\top$ and the device reported $\bot$. This is the *false negative*: the noisy bit missed. The spam filter dropped a spam message into the inbox. The coin came up tails on a trial that should usually

have produced heads. The wire flipped a transmitted $\top$ into a $\bot$. Whatever the mechanism, the system replied with the wrong answer in the $\top$-to-$\bot$ direction.

Truth $\bot$, report $\top$. The input was $\bot$ and the device reported $\top$. This is the *false positive*: the noisy bit fired when it should have stayed silent. The spam filter flagged a legitimate message. The coin came up heads on a trial when heads was unlikely. The wire flipped a $\bot$ into a $\top$.

Truth $\bot$, report $\bot$. The input was $\bot$ and the device reported $\bot$. The *correct negative*: the noisy bit stayed silent when it should have. The ham message passed through, the unlikely-heads trial produced tails as expected, the $\bot$-bit traveled the wire unchanged.

Those are the four cells. Their names are exactly the four labels of the contingency table in Section 1.2: TP, FN, FP, TN. This is not a coincidence; it is the same four-way split seen from a different angle. The contingency table of Section 1.2 records what happens when we run the noisy bit n times and *count* how many trials land in each cell. The outcome matrix above records the *cells themselves*, abstracted from any particular run. The four cells of the joint distribution and the four cells of the empirical contingency table are the same four cells. The empirical table is a sample drawn from the joint distribution over those cells.

Why this is a "universe"

The input domain $\{\top, \bot\}$ has exactly two elements. We will call this set the *Boolean universe* and write it $U = \{\top, \bot\}$. The word *universe* is borrowed from set-theoretic usage: it is the ambient set out of which subsets are formed and over which predicates are evaluated. For our purposes the Boolean universe is the smallest non-trivial universe of interest. With one element, no nontrivial predicate exists. With two, we can already ask whether an input is one specific value or the other, which is the simplest discrimination a classifier can perform.

The smallness is the point. Part I of this book develops the algebra of this universe and nothing larger. Every result, every composition rule, every error-rate formula in the chapters that follow is a statement about predicates whose input is a single Boolean. The reason for that restriction is pedagogical and structural: the Boolean universe is the *atom* of the framework, and every richer universe (multi-bit inputs, finite enumerations, arbitrary algebraic types) is built up from Booleans by Kronecker products in Part II and beyond. If we understand the algebra of the four cells, we understand a great deal about systems that look more complicated on the

surface but reduce, on inspection, to compositions of noisy bits.

The deferral

A fully formal description of the four cells would assign each one a probability mass. Given the rates of Section 1.2 and the prevalence of $\top$ in the input stream, the four cells receive four numbers that sum to one. The resulting object is a joint distribution over the four pairs (truth, report). Multiplying it out involves the rates ε, ω, τ, η, and the prior probability of a $\top$-input. We will not write that joint distribution here.

The reason for the deferral is structural, not coy. The formal definition of the object we are circling, the type whose instances are exactly the noisy bits with two free rate parameters, lives in Section 4.5. That section earns the formal step by first introducing the two axioms that make the type well-posed (in chapter 4 §§4.2 and 4.3) and then proving the Kronecker factorization that justifies the parameter economy (chapter 4 §4.4). Writing the joint PMF in §1.3 would either repeat that machinery or assert it without support. Both are bad pedagogy. The honest path is to acknowledge that the four cells admit a joint distribution, name where that distribution gets written down, and continue. Section 4.5 returns to this picture with the formal definition; the table of joint probabilities, the channel matrix, and the worked biased-coin instance all appear there.

A qualitative example: the biased coin

The biased coin from Section 1.1 runs through the book, and it is worth seeing what the four cells look like for a specific instance. Take $\varepsilon = 0.1$ and $\omega = 0.2$: the coin reports a $\top$ when the truth is $\bot$ one trial in ten on average, and reports a $\bot$ when the truth is $\top$ one trial in five on average.

In the outcome matrix this means the following. Of every five $\top$-trials, roughly four land in the *correct positive* cell and roughly one lands in the *false negative* cell: the true-positive rate is $\tau = 1 - \omega = 0.8$ and the false-negative rate is $\omega = 0.2$. Of every ten $\bot$-trials, nine land in the *correct negative* cell and one lands in the *false positive* cell: $\eta = 1 - \varepsilon = 0.9$ and $\varepsilon = 0.1$.

These four conditional shares (four-fifths, one-fifth, one-tenth, nine-tenths) say what fraction of each *row* the noisy bit places in each *column*. They do not say what fraction of all trials lands in each cell, because that fraction also depends on how many trials had truth $\top$ versus truth $\bot$ in the first place. The bridge from row-conditional rates to outright cell probabilities is

exactly the joint distribution we are deferring. Section 4.5 takes the bridge.

Where the chapter goes from here

This section has named four cells of the joint distribution but refused to write down the joint distribution itself. Section 1.4 takes the next concrete step: you can *measure* what is in those cells, by sampling, even before Section 4.5 writes the joint formula. Estimation and formalization meet later; in Section 1.4 we start estimating. That returns the chapter to concrete arithmetic and sets up the question, picked up by chapters 2 through 4, of what an algebra of these noisy bits looks like.

1.4 Sampling and Estimating Rates

Section 1.2 defined what ε and ω mean as conditional rates read off a contingency table. Section 1.3 walked the joint structure of the four cells of that table. Both treated the rates as known. In practice they are not. A spam filter arrives without a label saying "my false-positive rate is 0.03"; a randomized algorithm comes with a worst-case error bound but not a tight one; a noisy channel must be characterized from observed traffic. The honest question is: given a finite sample of trials, how do we estimate the rates, and how confident should we be in the estimate?

This section answers in three short pieces. Binomial counts give the estimator. The central limit theorem gives an approximate confidence interval. A notebook simulation lets the reader watch both work and fail. The full theoretical treatment of the estimators (bias, variance, efficiency, alternative interval forms) lives in chapter 7 (Part II), which earns the formal step after the algebra of approximate sets is in place.

Binomial counts

Run a noisy bit n times. Of those n trials, suppose n_0 have truth $\bot$ and $n_1 = n - n_0$ have truth $\top$. The two row totals of the contingency table of Table 1.1 are n_0 and n_1. For each of the n_0 trials with truth $\bot$, the noisy bit independently reports $\top$ with probability ε; the count of such reports is the false-positive count FP. Independence across the n_0 trials makes the count a binomial random variable,

$$\mathrm{FP} \;\sim\; \mathrm{Binomial}(n_0, \varepsilon).$$

The same construction on the n_1 trials with truth $\top$ gives $\mathrm{FN} \sim \mathrm{Binomial}(n_1, \omega)$.

The natural empirical estimators are the row-conditional proportions

$$\hat{\varepsilon} \;=\; \frac{\mathrm{FP}}{n_0}, \qquad \hat{\omega} \;=\; \frac{\mathrm{FN}}{n_1}.$$

These are exactly the formulas of Section 1.2 with empirical counts in the numerator. The estimators have two properties worth stating at this stage and deferring the proof for later. First, each estimator is unbiased: $\mathbb{E}[\hat{\varepsilon}] = \varepsilon$ and $\mathbb{E}[\hat{\omega}] = \omega$. Second, the variances are

$$\mathrm{Var}(\hat{\varepsilon}) \;=\; \frac{\varepsilon(1-\varepsilon)}{n_0}, \qquad \mathrm{Var}(\hat{\omega}) \;=\; \frac{\omega(1-\omega)}{n_1}.$$

Both shrink as $1/n_0$ or $1/n_1$, so the estimator improves with more trials of the corresponding truth class. The variance numerator $\varepsilon(1-\varepsilon)$ peaks at $\varepsilon = 1/2$, where the noisy bit is hardest to characterize, and vanishes at $\varepsilon \in \{0, 1\}$, where a single trial settles the matter. Chapter 7 derives both facts.

A confidence interval

For large n_0, the central limit theorem applies to the standardized estimator. The result is that $\hat{\varepsilon}$ is approximately Normal with mean ε and variance $\varepsilon(1-\varepsilon)/n_0$. Replacing the unknown ε in the variance with its empirical estimate gives the standard Wald confidence interval at level $1-\alpha$,

$$\hat{\varepsilon} \;\pm\; z_{\alpha/2}\sqrt{\frac{\hat{\varepsilon}(1-\hat{\varepsilon})}{n_0}},$$

where $z_{\alpha/2}$ is the upper-$\alpha/2$ quantile of the standard Normal. The analogous interval for ω replaces $\hat{\varepsilon}$ with $\hat{\omega}$ and n_0 with n_1.

The Wald form is convenient and asymptotically correct, but it is also known to under-cover at small n_0 when ε sits near 0 or 1. The interval can even extend below 0 or above 1, which is a sign that the Normal approximation has broken down. Alternative intervals (Wilson, Clopper-Pearson, Jeffreys) handle the boundary case better. Chapter 7 develops the full theory; for now the Wald interval is the existence statement that an interval is available.

The chapter 1 notebook

The accompanying notebook `python/notebooks/ch01-noisy-bit.ipynb` makes the abstract statements concrete. It simulates a binary symmet-

ric channel with a known crossover probability and lets the reader vary the sample size across several orders of magnitude.

The notebook does four things. First, it runs many independent replications at a fixed true ε for each sample size and plots $\hat{\varepsilon}$ as a function of n_0 for n_0 sweeping from tens of samples to tens of thousands. The plot shows the empirical estimator settling toward the true rate, with a visible $1/\sqrt{n_0}$ band of fluctuation traced by the spread of the replications. The unbiasedness statement of the previous subsection is visible in the centering of the band; the variance formula is visible in its width.

Second, at a single fixed n_0 it runs many independent replications, constructs the Wald interval at the 95% level for each, and counts the fraction of intervals that actually contain the true ε. The CLT promises this fraction will approach 95%; the reader sees it land close to nominal at a moderate sample size.

Third, it sweeps n_0 across several orders of magnitude and plots the empirical coverage at each size. For small n_0 the coverage falls below 95%, exactly the under-coverage promised above, and the gap closes as n_0 grows. The reader watches the asymptotic guarantee fail in the regime where it is supposed to fail.

Fourth, it sweeps the true ε from 0.5 down to 0.01 at a fixed n_0 and shows the coverage gap widening as the rate approaches the boundary. The pattern is a small but real warning: an estimator that is asymptotically correct can still mislead at the sample sizes practitioners actually use. The negative-space chapters of Part III return to that point with examples in which the Wald approximation fails by enough to matter.

What comes next

We can now define a noisy bit (Section 1.2), walk its joint structure (Section 1.3), and estimate its rates from finite data with calibrated uncertainty. The chapter has done its concrete work. Section 1.5 zooms back out and explains why this single, narrow object, the noisy bit on two values with two free rates, deserves to be called the framework's atom and not merely its first example.

1.5 Why "Atom"?

The noisy bit has been built up carefully over four sections. We have seen its three faces (Section 1.1), named its two rates (Section 1.2), walked its joint structure (Section 1.3), and shown how to estimate the rates from

finite samples (Section 1.4). The opening overview of the chapter promised something more: the noisy bit would be the framework's *atom*, not merely its first example. It is worth pausing on what that word is doing here, because the rest of the book takes the claim as a working hypothesis.

A noisy bit is atomic in a precise sense. It has no internal Bernoulli structure to peel back. The input is a single Boolean value, the output is a single Boolean value, and the entire behavior is captured by two numbers, the false-positive rate ε and the false-negative rate ω. You cannot decompose a noisy bit into smaller noisy parts, because a smaller part would have a sub-Boolean input or output, and there is nothing below a Boolean. The atom is the place where the internal accounting stops and the external accounting begins. Every larger object the book builds, from chained channels to noisy AND gates to randomized algorithms to approximate sets to general algebraic types, will turn out to be assembled, by some specific composition rule, from noisy bits. The atom is the smallest object that the framework's machinery applies to without itself invoking the machinery.

A second sense of atomicity is worth flagging now. Section 2.4 will define the *model order* of a noisy system: the number of equivalence classes the input domain splits into. A noisy bit has model order two ($B_1 = \{\top\}$, $B_2 = \{\bot\}$), the smallest non-trivial value; everything richer is built by composition from objects whose model order is exactly two. The noisy bit is atomic not only because Booleans are indivisible but because its channel matrix has the smallest non-trivial structure any noisy system can have.

That commitment is empty until we discharge it. The next three chapters do the discharging in three different directions.

Chapters 2 through 4: the algebra of the atom

Chapter 2 (Chapter 2) gives the noisy bit a formal shape. The two-rate description of Section 1.2 becomes a 2×2 row-stochastic matrix Q. The binary symmetric channel, where $\varepsilon = \omega$, is one specialization; the asymmetric binary channel, where the two rates may differ, is the general case; the Z-channel, where one rate is forced to zero, is a useful boundary. The chapter introduces *model order* as a way to talk about how many distinct rates a channel actually has, and shows that the noisy bit sits at model order two, the minimum non-trivial value. Chapter 2 also takes the first step in composition: channels can be chained, and the matrix product of two row-stochastic matrices is another row-stochastic matrix, so a chain of channels is itself a channel. The atom is closed under serial composition.

Chapter 3 (Chapter 3) raises the question that classical logic forgot to

ask. AND, OR, and NOT are familiar operations on Boolean values. What do they do to noisy bits? If the two operands of an AND each carry their own ε and ω, what are the effective rates of the result? The answer is the composition theorem (Section 3.3): the output is itself a noisy bit, with rates computable in closed form from the rates of the operands. The Boolean operations stay closed when their operands stop being exact. The chapter develops the closed-form rules and applies them to several worked examples. The atom is closed under the operations of Boolean algebra.

Chapter 4 (Chapter 4) axiomatizes. Chapters 1 through 3 silently assume that error events at distinct inputs are independent, and that the rates themselves do not drift across the universe of inputs. Chapter 4 names these as Axiom 1 (Section 4.2, element-wise independence) and Axiom 2 (Section 4.3, conditional independence of per-block rates). The Kronecker factorization theorem (Section 4.4) shows what those two axioms purchase: the joint distribution over a block of inputs factors as a tensor product of the per-element channel matrices, which is what makes the per-element description sufficient. Chapter 4 then writes the formal definition Bernoulli[bool] $= (X, Y, Q)$ (Section 4.5), where X is the input domain, Y is the output report, and Q is the channel matrix that chapter 2 will introduce. The triple ties the chapter 1, chapter 2, and chapter 4 threads together in a single object, and chapter 4 shows that every concrete object chapters 1 through 3 worked with is an instance of it. The atom is characterized axiomatically, not just by example.

Miller-Rabin: a familiar algorithm in unfamiliar dress

One example deserves its own preview because it is the first place the framework will pay an unfamiliar dividend. The Miller-Rabin primality test is among the most widely deployed randomized algorithms in computer science; every cryptographic library uses it. The standard treatment presents Miller-Rabin as a clever piece of number theory attached to a clever piece of probability. Section 3.4 takes a different view. One round of Miller-Rabin is exactly a noisy bit, with $\omega = 0$ (a prime never fails a witness test) and $\varepsilon \leq 1/4$ (the strong-liar count for any composite is at most a quarter of the witness pool). The k-round test is the AND of k such noisy bits, and the famous bound $\varepsilon_k \leq (1/4)^k$ is not a separate piece of analysis but a direct application of the composition theorem of Section 3.3. The number theory contributes the per-round $1/4$ bound; everything else is the algebra of the atom. We will see the calculation in full in chapter 3.

Part II: sets out of atoms

Chapter 1 has shown what a single noisy bit looks like. A natural next question is what happens when many of them are arranged into a structure. Part II asks the version of this question that membership-test data structures invite. A set S over a universe U induces, for each element $u \in U$, a Boolean query, namely "is u in S?". An approximate set is one in which each such query is a noisy bit. Bloom filters, the canonical example, are a recipe for producing a noisy-bit instance per universe element with one-sided error ($\omega = 0$) and a tunable ε. The atom becomes a building block, and the project of Part II is the algebra of the resulting structures: how set operations propagate error rates, how classification measures quantify quality, and how much space an approximate set must consume to hit a given accuracy target. Each of these threads is grounded in the atom but is not reducible to it; the atom is the unit, not the whole story.

Part III: lifting the atom past Boolean

The Boolean domain is convenient but not fundamental. Once the machinery of Part II is in place, the natural question is whether the same construction works for richer codomains. Part III replaces $\{\top, \bot\}$ with an arbitrary algebraic type T. Approximate maps, approximate relations, and approximate values of structured types all become available. The two-element model order of chapter 2 ceases to be a ceiling. The atom of chapter 1 is the simplest non-trivial special case of a much larger family of atoms, one for each algebraic type, and the closures and composition rules of Part I generalize accordingly. Where Part II takes the atom and builds a collection out of it, Part III takes the atom and changes the alphabet it works in.

Closing

This has been a forward glance. The chapter ahead, namely Chapter 2, picks up where Section 1.4 left off and gives the noisy bit its first formal vocabulary. Before that, the next and final section of this chapter, the bibliographic notes, names the antecedents of what we have just done: Shannon's channel framing, the contingency-table tradition from medical statistics, the precision-and-recall vocabulary from information retrieval, and the original Miller and Rabin papers on primality testing. The historical record will tell us that almost every component of chapter 1 was already in use somewhere; what is new is the decision to treat them all as one object and to call that object an atom. Chapter 1 has done what chapter 1 should do, namely

shown the framework's atom, named it, and pointed at the algebra the rest of Part I unfolds; everything that follows is composition, and the atom is what gets composed.

Bibliographic Notes

Shannon and the channel framing. The noisy-channel reading of Face 2 in Section 1.1—and the binary symmetric channel that opens Chapter 2 (Section 2.1)—descend directly from Shannon [Sha48]. Shannon's 1948 paper introduced the discrete memoryless channel as the mathematical object for noisy communication: a conditional distribution $P(\text{output} \mid \text{input})$ together with a capacity defined as the supremum of the mutual information between input and output. The binary symmetric channel is among the simplest of Shannon's channels, and it remains the standard pedagogical entry point to the channel framework, both in his original paper and in essentially every information-theory textbook that has followed.

The contingency-table tradition. The 2×2 contingency table that anchors Section 1.2 predates the modern probability framing of this book by decades. It grew up in epidemiology, biostatistics, and medical diagnostic testing, where the natural objects of study are tests with a true-positive (sensitivity) and a true-negative (specificity) rate, and where the population prevalence of the underlying condition is part of every working calculation. The vocabulary of sensitivity, specificity, and the cells of the 2×2 table travelled from clinical screening into machine learning largely unchanged; what differs across fields is which marginal the practitioner conditions on, and which derived rate gets the spotlight. The cross-walk at the end of Section 1.2—from (ε, ω) to precision and recall—is the modern statement of a much older bilingual conversation between epidemiology and signal detection.

Precision, recall, and information retrieval. The vocabulary of *precision* and *recall* arrived in this book by a different route, from information retrieval. In the IR setting one issues a query against a corpus and asks: of the documents returned, what fraction are relevant (precision); of the relevant documents in the corpus, what fraction were returned (recall). The textbook reference is Manning, Raghavan, and Schütze [MRS08], whose Chapter 8 develops the precision-recall vocabulary, the precision-recall curve, and the F-measure as the standard combined score. Chapter 1's emphasis on

the difference between (ε, ω) and (precision, recall)—that the first pair is intrinsic to a noisy bit and the second depends also on the workload—is the contingency-table analogue of the IR distinction between a system's intrinsic discriminating ability and its observed behaviour on a given query mix.

Miller, Rabin, and the primality tease. The Miller-Rabin algorithm previewed in Section 1.5 and developed in Section 3.4 has two distinct intellectual parents. Miller [Mil76] introduced the witness-based structure under the Extended Riemann Hypothesis; Rabin [Rab80] removed the hypothesis by randomizing the choice of witness and proved the unconditional $1/4$ bound on the per-round false-positive rate. Chapter 3's reading of Miller-Rabin—one round of the test as a Bernoulli[bool] with $\omega = 0$ and $\varepsilon \leq 1/4$, the k-round protocol as a composition—restates that result in the language of this text. The full historical record, the proof of the $1/4$ bound, and the companion treatment of randomized algorithms more broadly are taken up in Section 3.5.

The Bernoulli papers. The atom of this chapter, the noisy bit, is the order-2 Boolean instance of the formal Bernoulli framework developed in our companion papers. The two-axiom model and the rate-propagation theorems originate in Towell [Tow26a]; the extension from approximate sets to approximate values over arbitrary algebraic types—and, in particular, the treatment of Bernoulli[bool] as a value-with-error type—is the subject of Towell [Tow26d]. The present book [Tow26b] reorganizes that material as a textbook, starting from the noisy bit, building outward to sets, maps, relations, and the algebraic types that the companion paper studies in their full generality. Chapter 1's contribution is the decision to treat the biased coin, the binary symmetric channel, and the classifier verdict as three faces of one object and to call that object an atom; almost every other ingredient of the chapter has a longer history in one of the traditions noted above.

Exercises

1. ⋆ **Confusion matrix to rate conversion.** A binary classifier is evaluated on a labelled test set and produces the following counts: TP = 90, FP = 10, FN = 10, TN = 90.

   (a) Compute the empirical false-positive rate $\hat{\varepsilon}$ and false-negative rate $\hat{\omega}$ using the row-conditional proportions from Section 1.2.

Be explicit about which counts go in the numerator and which in the denominator.

(b) Compute the precision $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ and the recall $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.

(c) In one sentence, explain why $\hat{\varepsilon}$ and precision are *different* numbers even though both involve the false-positive count. (*Hint:* which marginal does each one condition on?)

2. ⋆ **Three real-world noisy bits and one non-example.** The three faces of Section 1.1 (BSC, biased coin, classifier verdict) are stand-ins for many real systems. Name *three* systems of your own that you think of as noisy bits in the sense of Section 1.2 (a Boolean-output system with stable per-input rates).

(a) For each of the three, identify what plays the role of the Boolean input and Boolean output, then write a rough estimate of ε and ω.

(b) Construct one explicit *non-example* whose per-input error rates are not stable: they change over time, or depend on which queries came before. The error behaviour is real, but the rates are not the stable numbers the noisy-bit model of Section 1.2 assumes.

(c) In one sentence, identify the stability property of the noisy-bit model that your non-example violates. We return to non-examples like this in chapter 16, which makes the failure modes of the model its explicit subject.

3. ⋆ **Joint table to rates.** A noisy bit is described by the joint table below, where the truth has prior $\Pr\{X=\top\}=0.3$ and $\Pr\{X=\bot\}=0.7$:

	$Y=\top$	$Y=\bot$
$X=\top$	0.27	0.03
$X=\bot$	0.07	0.63

The four cell entries are joint probabilities $\Pr\{X,Y\}$, and they sum to 1.

(a) Verify that the row sums match the priors.

(b) Compute the row-conditional false-positive rate $\varepsilon = \Pr\{Y = \top | X = \bot\}$ and the false-negative rate $\omega = \Pr\{Y = \bot | X = \top\}$ by dividing the appropriate cell by its row sum (Section 1.2).

(c) Confirm that ε and ω are properties of the noisy bit itself, not of the prior $\Pr\{X = \top\}$: recompute them for a different prior (say $\Pr\{X = \top\} = 0.5$) with the same conditional behaviour, and check that they are unchanged. In one sentence, explain why this independence from the prior is what lets the noisy-bit model get by with two rates.

4. ⋆⋆ **Empirical estimation under varying n (notebook-tied).** Open the chapter 1 companion notebook `python/notebooks/ch01-noisy-bit.ipynb`. Use the binary symmetric channel simulator with true crossover $\varepsilon = 0.10$ and run the estimator at $n \in \{10, 100, 1000, 10000\}$.

   (a) Tabulate $\hat{\varepsilon}$ for each value of n from a single run, and the absolute deviation $|\hat{\varepsilon} - \varepsilon|$ from the true rate.

   (b) Plot $\hat{\varepsilon}$ as a function of n on a log-n axis, with the true rate drawn as a horizontal reference line. Describe in one sentence how the spread of the estimator around the true rate shrinks with n.

   (c) Compare the rate at which the empirical deviation shrinks to the predicted standard error $\sqrt{\varepsilon(1-\varepsilon)/n}$ from Section 1.4. Is the observed convergence consistent with a $1/\sqrt{n}$ shrinking band?

5. ⋆⋆ **The trivial biased coin.** A trivial biased coin is a noisy bit that ignores the input entirely: it returns $\top$ with probability p and $\bot$ with probability $1 - p$, regardless of whether the truth is $\top$ or $\bot$.

   (a) Using the definitions in Section 1.2, write ε and ω as functions of p.

   (b) At what value of p is the trivial biased coin *useless* for the classification task, in the sense that ε and ω are both equal? Compute that pair of rates.

   (c) At what values of p does the trivial biased coin achieve its *best* behaviour, in the sense that one of the two rates is exactly zero? Which rate, at which value of p?

   (d) Comment briefly on the asymmetry: a constant function is a perfect classifier for one truth class and a maximally bad one for the other. The biased-coin parameterization makes this trade-off visible.

6. ⋆⋆ **NOT swaps the two rates.** Let b be a noisy bit with rates (ε, ω). Define $b' = \neg b$ to be the noisy bit obtained by swapping the

output: whenever b reports $\top$, b' reports $\bot$, and vice versa. (No additional randomness is introduced; b' is a deterministic relabelling of the output of b.)

(a) Write out the four conditional probabilities $\Pr\{Y = \top | X = \top\}$, $\Pr\{Y = \bot | X = \top\}$, $\Pr\{Y = \top | X = \bot\}$, $\Pr\{Y = \bot | X = \bot\}$ for b, following the row-conditional convention of Section 1.2.

(b) Apply the output swap to b and write out the same four conditional probabilities for b'.

(c) Read off the rates of b'. Show that $\varepsilon' = \omega$ and $\omega' = \varepsilon$.

(d) Conclude that NOT acts on the parameter pair as the swap $(\varepsilon, \omega) \mapsto (\omega, \varepsilon)$. In one sentence, interpret this geometrically as a reflection in the parameter plane $[0, 1]^2$.

7. $\star\star\star$ **Threshold sweep and the error-rate trade-off curve.** A score-based classifier produces a score $s(x) \in [0, 1]$ on each input x and reports $\top$ if and only if $s(x) > \theta$ for a threshold θ. For each value of θ, the system is a noisy bit with rates $\varepsilon(\theta)$ and $\omega(\theta)$ that depend on θ.

(a) As θ increases from 0 to 1, describe what happens to $\varepsilon(\theta)$ and $\omega(\theta)$: which one increases, which one decreases, and why. (You do not need a formula; reason from the threshold rule of Section 1.2.)

(b) At $\theta = 0$ the classifier reports $\top$ for almost every input; at $\theta = 1$ it reports $\bot$ for almost every input. Give the (ε, ω) pair at each of these two extremes.

(c) The curve traced by $(\varepsilon(\theta), \omega(\theta))$ as θ sweeps from 0 to 1 is the trade-off curve between the two error rates. The standard *ROC curve* plots the true-positive rate against the false-positive rate, which is the same information re-axed. Sketch (in words) the shape of the (ε, ω) trade-off curve, and explain why a classifier cannot in general drive both rates to zero at once.

8. $\star\star$ **Sample-size formula via the CLT bound.** Section 1.4 gives the variance of the empirical false-positive estimator as $\mathrm{Var}\,(()\,\hat{\varepsilon}) = \varepsilon(1 - \varepsilon)/n_0$. Suppose we want the standard error to satisfy $\sqrt{\mathrm{Var}\,(()\,\hat{\varepsilon})} \leq \sigma$ for some target $\sigma > 0$.

(a) Solve the inequality $\sqrt{\varepsilon(1 - \varepsilon)/n_0} \leq \sigma$ for n_0. Write the smallest integer n_0 that satisfies the bound as a function of ε and σ.

(b) Evaluate n_0 for the practical target $\sigma = 0.01$ at $\varepsilon \in \{0.01, 0.05, 0.10, 0.50\}$. Tabulate the four sample-size requirements.

(c) Identify the value of ε that requires the largest sample size. In one sentence, explain why this maximum sits at $\varepsilon = 1/2$ rather than at the boundary (cf. Section 1.4, which observes that the variance numerator peaks at 1/2).

9. ⋆⋆ **Informal LLN argument for $\hat{\varepsilon} \to \varepsilon$.** Argue informally that under iid sampling, the empirical estimator $\hat{\varepsilon}$ from Section 1.4 converges to the true rate ε in probability as $n_0 \to \infty$.

(a) State precisely the iid assumption you are using: each of the n_0 negative-truth queries produces a Bernoulli(ε) output independent of all the others. Write the empirical count FP as a sum of n_0 such Bernoulli variables.

(b) Recall (or look up) Chebyshev's inequality: for any random variable Z with finite variance and any $\delta > 0$, $\Pr\{|Z - \mathbb{E}[Z]| \geq \delta\} \leq \mathrm{Var}\,(()\,Z)/\delta^2$. Apply this to $Z = \hat{\varepsilon}$, using the unbiasedness statement $\mathbb{E}[\hat{\varepsilon}] = \varepsilon$ and the variance formula $\mathrm{Var}\,(()\,\hat{\varepsilon}) = \varepsilon(1-\varepsilon)/n_0$ from Section 1.4.

(c) Show that the right-hand side of the Chebyshev bound goes to zero as $n_0 \to \infty$ for any fixed $\delta > 0$. Conclude that $\hat{\varepsilon} \to \varepsilon$ in probability.

(d) In one sentence, identify which assumption from part (a) is most likely to fail in a real data system, and what kind of correlation across queries would prevent the Chebyshev argument from going through.

10. ⋆⋆⋆ **Stable rates, correlated errors.** The noisy-bit model rests on two assumptions: the per-input rates are stable across queries, and the noise events on different queries are independent. Construct an explicit Boolean-input, Boolean-output system with *stable* per-input rates whose errors are nonetheless *correlated* across queries, so that a naive iid estimator misjudges its behaviour.

(a) Construct the system. Specify the input domain (Boolean), the output domain (Boolean), and the mechanism that determines the output given the input. The mechanism must produce a well-defined Boolean output on every query and must have well-defined, stable per-input rates ε and ω, but the error events on successive queries must be statistically dependent in a stated way.

(b) Identify precisely how the independence assumption fails: describe the dependence between the error event on one query and the error event on the next, and confirm in one sentence that the per-input rates themselves are still stable (so it is independence, not stability, that breaks).

(c) Argue that a naive observer who computes $\hat{\varepsilon}$ and $\hat{\omega}$ from a sample of queries, and then treats those estimates as if the queries were iid, would misjudge the system. Be specific: the point estimates may be fine, but what quantity (an error bar, a coverage claim, a sample-size requirement) would they get wrong, and in which direction?

(d) Chapter 16 is the negative-space chapter of the book: its explicit subject is what fails to fit the model. Your construction is a candidate worked example for that chapter; in one sentence, state the conceptual lesson your example would carry there.

11. ⋆⋆⋆ **Stress test of the notebook coverage estimator.** The chapter 1 notebook `python/notebooks/ch01-noisy-bit.ipynb` runs a Wald-interval coverage check that empirically counts the fraction of 95% intervals containing the true ε. Section 1.4 flags the under-coverage caveat: the Wald form is asymptotically correct but is known to under-cover at small n_0 when ε sits near 0 or 1.

(a) Identify parameter regimes (combinations of n_0 and ε) under which you predict the empirical coverage will fall visibly below 0.95. Justify your prediction by reference to the under-coverage caveat in Section 1.4.

(b) Run the notebook's coverage estimator in two regimes of your choosing: one where you predict good coverage (close to 0.95) and one where you predict bad coverage. Report the two empirical coverage numbers.

(c) Explain in one paragraph *why* the Wald form under-covers in the bad regime. The argument should reference the Normal approximation underlying the Wald form and identify what fails when ε sits near a boundary or n_0 is small. (No rigorous derivation is needed; the explanation should be sound.)

(d) State which of the alternative intervals named in Section 1.4 (Wilson, Clopper-Pearson, Jeffreys) you would expect to perform better in your bad-coverage regime, and briefly explain the intuition. Full derivations are deferred to chapter 7.

Chapter 2

Binary Channels and Model Order

After this chapter you will be able to:

- Define a binary channel formally and write its 2×2 transition matrix.
- Distinguish symmetric (BSC) from asymmetric (Z-channel and general) binary channels and connect each to model order.
- Compose two binary channels via matrix product and read off the resulting channel.
- State the Boolean ceiling at order 2: any predicate over the input domain $\{\top, \bot\}$ has model order at most 2.

2.1 The Binary Symmetric Channel

One bit, one channel

Section 1.1 introduced three framings of the noisy bit: a biased coin, a noisy communication channel, and a classifier's verdict. All three share a common structure that deserves a name and a precise description. The *binary symmetric channel* is that common structure, stripped to its essentials.

The input is a single Boolean value: $\top$ ("in the set", "heads", "positive") or $\bot$ ("not in the set", "tails", "negative"). The output is also Boolean. The channel transmits the input faithfully most of the time, but occasionally flips it. The defining property of the *symmetric* channel is that the flip

probability is the same regardless of which input is being transmitted: a positive is misreported as negative with the same probability that a negative is misreported as positive.

This symmetry is a genuine constraint. Most real noisy Boolean systems do not satisfy it: a spam filter is far more likely to mis-classify legitimate mail as spam (false positive) than to miss actual spam (false negative). The BSC is the ideal symmetric case, and understanding it precisely is the right starting point before the asymmetric generalization of Section 2.2.

Definition 2.1.1 (Binary Symmetric Channel)**.** *A* binary symmetric channel *with crossover probability* $\varepsilon \in (0,1)$ *is a random map* $\phi : \{\top, \bot\} \to \{\top, \bot\}$ *satisfying*

$$\Pr\{\phi(x) \neq x\} \;=\; \varepsilon \quad \textit{for every } x \in \{\top, \bot\}.$$

The parameter ε *is the* crossover probability*: the probability that the output differs from the input.*

Two things are worth noting immediately. First, the crossover probability is the same for both inputs: $\Pr\{\phi(\top) \neq \top\} = \Pr\{\phi(\bot) \neq \bot\} = \varepsilon$. This is the symmetry. Second, ε plays the role of both the false-positive rate and the false-negative rate simultaneously: in the Bernoulli framework's notation from Section 1.1, the BSC has $\varepsilon = \omega$ (using the convention that ε is the rate of misreporting a negative as positive and ω is the rate of misreporting a positive as negative). The BSC is the special case where these two rates collapse to a single parameter.

The channel matrix

A single parameter and a prose definition are useful, but a matrix representation is more useful still. It displays all four input-output probabilities at once, supports direct computation, and generalizes cleanly to the asymmetric case in Section 2.2 and to multi-query systems in Chapter 4.

We organize the channel as a 2×2 matrix Q, called the *channel transition matrix* or simply the *channel matrix*. The convention throughout this book is:

Notation. *The channel matrix* Q *for a binary channel is a* 2×2 *stochastic matrix.* ***Rows*** *are indexed by the input block: row 1 corresponds to block* $B_1 = \{\top\}$ *(positives), row 2 to block* $B_2 = \{\bot\}$ *(negatives).* ***Columns*** *are indexed by the output label: column 1 corresponds to output* $\top$ *(reported positive), column 2 to output* $\bot$ *(reported negative). The* (i, j)*-entry of* Q *is*

the probability of observing output j given that the input belongs to block B_i. Each row sums to 1: Q is row-stochastic.

Under this convention, the four entries of Q have specific names in the confusion-table vocabulary. For the order-2 Boolean model with $B_1 = \{\top\}$ and $B_2 = \{\bot\}$:

	output $\top$	output $\bot$
input $\top$ (B_1)	τ (true-positive rate)	ω (false-negative rate)
input $\bot$ (B_2)	ε (false-positive rate)	η (true-negative rate)

So the general order-2 channel matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where we use $\tau = 1 - \omega$ and $\eta = 1 - \varepsilon$ from the row-stochastic constraint.

For the BSC, the symmetry constraint $\varepsilon = \omega$ collapses this to a single parameter:

$$Q_{\mathrm{BSC}} \;=\; \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Both rows sum to 1. Both diagonal entries equal $1-\varepsilon$ (correct output). Both off-diagonal entries equal ε (flip). The two rows are simply permutations of each other, which is the matrix expression of symmetry. The BSC has a single free parameter; the general order-2 channel has two. We return to this reduction in Section 2.4.

Reading the matrix

Before computing with Q, it is worth reading it carefully for a small ε. Take $\varepsilon = 0.05$ as a concrete case.

$$Q \;=\; \begin{pmatrix} 0.95 & 0.05 \\ 0.05 & 0.95 \end{pmatrix}.$$

Reading row 1: a positive input ($\top$) is correctly reported as positive ($\top$) with probability 0.95 and incorrectly reported as negative ($\bot$) with probability 0.05. Reading row 2: a negative input ($\bot$) is incorrectly reported as positive ($\top$) with probability 0.05 (a false positive) and correctly reported as negative ($\bot$) with probability 0.95. The channel is "mostly right" with a small error floor.

The channel matrix carries more information than the crossover probability alone: it tells you the probability of every input-output pair simultaneously, without requiring you to remember which rate is which or convert between TPR and FNR in your head. Once we introduce multiple queries in Section 2.3 and Section 4.4, the matrix form pays for itself many times over.

Worked example: BSC with $\varepsilon = 0.05$

We verify all four probabilities and three derived quantities for $\varepsilon = 0.05$.

The matrix.

$$Q = \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix} = \begin{pmatrix} 0.95 & 0.05 \\ 0.05 & 0.95 \end{pmatrix}.$$

Row sums. Row 1: $0.95 + 0.05 = 1.00$. Row 2: $0.05 + 0.95 = 1.00$. Both rows sum to 1, confirming that Q is row-stochastic.

Single-query error probability. If the input is drawn uniformly from $\{\top, \bot\}$, each with probability $1/2$, the probability of an incorrect output is

$$\Pr\{\text{error}\} = \tfrac{1}{2}\cdot\Pr\{\phi(\top) \neq \top\} + \tfrac{1}{2}\cdot\Pr\{\phi(\bot) \neq \bot\} = \tfrac{1}{2}\cdot 0.05 + \tfrac{1}{2}\cdot 0.05 = 0.05.$$

Under a uniform prior on the input, the error probability equals the crossover probability ε. This is not a coincidence: symmetry means the marginal error rate does not depend on the input distribution, as long as $\varepsilon = \omega$.

Two specific joint probabilities. Suppose the input is positive ($\top$). The probability of a correct positive output is $Q_{11} = 0.95$. The probability of a false-negative output is $Q_{12} = 0.05$.

Now suppose the input is negative ($\bot$). The probability of a false positive is $Q_{21} = 0.05$. The probability of a correct negative is $Q_{22} = 0.95$.

Accuracy. For a uniform prior, accuracy $= \frac{1}{2}\cdot 0.95 + \frac{1}{2}\cdot 0.95 = 0.95 = 1 - \varepsilon$. As expected, the BSC with crossover 0.05 has 95% accuracy under a uniform input distribution.

The BSC as the symmetric special case of order 2

The source papers in the Bernoulli framework use two conventions that are worth aligning before proceeding. Some sections call the BSC a "first-order" model (one free parameter), while Chapter 4 calls it the *symmetric special*

case of the order-2 model. These are both correct, depending on what is being counted.

Model order, formalized in Section 2.4, counts the number of distinct partition blocks in the input domain, not the number of free parameters. The Boolean input domain $\{\top, \bot\}$ has two elements, and in the order-2 model these form two blocks: $B_1 = \{\top\}$ (positives) and $B_2 = \{\bot\}$ (negatives). The model has two blocks and therefore model order 2, regardless of whether the BSC's symmetry constraint $\varepsilon = \omega$ further reduces the free parameter count from 2 to 1.

Put differently: model order is about the partition of the input domain (how many distinct error regimes exist), while parameter count is about how many numbers you need to write down (which may be less than the model order, if the channel is symmetric or otherwise constrained). The BSC is order 2 with one free parameter; the general asymmetric channel of Section 2.2 is order 2 with two free parameters.

Section 4.3 makes the connection to partition blocks explicit in the context of Axiom 2. The convention $B_1 = \{\top\}$, $B_2 = \{\bot\}$ used there matches the row labeling of Q throughout this chapter and in Chapter 4.

Connection to chapter 1: the three faces revisited

Section 1.1 presented three framings of the same abstract object. We can now describe each one precisely using the BSC channel matrix.

The biased coin. A coin with heads probability $1-\varepsilon$ and tails probability ε, where "heads" is the correct output, is a BSC. The two sides of the coin are the two elements of $\{\top, \bot\}$; the flip probability is ε for both sides, because the coin is symmetric. The channel matrix is exactly Q_{BSC} above. We generalize to an asymmetric coin, with different head-bias and tail-bias, in Section 2.2.

The noisy communication channel. This is Shannon's BSC in its original form [Sha48]: a bit sent over a channel where each transmitted bit is flipped independently with probability ε. The input alphabet is $\{0, 1\}$ and the channel matrix is Q_{BSC} with the mapping $1 \leftrightarrow \top$ and $0 \leftrightarrow \bot$.

The classifier verdict. A binary classifier with equal false-positive and false-negative rates is a BSC. In practice, classifiers rarely have $\varepsilon = \omega$; the symmetric case is a simplifying assumption. When the two rates are unequal, the classifier is better modeled as an asymmetric binary channel, the subject of Section 2.2.

The unifying point: all three framings share the same channel matrix Q_{BSC} with the same single crossover parameter ε. The names differ; the probability model is identical.

What makes symmetry a strong constraint

The BSC's symmetry constraint, $\varepsilon = \omega$, is restrictive but often useful as a first approximation. Three properties follow from it.

First, the channel matrix Q_{BSC} is a *circulant* matrix: its rows are cyclic permutations of each other. This makes the algebra of composing BSCs, discussed in Section 2.3, particularly clean.

Second, the *capacity* of the BSC, defined as the maximum mutual information between input and output, is $1 - H_2(\varepsilon)$ bits per use, where $H_2(\varepsilon) = -\varepsilon \log_2 \varepsilon - (1-\varepsilon)\log_2(1-\varepsilon)$ is the binary entropy function. Capacity is maximized at $\varepsilon = 0$ (a perfect channel, transmitting one bit per use without error) and at $\varepsilon = 1$ (a perfectly anti-correlated channel, which also transmits one bit per use after relabeling outputs); it is minimized at $\varepsilon = 1/2$, where the output is statistically independent of the input and no information is transmitted. Exercise 4 works through the derivation.

Third, the BSC is its own *inverse channel* in the following sense: composing a BSC with crossover ε with another BSC with the same crossover ε yields a BSC with crossover $2\varepsilon(1-\varepsilon)$, which is strictly greater than ε for $0 < \varepsilon < 1/2$. Two imperfect copies of the same symmetric channel make things worse, not better. The formula for composed BSC crossovers is derived in full in Section 2.3.

Notation in this chapter

Throughout chapters 2 and beyond, the following convention applies:

- Q always denotes a channel transition matrix, with rows indexed by input block and columns indexed by output label.
- ε (macro: `\fprate`, rendered as ε) is the false-positive rate: the probability that a negative input is reported as positive.
- ω (macro: `\fnrate`, rendered as ω) is the false-negative rate: the probability that a positive input is reported as negative.
- $\tau = 1 - \omega$ is the true-positive rate, and $\eta = 1 - \varepsilon$ is the true-negative rate.

- For the BSC, $\varepsilon = \omega$, and we use ε for the single crossover parameter throughout.

The source papers in this research program sometimes use (FPR, TPR) rather than (FPR, FNR) as the parameter pair. The two choices are equivalent: $\text{TPR} = 1 - \text{FNR}$ is a deterministic function, so any statement about one pair translates directly to the other.[1]

Looking ahead

The BSC is the symmetric special case of a more general object. In Section 2.2 we lift the constraint $\varepsilon = \omega$ and allow the two error rates to be independent: the resulting *asymmetric binary channel* covers Bloom filters (where $\omega = 0$ by construction) and medical tests (where FPR and FNR can be tuned independently by adjusting a threshold). In Section 2.3 we ask what happens when two channels are applied in sequence, and derive the composed channel's matrix from the two input matrices via matrix multiplication. In Section 2.4 we formalize what we have been calling "order 2" and prove the Boolean ceiling theorem: because the Boolean domain has exactly two elements, every binary channel has model order at most 2.

2.2 The Asymmetric Binary Channel

Lifting the symmetry constraint

The Binary Symmetric Channel of Section 2.1 is a useful starting point, but its defining property—equal error rates for both inputs—is rarely satisfied in practice. A spam filter flags legitimate mail as spam far more often than it lets genuine spam through. A Bloom filter returns false positives but never false negatives. A medical screening test is deliberately tuned to minimize one kind of error at the cost of the other.

These systems are not defective BSCs; they are different channels altogether. When we allow the false-positive rate ε and the false-negative rate ω to be independent parameters, we obtain the *asymmetric binary channel*: the same two-element input and output domains, the same row-stochastic channel matrix Q, but no constraint tying ε to ω.

[1] Specifically, the companion paper [Tow26a] uses (FPR, TPR) in places. In converting source material to this textbook, we have replaced that convention with (FPR, FNR) throughout, using the alex.sty macros ε and ω.

Definition 2.2.1 (General asymmetric binary channel)**.** *An* asymmetric binary channel *with false-positive rate* $\varepsilon \in [0,1)$ *and false-negative rate* $\omega \in [0,1)$ *is a random map* $\phi : \{\top, \bot\} \to \{\top, \bot\}$ *satisfying*

$$\Pr\{\phi(\top) \neq \top\} = \omega,$$
$$\Pr\{\phi(\bot) \neq \bot\} = \varepsilon.$$

The channel transition matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where row 1 corresponds to input block $B_1 = \{\top\}$ *(positives), row 2 to block* $B_2 = \{\bot\}$ *(negatives), and columns to output labels* $\top$ *(left) and* $\bot$ *(right), following the convention established in Section 2.1.*

Two special cases are immediately visible in the matrix. Setting $\omega = \varepsilon$ recovers the BSC of Definition 2.1.1: the two rows become identical up to a reflection, and the single crossover parameter ε is enough to describe the channel. Setting $\omega = 0$ makes row 1 become $(1, 0)$: a positive input is always reported correctly. This is the *Z-channel*, the subject of the next subsection. The name of each special case and its connection to a familiar system is spelled out in full in Section 2.4.

The Z-channel: one-sided error

The most widely encountered asymmetric channel in computer science is the *Z-channel*. In the variant used here, the transmission $\top \to \top$ is perfect, and the only error is $\bot \to \top$: a negative input incorrectly reported as positive.[2]

Definition 2.2.2 (Z-channel)**.** *A* Z-channel *with false-positive rate* $\varepsilon \in (0,1)$ *is an asymmetric binary channel with* $\omega = 0$*. Its channel matrix is*

$$Q_Z \;=\; \begin{pmatrix} 1 & 0 \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The Z-channel has a sharp asymmetry: false negatives are impossible ($\omega = 0$), while false positives occur with probability ε. This makes an output of $\bot$ fully reliable—it cannot be a mistake—while an output of $\top$ is uncertain.

[2]Information-theory texts often define the Z-channel with the opposite asymmetry—false negatives only, no false positives ($\omega > 0$, $\varepsilon = 0$), so that $\bot \to \bot$ is perfect and the only error is $\top \to \bot$. Both variants are equivalent up to relabeling of inputs and outputs. The convention here aligns with the Bloom filter and other positive approximate sets, where the false-positive direction is the natural error mode.

Bloom filters as Z-channels. The Bloom filter is the canonical Z-channel instance from data structures. A Bloom filter answers membership queries for a set S: given an element x, it returns either *present* ($\top$) or *absent* ($\bot$). The filter is constructed so that

$$x \in S \implies \text{Bloom returns } \top \quad \text{(no false negatives)},$$

while a non-member $x \notin S$ may still trigger $\top$ with some probability ε (a false positive). In the channel-matrix language, the Bloom filter is exactly a Z-channel: $\omega = 0$ by construction, ε controlled by the filter's bit-width and hash count. When the Bloom filter returns absent ($\bot$), the answer is certain; when it returns present ($\top$), the answer is probabilistic. This semantic asymmetry—reliable "no", uncertain "yes"—is the hallmark of all Z-channel systems.

The biased coin, revisited

Section 1.1 introduced the biased coin as one framing of the noisy bit. In Section 2.1 we identified it as a BSC: a coin with the same flip probability for heads and tails. But a coin can be *doubly biased*: the probability of landing heads on a flip from heads may differ from the probability of landing tails on a flip from tails. Such a coin is not a BSC; it is an asymmetric binary channel.

Concretely: let $\top$ represent heads and $\bot$ represent tails. Suppose a flipped-heads coin shows tails with probability ω, and a flipped-tails coin shows heads with probability ε, where $\omega \neq \varepsilon$. The channel matrix for this coin is exactly Q from Definition 2.2.1. The BSC of Section 2.1 was the special case $\omega = \varepsilon$, where the double bias collapses to a single bias. The asymmetric channel is the general case that was always there, waiting for ω and ε to differ.

Physical examples are easy to construct. A coin that spins on a surface comes to rest heads or tails with probabilities that depend on its initial face. A weighted die, approximated as a two-sided object, has different effective flip rates for each face. A market that can only move a small amount in one direction (a price floor or ceiling) is an asymmetric channel on the direction of movement. In every case, the natural description is an asymmetric binary channel with two independent parameters ε and ω.

Medical tests as asymmetric channels

Medical screening tests illustrate the asymmetric channel in a domain where the two error types have very different consequences. A test for a serious

disease might be deliberately calibrated to err on the side of sensitivity: it flags nearly every true case (ω small) at the cost of a higher false-alarm rate (ε larger). A confirmatory test later in the diagnostic pipeline does the opposite: it trades sensitivity for specificity (ε small, ω possibly larger).

In both cases, the test is fully described by the matrix Q from Definition 2.2.1. The *screening test* has small ω and larger ε; the *confirmatory test* has small ε and larger ω. Neither is a BSC. The channel-matrix representation makes the trade-off visible at a glance: the top row (positives) and bottom row (negatives) can be tuned independently by adjusting the decision threshold.

This is precisely the receiver operating characteristic (ROC) curve familiar from binary classification: as the threshold moves, (ε, ω) traces a curve in the unit square $[0,1]^2$, and each point on the curve is a distinct asymmetric binary channel.

Worked example: $\varepsilon = 0.05$, $\omega = 0.10$

To make the asymmetric channel concrete, we work through the channel matrix and several derived quantities for $\varepsilon = 0.05$ and $\omega = 0.10$.

The matrix.

$$Q = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix} = \begin{pmatrix} 0.90 & 0.10 \\ 0.05 & 0.95 \end{pmatrix}.$$

Row sums. Row 1: $0.90 + 0.10 = 1.00$. Row 2: $0.05 + 0.95 = 1.00$. Both rows sum to 1, confirming that Q is row-stochastic.

Reading the matrix. A positive input ($\top$) is correctly reported as positive with probability 0.90 (true-positive rate $\tau = 1 - \omega$) and incorrectly reported as negative with probability 0.10 (false-negative rate ω). A negative input ($\bot$) is incorrectly reported as positive with probability 0.05 (false-positive rate ε) and correctly reported as negative with probability 0.95 (true-negative rate $\eta = 1 - \varepsilon$). The two error rates are distinct: $\omega = 0.10 \neq 0.05 = \varepsilon$. This channel is not a BSC.

Error-rate asymmetry. The ratio of the two error rates is $\omega/\varepsilon = 0.10/0.05 = 2$: false negatives are twice as likely as false positives. For the BSC with $\varepsilon = \omega = 0.075$ (the mean of the two rates), the matrix would be

$$Q_{\mathrm{BSC}} = \begin{pmatrix} 0.925 & 0.075 \\ 0.075 & 0.925 \end{pmatrix},$$

which has identical off-diagonal entries. Comparing this with the asymmetric Q above makes the structural difference plain: the off-diagonal entries 0.10 and 0.05 are a factor of 2 apart.

Joint probabilities under a uniform prior. If the input is drawn uniformly from $\{\top, \bot\}$ (prior probability 1/2 for each), the joint probabilities of each input-output pair are:

	output $\top$	output $\bot$	total
input $\top$	$\frac{1}{2} \cdot 0.90 = 0.450$	$\frac{1}{2} \cdot 0.10 = 0.050$	0.500
input $\bot$	$\frac{1}{2} \cdot 0.05 = 0.025$	$\frac{1}{2} \cdot 0.95 = 0.475$	0.500
total	0.475	0.525	1.000

Accuracy. Under the uniform prior, accuracy is the sum of the diagonal joint probabilities: $0.450+0.475 = 0.925$. This is the average of $\tau = 0.90$ and $\eta = 0.95$: accuracy $= \frac{1}{2}(0.90 + 0.95) = 0.925$. Compare this with the BSC above: the BSC would achieve accuracy 0.925 as well, but by having both error rates at 0.075 rather than one at 0.05 and one at 0.10. The asymmetric channel and this particular BSC have the same accuracy under a uniform prior, but their error structures are different—a point that matters when the prior is not uniform or when the two error types carry different costs.

The asymmetric channel as the general order-2 case

The BSC has one free parameter (ε, with the constraint $\varepsilon = \omega$ reducing the two-parameter family to one). The asymmetric binary channel has two free parameters (ε and ω, with no constraint between them). Section 2.4 formalizes what "order 2" means: the Boolean input domain $\{\top, \bot\}$ partitions into two blocks, $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, each with its own per-block error rate. The asymmetric channel assigns ω to B_1 and ε to B_2 independently; the BSC additionally forces $\omega = \varepsilon$.

In this sense, the BSC is not a different type of channel from the asymmetric channel: it is the *symmetric special case* of the same order-2 structure. The constraint $\omega = \varepsilon$ reduces the number of free parameters from 2 to 1 without changing the partition of the input domain.

The distinction between model order (the number of partition blocks) and parameter count (how many numbers are needed to describe the channel) is a recurring theme. Section 2.4 makes it precise and proves the Boolean ceiling theorem: because the Boolean domain has exactly two elements, any binary channel has model order at most 2. There is no room for a more fine-grained partition, no matter how many free parameters the channel has.

Looking ahead

The Z-channel and general asymmetric channel are the natural habitat of most real-world approximate Boolean systems. In Section 2.3 we compose two binary channels—which may each be symmetric, asymmetric, or Z-channel—via matrix multiplication and read off the resulting channel's error rates directly from the product. The asymmetric channel's two-parameter freedom makes the composition richer than the BSC case, and the matrix formalism handles both cases uniformly.

2.3 Composition of Channels

Channels in series

The channels we have studied so far—the BSC of Section 2.1 and the asymmetric binary channel of Section 2.2—model a single noisy step. Many real systems chain two or more such steps together. A signal passes through a first amplifier that introduces some noise, then through a second that adds more. A classification pipeline applies one model to filter candidates and a second to rank them. A Bloom filter checks membership; a downstream system then applies its own fallible test to the reported positives.

In each case we have a *serial pipeline*: an element enters the first channel Q_1, its output becomes the input to the second channel Q_2, and the combined system produces a final output. We want to know: what is the channel matrix of the composed system?

Definition 2.3.1 (Serial composition of binary channels)**.** *Let Q_1 and Q_2 be two binary channel matrices. The* serial composition $Q_2 \circ Q_1$ *is the channel formed by applying Q_1 first and Q_2 second. Its channel matrix is the ordinary matrix product*

$$Q_{\text{total}} \;=\; Q_2\, Q_1.$$

The (i, j)-entry of Q_{total} is the probability of observing final output j given that the original input belongs to block B_i.

The right-to-left convention $Q_2 Q_1$ is standard: in matrix notation, the *rightmost* factor is applied first. The input vector (or row) is transformed by Q_1, and the intermediate output is then transformed by Q_2. The law of total probability is what makes the product work:

$$(Q_{\text{total}})_{ij} \;=\; \sum_{k} (Q_1)_{ik}\,(Q_2)_{kj}.$$

Reading this sum: $(Q_1)_{ik}$ is the probability that the first channel maps input block B_i to intermediate output k; $(Q_2)_{kj}$ is the probability that the second channel maps intermediate input k to final output j. Summing over all intermediate values k gives the total probability that B_i produces final output j, which is exactly the (i, j)-entry of the composed channel.

Worked example: composing two BSCs

The Binary Symmetric Channel of Section 2.1 is the cleanest test case for composition. We show that composing two BSCs always yields a third BSC, and we derive the composed crossover probability explicitly.

Let Q_1 be a BSC with crossover ε_1 and Q_2 be a BSC with crossover ε_2:

$$Q_1 \;=\; \begin{pmatrix} 1-\varepsilon_1 & \varepsilon_1 \\ \varepsilon_1 & 1-\varepsilon_1 \end{pmatrix}, \qquad Q_2 \;=\; \begin{pmatrix} 1-\varepsilon_2 & \varepsilon_2 \\ \varepsilon_2 & 1-\varepsilon_2 \end{pmatrix}.$$

We compute $Q_{\text{total}} = Q_2 Q_1$ directly:

$$\begin{aligned} Q_2 Q_1 &= \begin{pmatrix} 1-\varepsilon_2 & \varepsilon_2 \\ \varepsilon_2 & 1-\varepsilon_2 \end{pmatrix} \begin{pmatrix} 1-\varepsilon_1 & \varepsilon_1 \\ \varepsilon_1 & 1-\varepsilon_1 \end{pmatrix} \\ &= \begin{pmatrix} (1-\varepsilon_2)(1-\varepsilon_1) + \varepsilon_2\varepsilon_1 & (1-\varepsilon_2)\varepsilon_1 + \varepsilon_2(1-\varepsilon_1) \\ \varepsilon_2(1-\varepsilon_1) + (1-\varepsilon_2)\varepsilon_1 & \varepsilon_2\varepsilon_1 + (1-\varepsilon_2)(1-\varepsilon_1) \end{pmatrix}. \end{aligned}$$

Observe that the top-left and bottom-right entries are equal, and the top-right and bottom-left entries are equal. That is precisely the structure of a BSC matrix. Denote the off-diagonal entry as $\varepsilon_{\text{total}}$:

$$\begin{aligned} \varepsilon_{\text{total}} &= (1-\varepsilon_2)\varepsilon_1 + \varepsilon_2(1-\varepsilon_1) \\ &= \varepsilon_1 - \varepsilon_1\varepsilon_2 + \varepsilon_2 - \varepsilon_1\varepsilon_2 \\ &= \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2. \end{aligned} \tag{2.1}$$

So the composed channel is

$$Q_{\text{total}} \;=\; \begin{pmatrix} 1-\varepsilon_{\text{total}} & \varepsilon_{\text{total}} \\ \varepsilon_{\text{total}} & 1-\varepsilon_{\text{total}} \end{pmatrix},$$

a BSC with crossover

$$\varepsilon_{\text{total}} \;=\; \varepsilon_1(1-\varepsilon_2) + (1-\varepsilon_1)\varepsilon_2.$$

This is the formula stated in Definition 2.3.1: two BSCs in series produce a third BSC.

Reading the composition formula

The formula $\varepsilon_{\text{total}} = \varepsilon_1(1-\varepsilon_2) + (1-\varepsilon_1)\varepsilon_2$ has a direct probabilistic interpretation. An error in the composed channel occurs if and only if *exactly one* of the two channels flips its input. There are two ways this can happen:

- The first channel flips (probability ε_1) and the second does not (probability $1-\varepsilon_2$), contributing $\varepsilon_1(1-\varepsilon_2)$.
- The first channel does not flip (probability $1-\varepsilon_1$) and the second does (probability ε_2), contributing $(1-\varepsilon_1)\varepsilon_2$.

If both channels flip, the two errors cancel: the input is back to its original value. If neither flips, the input passes through unchanged. The composed error is therefore the XOR of the two individual errors, and the probability of the XOR of two independent Bernoulli trials with parameters ε_1 and ε_2 is exactly $\varepsilon_1(1-\varepsilon_2) + (1-\varepsilon_1)\varepsilon_2$.

Two channels make things worse. For $0 < \varepsilon_1, \varepsilon_2 < 1/2$, we have

$$\varepsilon_{\text{total}} = \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2 > \varepsilon_1.$$

Adding a second channel in series strictly increases the crossover probability (unless one of the channels is perfect, $\varepsilon = 0$, or one is a maximal-error channel, $\varepsilon = 1$). Two imperfect BSCs in series are noisier than either alone. The only way to reduce the crossover in a serial pipeline is to make the intermediate channel a *corrector*, which is outside the binary-channel model we study here.

Numerical example. Take $\varepsilon_1 = 0.05$ and $\varepsilon_2 = 0.10$. Then

$$\varepsilon_{\text{total}} = 0.05 \cdot 0.90 + 0.95 \cdot 0.10 = 0.045 + 0.095 = 0.140.$$

The composed channel has crossover 0.14, strictly larger than either 0.05 or 0.10. As a matrix:

$$Q_{\text{total}} = \begin{pmatrix} 0.86 & 0.14 \\ 0.14 & 0.86 \end{pmatrix}.$$

Both rows sum to 1 and the off-diagonal entries are equal—the composed channel is indeed a BSC.

General asymmetric channels compose by the same rule

The BSC is the symmetric special case. The general asymmetric binary channel of Definition 2.2.1 follows the same rule: the composed channel matrix is the matrix product Q_2Q_1, and the composed error rates follow from the law of total probability exactly as above.

Let Q_1 have false-positive rate ε_1 and false-negative rate ω_1, and let Q_2 have ε_2 and ω_2:

$$Q_1 = \begin{pmatrix} 1-\omega_1 & \omega_1 \\ \varepsilon_1 & 1-\varepsilon_1 \end{pmatrix}, \qquad Q_2 = \begin{pmatrix} 1-\omega_2 & \omega_2 \\ \varepsilon_2 & 1-\varepsilon_2 \end{pmatrix}.$$

The composed matrix Q_2Q_1 yields combined rates (reading off the $(1,2)$-entry for the false-negative rate and the $(2,1)$-entry for the false-positive rate):

$$\begin{aligned} \omega_{\text{total}} &= (1-\omega_2)\omega_1 + \omega_2\varepsilon_1, \\ \varepsilon_{\text{total}} &= (1-\varepsilon_2)\varepsilon_1 + \varepsilon_2(1-\omega_1). \end{aligned}$$

This is Theorem 3.7 from the companion paper [Tow26a], stated there in terms of the approximate-identity function composition $\widetilde{\mathrm{id}}_{\varepsilon,\tau} \circ \widetilde{\mathrm{id}}_{\varepsilon',\tau'}$. The matrix-product derivation above is the channel-matrix reformulation of the same result.

Note that the composition of two asymmetric channels is *not* generally a BSC even when the individual channels happen to have equal overall crossover rates: the asymmetry in Q_1 and Q_2 generally produces different false-positive and false-negative rates in Q_2Q_1. The BSC is the special case in which the symmetry $\varepsilon = \omega$ is preserved under composition because both inputs to the formula are symmetric, which forces $\varepsilon_{\text{total}} = \omega_{\text{total}}$.

Serial composition vs. parallel queries

It is essential to distinguish the *serial* composition of this section from the *parallel* query structure studied in Section 4.4.

- **Serial composition** (this section): an element passes through channel Q_1, and the *output* of Q_1 becomes the *input* to channel Q_2. The combined system acts on a single element, producing a single final output. The joint channel matrix is the ordinary matrix product Q_2Q_1, which is a 2×2 matrix (the same size as each factor).

- **Parallel queries** (Section 4.4): m independent queries are applied to m *distinct* elements, each through its own copy of the channel. The combined output is an m-tuple. The joint channel matrix is the Kronecker product $Q_1 \otimes Q_2 \otimes \cdots \otimes Q_m$, which is a $2^n \times 2^m$ matrix, growing *exponentially* in m.

The two compositions are structurally different and answer different questions. Serial composition (matrix product) asks: what is the end-to-end error rate when one channel's output feeds another? Parallel composition (Kronecker product) asks: what is the joint output distribution when we query m elements simultaneously?

Theorem 4.4.1 shows that the Kronecker product structure arises because Axioms 1 and 2 force independence across parallel queries. The matrix product of this section arises from the law of total probability applied to the intermediate state in a serial pipeline. These are complementary, not competing, structures. When both matter—for instance, when k parallel pipelines each consist of multiple stages in series—the combined channel is a Kronecker product of composed matrices:

$$\left(Q_2^{(1)} Q_1^{(1)}\right) \otimes \left(Q_2^{(2)} Q_1^{(2)}\right) \otimes \cdots ,$$

where each parenthesized factor is a serial composition computed by ordinary matrix multiplication. The Kronecker product is applied *across* parallel pipelines; the ordinary product is applied *within* each pipeline.

Composition is associative but not commutative

An immediate algebraic consequence of the matrix-product interpretation is that serial channel composition is *associative*: composing three channels as $(Q_3 Q_2) Q_1$ or $Q_3 (Q_2 Q_1)$ gives the same result, since matrix multiplication is associative. This means that chaining any number of binary channels in series is unambiguous: the order of application matters, but the grouping does not.

Commutativity, however, fails in general. The product $Q_2 Q_1$ need not equal $Q_1 Q_2$: applying the first channel before the second is not the same as applying the second before the first, unless the two channels have some special symmetry.

For the BSC, commutativity *does* hold: two BSC matrices commute because each is a polynomial in the same circulant permutation matrix. Concretely, if Q_1 and Q_2 are both BSCs, then $Q_2 Q_1 = Q_1 Q_2$ by the symmetry of the composition formula $\varepsilon_{\text{total}} = \varepsilon_1(1 - \varepsilon_2) + (1 - \varepsilon_1)\varepsilon_2$, which is

symmetric in ε_1 and ε_2. This symmetry is a property of the BSC's circulant structure and is *lost* in the asymmetric channel.

Exercise 2.7 constructs an explicit pair of asymmetric channels that do not commute, illustrating that the order of channels in a serial pipeline matters outside the BSC special case.

The stationary point

For a BSC with crossover $\varepsilon < 1/2$, the composition formula shows that $\varepsilon_{\text{total}} = \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2 > \max(\varepsilon_1, \varepsilon_2)$ whenever $0 < \varepsilon_1, \varepsilon_2 < 1/2$. As we chain more and more copies of the same BSC in series, the crossover approaches the stationary value $1/2$ from below. At $\varepsilon = 1/2$, the channel output is statistically independent of the input: the channel transmits no information. This convergence to the stationary point mirrors the figure in the companion paper [Tow26a] (Figure 3.1), which shows both TPR and FPR converging to the stationary value $\varepsilon/(\varepsilon+\omega)$ under repeated composition of the general asymmetric channel.

Looking ahead: two perspectives on the same idea

Channel composition via matrix product is the channel-theoretic perspective on a result that chapter 3 will reframe algebraically. Section 3.3 will show that composing k approximate identity channels in series produces a composed false-positive rate (and, symmetrically, a composed hit rate η_{total}) satisfying

$$\eta_{\text{total}} \;=\; 1 - \prod_{i=1}^{k} (1 - \eta_i),$$

where η_i is the hit rate of the i-th stage. This is the same calculation as the matrix product, viewed through the Boolean-algebra lens: the composition theorem in chapter 3 is the algebraic restatement of what Definition 2.3.1 computes.

At the other end of the formalism, Section 4.4 and Theorem 4.4.1 give the *parallel* analog: m independent channels queried on m distinct elements produce a joint confusion matrix that is the Kronecker product $Q_1 \otimes \cdots \otimes Q_m$. The Kronecker product is the structure forced by Axioms 1 and 2; the matrix product of this section needs neither axiom, only the law of total probability.

Understanding which composition is relevant—serial (matrix product) or parallel (Kronecker product)—is one of the key skills this book develops. The corollary Corollary 4.4.1.1 gives the parallel-query factorization; the

formula of this section gives the serial-pipeline composition. Both structures appear throughout Parts II and III.

2.4 Model Order

Two channels, one question

We have now seen two versions of the binary channel. The Binary Symmetric Channel of Section 2.1 has a single crossover parameter ε, constrained so that both inputs flip with the same probability. The asymmetric binary channel of Section 2.2 lifts that constraint and allows two independent parameters ε and ω, one for each input block. When we compose two such channels in series (Section 2.3), the resulting channel inherits the two-block structure, regardless of how many parameters either constituent channel carries.

Why exactly two blocks? And could a binary channel have more? These questions are not about specific channels; they are about the structure of the input domain. The answer is given by the concept of *model order*, which we define in this section, connect to the channels developed in §§2.1–2.3, and cap with a theorem.

Partitions and blocks

A *partition* of a set S is a collection of non-empty, pairwise disjoint subsets $B_1, \ldots, B_n \subseteq S$ whose union is S:

$$B_i \cap B_j = \emptyset \text{ for } i \neq j, \qquad B_1 \cup B_2 \cup \cdots \cup B_n = S.$$

Each subset B_i is called a *block*. A partition tells us which elements are "the same" for the purpose of the model: elements in the same block are treated identically.

In the Bernoulli framework, a partition of the input domain is central to the channel model. An element x in block B_i uses the per-block error rate assigned to B_i; elements in different blocks may use different rates. This is precisely the structure of Axiom 2 in Section 4.3: the per-block rates $\{\alpha_1, \ldots, \alpha_n\}$ are conditionally independent given the objective set, one rate per block B_i.

Definition 2.4.1 (Model order)**.** *Let $\phi : D \to D$ be a binary channel with input domain D. A* partition of D compatible with ϕ *is a partition $\{B_1, \ldots, B_n\}$ of D such that every element within the same block B_i has the*

same conditional error probability:

$$\Pr\{\phi(x) \neq x\} = \alpha_i \quad \textit{for all } x \in B_i.$$

The model order *of ϕ is the number of blocks in the finest compatible partition of D: the partition that assigns distinct blocks to elements with distinct error probabilities and groups together only those elements that genuinely share a common rate. We denote the model order by n.*

Two remarks on this definition. First, the finest compatible partition is well-defined: we can always take the coarsest partition (all of D as one block) if the error probability is the same for every input, or refine to singletons if every input has its own distinct rate. The model order is the count of blocks in the finest such partition. Second, model order is a property of the channel as a whole, not of any particular parameterization: it reflects the true number of distinct error regimes in the input domain, regardless of how the channel is described.

Model order 2 for the Boolean domain

The Boolean input domain is $D = \{\top, \bot\}$. There are exactly two elements. The possible partitions of D are:

1. The *trivial partition*: one block $\{B_1\} = \{\top, \bot\}$, both inputs in a single block. This means $\Pr\{\phi(\top) \neq \top\} = \Pr\{\phi(\bot) \neq \bot\}$: the same error rate for both inputs.

2. The *discrete partition*: two blocks $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, each input in its own block. This means each input has its own error rate, which need not be equal.

No other partition exists. The input domain has two elements, and a partition of a 2-element set has at most two blocks. The model order of any binary channel is therefore at most 2.

The two channels from §§2.1–2.2 correspond exactly to these two partitions.

Model order 1: the trivial partition. When a binary channel assigns the same error rate ε to both inputs, only one block is needed: $\{B_1\} = \{\top, \bot\}$. The channel matrix is

$$Q = \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

which is exactly the BSC of Definition 2.1.1. A single shared rate ε describes the channel; the model order is 1.

Model order 2: the discrete partition. When the two inputs have distinct error rates $\varepsilon \neq \omega$, no single block can cover both; the finest compatible partition has two blocks, $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, and the model order is 2. The channel matrix is

$$Q = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

which is the general asymmetric channel of Definition 2.2.1. Two parameters, ε and ω, are required.

The BSC as the symmetric special case of order 2. There is a subtlety that is easy to overlook. The BSC has model order 1: its finest compatible partition has a single block because both inputs have the same error rate ε. We can, however, always embed a model-order-1 channel into the order-2 framework by using the discrete partition $B_1 = \{\top\}$, $B_2 = \{\bot\}$ and then imposing the additional constraint $\varepsilon = \omega$. This is exactly how Section 2.1 described the BSC: as an order-2 channel with the symmetry constraint. Saying "the BSC is the symmetric special case of the order-2 model" means that we work in the order-2 channel family (two blocks, two parameters) and then restrict to the symmetric subcase where the two parameters are equal.

Both descriptions are correct: the BSC's finest compatible partition has one block (model order 1 by the definition above), but it is also a degenerate member of the two-parameter, two-block family (the symmetric special case of order 2). The distinction matters primarily when comparing channels across families: the BSC genuinely lives in the order-1 class by partition count, and the asymmetric channel genuinely requires order 2. Practically, we will work in the order-2 framework throughout chapters 2–4, treating the BSC as a constrained member of that family, because the two-block partition is the natural one for the Boolean domain.

The Boolean ceiling theorem

The preceding analysis leads immediately to the central theorem of this section.

Theorem 2.4.1 (Boolean ceiling)**.** *Any binary channel with input domain* $\{\top, \bot\}$ *has model order at most* 2*.*

Proof. Let $\phi : \{\top, \bot\} \to \{\top, \bot\}$ be any binary channel. We must show that every compatible partition of the input domain $D = \{\top, \bot\}$ has at most 2 blocks.

A partition of D is a collection of non-empty, pairwise disjoint subsets whose union is $D = \{\top, \bot\}$. Since $|D| = 2$, any partition can have at most $|D| = 2$ blocks: a partition with more than 2 blocks would require more than 2 elements in the union, but $|D| = 2$. Concretely, the partitions of a 2-element set are:

1. $\big\{\{\top, \bot\}\big\}$: the trivial partition, one block.
2. $\big\{\{\top\},\ \{\bot\}\big\}$: the discrete partition, two blocks.

No other partition exists. The model order, defined as the number of blocks in the finest compatible partition, is therefore either 1 or 2, and in particular is at most 2. □

The proof is elementary because the claim is a direct consequence of the input domain's cardinality. A partition of any k-element set has at most k blocks; for the Boolean domain, $k = 2$, giving the ceiling of 2. The real content of the theorem is that this ceiling is achieved: the asymmetric channel of Section 2.2 is a binary channel with model order exactly 2, so the bound is tight. And the BSC shows that the ceiling need not be reached: a binary channel can have model order 1 when the two inputs happen to share a common error rate.

Model order vs. parameter count

Model order and parameter count are related but distinct, and conflating them is one of the most common sources of confusion in this framework. It is worth being explicit.

- **Model order** is the number of partition blocks in the finest compatible partition of the input domain. It is a structural property of the channel, determined by how many distinct error regimes actually exist in D. For the Boolean domain, model order is at most 2 by the Boolean ceiling theorem.
- **Parameter count** is the number of free real numbers required to specify the channel. For a single Boolean channel, an n-th order model has n free parameters (one per block), but the total parameter count

for m joint queries grows with m: under Axioms 1 and 2, the per-source-block convention gives $n(2^m - 1)$ parameters in general, reduced by Kronecker factorization to nm (Corollary 4.4.1.2).

The key distinction: model order is about partition granularity (how many distinct error regimes), while parameter count is about how many numbers are needed to specify the model (which depends on the axioms and the query structure). Confusion matrices and model order are orthogonal concepts: model order constrains which rows of Q are identical (or more precisely, which inputs can be put in the same block), while the independence axioms constrain the internal structure of each row and how rows interact across multiple queries.

Three examples make the distinction concrete.

Example 1: BSC, single query. Model order 1 (one block, $\varepsilon = \omega$). Parameter count: 1.

Example 2: Asymmetric channel, single query. Model order 2 (two blocks, $\varepsilon \neq \omega$). Parameter count: 2.

Example 3: Asymmetric channel, m queries under Axioms 1 and 2. Model order 2. Parameter count under axioms: $2m$ (two parameters per query, by Corollary 4.4.1.2). Parameter count without axioms, per-source-block convention: $2(2^m - 1)$.

In Example 3, model order does not change with m: the channel is still order 2, with two blocks, regardless of the query count. What changes with m is the parameter count. The axioms drive the dramatic reduction from $2(2^m - 1)$ to $2m$: without Axiom 1 (element-wise independence) and Axiom 2 (cross-block rate independence, Axiom 2), the Kronecker factorization of Corollary 4.4.1.2 would not hold. The axioms are doing all the work; model order is not.

This clarification matters for reading the companion papers correctly. Some sections of the source papers use "order" loosely to refer to the number of free parameters in the single-query setting, where both quantities agree (an n-th order model on one Boolean query has n parameters). In the multi-query setting, or whenever the independence axioms are in view, it is essential to keep the two concepts separate. Throughout this book, *model order* always means the number of partition blocks, as in Definition 2.4.1.

How the channels of §§2.1–2.3 fit

We can now locate every channel of this chapter within the model-order framework.

The BSC (Section 2.1). Model order 1 by finest-partition count, but studied in the order-2 framework (as the symmetric special case). Channel matrix: Q_{BSC} with $\varepsilon = \omega$. Free parameters in Q: 1.

The asymmetric binary channel (Section 2.2). Model order 2. Channel matrix: general Q with independent ε and ω. Free parameters in Q: 2.

The Z-channel (Definition 2.2.2). Model order 2 (the two blocks have genuinely different rates: $\omega = 0$ for $B_1 = \{\top\}$ and $\varepsilon > 0$ for $B_2 = \{\bot\}$). Free parameters in Q: 1 (only ε, since ω is known to be 0). This illustrates that model order and free parameter count are independent: the Z-channel has model order 2 but only one free parameter.

Composed channels (Section 2.3). Composing two binary channels produces a binary channel. The model order of the composed channel is at most 2 (by the Boolean ceiling theorem, since the output is still a binary channel over the same domain $\{\top, \bot\}$). The parameter count of the composed channel depends on the input channels' rates: the BSC-BSC composition gives one parameter ($\varepsilon_{\text{total}}$), while the asymmetric-asymmetric composition generally gives two.

Worked illustration: ordering by blocks

The following table organizes the four channels of this chapter by their block structure under the discrete partition $\{B_1, B_2\} = \{\{\top\}, \{\bot\}\}$:

Channel	α_1 (rate for $B_1 = \{\top\}$)	α_2 (rate for $B_2 = \{\bot\}$)	Model order
BSC	ε	ε $(= \alpha_1)$	1
Z-channel	0	ε $(\neq 0)$	2
Asymmetric	ω	ε (free)	2
Perfect channel	0	0	1

Reading the table: the BSC's two blocks share the same rate ε, so the finest compatible partition merges them into one block, giving model order 1. The perfect channel (zero error everywhere) likewise merges both blocks, giving model order 1. The Z-channel and asymmetric channel have $\alpha_1 \neq \alpha_2$, so the finest compatible partition has two blocks, giving model order 2.

The table also makes visible why the Z-channel has model order 2 despite having only one free parameter (ε): the two blocks have different rates (0 and $\varepsilon > 0$), so the finest partition must keep them separate. Parameter

count reflects how many numbers we write down; model order reflects how many blocks are needed.

Axiom 2 and the per-block structure

The partition-block structure of model order connects directly to the formal treatment in Section 4.3. Axiom 2 (Axiom 2) asserts that the per-block error rates $\alpha_1, \ldots, \alpha_n$ are conditionally independent given the objective set. In the order-2 Boolean case, this reduces to:

$$f(\varepsilon, \omega|A) = f(\varepsilon|A) \cdot f(\omega|A),$$

the independence of the false-positive rate (rate for block $B_2 = \{\bot\}$) and the false-negative rate (rate for block $B_1 = \{\top\}$). The model order is what determines the dimension of this independence claim: order-2 gives a two-factor joint density; order-n gives an n-factor joint density.

The Boolean ceiling theorem limits this: because the Boolean domain has model order at most 2, the Axiom 2 independence claim for Boolean channels involves at most two independent rates. This makes the framework particularly tractable: every Boolean channel can be described by at most two parameters, and the Kronecker factorization of Corollary 4.4.1.2 reduces the multi-query parameter count to $2m$ rather than $2(2^m - 1)$.

Why the ceiling matters

The Boolean ceiling theorem is not a limitation of the framework; it is a feature. Any predicate over $\{\top, \bot\}$ lives in a world with exactly two possible error regimes: one for each element of the domain. No matter how complex the underlying computation, no matter how many intermediate steps a channel goes through, the final Boolean predicate has model order at most 2. The two-block partition $\{B_1, B_2\} = \{\{\top\}, \{\bot\}\}$ is the universal structure for Boolean channels.

This has a practical consequence. When we apply the Bernoulli framework to a new Boolean predicate—a membership test, a primality check, a classifier's verdict—we know in advance that the predicate is either order 1 (if the error probability is the same for all inputs) or order 2 (if the error probabilities differ by input sign). There are no other cases. The Boolean ceiling closes off the space of possible structures and reduces the question of model identification to a single binary choice.

Chapter 9 will revisit the model-order concept for input domains with more than two elements: when $|D| = k$ for $k > 2$, the same partition-block

machinery applies and the ceiling rises to k, but the analysis of channels, parameter counts, and independence axioms remains structurally the same.

Looking ahead

Section 4.3 placed the partition-block structure at the center of Axiom 2. Corollary 4.4.1.2 showed how the Kronecker factorization drives the parameter count from $2(2^m - 1)$ down to $2m$ for order-2 Boolean predicates under the axioms. In Part II we will see how the order-2 Boolean case interacts with the Bernoulli set operations: union, intersection, and complement all produce channels whose model order is at most 2 (because they remain binary channels over the Boolean domain). The Boolean ceiling theorem is why the entire Boolean algebra of Bernoulli sets remains tractable.

The broader setting, where the input domain has cardinality greater than 2, belongs to Part III. A single sentence is enough for now: the same partition-block machinery that gives the Boolean ceiling at order 2 generalizes to n-ary channels with model order at most $|D|$, covered in Chapter 9.

Bibliographic Notes

Shannon and the channel framework. The channel matrix formalism used throughout this chapter descends directly from Shannon [Sha48]. Shannon's paper introduced the discrete memoryless channel as the mathematical object for studying noisy communication: a channel is specified by a conditional distribution $P(\text{output} \mid \text{input})$, and the capacity of the channel is the maximum mutual information between input and output over all input distributions. The binary symmetric channel of Section 2.1 is one of Shannon's central running examples; the crossover parameter ε and the capacity formula $1 - H(\varepsilon)$ are both in that 1948 paper. The treatment of the asymmetric binary channel in Section 2.2 is a natural extension of Shannon's framework to the case where the two input symbols face independent error probabilities. For a modern textbook development of binary channels, channel capacity, and the channel coding theorem, see Cover and Thomas [CT06], especially Chapters 7 and 8.

The Bernoulli papers. The connection between binary channels and the Bernoulli framework is drawn in Towell [Tow26a], which introduces the two-axiom model and proves the composition theorem that Section 2.3 reformulates in matrix-product language. The matrix-product form—that serial

composition of two binary channels corresponds to matrix multiplication of their channel matrices—is a reorganization of the rate-propagation formula in that paper; see the companion notes in Section 4.6 for a precise account of what is new in this text and what descends from the companion papers.

The model-order treatment of Section 2.4, and in particular the definition of model order as the number of partition blocks in the finest compatible partition of the input domain (Definition 2.4.1), follows Towell [Tow26d]. That paper extends the Bernoulli framework from Boolean sets to approximate values over arbitrary algebraic types; the order-2 Boolean case developed in this chapter is the simplest instance of its general theory. The Boolean Ceiling Theorem (Theorem 2.4.1)—that any binary channel over $\{\top, \bot\}$ has model order at most 2—is explicit in the algebraic-types paper as the base case of the more general statement that a channel over a k-element domain has model order at most k.

Model order versus degrees of freedom. A clarification that deserves explicit attention is the distinction between model order and parameter count, developed in Section 2.4 under the heading "Model order vs. parameter count." This distinction is easy to miss in the source papers, because for a single Boolean query the two quantities agree: an n-th order channel on one Boolean input has n partition blocks and also n free parameters, one per block. The difference surfaces in the multi-query setting and is worth recording here, as it is flagged as a source of confusion in the companion repository's planning notes (`papers/CLAUDE.md`, Terminology section). When m queries are issued jointly, a general order-n channel on m inputs has $n(2^m - 1)$ parameters in the per-source-block convention, but the independence axioms (Axioms 1 and 2) force a Kronecker factorization that reduces this to nm. The model order does not change: it remains n, determined by the partition of the single-query input domain. What changes is the parameter count, and the axioms are entirely responsible for the reduction. Model order constrains which rows of the channel matrix Q can be identified; the axioms constrain how the channel matrix factorizes across queries. These are orthogonal constraints.

Further reading. Chapter 4's Bibliographic Notes (Section 4.6) provide additional context for the Kronecker product and its role in the multi-query factorization, and for the relationship between the Bernoulli axioms and the memoryless channel. Readers who want a deeper grounding in information theory beyond what Part I requires will find Cover and Thomas [CT06] the

standard reference.

Exercises

1. ⋆**Writing the channel matrix.** Let $\varepsilon = 0.07$ and $\omega = 0.12$. Write the 2×2 channel matrix Q for the asymmetric binary channel (Definition 2.2.1) with these parameters. Verify that both rows sum to 1 (row-stochastic condition). Then write the matrix for the special case $\varepsilon = \omega = 0.07$ and identify which named channel (Definition 2.1.1) it gives.

2. ⋆**Composing two BSCs by hand.** Let Q_1 be a BSC with crossover $\varepsilon_1 = 0.05$ and Q_2 be a BSC with crossover $\varepsilon_2 = 0.10$ (Definition 2.1.1).

   (a) Write out Q_1 and Q_2 explicitly as 2×2 matrices.

   (b) Compute the matrix product $Q_{\text{total}} = Q_2\, Q_1$ entry by entry (Definition 2.3.1).

   (c) Using the formula $\varepsilon_{\text{total}} = \varepsilon_1(1 - \varepsilon_2) + (1 - \varepsilon_1)\varepsilon_2$ from Equation (2.1), verify that $\varepsilon_{\text{total}} = 0.14$ and that Q_{total} is a BSC with this crossover.

3. ⋆**Identifying model order.** For each of the following systems, state the model order (Definition 2.4.1) and justify your answer in one sentence.

   (a) A BSC with crossover $\varepsilon = 0.03$ (Definition 2.1.1).

   (b) A Bloom filter (Definition 2.2.2).

   (c) An asymmetric binary channel with $\varepsilon = 0.05$ and $\omega = 0.08$ (Definition 2.2.1).

   (d) The composition of a BSC followed by a Z-channel (Definition 2.3.1).

4. ⋆⋆ **Deriving BSC capacity.** The capacity of a channel is the maximum mutual information $I(X;Y)$ between input X and output Y over all input distributions. Let $p = \Pr\{X = \top\}$ be the prior probability of a positive input, so $\Pr\{X = \bot\} = 1 - p$.

   (a) For a BSC with crossover ε (Definition 2.1.1), write out $\Pr\{Y = \top\}$ and $\Pr\{Y = \bot\}$ in terms of p and ε.

   (b) Recall that mutual information satisfies $I(X;Y) = H(Y) - H(Y|X)$, where H denotes binary entropy. Show that $H(Y|X) = H_2(\varepsilon)$ regardless of p, where $H_2(\varepsilon) = -\varepsilon \log_2 \varepsilon - (1 - \varepsilon)\log_2(1 - \varepsilon)$.

(c) Show that $H(Y)$ is maximized at $p = 1/2$, and conclude that the BSC capacity is $1 - H_2(\varepsilon)$.

(d) Evaluate the capacity at $\varepsilon = 0.05$ (the running example from Section 2.1) and at $\varepsilon = 0.5$. Interpret both values.

5. ⋆⋆**Bloom filter as a Z-channel.** The Bloom filter satisfies $\omega = 0$: it never reports a false negative (Definition 2.2.2).

(a) Write the channel matrix Q for a Bloom filter with false-positive rate $\varepsilon = 0.05$ and verify that the matrix satisfies $\omega = 0$.

(b) A query returns "not in set." Show that, under the Z-channel model (Section 2.2), this answer carries no uncertainty: the true membership status is determined exactly.

(c) Argue that no amount of serial composition of Z-channels (Definition 2.3.1) with $\omega = 0$ can produce a non-zero false-negative rate. Use the matrix-product form to support your argument.

(*Hint for (c):* consider what the composition formula from Equation (2.1) becomes when $\omega_1 = \omega_2 = 0$.)

6. ⋆⋆**The composition of two BSCs is a BSC.** Prove the general case of the composition result implicit in Equation (2.1): for any two BSC matrices Q_1 and Q_2 with crossovers ε_1 and ε_2, the product $Q_{\text{total}} = Q_2 Q_1$ is again a BSC.

(a) A 2×2 stochastic matrix is a BSC (Definition 2.1.1) if and only if both of its off-diagonal entries are equal. Show that Q_{total} satisfies this condition by multiplying the two symmetric matrices and comparing off-diagonal entries.

(b) Identify the crossover of Q_{total} in terms of ε_1 and ε_2.

(c) The composition of BSCs is therefore *closed* in the class of BSCs. Give a one-sentence interpretation: why is closure a useful property for building serial pipelines out of symmetric components?

7. ⋆⋆**When does an asymmetric channel reduce to a BSC?** An asymmetric binary channel (Definition 2.2.1) has parameters ε and ω.

(a) State the algebraic condition on (ε, ω) that makes the asymmetric channel's matrix equal to a BSC matrix (Definition 2.1.1).

(b) In the (ε, ω) parameter space $[0, 1)^2$, describe the set of parameter pairs that correspond to BSCs. What geometric object is this set?

(c) At the boundary of this set, perturb ω by a small amount $\delta > 0$ while fixing ε. Compute the new Q and identify the model order (Definition 2.4.1) of the resulting channel.

(d) Explain in one paragraph why the BSC is a measure-zero special case within the space of asymmetric channels, and what that implies for real-world data systems.

8. ⋆⋆**Constructing a model-order-2 channel from two model-order-1 channels.** Two deterministic channels each have model order 1: the identity channel (crossover $\varepsilon = 0$) and the complementing channel (crossover $\varepsilon = 1$). Both are trivially order-1 BSCs.

   (a) Compute the serial composition of the identity channel followed by the complementing channel (Definition 2.3.1). What channel does the composition produce?

   (b) Now consider a non-degenerate construction: compose a BSC with crossover $\varepsilon_1 = 0.1$ with an asymmetric channel having $\varepsilon_2 = 0.2$, $\omega_2 = 0.3$. Compute $Q_{\text{total}} = Q_2 Q_1$ and verify the model order (Definition 2.4.1) of the result.

   (c) Describe in general how serial composition (Section 2.3) affects the partition structure. Does composing two order-n channels always yield an order-n channel? Does it always yield an order-m channel for some fixed m related to the input domain?

   (*Hint:* focus on the input domain of the composed channel, not the intermediate domain.)

9. ⋆⋆⋆**Rigorous proof of the Boolean ceiling.** Theorem 2.4.1 states that any binary channel over the input domain $\{\top, \bot\}$ has model order at most 2. Give a self-contained proof from the definitions.

   (a) State the definition of model order (Definition 2.4.1) precisely: a partition of $\{\top, \bot\}$ is a collection of non-empty pairwise-disjoint subsets whose union is $\{\top, \bot\}$. List all distinct partitions of the two-element set $\{\top, \bot\}$.

   (b) Show that no partition of $\{\top, \bot\}$ can have more than 2 blocks. (The argument is purely combinatorial: a partition into k blocks of a 2-element set requires $k \leq 2$.)

   (c) Conclude that the model order of any channel over $\{\top, \bot\}$ is at most 2, and give the two distinct channel classes that correspond

to model order 1 (the BSC) and model order 2 (the general asymmetric channel).

(d) Contrast with the n-ary case: state, without proving, how the ceiling changes for an input domain of cardinality k (foreshadows Chapter 9 without entering Part III content).

10. $\star\star\star$**Channel composition: associative but not commutative.** Let Q_1, Q_2, Q_3 be binary channel matrices and define serial composition as matrix product (Definition 2.3.1).

(a) *Associativity.* Prove that $(Q_3Q_2)Q_1 = Q_3(Q_2Q_1)$. (This follows directly from associativity of matrix multiplication; state the argument clearly and identify what it implies for building pipelines of arbitrary length.)

(b) *Non-commutativity by counterexample.* Let Q_1 be a BSC with crossover $\varepsilon_1 = 0.1$ and Q_2 be an asymmetric channel with $\varepsilon_2 = 0.4$ and $\omega_2 = 0.1$. Compute Q_2Q_1 and Q_1Q_2 explicitly. Show that they are not equal.

(c) Interpret the non-commutativity result: the order in which two noisy stages appear in a serial pipeline matters. Give a one-sentence practical implication for designing a two-stage classification pipeline where one stage has higher false-positive rate and the other has higher false-negative rate.

11. $\star\star\star$**A 3-input partition and model order 3.** The binary domain $\{\top, \bot\}$ supports at most model order 2 (Theorem 2.4.1). This exercise explores what happens when the input domain grows.

(a) Let the input domain be $\{1, 2, 3\}$ (a 3-element set). List all distinct partitions of this domain. How many distinct values of model order are possible?

(b) Write a 3×2 channel matrix Q for a channel from the 3-element input domain to the binary output domain $\{\top, \bot\}$, choosing a partition into 2 blocks $B_1 = \{1, 2\}$ and $B_2 = \{3\}$ with per-block error rates $\varepsilon_1 = 0.05$, $\varepsilon_2 = 0.20$. Verify that both rows in each block share the same entries and that the matrix is row-stochastic.

(c) Write a second channel matrix using the 3-block partition $B_1 = \{1\}$, $B_2 = \{2\}$, $B_3 = \{3\}$ with rates $\varepsilon_1 = 0.05$, $\varepsilon_2 = 0.15$, $\varepsilon_3 = 0.30$. State the model order of this channel.

(d) Explain in one paragraph why the Kronecker factorization (Theorem 4.4.1) discussed in the context of parallel queries (chapter 4 §4.4) extends naturally to n-ary input domains, and identify the one change in the parameter count. (This is a preview of Chapter 9; no detailed proof is expected.)

Chapter 3

Boolean Algebra Under Noise

After this chapter you will be able to:

- Derive AND, OR, and NOT under noise from first principles, using the per-block rate parameters of the input Bernoulli[bool] instances.
- State and apply the composition theorem $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$ for chained identities-with-error.
- Analyze Miller-Rabin's k-round error from first principles by treating one witness as a Bernoulli[bool] instance and composing.
- Locate the hidden independence assumption that Chapter 4 will make explicit.

3.1 AND Under Noise

We have, by now, a clear picture of a single noisy bit: a Bernoulli[bool] instance with a false-positive rate ε and a false-negative rate ω, as defined formally in Definition 4.5.1. The binary symmetric channel of Section 2.1 was the symmetric special case $\varepsilon = \omega$. The biased-coin running example, introduced in Section 1.1, is the general asymmetric case.

The first question chapter 3 must answer is: what happens when we combine two noisy bits using a Boolean operation? AND is the natural place to start, because it is the most commonly composed gate in practice, the one behind the Bloom filter's k-hash membership query, and the operation

that underlies Miller-Rabin's k-round test. Once we understand AND under noise, OR and NOT follow without new machinery, as Section 3.2 shows.

Setup: two independent noisy bits

Let $\tilde{b}_1$ and $\tilde{b}_2$ be two Bernoulli[bool] instances with rates $(\varepsilon_1, \omega_1)$ and $(\varepsilon_2, \omega_2)$ respectively. Each instance has its own channel matrix:

$$Q_i \;=\; \begin{pmatrix} 1-\omega_i & \omega_i \\ \varepsilon_i & 1-\varepsilon_i \end{pmatrix}, \quad i = 1, 2.$$

Row 1 of Q_i governs positive inputs (true value $\top$); row 2 governs negative inputs (true value $\bot$). Columns are the output labels $\top$ and $\bot$, following the convention from Section 2.1.

We apply the AND function to the two observed outputs. The *true* value of the AND is the AND of the two *true* inputs. We want to know: what is the probability that the AND of the noisy observations agrees with the AND of the true values?

Axioms invoked: a flag before we compute

Before proceeding to the case analysis, we must flag the independence assumptions the derivation below will silently use. We state them explicitly here because Part I's aim is to make these assumptions visible, not merely convenient.

Axiom 1 (Section 4.2, Axiom 1) asserts that error events at distinct inputs to a single predicate are mutually independent. In the AND setting, $\tilde{b}_1$ and $\tilde{b}_2$ evaluate two distinct input elements. Axiom 1 therefore licenses the factorization: the event that $\tilde{b}_1$ makes an error is independent of the event that $\tilde{b}_2$ makes an error. Every product of probabilities in the case analysis below rests on this factorization.

Axiom 2 (Section 4.3, Axiom 2) asserts that the per-block error rates are conditionally independent of each other given the objective set. For $\tilde{b}_1$, this means ε_1 and ω_1 are independent parameters; likewise for $\tilde{b}_2$. As a consequence (Proposition 4.3.1), the channel matrix Q_i is stable across queries: the rate seen by one query of $\tilde{b}_1$ is the same rate that applies to any other query of the same instance. This is what allows us to read the rates ε_i and ω_i as fixed constants throughout the case analysis, rather than tracking a time-varying stochastic process.

Both axioms are in play. Axiom 1 handles the across-instance independence; Axiom 2 handles the within-instance rate stability. Chapter 4 proves

both axioms in full (Section 4.2 and Section 4.3) and shows, in Section 4.4, that the two together are exactly the content of the Kronecker factorization theorem. What we do now is use these axioms deliberately, not accidentally.

Truth table for AND: four cases

The AND of two Boolean values is $\top$ if and only if both inputs are $\top$. There are four combinations of true inputs; we analyze each.

True b_1	True b_2	Correct AND output	Pr[AND output $= \top$]
$\top$	$\top$	$\top$	$(1-\omega_1)(1-\omega_2)$
$\top$	$\bot$	$\bot$	$(1-\omega_1)\varepsilon_2$
$\bot$	$\top$	$\bot$	$\varepsilon_1(1-\omega_2)$
$\bot$	$\bot$	$\bot$	$\varepsilon_1\varepsilon_2$

The rightmost column gives Pr[observed AND $= \top$] for each true-input combination. It does not give the probability that the AND is *correct*; that requires also knowing which output is correct for that row. Let us work through each case in turn.

Case 1: both true inputs are $\top$. The correct output is $\top$. For the AND of observed outputs to equal $\top$, both $\tilde{b}_1$ and $\tilde{b}_2$ must report $\top$. Each does so correctly with probability $1-\omega_i$ (the true-positive rate). By Axiom 1, the two events are independent:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \top, b_2 = \top] \ = \ (1-\omega_1)(1-\omega_2).$$

This is the AND's true-positive probability.

Case 2: $b_1 = \top$, $b_2 = \bot$. The correct output is $\bot$ (since $b_2 = \bot$). For the AND of observed outputs to incorrectly equal $\top$, $\tilde{b}_1$ must report $\top$ (correctly, with probability $1-\omega_1$) *and* $\tilde{b}_2$ must report $\top$ (incorrectly, a false positive, with probability ε_2). By Axiom 1:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \top, b_2 = \bot] \ = \ (1-\omega_1)\varepsilon_2.$$

This is a false positive conditional on the input pattern $(\top, \bot)$.

Case 3: $b_1 = \bot$, $b_2 = \top$. Symmetric to Case 2. The AND output is $\top$ incorrectly when $\tilde{b}_1$ makes a false positive and $\tilde{b}_2$ reports correctly:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \bot, b_2 = \top] \ = \ \varepsilon_1(1-\omega_2).$$

Case 4: both true inputs are $\bot$. The correct output is $\bot$. The AND output is $\top$ incorrectly only when both $\tilde{b}_1$ and $\tilde{b}_2$ make false positives simultaneously:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \bot, b_2 = \bot] \ = \ \varepsilon_1\varepsilon_2.$$

Closed-form output rates

The AND gate's output is a new Boolean value. To characterize it as a Bernoulli[bool] instance, we need to identify its false-positive rate and false-negative rate with respect to the *true AND output*.

The true AND output is $\top$ if and only if $b_1 = b_2 = \top$. The true AND output is $\bot$ in all three remaining cases.

False-negative rate of AND. A false negative occurs when the true output is $\top$ (meaning $b_1 = b_2 = \top$) but the observed AND is $\bot$. From Case 1, the probability that the observed AND is $\top$ when the true output is $\top$ is $(1 - \omega_1)(1 - \omega_2)$. Therefore, the false-negative rate is:

$$\omega_\wedge \;=\; 1 - (1 - \omega_1)(1 - \omega_2) \;=\; \omega_1 + \omega_2 - \omega_1\omega_2. \tag{3.1}$$

This is the probability that at least one of the two instances reports $\bot$ when the true value is $\top$. The union-of-events formula applies because both must succeed for the AND to succeed.

False-positive rate of AND. A false positive occurs when the true output is $\bot$ (meaning at least one of b_1, b_2 is $\bot$) but the observed AND is $\top$. The true output is $\bot$ for three input patterns: $(\top, \bot)$, $(\bot, \top)$, and $(\bot, \bot)$.

The observed AND is $\top$ in each of these three cases with the probabilities from Cases 2, 3, and 4 above. The false-positive rate of the AND gate is therefore a mixture over these three patterns, weighted by their prior probabilities. Let π denote the prior probability that $b_1 = \top$, and let $\pi_1 = \pi$, $\pi_2 = \pi$ (assuming the two bits are drawn from the same prior for simplicity; the general formula follows with π_1 and π_2 distinct).

The conditional-on-pattern false-positive events are:

$$\begin{aligned}
&\Pr[\wedge = \top \mid b_1 = \top, b_2 = \bot] = (1 - \omega_1)\varepsilon_2,\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \top] = \varepsilon_1(1 - \omega_2),\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \bot] = \varepsilon_1\varepsilon_2.
\end{aligned}$$

These are three distinct conditional false-positive probabilities, one per input pattern. In the Bernoulli framework, the false-positive rate of the AND gate is the probability that the observed AND is $\top$ given that the true AND is $\bot$. Because the three patterns that yield true AND $= \bot$ have different likelihoods of a false $\top$ output, the effective false-positive rate of the AND gate depends on the input distribution.

For the special case where both bits are independently queried about the same underlying question (both $\pi_1 = \pi_2 = \pi$, and the two bits are drawn

independently), the marginal false-positive rate is:

$$\varepsilon_\wedge(\pi) \;=\; \frac{\pi(1-\pi)(1-\omega_1)\varepsilon_2 + (1-\pi)\pi\,\varepsilon_1(1-\omega_2) + (1-\pi)^2\varepsilon_1\varepsilon_2}{\pi(1-\pi) + (1-\pi)\pi + (1-\pi)^2}. \quad (3.2)$$

The denominator is the probability that the true AND is $\bot$, which equals $1-\pi^2$.

The input-distribution-free bounds. In most practical analyses, we do not know the exact prior π. What we can say regardless of π is the following. Since we are taking the AND of two outputs, the AND gate can only output $\top$ if both inputs report $\top$. This means:

$$\varepsilon_\wedge \;\leq\; \min(\varepsilon_1, \varepsilon_2),$$

since the AND gate can falsely report $\top$ only when both individual instances report $\top$, which is at least as unlikely as each one doing so alone. The upper bound follows from the cases directly: no single case has a false-positive probability exceeding $\max(\varepsilon_1, \varepsilon_2)$, and the worst case (Case 2 with $\omega_1 = 0$) approaches ε_2.

The most important special case is when both instances are independent, identically noisy copies: $\varepsilon_1 = \varepsilon_2 = \varepsilon$, $\omega_1 = \omega_2 = \omega$. The worst-case false-positive input pattern is the $(\bot, \bot)$ case, giving ε^2. For the $(\top, \bot)$ and $(\bot, \top)$ patterns, the false-positive probability is $(1-\omega)\varepsilon$, which for small ω is approximately ε. The effective false-positive rate is therefore bounded above by $(1-\omega)\varepsilon$, and the dominant pattern depends on the input prior. When both bits query the same two-valued population symmetrically, the AND gate reduces the false-positive rate dramatically compared to a single query: the $(\bot, \bot)$ contribution goes as ε^2, which for small ε is negligible. This is the mechanism behind Miller-Rabin's exponential error reduction, which we examine in Section 3.4.

AND as a Bernoulli[bool] **operation.** The AND gate's output is a new Boolean value. Its false-negative rate $\omega_\wedge$ in (3.1) depends only on the input rates, not on the input prior. Its false-positive rate $\varepsilon_\wedge$ depends on the input prior through (3.2), unless we are in a special case that eliminates the dependence (for example, the $(1-\omega)\varepsilon_2$ bound applies when $b_1 = \top$ is guaranteed).

The conclusion is that AND is a binary operation on Bernoulli[bool] instances that produces a new Bernoulli[bool] instance. The type is closed under AND: each output is characterized by a pair of rates, derived from the input rates via the formulas above. The derivation invoked both axioms

explicitly: Axiom 1 to factor probabilities across the two instances, and Axiom 2 to treat each instance's rates as stable constants throughout the computation.

Worked example: AND of two noisy coin observers

The biased-coin thread from Section 1.1 returns here with its most concrete appearance so far. We have two observers of the same coin flip.

Setup. A coin lands heads ($\top$) with probability $\pi = 0.5$ (fair coin). Two independent observers $\tilde{b}_1$ and $\tilde{b}_2$ each report the outcome, but with different noise levels:

- Observer 1: $\varepsilon_1 = 0.10$, $\omega_1 = 0.05$ (more likely to miss heads than to hallucinate heads).
- Observer 2: $\varepsilon_2 = 0.08$, $\omega_2 = 0.12$ (more likely to miss heads than to hallucinate, but with different magnitudes).

We declare "heads" (output $\top$) only if both observers agree: output $\tilde{b}_1 \wedge \tilde{b}_2$.

False-negative rate of AND. From (3.1):

$$\omega_\wedge = \omega_1 + \omega_2 - \omega_1\omega_2 = 0.05 + 0.12 - (0.05)(0.12) = 0.17 - 0.006 = 0.164.$$

Requiring both observers to report "heads" when the coin actually landed heads raises the false-negative rate from the individual rates of 5% and 12% to 16.4%. This is the cost of the AND gate: it is harder to get two correct $\top$ reports simultaneously than just one.

Conditional false-positive rates, by pattern.

$$\begin{aligned}
&\Pr[\wedge = \top \mid b_1 = \top, b_2 = \bot] = (1 - 0.05)(0.08) = 0.95 \times 0.08 = 0.076,\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \top] = (0.10)(1 - 0.12) = 0.10 \times 0.88 = 0.088,\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \bot] = (0.10)(0.08) = 0.008.
\end{aligned}$$

The AND gate is hardest to fool when both true values are $\bot$: the probability of a false $\top$ output drops to 0.8%, compared to 7.6% or 8.8% when one true value is $\top$.

Effective false-positive rate (fair coin, $\pi = 0.5$). With a fair coin, the

four true-input patterns have probabilities:

$$\begin{aligned}
\Pr[b_1 = \top, b_2 = \top] &= 0.5 \times 0.5 = 0.25,\\
\Pr[b_1 = \top, b_2 = \bot] &= 0.5 \times 0.5 = 0.25,\\
\Pr[b_1 = \bot, b_2 = \top] &= 0.5 \times 0.5 = 0.25,\\
\Pr[b_1 = \bot, b_2 = \bot] &= 0.5 \times 0.5 = 0.25.
\end{aligned}$$

The true AND is $\bot$ when any of the last three patterns hold, giving $\Pr[\text{true AND} = \bot] = 0.75$. The marginal probability that the observed AND is $\top$ when the true AND is $\bot$ is the weighted average:

$$\begin{aligned}
\text{numerator} &= 0.25 \times 0.076 + 0.25 \times 0.088 + 0.25 \times 0.008\\
&= 0.019 + 0.022 + 0.002 = 0.043.
\end{aligned}$$

The effective false-positive rate is:

$$\varepsilon_\wedge \;=\; \frac{0.043}{0.75} \;\approx\; 0.057.$$

Comparison to individual rates. The two individual observers have $\varepsilon_1 = 0.10$ and $\varepsilon_2 = 0.08$. Their AND gate reduces the effective false-positive rate to approximately 0.057 under a fair-coin prior. The reduction is meaningful but not dramatic, because the dominant false-positive patterns are those where one true value is $\top$ (so only one observer needs to make an error); only the all-negative pattern benefits from requiring both observers to err simultaneously.

The false-negative rate rises from 5% and 12% individually to 16.4%. The AND gate trades false-negative accuracy for false-positive suppression. In applications where false positives are costly and false negatives are tolerable, this is a favorable trade. In applications where false negatives are costly (a safety system that must not miss a $\top$ event), requiring both observers to agree is counterproductive.

Summary of the example. Two Bernoulli[bool] observers with rates $(0.10, 0.05)$ and $(0.08, 0.12)$ combine under AND to produce a new Bernoulli[bool] instance with $\omega_\wedge = 0.164$ and $\varepsilon_\wedge \approx 0.057$ (at $\pi = 0.5$). The rates follow closed-form formulas derived from the four truth-table cases. Both Axiom 1 and Axiom 2 were invoked: Axiom 1 to factor the joint probability of both observers reporting $\top$ into a product of individual probabilities; Axiom 2 to treat each observer's rates as stable constants rather than time-varying quantities.

Looking ahead

The AND derivation gives us one gate. Section 3.2 will derive OR and NOT under the same methodology: NOT is elementary (it swaps ε and ω), and OR follows via De Morgan's law applied to noisy gates. Together, AND, OR, and NOT generate all of Boolean algebra.

Once we have all three gates, Section 3.3 asks what happens when we chain them: if AND has false-positive rate $\varepsilon_\wedge$, and we then AND the result with a third noisy bit, what is the total false-positive rate? The answer is the composition theorem, $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$, which describes how error rates accumulate in chains of identity-with-error operations.

That composition theorem is precisely what makes Miller-Rabin's k-round bound derivable from first principles, as Section 3.4 will show. The k-round Miller-Rabin test is, in the framework's language, a k-fold AND of Bernoulli[bool] witnesses. The derivation we just completed for two witnesses extends directly to k; the formulas are the same; only the indices change.

3.2 OR and NOT

Section 3.1 derived what happens when we AND two Bernoulli[bool] instances. We found that AND is a binary operation on the type: two instances with rates $(\varepsilon_1, \omega_1)$ and $(\varepsilon_2, \omega_2)$ combine to produce a new instance with closed-form rates $\varepsilon_\wedge$ and $\omega_\wedge$. The computation invoked both axioms: Axiom 1 to factor joint probabilities, and Axiom 2 to treat the per-instance rates as stable constants.

OR and NOT call for the same treatment. As we will see, NOT is immediate from the channel-matrix perspective, and OR follows from De Morgan's law applied to noisy gates. We then verify the De Morgan derivation by re-deriving OR from the truth table directly, confirming that both paths agree. The section closes by identifying what the three gates together buy us: closure of Bernoulli[bool] under all of Boolean algebra.

NOT under noise

Let $\tilde{b}$ be a Bernoulli[bool] instance with channel matrix

$$Q = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Its NOT, $\neg\tilde{b}$, reports the complement of the observed output. What are the rates of the complemented instance?

We answer this by tracing the meaning of the four outcomes. If the true value of $\tilde{b}$ is b and the observed output is $\hat{b}$, then $\neg\tilde{b}$ has true value $\neg b$ and observed output $\neg\hat{b}$.

False-positive rate of $\neg\tilde{b}$. A false positive of $\neg\tilde{b}$ is the event that the true value of $\neg b$ is $\bot$—meaning the true value of b is $\top$—while the observed output of $\neg\tilde{b}$ is $\top$—meaning the observed output of $\tilde{b}$ is $\bot$. In other words, a false positive of $\neg\tilde{b}$ is a *false negative* of $\tilde{b}$. The rate is:

$$\varepsilon_\neg \;=\; \omega.$$

False-negative rate of $\neg\tilde{b}$. A false negative of $\neg\tilde{b}$ is the event that the true value of $\neg b$ is $\top$—meaning $b = \bot$—while the observed output of $\neg\tilde{b}$ is $\bot$—meaning $\hat{b} = \top$. In other words, a false negative of $\neg\tilde{b}$ is a *false positive* of $\tilde{b}$. The rate is:

$$\omega_\neg \;=\; \varepsilon.$$

Summary. The NOT operation acts on the channel matrix by swapping the roles of false positives and false negatives:

$$Q_{\neg\tilde{b}} \;=\; \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \omega & 1-\omega \end{pmatrix}. \tag{3.3}$$

In rate notation: $(\varepsilon, \omega) \overset{\neg}{\mapsto} (\omega, \varepsilon)$.

This is the simplest operation in the Boolean algebra of noisy bits. The channel matrix is not recomputed from a truth table; it is obtained by reading the rows of Q in the opposite order. Complementing a predicate swaps the positive and negative blocks; the row that formerly governed positives now governs negatives, and vice versa.

NOT is its own inverse. Applying NOT twice returns the original:

$$(\varepsilon_\neg, \omega_\neg) \;=\; (\omega, \varepsilon) \;\overset{\neg}{\mapsto}\; (\varepsilon, \omega).$$

In channel-matrix terms, the row-swap (3.3) applied twice restores Q exactly. More precisely, NOT is an *involution* on Bernoulli[bool]: $(\neg)^2 = \mathrm{id}$.

This is the noisy-gate analog of the classical fact $\neg\neg b = b$. Under noise, the equality is at the level of the channel matrix, not the individual observation: applying NOT twice to a Bernoulli[bool] instance returns an instance with the same rates as the original, not the same realized output on any given query.

OR via De Morgan's law

De Morgan's law says $b_1 \vee b_2 = \neg(\neg b_1 \wedge \neg b_2)$. We apply this identity at the gate level, using NOT (derived above) and AND (from Section 3.1).

Let $\tilde{b}_1$ and $\tilde{b}_2$ be two independent Bernoulli[bool] instances with rates $(\varepsilon_1, \omega_1)$ and $(\varepsilon_2, \omega_2)$ respectively.

Step 1: Negate each input. By the NOT formula above:

$$\neg\tilde{b}_1 \text{ has rates } (\omega_1, \varepsilon_1),$$
$$\neg\tilde{b}_2 \text{ has rates } (\omega_2, \varepsilon_2).$$

Step 2: AND the negated inputs. We apply the AND formula from Section 3.1 to $\neg\tilde{b}_1$ and $\neg\tilde{b}_2$ with their swapped rates. For two Bernoulli[bool] instances with rates (a_1, c_1) and (a_2, c_2), the AND gate's false-negative rate is $c_1 + c_2 - c_1 c_2$ (from Equation (3.1)).

For the AND of $\neg\tilde{b}_1$ and $\neg\tilde{b}_2$, the roles of ε and ω are swapped relative to the original instances, so $a_i = \omega_i$ and $c_i = \varepsilon_i$:

$$\omega_{\neg\wedge\neg} \;=\; \varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2. \tag{3.4}$$

The FPR of the AND of the negated inputs is, again from the AND formula, now with the input FPRs equal to ω_1, ω_2 (the swapped values). It depends on the input prior over the negated values; but we will see in Step 3 that the outer NOT maps it to the OR's FNR, which we handle directly from the truth table.

Step 3: Negate the result. The outer NOT swaps the rates of $\neg\tilde{b}_1 \wedge \neg\tilde{b}_2$:

$$\varepsilon_\vee \;=\; \omega_{\neg\wedge\neg} \;=\; \varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2, \tag{3.5}$$

and $\omega_\vee = \varepsilon_{\neg\wedge\neg}$.

The FPR of the OR gate, $\varepsilon_\vee = \varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2$, is the inclusion-exclusion formula for the probability that at least one of two independent events occurs. It is input-distribution-free: it depends only on the individual false-positive rates, not on the prior over input patterns. This is the noisy-gate analog of the classical fact that $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$ for independent events.

OR via the direct truth table

We now re-derive the OR false-positive rate directly from the truth table, to confirm that both paths yield the same result.

The OR of the observed outputs is $\top$ if *at least one* of $\hat{b}_1$, $\hat{b}_2$ is $\top$. The true OR output, $b_1 \vee b_2$, is $\top$ if at least one of b_1, b_2 is $\top$.

True b_1	True b_2	Correct OR output	Pr[OR output $= \bot$]
$\top$	$\top$	$\top$	$\omega_1\omega_2$
$\top$	$\bot$	$\top$	$\omega_1(1-\varepsilon_2)$
$\bot$	$\top$	$\top$	$(1-\varepsilon_1)\omega_2$
$\bot$	$\bot$	$\bot$	$(1-\varepsilon_1)(1-\varepsilon_2)$

The rightmost column gives Pr[observed OR $= \bot$] for each true-input combination (the OR is $\bot$ only when both observed outputs are $\bot$, which by Axiom 1 has probability equal to the product of the individual $\bot$-reporting probabilities).

False-positive rate of OR. A false positive of the OR gate occurs when the true OR is $\bot$—meaning both true inputs are $\bot$—but the observed OR is $\top$. This is the complement of both observed outputs being $\bot$:

$$\varepsilon_\vee \;=\; \Pr[\hat{b}_1 = \top \text{ or } \hat{b}_2 = \top \mid b_1 = \bot, b_2 = \bot] \;=\; 1-(1-\varepsilon_1)(1-\varepsilon_2) \;=\; \varepsilon_1+\varepsilon_2-\varepsilon_1\varepsilon_2. \tag{3.6}$$

This matches (3.5) from the De Morgan derivation exactly.

False-negative rate of OR. A false negative of the OR gate occurs when the true OR is $\top$—meaning at least one true input is $\top$—but the observed OR is $\bot$. The observed OR is $\bot$ when *both* observed outputs are $\bot$.

The three cases where the true OR is $\top$ are $(\top, \top)$, $(\top, \bot)$, and $(\bot, \top)$:

$$\begin{aligned}
\Pr[\hat{b}_1 = \bot, \hat{b}_2 = \bot \mid b_1 = \top, b_2 = \top] \;&=\; \omega_1\omega_2, \\
\Pr[\hat{b}_1 = \bot, \hat{b}_2 = \bot \mid b_1 = \top, b_2 = \bot] \;&=\; \omega_1(1-\varepsilon_2), \\
\Pr[\hat{b}_1 = \bot, \hat{b}_2 = \bot \mid b_1 = \bot, b_2 = \top] \;&=\; (1-\varepsilon_1)\omega_2.
\end{aligned}$$

As with the AND's false-positive rate, the OR's false-negative rate depends on the input distribution: different mixtures of the three patterns yield different marginal FNR values. For the special case $\varepsilon_1 = \varepsilon_2 = 0$ (no false positives), the three conditional FNRs simplify to $\omega_1\omega_2$, ω_1, and ω_2 respectively. The dominant pattern is then whichever positive-input combination is most common: if most queries have exactly one positive input, the FNR of OR is close to ω_1 or ω_2 individually (OR requires only one correct positive report, but it still misses when the positive input's reporter makes a false negative).

In contrast, the FPR of OR (3.6) is fully determined by the individual FPRs alone: the only way the true OR is $\bot$ is if both true inputs are $\bot$, and

the only way the observed OR is then $\top$ is if at least one of the reporters makes a false positive. Both the input pattern and the conditional false-positive probability are distribution-free. This asymmetry between the OR's FPR and FNR is the dual of the AND's asymmetry: AND has a distribution-free FNR and a distribution-dependent FPR; OR has a distribution-free FPR and a distribution-dependent FNR.

Summary of the three gates

Gate	FPR	FNR
$\tilde{b}_1 \wedge \tilde{b}_2$ (AND)	depends on input prior (see §3.1)	$\omega_1 + \omega_2 - \omega_1\omega_2$
$\neg\tilde{b}$ (NOT)	ω	ε
$\tilde{b}_1 \vee \tilde{b}_2$ (OR)	$\varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2$	depends on input prior

The table crystallizes the fundamental duality between AND and OR. Under noise, AND suppresses false positives at the cost of inflating false negatives (the FNR of AND grows monotonically with the number of ANDed instances), while OR suppresses false negatives at the cost of inflating false positives (the FPR of OR grows monotonically with the number of ORed instances). NOT is the exchange operator that converts one regime into the other.

Worked check: consistency at $\varepsilon = \omega$. For the symmetric case $\varepsilon_1 = \varepsilon_2 = \varepsilon$ and $\omega_1 = \omega_2 = \omega$:

$$\omega_\wedge = 2\omega - \omega^2,$$
$$\varepsilon_\vee = 2\varepsilon - \varepsilon^2.$$

For a BSC with crossover probability $\varepsilon = \omega$, these are symmetric: the AND gate inflates errors for positives by the same formula as the OR gate inflates errors for negatives. De Morgan's duality is exact, as expected.

Boolean algebra closure under noise

AND, OR, and NOT together generate all of two-valued Boolean algebra. Every Boolean function of n variables can be expressed as a formula built from AND, OR, and NOT (this is the completeness of the connective basis $\{\wedge, \vee, \neg\}$). We have now derived the channel matrices for each of the three generators. The key structural observation is the following.

Proposition 3.2.1 (Closure of Bernoulli[bool] under Boolean operations)**.** *If* $\tilde{b}_1, \ldots, \tilde{b}_n$ *are* Bernoulli[bool] *instances, then every Boolean formula* $\varphi(\tilde{b}_1, \ldots, \tilde{b}_n)$ *built from AND, OR, and NOT is also a* Bernoulli[bool] *instance. Its channel matrix has entries expressible as polynomials in the input rates* $\varepsilon_1, \omega_1, \ldots, \varepsilon_n, \omega_n$*.*

Proof sketch. By induction on the structure of φ. The base case is a single instance $\tilde{b}_i$, which is Bernoulli[bool] by assumption. For the inductive step:

NOT: if $\tilde{b}$ is Bernoulli[bool] with rates (ε, ω), then $\neg\tilde{b}$ is Bernoulli[bool] with rates (ω, ε) by the derivation above.

AND: if $\tilde{b}_1$ and $\tilde{b}_2$ are Bernoulli[bool] instances, then $\tilde{b}_1 \wedge \tilde{b}_2$ is Bernoulli[bool] by the derivation in Section 3.1, with FNR given by (3.1) and FPR by (3.2). Both are polynomial functions of the input rates.

OR: by De Morgan's law, $\tilde{b}_1 \vee \tilde{b}_2 = \neg(\neg\tilde{b}_1 \wedge \neg\tilde{b}_2)$. The outer and inner NOTs each preserve Bernoulli[bool] membership (by the NOT case); the AND of the negated instances is Bernoulli[bool] (by the AND case). The composition is therefore Bernoulli[bool].

Since every Boolean formula is built by finitely many applications of AND, OR, and NOT, the induction terminates and the result holds. □

Closure is not merely a formal nicety. It is what makes the Bernoulli framework composable: we can build complex Boolean circuits from noisy gates and know that every intermediate and final result is still a Bernoulli[bool] instance. Error propagation through the circuit is computable in closed form; it does not require re-deriving the framework from scratch at each stage.

Looking ahead to composition

The OR gate illustrates a structural point about the composition theorem, which Section 3.3 will state precisely.

The FPR of OR is $\varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2$. For small ε_1 and ε_2, this is approximately $\varepsilon_1 + \varepsilon_2$: error rates *add* under OR. Equivalently, the probability of no error is $(1 - \varepsilon_1)(1 - \varepsilon_2)$: no-error probabilities *multiply*.

The AND gate, dually, has FNR equal to $\omega_1 + \omega_2 - \omega_1\omega_2$: a symmetric formula showing that false-negative error rates add under AND. The no-false-negative probability $(1 - \omega_1)(1 - \omega_2)$ again factors as a product.

Both formulas are instances of the same pattern. When we chain k independent identity-with-error stages—each of which fails independently with rate η_i—the total no-error probability is $\prod_i(1 - \eta_i)$, and the total error rate is $\eta_{\text{total}} = 1 - \prod_i(1 - \eta_i)$. Section 3.3 proves this formula in its

general form and shows that the AND and OR formulas above are special cases. Once we have the composition theorem in hand, Miller-Rabin's k-round error bound becomes immediate: k rounds of witness-ANDing drive the FPR to $(1/4)^k$ because each round is an independent stage of error suppression, and the no-error probability multiplies.

The gates derived in this section—NOT, OR, AND, and their interaction via De Morgan—are the moving parts of that composition argument. They are also the gates that give the Bernoulli framework its claim to being a complete algebra, not merely a collection of special-purpose error-rate formulas.

3.3 The Composition Theorem

Section 3.1 showed that the AND of k independent Bernoulli[bool] witnesses drives the false-positive rate down: requiring all k to agree on $\top$ makes a false $\top$ output unlikely. Section 3.2 showed the dual: the OR of k witnesses drives the false-negative rate down. Both sections closed with a preview of the same structural pattern: the probability that the chain produces *no* error factors as a product.

This section makes that pattern precise. We state and prove the *composition theorem*: for k independent identity-with-error stages chained in series, the end-to-end error rate is

$$\eta_{\text{total}} = 1 - \prod_{i=1}^{k} (1 - \eta_i),$$

where η_i is the per-stage error rate. We then examine what this formula does and does not capture, paying particular attention to the BSC running thread—because the BSC has a subtlety, a cancellation structure, that the composition theorem does not capture. Understanding that gap is essential to using the theorem correctly.

Setup: chaining identity-with-error stages

The model is simple. We have an original Boolean value $b \in \{\top, \bot\}$. A single *identity-with-error* stage passes b through a channel that reports the correct value with probability $1 - \eta_i$ and reports the wrong value with probability η_i. Call the stage's output $\hat{b}_i$. The stage is not a BSC in general: we will say more shortly about what the error rate η_i means. For now, think of it as the probability that the stage makes *any* error, regardless of direction.

We chain k such stages: $b \to \hat{b}_1 \to \hat{b}_2 \to \cdots \to \hat{b}_k$. The final output $\hat{b}_k$ is what we observe. The question is: what is the probability that $\hat{b}_k \neq b$?

For this question to have a clean answer, we need a clean independence structure. We assume:

1. **Chain-stage independence.** The error event at stage i is independent of the error event at every other stage $j \neq i$. This is a chain-level assumption: it says that the k stages do not share error-inducing correlations along the pipeline.

2. **Each error is one-directional.** Once stage i makes an error, the subsequent stages in the chain receive the wrong value and propagate it. Errors do not cancel. This is the identity-with-error regime: any single error in the chain corrupts the end-to-end output.

The second assumption is worth flagging before the theorem, not after. It is the source of the BSC subtlety we will return to.

The composition theorem

Theorem 3.3.1 (Composition theorem, identity-with-error chains)**.** *Let b be a Boolean value. Let $\hat{b}_1, \hat{b}_2, \ldots, \hat{b}_k$ be the successive outputs of k chained identity-with-error stages, where stage i has error rate $\eta_i \in [0, 1]$ and the error events across stages are mutually independent.*

Under the identity-with-error assumption (any single error in the chain corrupts the end-to-end output), the end-to-end error rate satisfies

$$\eta_{\text{total}} \;=\; 1 - \prod_{i=1}^{k} (1 - \eta_i). \tag{3.7}$$

Equivalently, letting $\eta_i = 1 - \eta_i$ denote the per-stage correctness probability*, the end-to-end correctness probability is*

$$\eta_{\text{total}} \;=\; \prod_{i=1}^{k} \eta_i. \tag{3.8}$$

Proof. We prove the correctness-probability form (3.8) by induction on k, then recover (3.7) by complementation.

Base case, $k = 1$. A single stage with error rate η_1 has correctness probability $\eta_1 = 1 - \eta_1$. The formula gives $\prod_{i=1}^{1} \eta_i = \eta_1$. Correct.

Inductive step. Assume the result holds for $k-1$ stages: the first $k-1$ stages in the chain produce a correct output with probability $\eta_1 \cdots \eta_{k-1}$. Now append stage k with error rate η_k.

Under the identity-with-error assumption, the final output $\hat{b}_k$ equals b if and only if *both* of the following hold:

- The output of the first $k-1$ stages is correct, with probability $\eta_1 \cdots \eta_{k-1}$ (by the inductive hypothesis).
- Stage k does not make an error, with probability η_k.

By chain-stage independence, these two events are independent. Their joint probability is:

$$\eta_{\text{total}} \;=\; (\eta_1 \cdots \eta_{k-1}) \cdot \eta_k \;=\; \prod_{i=1}^{k} \eta_i.$$

Taking the complement, the end-to-end error rate is

$$\eta_{\text{total}} = 1 - \eta_{\text{total}} = 1 - \prod_{i=1}^{k} (1 - \eta_i). \qquad \square$$

Remark on the identity-with-error assumption. The inductive step rests on the phrase "if and only if both." This is only valid when any error at any stage is fatal to correctness. If stage k receives an already-wrong value and makes an error, it *could* accidentally correct the previous error—returning to the right answer. The composition theorem excludes this possibility by assumption. For channels where cancellation is possible (notably the BSC), the formula (3.7) does not give the exact end-to-end error rate, and the matrix-product computation of Section 2.3 must be used instead. We return to this point with numerical evidence in the BSC worked example below.

Connection to the AND and OR formulas

Before the BSC example, let us confirm that the composition theorem unifies the formulas already derived in Sections 3.1 and 3.2.

AND as composition. In Section 3.1, the false-negative rate of AND was:

$$\omega_{\wedge} \;=\; 1 - (1 - \omega_1)(1 - \omega_2).$$

This is exactly (3.7) with $k = 2$ and $\eta_i = \omega_i$. Interpreting AND as a two-stage identity-with-error composition in the positive block: stage 1 fails to

deliver a $\top$ output with probability ω_1; stage 2 independently fails with probability ω_2; the AND of the two outputs is $\top$ only if both succeed. The identity-with-error assumption is satisfied in the positive block because if either stage makes a false negative (reports $\bot$ when the true value is $\top$), the AND output is $\bot$ regardless of the other stage. There is no cancellation in the $\top$ direction.

Extending to k independent Bernoulli[bool] instances all having true value $\top$, with false-negative rates $\omega_1, \ldots, \omega_k$: the k-fold AND has false-negative rate

$$\omega_{\wedge,k} \;=\; 1 - \prod_{i=1}^{k} (1 - \omega_i).$$

When all stages have the same rate ω, this simplifies to $1 - (1-\omega)^k$.

OR as composition. In Section 3.2, the false-positive rate of OR was:

$$\varepsilon_{\vee} \;=\; 1 - (1-\varepsilon_1)(1-\varepsilon_2).$$

This is (3.7) with $k = 2$ and $\eta_i = \varepsilon_i$. Interpreting OR as a two-stage identity-with-error composition in the negative block: stage 1 makes a false positive with probability ε_1; stage 2 independently with probability ε_2; the OR output is $\bot$ only if both avoid false positives. The identity-with-error assumption holds in the $\bot$ direction because once either stage makes a false positive (reports $\top$ when the true value is $\bot$), the OR output is $\top$ regardless.

Extending to k stages with false-positive rates $\varepsilon_1, \ldots, \varepsilon_k$: the k-fold OR has false-positive rate

$$\varepsilon_{\vee,k} \;=\; 1 - \prod_{i=1}^{k} (1 - \varepsilon_i).$$

The BSC running thread: where the formula applies and where it does not

The binary symmetric channel (BSC) from Section 2.3 is the principal running example for composition in this book. We use it here to illustrate both what the composition theorem captures and, critically, where its identity-with-error assumption breaks down.

What chapter 2 §2.3 derived. Section 2.3 computed the crossover probability of two BSCs in series using the matrix product $Q_{\text{total}} = Q_2 Q_1$. The result (Equation (2.1)) was:

$$\varepsilon_{\text{total}} \;=\; \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2.$$

For equal crossovers $\varepsilon_1 = \varepsilon_2 = \varepsilon$, this becomes:

$$\varepsilon_{\text{total}} \;=\; 2\varepsilon - 2\varepsilon^2. \tag{3.9}$$

What the composition theorem gives. If we naively apply (3.7) with $\eta_i = \varepsilon$ for both stages and $k = 2$, we obtain:

$$\eta_{\text{total}} \;=\; 1 - (1-\varepsilon)^2 \;=\; 2\varepsilon - \varepsilon^2. \tag{3.10}$$

These two formulas are different.

$$\underbrace{2\varepsilon - 2\varepsilon^2}_{\text{correct BSC composition, §2.3}} \quad \neq \quad \underbrace{2\varepsilon - \varepsilon^2}_{\text{naive §3.3 formula}} \quad .$$

For $\varepsilon = 0.1$:

$$\begin{aligned} \text{correct (§2.3):}\ & 2(0.1) - 2(0.01) = 0.18, \\ \text{naive (§3.3):}\ & 2(0.1) - (0.01) = 0.19. \end{aligned}$$

The naive formula overestimates the BSC's end-to-end error rate.

The discrepancy is not a bug in either formula. It reveals the identity-with-error assumption in action.

The cancellation structure of the BSC. In a BSC, a "flip" swaps $\top \leftrightarrow \bot$. Two flips cancel: $\top \xrightarrow{\text{flip}} \bot \xrightarrow{\text{flip}} \top$. The end-to-end output is wrong if and only if an *odd* number of flips occurred across the chain. The end-to-end output is right if an *even* number of flips occurred (including zero).

For $k = 2$ stages with equal crossover ε, the parity decomposition is:

$$\begin{aligned} \Pr[\text{exactly 0 flips}] &= (1-\varepsilon)^2, \\ \Pr[\text{exactly 1 flip}] &= 2\varepsilon(1-\varepsilon), \\ \Pr[\text{exactly 2 flips}] &= \varepsilon^2. \end{aligned}$$

The end-to-end error probability is the odd-flip probability:

$$\Pr[\text{odd number of flips}] = 2\varepsilon(1-\varepsilon) = 2\varepsilon - 2\varepsilon^2.$$

This matches the §2.3 formula (3.9) exactly. The "at-least-one-flip" probability that the naive §3.3 formula computes is

$$\Pr[\text{at least one flip}] = 1 - (1-\varepsilon)^2 = 2\varepsilon - \varepsilon^2,$$

which includes the event that both stages flip—an event that *cancels* to correctness. The composition theorem counts this event as an error; the BSC does not. Hence the overestimate.

Summary of the distinction. The composition theorem formula $\eta_{\text{total}} = 1 - \prod(1 - \eta_i)$ answers the question: *what is the probability that at least one stage in the chain makes an error?* For the identity-with-error model—where a single error corrupts the end-to-end output irrecoverably—this is also the end-to-end error rate. But for the BSC, two errors cancel, so the relevant question is: *what is the probability that an odd number of stages flip?* The BSC composition formula from Section 2.3 answers this second question.

The two questions agree only in special cases:

- **One-sided error channels: FNR $= 0$.** If $\omega = 0$ (a positive approximate set), then stage i can only make errors in one direction: a false positive cannot be "uncorrected" by a later false positive, because the false positive takes a $\perp$ input to $\top$, and a subsequent channel with FNR $= 0$ will report $\top$ correctly. There is no cancellation in the positive direction. The composition theorem formula gives the exact accumulated false-positive rate.

- **One-sided error channels: FPR $= 0$.** Symmetrically, if $\varepsilon = 0$, the only errors are false negatives, and two false negatives do not cancel (a missed positive remains missed). The composition theorem applies.

- **AND / OR gates.** As shown above, AND with FNR rates and OR with FPR rates are both in the identity-with-error regime in their respective dominant error direction. No cancellation is possible.

- **Miller-Rabin's k-round test.** The Miller-Rabin witness test has FNR $= 0$ per round (it never falsely declares a prime composite). The only error is a false positive, and a false positive cannot be "uncanceled" by subsequent rounds—each round independently either detects the compositeness or doesn't. The composition theorem applies directly: $\varepsilon_k \leq (1/4)^k$. We work through this in Section 3.4.

For the BSC, neither condition holds: the BSC has $\varepsilon = \omega$, and two flips cancel. The correct composition formula is the one in Section 2.3: matrix product, not the composition theorem of this section. This is not a contradiction; it is two different theorems answering two different questions about two different models.

The channel-matrix product view

The matrix-product computation in Section 2.3 gives the BSC composition formula (with cancellation): two flips that cancel return the chain to the correct answer, and the resulting end-to-end crossover is $\varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2$, not $1-(1-\varepsilon_1)(1-\varepsilon_2)$. The composition theorem of this section applies to a different underlying model: each stage either preserves correctness or destroys it, with no possibility of recovery ("once lost, stays lost"). The BSC's bidirectional flip structure is incompatible with this monotone correctness assumption. The two formulas describe different physical processes; neither is wrong, and they answer different questions about different models.

Sister structure: Kronecker for parallel queries

The composition theorem governs *serial* chains: one query passes through k stages in sequence. The Kronecker factorization theorem (Theorem 4.4.1) governs *parallel* queries: m queries on m distinct elements, each through its own copy of the channel.

In the serial case (this section), correctness probabilities multiply along the chain:

$$\eta_{\text{total}} \;=\; \eta_1 \cdot \eta_2 \cdots \eta_k.$$

In the parallel case (Section 4.4), the joint confusion matrix factors as a Kronecker product:

$$Q^{(m)} \;=\; Q_1 \otimes Q_2 \otimes \cdots \otimes Q_m.$$

These two factorizations arise from different independence structures. The serial factorization uses *chain-stage independence*: the error events at distinct stages of the pipeline are independent. The Kronecker factorization uses *Axiom 1* (Axiom 1): the error events at distinct input elements are independent. The two independence assumptions can both hold simultaneously—and in well-designed systems, they do—but they are logically distinct.

An important practical consequence: when k parallel pipelines each consist of multiple stages in series, the combined channel matrix is

$$\left(Q_2^{(1)}Q_1^{(1)}\right) \otimes \left(Q_2^{(2)}Q_1^{(2)}\right) \otimes \cdots,$$

as Section 2.3 noted. The serial composition (matrix product) applies *within* each pipeline; the Kronecker product applies *across* parallel pipelines. The composition theorem of this section computes the correctness probability

of each pipeline's serial chain; the Kronecker theorem assembles the joint confusion matrix for the m parallel pipelines together.

A note on chain-stage independence vs. element-wise independence

The composition theorem assumes chain-stage independence. Axiom 1 (Axiom 1) guarantees element-wise independence: error events at distinct input elements are independent. These are closely related but operationally distinct.

In the Miller-Rabin example of Section 3.4, the k witness evaluations are drawn from a pool of witnesses. If we treat each witness as a distinct element of a predicate over the witness pool—which is the right recasting, as Section 3.4 details—then Axiom 1 licenses independence across witnesses. That independence is also chain-stage independence for the k-AND composition: the k AND stages each involve a distinct element, and their error events are independent by Axiom 1. In this setting, Axiom 1 implies chain-stage independence, and the composition theorem follows.

In contrast, consider a pipeline where stage i and stage j query the *same* element of the same Bernoulli predicate. If the implementation is deterministic (same input, same output), the two stages are perfectly correlated, not independent. Axiom 1 does not apply across repeated queries of the same element (Chapter 16 discusses this negative space). The composition theorem would not apply either: the chain-stage independence assumption fails when stages share elements.

The honest position is this: chain-stage independence is a structural property of the pipeline, not a consequence of Axiom 1 alone. In well-behaved implementations where each stage of the pipeline queries a distinct element, Axiom 1 implies chain-stage independence and the composition theorem holds. In pipelines where stages share elements or are correlated through shared randomness, the theorem requires additional justification or does not apply.

Worked numerical example: a 4-stage AND chain

To make the composition theorem concrete before we deploy it in Miller-Rabin, here is a worked example with four stages.

Let $\tilde{b}_1, \tilde{b}_2, \tilde{b}_3, \tilde{b}_4$ be independent Bernoulli[bool] instances, each a positive approximate set (FNR $= 0$), with false-positive rates:

$$\varepsilon_1 = 0.20, \quad \varepsilon_2 = 0.15, \quad \varepsilon_3 = 0.10, \quad \varepsilon_4 = 0.25.$$

We take their four-fold AND: declare $\top$ only if all four report $\top$.

Since $\omega_i = 0$ for all stages (positive approximate set), the four-fold AND has $\omega_\wedge = 0$ as well: the AND never falsely suppresses a true $\top$ (every stage correctly passes $\top$ inputs through).

The composition theorem gives the false-positive rate of the AND. With $\eta_i = \varepsilon_i$:

$$\eta_{\text{total}} = (1 - 0.20)(1 - 0.15)(1 - 0.10)(1 - 0.25) = 0.80 \times 0.85 \times 0.90 \times 0.75.$$

Computing step by step:

$$\begin{aligned} 0.80 \times 0.85 &= 0.680, \\ 0.680 \times 0.90 &= 0.612, \\ 0.612 \times 0.75 &= 0.459. \end{aligned}$$

So $\eta_{\text{total}} = 0.459$ and:

$$\varepsilon_{\wedge,4} = 1 - 0.459 = 0.541.$$

Is this better than each individual stage? Yes: the individual stages have $\varepsilon_i \in \{0.10, 0.15, 0.20, 0.25\}$. The AND gate has $\varepsilon_{\wedge,4} \approx 0.541$—actually worse than each individual stage on its own.

This counterintuitive result is not a paradox. The stages in this example are quite noisy (ε_i up to 0.25). The AND of four noisy stages accumulates the FNR in the positive direction (here zero, since FNR = 0) and pushes the FPR down *only* when $\varepsilon_i < 0.5$ and the product $\prod(1 - \varepsilon_i)$ is large. With four stages each above 0.10, the product of TNRs is only 0.459, so the AND gate's FPR is 0.541. To get an AND gate with FPR below any individual stage, we need the product of individual TNRs to exceed the best individual TNR, i.e., the other stages must compensate.

For comparison, if all four stages had $\varepsilon = 0.25$:

$$\varepsilon_{\wedge,4} = 1 - (0.75)^4 = 1 - 0.3164 = 0.684.$$

And if all four stages had $\varepsilon = 0.05$:

$$\varepsilon_{\wedge,4} = 1 - (0.95)^4 = 1 - 0.8145 = 0.185.$$

With low per-stage FPR, the AND gate's FPR is substantially lower than any individual stage. This is the regime Miller-Rabin exploits: with $\varepsilon \leq 1/4$ per round, a k-round AND achieves $\varepsilon_k \leq (1/4)^k$.

Looking ahead to Miller-Rabin

The composition theorem establishes the machinery. Section 3.4 deploys it against a canonical example: Miller-Rabin witnesses are Bernoulli[bool] instances with FNR $= 0$ and FPR $\leq 1/4$, so k independent rounds give the bound $\varepsilon_k \leq (1/4)^k$ from (3.7), a direct instance of the framework developed in this section.

3.4 Miller-Rabin: A Worked Walkthrough

Part I has built up a small but complete toolkit: Bernoulli[bool] instances with error rates (ε, ω), the Boolean algebra over noisy bits (Section 3.1), and the composition theorem that governs how error rates accumulate when instances are combined (Section 3.3). This section deploys that toolkit in full against a canonical example from number theory.

The Miller-Rabin primality test is one of the most widely used randomized algorithms in practice. It decides, with adjustable confidence, whether an integer n is prime. We will show that the algorithm IS a k-fold Bernoulli[bool] composition. Every formula in the Miller-Rabin analysis—the per-round false-positive bound $\varepsilon \leq 1/4$, the k-round bound $\varepsilon_k \leq (1/4)^k$, and the design rule "choose k large enough"—falls directly out of the framework developed in §§3.1–3.3. The algorithm contributes one non-trivial mathematical fact; everything else is the composition theorem applied.

Step 1: The witness predicate

Fix a positive odd integer $n > 3$ whose primality we wish to decide. A single *Miller-Rabin witness test* proceeds as follows.

Write $n - 1 = 2^r \cdot d$ with d odd and $r \geq 1$. Choose a witness $a \in \{2, 3, \ldots, n-2\}$ uniformly at random. Compute the sequence

$$a^d \bmod n, \quad a^{2d} \bmod n, \quad a^{4d} \bmod n, \quad \ldots, \quad a^{2^r d} \bmod n.$$

Declare n **composite** (the definitive outcome) if none of these values equals $\pm 1 \pmod n$, or more precisely: if $a^d \not\equiv \pm 1$ and $a^{2^j d} \not\equiv -1$ for all $j \in \{1, \ldots, r-1\}$. Declare n **probably prime** (the uncertain outcome) otherwise.

The test returns one of two verdicts:

- **"composite":** n is definitely not prime. No false negatives here—a prime number always passes the test (Fermat's little theorem guaran-

tees $a^{n-1} \equiv 1 \pmod{n}$ for any a coprime to n, and the sequence of square roots must eventually exhibit a trivial square root of 1).

- **"probably prime":** n might be prime, or might be a composite that "fooled" witness a. The latter is possible for composites with large *strong-liar sets*.

This is already the shape of a Bernoulli[bool] instance, as we make precise next.

Step 2: Characterizing the test as Bernoulli[bool] with FNR $= 0$, FPR $\leq 1/4$

The witness test is a predicate on an integer n:

$$W_a(n) \;=\; \begin{cases} \text{composite} & \text{if } a \text{ certifies compositeness,} \\ \text{probably prime} & \text{otherwise.} \end{cases}$$

Identify "composite" with $\bot$ (false, n is not prime) and "probably prime" with $\top$ (positive result, n might be prime). The true value of the predicate is whether n is actually prime.

In these terms, two error events are possible:

- **False negative (ω):** n is prime but $W_a(n) =$ composite. This cannot happen: if n is prime, then $a^d \equiv \pm 1 \pmod{n}$ or some $a^{2^j d} \equiv -1 \pmod{n}$, and the test correctly declares n probably prime. So $\omega = 0$.
- **False positive (ε):** n is composite but $W_a(n) =$ probably prime. This can happen when a is a *strong liar* for n: an element of $\{2, \ldots, n-2\}$ that passes the Miller-Rabin test even though n is composite.

The Miller-Rabin theorem states that for any composite n, the number of strong liars in $\{2, \ldots, n-2\}$ is at most $(n-3)/4$. A uniformly random witness therefore satisfies:

$$\varepsilon \;\leq\; \frac{1}{4}. \tag{3.11}$$

The $1/4$ bound is sharp: some composites (products of two primes of similar size) come close to having $(n-3)/4$ strong liars.

The single-round witness test is therefore a Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$ (see Definition 4.5.1 for the formal type).

Predicate recasting: fitting Axiom 1. There is a subtle point here that deserves explicit attention, foreshadowed in Section 4.2 and Axiom 1.

The natural description of the Miller-Rabin test is: fix a single candidate n, and draw k independent random witnesses $a_1, a_2, \ldots, a_k \in \{2, \ldots, n-2\}$. In this framing, randomness enters via the witness draw, not via the input. The input n is *fixed* across all rounds. But Axiom 1 is a statement about distinct *input elements* to a single predicate: the error events at distinct inputs are independent.

To apply Axiom 1 cleanly, we recast the test. Define a new predicate

$$\tilde{P}\colon \{2, \ldots, n-2\} \;\to\; \{\text{composite, probably prime}\}$$

where $\tilde{P}(a)$ is the verdict of the witness test applied to n with witness a. Now the witness pool $\{2, \ldots, n-2\}$ plays the role of the *universe*, and distinct witnesses $a_i \neq a_j$ are distinct input elements. Axiom 1 is the claim that the error event at a_i (the event "a_i is a strong liar for n") is independent of the error event at a_j (the event "a_j is a strong liar for n"). Whether this independence actually holds for a given composite n is a fact about the structure of n's strong-liar set, not something that Axiom 1 grants by fiat. We return to this point in Step 5. What the recasting achieves is structural: distinct witnesses are now distinct input elements, and the Axiom 1 framework applies to the recast predicate $\tilde{P}$ in the same way it applies to any other Bernoulli[bool] predicate.

The k-round Miller-Rabin test is then the AND of k evaluations of $\tilde{P}$: declare "probably prime" only if all k witnesses return "probably prime". This is a k-fold AND of Bernoulli[bool] instances, exactly the operation analyzed in Section 3.1.

Step 3: Applying the composition theorem for k rounds

With the predicate recasting in place, the composition theorem (Section 3.3, Theorem 3.3.1) applies directly.

Each round is a Bernoulli[bool] instance with $\omega_i = 0$ and $\varepsilon_i \leq 1/4$. The k-fold AND has:

False-negative rate of the k-round test. Since $\omega_i = 0$ for all rounds, any prime will pass every round correctly. The AND of zero-false-negative stages has zero false-negative rate:

$$\omega_k \;=\; 1 - \prod_{i=1}^{k}(1-\omega_i) \;=\; 1 - 1^k \;=\; 0.$$

The k-round test never misidentifies a prime as composite, regardless of k.

False-positive rate of the k-round test. For the FNR $= 0$ (one-sided error) case, the composition theorem applies without the BSC cancellation caveat noted in Section 3.3: a false positive at any round cannot be "corrected" by a later round, because subsequent rounds with $\omega = 0$ correctly pass $\top$ inputs through. The k-round false-positive rate satisfies:

$$\varepsilon_k \;\leq\; \varepsilon^k \;\leq\; \left(\frac{1}{4}\right)^k. \tag{3.12}$$

Here the first inequality uses the fact that, when $\omega_i = 0$, the composition theorem's "at least one error" formula for the false-positive rate reduces to the product $\prod_{i=1}^{k} \varepsilon_i$, since each round making a false positive is the only way the chain can make a false positive.

To see this: the k-round AND declares "probably prime" if and only if every round declares "probably prime". For a composite n, this happens if and only if every witness a_i is a strong liar. By the Axiom 1 recasting, the events "a_i is a strong liar" are independent (under the worst-case assumption over n). The probability that all k witnesses are strong liars is at most $(1/4)^k$.

Equivalently: the correctness probability of the k-round test (the probability of correctly declaring a composite as composite) is at least $1 - (1/4)^k$, and the correctness probability of correctly identifying a prime as probably prime is exactly 1.

The three key error bounds for standard choices of k:

k	$\varepsilon_k \leq (1/4)^k$	Approximate
1	$1/4$	0.250
5	$(1/4)^5$	$\approx 9.8 \times 10^{-4}$
20	$(1/4)^{20}$	$\approx 9.1 \times 10^{-13}$

At $k = 20$ rounds, the probability of a false positive is below 10^{-12}: for practical purposes, the test is definitive.

Step 4: Pseudocode and numerical example

Worked example: $n = 341 = 11 \times 31$.

We choose $n = 341$ because it is a pedagogically useful case: 341 is a *Fermat pseudoprime to base 2*, meaning the naive Fermat test ($2^{n-1} \equiv 1 \pmod{n}$?) would incorrectly pass it. Indeed, $2^{340} \equiv 1 \pmod{341}$, so Fermat fails. Miller-Rabin does not.

Setup. We have $n - 1 = 340$. Factoring out powers of 2: $340 = 4 \times 85 = 2^2 \times 85$. So $r = 2$ and $d = 85$.

Algorithm 1: Miller-Rabin primality test (k rounds)

input : n: odd integer > 3; k: number of rounds ($k \geq 1$)
output : COMPOSITE (definitive) or PROBABLY-PRIME (with FPR $\leq (1/4)^k$)

```
1  function MillerRabin(n, k)
2      write n − 1 = 2^r · d with d odd;
3      for i ← 1 to k do
4          choose a uniformly at random from {2, ..., n − 2};
5          x ← a^d mod n;
6          if x = 1 or x = n − 1 then
7              continue ;            // witness a passes; try next
8          end
9          for j ← 1 to r − 1 do
10             x ← x^2 mod n;
11             if x = n − 1 then
12                 break ;            // found −1; witness passes
13             end
14         end
15         if x ≠ n − 1 then return COMPOSITE;
16     end
17     return PROBABLY-PRIME;
```

Witness $a = 2$. Compute the sequence:

$$a^d \bmod n = 2^{85} \bmod 341 = 32.$$
$$a^{2d} \bmod n = 32^2 \bmod 341 = 1024 \bmod 341 = 1.$$

The test checks: is $a^d \equiv \pm 1 \pmod{n}$? We have $32 \not\equiv 1$ and $32 \not\equiv 340 \equiv -1 \pmod{341}$. So we proceed to squaring.

Next, is $a^{2d} \equiv -1 \pmod{n}$? We have $1 \not\equiv 340 \pmod{341}$. We have exhausted all $r - 1 = 1$ squarings without finding -1.

Therefore, $2^{85} \equiv 32 \pmod{341}$ with $32 \neq \pm 1$, and $32^2 \equiv 1 \pmod{341}$. This is a *non-trivial square root of 1* modulo 341: a value $x \neq \pm 1$ with $x^2 \equiv 1$. The existence of a non-trivial square root of 1 is a certificate of compositeness (if n were prime, the only square roots of 1 in $\mathbb{Z}/n\mathbb{Z}$ would be ± 1). The algorithm returns COMPOSITE.

Why does 341 fool Fermat but not Miller-Rabin? The Fermat test checks $2^{340} \equiv 1 \pmod{341}$, which holds—341 is a Fermat pseudoprime. Miller-

Rabin is stronger: it checks not just $a^{n-1} \equiv 1$, but whether the sequence of square roots leading to 1 goes through -1 at some point. For a prime p, every square root of 1 in $\mathbb{Z}/p\mathbb{Z}$ is ± 1. For the composite 341, the Chinese Remainder Theorem produces elements that square to 1 without being ± 1 modulo 341; witness $a = 2$ exposes one of them.

Error bounds for $n = 341$. For a composite with $r = 2$, $d = 85$, and k rounds:

k	$\varepsilon_k \leq (1/4)^k$	Interpretation
1	0.250	1-in-4 chance of passing a random witness
5	$\approx 9.8 \times 10^{-4}$	Better than 1-in-1000
20	$\approx 9.1 \times 10^{-13}$	Effectively certain

In practice, $k = 20$ is the standard for cryptographic applications. At that level the false-positive probability is negligible compared to hardware error rates.

Step 5: The unification thesis

We are now in a position to state plainly what this section has shown.

Miller-Rabin primality testing is not a separate technique that happens to use randomness. It is an instance of the Bernoulli framework applied to a specific mathematical predicate.

- The single-round witness test is a Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$. This follows from the Miller-Rabin theorem about strong-liar counts, not from the framework itself.
- The k-round test is the k-fold AND of Bernoulli[bool] witnesses. This is the composition of Section 3.1.
- The bound $\varepsilon_k \leq (1/4)^k$ is the composition theorem (Theorem 3.3.1) applied to the one-sided-error (FNR $= 0$) special case.
- The design rule "choose k large enough to drive ε_k below a target ϵ" is the composition theorem solved for k: $k \geq \log_{1/4}(\epsilon) = -2\log_2(\epsilon)$.
- The correctness of this design rule is guaranteed by the framework given one input from the number theory: $\varepsilon \leq 1/4$ per round.

Every structural step above was a consequence of the framework. The only fact that Miller-Rabin contributes is the 1/4 bound on the per-round

false-positive rate—that is the mathematical content of the Miller-Rabin theorem. The rest is Boolean algebra over noisy bits.

On the independence assumption. The analysis in Step 3 applied Axiom 1 to the recast predicate $\tilde{P}$, treating the error events at distinct witnesses as independent. It is worth being precise about what this means.

The $(1/4)^k$ bound is a *worst-case* bound over all composites n. It says: for any composite n, at most $1/4$ of witnesses are strong liars. Therefore, if we sample witnesses uniformly and independently, the probability that all k land in the strong-liar set is at most $(1/4)^k$.

The independence here is *over the uniform random draw of witnesses*, not a statement about the structure of the strong-liar set itself. For a given composite n with a fixed strong-liar set $L_n \subseteq \{2, \ldots, n-2\}$, the event "$a_i \in L_n$" and the event "$a_j \in L_n$" are independent when a_i and a_j are drawn independently and uniformly from $\{2, \ldots, n-2\}$, because membership of a uniform random element in L_n is independent across draws. This is sampling-with-replacement independence, which holds by construction, not by any special property of L_n.

So the application of Axiom 1 here is clean: we draw distinct witnesses, and the error events are independent by the uniform sampling protocol. The bound $(1/4)^k$ uses both the $|L_n| \leq (n-3)/4$ size bound (the mathematical content of Miller-Rabin) and the uniform independent sampling (the protocol that makes Axiom 1 applicable to the recast predicate $\tilde{P}$). Neither alone is sufficient; both are needed.

What this section was not. We have not given the most general treatment of randomized primality tests. The Las Vegas versus Monte Carlo distinction—and the question of what a *zero-error* primality test would require—belongs to chapter 11 (Chapter 11), where the Bernoulli framework connects to the full theory of randomized algorithms. The present section had one goal: to show that the core of Miller-Rabin is the composition theorem, nothing more.

Looking ahead

The Miller-Rabin example sits at one end of a spectrum. At the other end sits the Bloom filter: also a k-fold Bernoulli[bool] composition, also with a per-round false-positive rate that the composition theorem drives down, but drawn from a different application domain (approximate set membership rather than primality). Section 3.5 will make this connection explicit and show how the Boolean algebra of this chapter generalizes, in a single

structural step, to the algebra of approximate sets.

3.5 Composition is the Bridge to Sets

We have now arrived, via AND, OR, NOT, the composition theorem, and the Miller-Rabin walkthrough, at a complete picture of Boolean algebra under noise. A Bernoulli[bool] instance is a noisy channel on a two-element type. The Boolean operations close this type: AND, OR, NOT applied to Bernoulli[bool] instances produce new Bernoulli[bool] instances with derived error rates. The composition theorem (Section 3.3, Theorem 3.3.1) governs how those rates accumulate when instances are chained. Miller-Rabin (Section 3.4) showed the payoff: one input from number theory, everything else from the composition theorem.

But bool is only one type. Part II of this book builds approximate data structures over richer types: sets, maps, relations. The connection from Bernoulli[bool] to Bernoulli[S] is not a conceptual leap; it is a structural consequence of the same composition machinery developed in this chapter. Three named bridges make this concrete.

Bridge 1: A Bloom filter's m hash functions are m-fold AND composition

A Bloom filter tests set membership approximately. The classical construction uses m independent hash functions $h_1, h_2, \ldots, h_m$, each mapping a query element x to a bit position in a bit array. An element x is declared a member of the set if and only if all m bit positions are set in the array. The m-bit AND of hash outcomes is the filter's membership query.

In the Bernoulli[bool] language: each hash lookup is a Bernoulli[bool] instance. For a true member x, every hash position is set, so each individual lookup returns $\top$ with probability 1 (no false negatives: $\omega_i = 0$ for each hash). For a non-member x, each hash position is set independently with probability $p = 1 - (1 - 1/n)^k$ (where n is the bit-array size and k is the count of elements inserted), giving a per-hash false-positive rate $\varepsilon_i = p$. The Bloom filter's reported membership—"all m hash positions are set"—is the m-fold AND:

$$\bigwedge_{i=1}^{m} \tilde{h}_i(x)$$

where $\tilde{h}_i(x)$ is the i-th hash lookup result. The AND of Bernoulli[bool] instances with $\omega_i = 0$ is itself a Bernoulli[bool] instance (shown in Section 3.1), and its false-positive rate is the product of the individual false-positive rates by the composition theorem:

$$\varepsilon_{\text{filter}} \;\leq\; \prod_{i=1}^{m} \varepsilon_i \;=\; p^m.$$

This is exactly the classical Bloom filter false-positive rate formula, derived here as a direct consequence of the Boolean algebra of noisy bits.

This is not an analogy. A Bloom filter membership query is, definitionally, an m-fold AND of Bernoulli[bool] instances. The derivation above does not add anything beyond what this chapter has already established; it merely identifies the Bloom filter as an instance of a pattern already in hand.

The difference from Miller-Rabin. Both the Bloom filter and Miller-Rabin are m-fold AND compositions of Bernoulli[bool] instances with $\omega = 0$. The structural pattern is identical; the sources differ. Miller-Rabin's per-round $\varepsilon \leq 1/4$ bound is a theorem from number theory. The Bloom filter's per-hash ε is a design parameter controlled by the bit-array size and insertion count. In both cases, the composition theorem (Equation (3.12) for Miller-Rabin; the formula above for the Bloom filter) converts a per-stage guarantee into an end-to-end guarantee.

Chapter 5 develops the Bloom filter and its variants in full, including optimal parameter selection, the standard analysis of false-positive rate as a function of bit-array size, and the failure modes that chapter 16 (Chapter 16) examines.

Bridge 2: Set-operation error rates are AND, OR, NOT error rates

Bridge 1 takes a Bloom filter query—a Bernoulli[bool] instance—and deduces its error rates from the composition theorem. Bridge 2 lifts the same reasoning one level: from queries on individual elements to operations on *sets*.

Let $\tilde{A}$ and $\tilde{B}$ be two approximate sets, each a Bernoulli[S] instance for some universe S. For any element $x \in \mathsf{S}$, the membership indicator $x \in \tilde{A}$ is a Bernoulli[bool] instance with rates $(\varepsilon_A, \omega_A)$, and similarly $x \in \tilde{B}$ has rates $(\varepsilon_B, \omega_B)$.

Set operations on approximate sets are defined pointwise: for each x,

$$\begin{aligned} x \in \tilde{A} \cap \tilde{B} &\text{ is computed as } (x \in \tilde{A}) \wedge (x \in \tilde{B}), \\ x \in \tilde{A} \cup \tilde{B} &\text{ is computed as } (x \in \tilde{A}) \vee (x \in \tilde{B}), \\ x \in \tilde{A}^c &\text{ is computed as } \neg(x \in \tilde{A}). \end{aligned}$$

The error rates for intersection, union, and complement on approximate sets are therefore exactly the AND, OR, NOT error rates derived in Section 3.1 and the sections following it. For intersection:

$$\begin{aligned} \varepsilon_\cap &= \varepsilon_A \cdot \varepsilon_B, \\ \omega_\cap &= 1 - (1-\omega_A)(1-\omega_B) = \omega_A + \omega_B - \omega_A\omega_B. \end{aligned}$$

For union:

$$\begin{aligned} \varepsilon_\cup &= \varepsilon_A + \varepsilon_B - \varepsilon_A\varepsilon_B, \\ \omega_\cup &= \omega_A \cdot \omega_B. \end{aligned}$$

For complement, the rates simply swap:

$$\varepsilon_{A^c} = \omega_A, \qquad \omega_{A^c} = \varepsilon_A.$$

These formulas derive from nothing more than the AND, OR, NOT error rate formulas in chapter 3, applied one element at a time. The per-element membership query is a Bernoulli[bool] instance; the set operation is the Boolean operation on those instances.

This derivation is not the full story. To treat $\tilde{A} \cap \tilde{B}$ as a Bernoulli[S] instance requires that the per-element rates be uniform across S—that $\varepsilon_\cap(x)$ and $\omega_\cap(x)$ be the same for every x. This is exactly the rate-stability condition that chapter 4's axioms address. For well-behaved approximate set data structures (Bloom filters, quotient filters, cuckoo filters), this uniformity holds by construction. For malicious inputs or adversarial query sequences, it may fail; chapter 16 examines those failure modes.

Chapter 6 derives these formulas in full generality, works through the rate uniformity conditions, and shows how the Boolean algebra of Bernoulli[bool] lifts to a Boolean algebra of Bernoulli[S] instances.

Bridge 3: Approximate function composition lifts Boolean composition to maps

Bridges 1 and 2 both remain within the Boolean type: the output of the membership query is a bit. Bridge 3 generalizes to non-Boolean output types.

Let $f\colon X \to Y$ be a function from some input type X to some output type Y. An *approximate function* $\tilde{f}\colon X \to Y$ is a function that may return incorrect values. When $Y = \text{bool}$, this is precisely the Bernoulli[bool] setting: the error events are false positives and false negatives. When Y is an arbitrary type, errors are deviations from $f(x)$ in the output space; their structure depends on Y.

The composition machinery of this chapter generalizes from $Y = \text{bool}$ to arbitrary Y in a single structural step. Suppose we have two approximate functions:

$$\tilde{f}\colon X \to Y, \qquad \tilde{g}\colon Y \to Z.$$

The composed approximate function $\tilde{g} \circ \tilde{f}\colon X \to Z$ accumulates error from both stages. In the bool case, this accumulation is governed by the composition theorem of Section 3.3: the end-to-end error rate is derived from the per-stage rates via $\eta_{\text{total}} = 1 - \prod_i (1 - \eta_i)$. For general Y, the end-to-end error is a convolution of the two error distributions in Y and Z respectively; the Boolean formula is the special case where those distributions are Bernoulli distributions on a two-element type.

When chapter 9 introduces Bernoulli[M] for a map type $M = \text{Map}[X, Y]$, the composition machinery of this chapter is the foundation. Every derivation in the bool case carries over to the map case with Y-valued channels replacing scalar bits: the channel matrix Q grows from 2×2 to $|Y| \times |Y|$, but the composition law $Q_{\text{total}} = Q_k \cdots Q_1$ is unchanged. The Boolean algebra of approximate sets is the $Y = \text{bool}$ fiber of the full map algebra.

This is the sense in which Boolean composition is the bridge: not a metaphor, but a degenerate case. The bool formulas of this chapter are the $Y = \text{bool}$ specializations of formulas that hold for all output types. Chapter 9 will make the generalization explicit.

The chapter in one sentence

The three bridges are not independent observations. They are consequences of a single fact: Boolean operations on Bernoulli[bool] instances produce new Bernoulli[bool] instances, with error rates governed by the composition theorem. Bridge 1 identifies Bloom filters as m-fold AND. Bridge 2 identifies set operations as pointwise Boolean operations on membership indicators. Bridge 3 identifies the bool case as a special fiber of map composition.

Put differently: the composition theorem is a general fact about chained channels. In this chapter we have seen it in the Boolean case. Part II and Part III will see it in more general cases. The machinery does not change; only the output type does.

What comes next. The bibliographic notes (Section 3.5) place the Miller-Rabin analysis, the Bloom filter connection, and the chapter's derivations in the literature. Chapter 4 (Chapter 4), which this chapter has cited throughout, formalizes the independence axioms that our derivations assumed; reading chapters 3 and 4 in sequence closes the loop. Part II then builds on both chapters to develop the full algebra of approximate sets and maps.

Bibliographic Notes

The original Miller-Rabin papers. The primality test at the center of Section 3.4 has two distinct intellectual parents. Miller [Mil76] showed, assuming the Extended Riemann Hypothesis, that a deterministic variant of the witness test runs in polynomial time and is correct for all sufficiently large n. The witness-based structure—compute $a^d \bmod n$ and test for a non-trivial square root of unity—originates in that paper. Rabin [Rab80] removed the unproven hypothesis: his contribution was to show that, when the witness a is chosen uniformly at random from $\{2, \ldots, n-2\}$, the probability that a composite n passes the test is at most $1/4$, unconditionally. This is the bound that makes the composed k-round test practical; no number-theoretic conjecture is required. Chapter 3's Bernoulli reading of Miller-Rabin—treating a single witness as a Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$—is a reformulation of Rabin's 1980 result in the language of this text.

Standard textbook treatments. Both the Miller-Rabin algorithm and the broader topic of randomized algorithms receive thorough coverage in two standard graduate texts. Cormen et al. [Cor+09] covers Miller-Rabin in Chapter 31 (Number-Theoretic Algorithms), including a proof of the $1/4$ FPR bound via the structure of strong liars modulo a composite; that proof provides the number-theoretic backing for the Bernoulli characterization in Section 3.4. Mitzenmacher and Upfal [MU17] covers randomized algorithms and probabilistic analysis more broadly, with particular attention to the concentration-of-error phenomena that arise when independent probabilistic tests are composed—the same setting as Section 3.3's chain model. Readers who want the full number-theoretic argument for the $1/4$ bound, or who want to see the Miller-Rabin test situated among other randomized primality tests, should consult either of these references before proceeding to Part III's treatment of randomized algorithms.

The Bernoulli papers and the composition theorem. The composition theorem of Section 3.3—that chaining k identities-with-error gives an end-to-end error rate $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$—is a Boolean-algebra reformulation of Theorem 3.7 in Towell [Tow26a]. That paper states the rate-propagation result in terms of the Bernoulli set model; Chapter 3 restates it at the Boolean-algebra layer, where the "set" is replaced by a single Bernoulli[bool] instance and the composition is explicitly a chain of identities-with-error rather than set-intersection operations. The AND, OR, and NOT rules of Section 3.1 are derived in Towell [Tow26d], which extends the Bernoulli framework from Boolean sets to approximate values over arbitrary algebraic types; the bool case is the base case of that general theory.

Three abstraction layers, one composition algebra. A thread that runs through Chapters 2, 3, and 4 is worth naming explicitly here. The same underlying composition algebra appears at three different abstraction layers:

- *Matrix product layer.* In Section 2.3 (Chapter 2), serial composition of two binary channels corresponds to matrix multiplication $Q_{\text{total}} = Q_2 Q_1$. The composition formula for the crossover probability follows from this product by reading off the $(1, 0)$ entry of the composed channel matrix.
- *Boolean-algebra layer.* In Section 3.3 (this chapter), the same composition appears as $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$ for chained identity-with-error gates. This is the Boolean-algebra rephrasing of the matrix-product formula; for a symmetric channel the two are not identical because BSC composition has a cancellation structure (two flips return to the original), but for one-way identities-with-error the product formula is exact.
- *Kronecker-product layer.* In Section 4.4 (Chapter 4), parallel composition of independent Bernoulli[bool] queries factors the joint confusion matrix as a Kronecker product (Theorem 4.4.1). The Kronecker product is the algebraic expression of the same independence structure that underlies the k-round product formula of Section 3.3.

The three layers are not three separate ideas; they are one idea viewed through three different algebraic lenses. Chapter 4's Bibliographic Notes (Section 4.6) situate the Kronecker layer in its own literature; Chapter 2's Bibliographic Notes (Section 2.4) situate the matrix-product layer. Seeing all three in the same framework is a contribution of this text.

Exercises

1. ⋆**Deriving OR rates from AND via De Morgan.** Let b_1 and b_2 be Bernoulli[bool] instances with false-positive rates ε_1, ε_2 and false-negative rates ω_1, ω_2. Using De Morgan's law $b_1 \vee b_2 = \neg(\neg b_1 \wedge \neg b_2)$ and the NOT rule from Section 3.2 (NOT swaps ε and ω), derive closed-form expressions for the false-positive and false-negative rates of $b_1 \vee b_2$. Verify that your answer matches the direct truth-table derivation given in Section 3.2.

2. ⋆**FPR after a 3-stage AND chain (worked).** Suppose three independent Bernoulli[bool] instances b_1, b_2, b_3 each have false-positive rate $\varepsilon = 0.20$ and false-negative rate $\omega = 0$.

   (a) Using the AND output-rate formula from Section 3.1, compute the false-positive rate ε_{12} of $b_1 \wedge b_2$. (*Hint:* with $\omega = 0$, several terms vanish; the only way to get a spurious $\top$ output is if both inputs report $\top$ when the true answer is $\bot$.)

   (b) Now compute the false-positive rate of $(b_1 \wedge b_2) \wedge b_3$, i.e., the full 3-stage AND.

   (c) Confirm that this agrees with the composition-theorem bound $\varepsilon_k \leq (\varepsilon)^k$ obtained by applying Theorem 3.3.1 in its special FNR-zero form, with $k = 3$ and the per-stage success probability $1-\varepsilon = 0.80$ on negative inputs.

3. ⋆**Computing Miller-Rabin's k for a target FPR.** Miller-Rabin guarantees a per-round false-positive rate of at most $\varepsilon_1 = 1/4$ (Equation (3.11)).

   (a) Using the k-round bound $\varepsilon_k \leq (1/4)^k$ (Equation (3.12)), find the smallest integer k such that $\varepsilon_k \leq 10^{-6}$. (*Hint:* take logarithms base 10 on both sides.)

   (b) Find the smallest k for $\varepsilon_k \leq 10^{-15}$ (a bound suitable for cryptographic applications).

   (c) Suppose a weaker test had $\varepsilon_1 = 1/2$ per round. How many rounds would be needed to achieve $\varepsilon_k \leq 10^{-6}$? Comment on the practical difference.

4. ⋆⋆**Verifying the composition theorem on a 4-stage example.** Consider a chain of four identity-with-error stages, each with per-stage error rate $\eta_i = 0.10$.

(a) Apply the composition theorem $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$ (Equation (3.7)) with $k = 4$ and $\eta_i = 0.10$ to find η_{total}.

(b) Verify the same answer by building up stage by stage: compute $\eta_{1\oplus 2} = 1 - (1 - \eta_1)(1 - \eta_2)$, then $\eta_{1\oplus 2\oplus 3}$, then the full 4-stage result. Confirm each step agrees with the cumulative formula.

(c) Let $p_{\text{correct}} = 1 - \eta_{\text{total}}$ be the probability that all four stages produce a correct output simultaneously. Express p_{correct} as $(1 - \eta)^k$ and evaluate. Interpret the result: what fraction of queries emerge error-free from a four-stage pipeline with 10% per-stage error?

5. ⋆⋆**NOT is its own inverse on** Bernoulli[bool]**.** Recall from Section 3.2 that the NOT operation on a Bernoulli[bool] instance with rates (ε, ω) swaps those rates, producing a new instance with rates (ω, ε).

(a) Apply NOT twice to a Bernoulli[bool] instance with rates (ε, ω). Show algebraically that the resulting rates are again (ε, ω).

(b) Interpret the involution $(\text{NOT})^2 = \text{id}$ in terms of the channel matrix Q from Definition 4.5.1: what matrix operation corresponds to NOT, and why does applying it twice return the original matrix?

(c) Does the same involution hold for AND and OR? That is, is there a notion of "applying AND to itself" that returns the identity? Explain briefly why the answer differs from the NOT case.

6. ⋆⋆**Implement Miller-Rabin in 20 lines and check empirical FPR.** Implement the Miller-Rabin primality test in the language of your choice. The pseudocode in Section 3.4 fits in roughly 15 lines; a full implementation with multi-round looping and result aggregation fits in 20.

(a) Test your implementation on the following inputs and report the outcome for each: $n = 2, 3, 4, 17, 100, 341, 561, 1009$. (Among these, 341 is a Fermat pseudoprime to base 2 but not a Miller-Rabin pseudoprime; 561 is a Carmichael number. Your implementation should correctly declare both composite with enough rounds.)

(b) To estimate the empirical false-positive rate, generate a random sample of 1,000 composite odd integers in the range $[10^4, 10^6]$

(e.g., products of two random primes). Run Miller-Rabin with $k = 1$ round on each and record how many are incorrectly declared "probably prime." Compare your empirical rate to the theoretical bound of $1/4$.

(c) Repeat part (b) with $k = 5$ and $k = 10$ rounds. Plot (or tabulate) empirical FPR vs. k. Does the empirical curve fall below the theoretical bound $(1/4)^k$ at each k?

7. ⋆⋆**Prove the composition theorem assuming Axiom 1.** The composition theorem (Theorem 3.3.1) states that k independent identity-with-error stages in series produce an overall error rate $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$.

   (a) State precisely what independence assumption is needed at each stage of the chain for the per-stage probabilities to multiply. In the language of Section 4.2, what does it mean for chain-stage error events to be mutually independent?

   (b) Prove the formula by induction on k: the base case $k = 1$ is immediate; for the inductive step, use the chain rule and the independence of the k-th stage from all previous stages to factor the joint error probability.

   (c) As noted in Section 3.3, this is *chain-stage independence*, which is distinct from the element-wise Axiom 1 (Axiom 1) even though both involve independence of error events. Write one paragraph explaining the operational difference: Axiom 1 governs *which elements* are queried; chain-stage independence governs *which stage* in the pipeline produces an error on a single query.

8. ⋆⋆**Construct a 3-bit ANDed-confidence example.** Let b_1, b_2, b_3 be three independent Bernoulli[bool] instances representing three noisy sensors, each observing whether a physical quantity x exceeds a threshold (true answer: $\top$). Suppose each sensor has $\varepsilon = 0.15$ and $\omega = 0.05$.

   (a) Compute the false-positive and false-negative rates of the AND of all three sensors, $b_1 \wedge b_2 \wedge b_3$, by successive application of the AND formula from Section 3.1. (*Hint:* compute $b_1 \wedge b_2$ first, then AND with b_3.)

   (b) Compute the false-positive and false-negative rates of the OR of all three sensors, $b_1 \vee b_2 \vee b_3$, using the De Morgan derivation from Section 3.2 or directly.

(c) The AND system emphasizes low FPR; the OR system emphasizes low FNR. In numerical terms, which objective does the ANDed configuration better achieve compared with a single sensor? Which objective is *worse* after ANDing? Justify your answer with the numbers from parts (a) and (b).

9. ⋆⋆⋆**Why the AND derivation in §3.1 tacitly assumes Axiom 2.** Re-read the derivation of the AND output rates in Section 3.1. The derivation treats ε_1 and ε_2 as fixed constants when writing the joint probability that both inputs produce a false $\top$.

   (a) Identify the exact expression in the derivation where the false-positive rates of the two inputs are multiplied together. Why does this step require that ε_1 and ε_2 be independent random quantities given the underlying set A? State which axiom (Axiom 2) licenses that factoring.

   (b) Suppose Axiom 2 (Axiom 2) fails for b_1 and b_2: specifically, suppose the false-positive rate of b_2 is a function of the false-positive rate of b_1, say $\varepsilon_2 = g(\varepsilon_1)$ for some nontrivial function g. Write a corrected expression for the false-positive rate of $b_1 \wedge b_2$ in this dependent case (in terms of $\mathbb{E}[\varepsilon_1 \cdot g(\varepsilon_1)]$ or a similar expectation).

   (c) Give a concrete scenario where this dependence arises naturally: for example, two sensors whose noise levels share a common environmental cause (correlated hardware failures, ambient temperature, etc.). Explain in two or three sentences why the simple product formula would overestimate the FPR reduction achieved by ANDing such sensors.

10. ⋆⋆⋆**De Morgan's laws hold for noisy Boolean gates.** The standard De Morgan laws are $\neg(A \wedge B) = \neg A \vee \neg B$ and $\neg(A \vee B) = \neg A \wedge \neg B$. Show that the same identities hold at the level of error rates for Bernoulli[bool] instances, using the rate-swapping NOT rule from Section 3.2 and the AND rate formulas from Section 3.1.

   (a) Start from the OR-via-De-Morgan derivation in Section 3.2. Write out the step-by-step substitution: apply NOT to each input, apply AND, then apply NOT to the result. Track the false-positive and false-negative rates through each step.

(b) Verify that the resulting rates for $\neg(b_1 \wedge b_2)$ equal those obtained by directly computing the rates of $(\neg b_1) \vee (\neg b_2)$, confirming the first De Morgan law.

(c) In classical Boolean algebra, De Morgan's laws are *definitional*: OR is *defined* to be NOT-AND-NOT. Is the same true in the Bernoulli framework, or is the agreement of rates a theorem that requires proof? Identify any assumption (such as the closure property in Proposition 3.2.1) that the argument needs.

11. $\star\star\star$**Constructing a system where the composition theorem fails.** The composition theorem (Theorem 3.3.1) requires that the per-stage error events be mutually independent. This exercise shows that the theorem can fail when that condition is violated – a preview of the negative-space discussion in Chapter 16.

(a) Construct a two-stage identity-with-error system in which the error events at the two stages are perfectly positively correlated: whenever stage 1 errs, stage 2 errs with probability 1, and whenever stage 1 is correct, stage 2 is also correct. Let the per-stage error rate be $\eta = 0.10$. Compute the true end-to-end error rate η_{total}.

(b) Compare to the composition-theorem prediction $\eta_{\text{total}}^{\text{thm}} = 1 - (1 - \eta)^2$. By how much does the actual error rate exceed the prediction, in absolute terms and as a ratio?

(c) Generalise to k perfectly correlated stages. Show that the actual error rate is η (the per-stage rate) regardless of k, while the composition-theorem prediction approaches 0 as $k \to \infty$. In one paragraph, explain what this failure mode means for designing multi-stage pipelines in practice: when is the independence assumption most likely to be violated, and what structural property of the underlying system causes the correlation?

Chapter 4

Two Axioms and Kronecker Factorization

After this chapter you will be able to:

- State Axioms 1 and 2 of the Bernoulli model formally.
- Prove the Kronecker factorization theorem and derive the parameter count drop from $n(2^m - 1)$ to $2m$.
- Write the formal definition of Bernoulli[bool] and recognize every example from chapters 1–3 as an instance.

4.1 Why Axioms?

The three chapters behind us built something real. We derived the error rates for AND and OR gates operating on noisy bits (Section 3.1), showed that composition of channels produces predictable compound rates, and in Section 3.3 proved the composition theorem: when k independent noisy channels are chained in series, their true-positive rates compose multiplicatively. We then applied that theorem immediately to Miller–Rabin primality testing, watching a per-round false-positive rate of at most $1/4$ collapse to $(1/4)^k$ after k rounds. The derivations were concrete and the results were correct.

But correct derivations can hide assumptions. Chapters 1–3 made *two* independence assumptions so naturally that we did not pause to name them. This chapter names them, states them precisely, and then shows exactly what mathematical structure they license. That structure is a precise struc-

tural theorem showing what those assumptions cost and what they buy; it is the formal payoff for the care we are about to take.

The first hidden assumption: element-wise independence

Recall the composition theorem from Section 3.3. When we wrote the true-negative rate for k channels chained in series as $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$, we were treating each channel's contribution to the end-to-end error as independent of every other channel's contribution. One query enters the chain; it passes through channel 1, whose output feeds channel 2, and so on; the overall error rate is a product of independent per-stage terms. No correlation across stages was allowed, and the product formula relied on that fact throughout.

A different scenario where Axiom 1 matters: two classifiers trained on overlapping data from the same source. Their errors on a new input x will tend to correlate: when one misclassifies x, the other is more likely to as well. The product formula breaks. The effective error rate is worse than the product formula predicts, because the errors reinforce rather than cancel.

We will call the assumption that justifies the product formula *Axiom 1.* Roughly, Axiom 1 asserts independence of error events across distinct input elements; the precise statement and its scope appear in Section 4.2. The practical effect is that joint error distributions factor across distinct queries, which is what made the composition theorem's product form possible. When Axiom 1 holds, every formula in chapters 1–3 that relies on independence is automatically justified. When Axiom 1 fails, those formulas become bounds at best and misleading at worst. Chapter 3's Miller–Rabin analysis, for instance, requires Axiom 1: the k witness tests must be independent for the per-round bound to compound as $(1/4)^k$. If the witnesses were correlated, the product would underestimate the true composite error rate.

The second hidden assumption: stable block error rates

The derivation of AND under noise in Section 3.1 used specific values ε and ω, the false-positive and false-negative rates, treating them as fixed properties of each input. We implicitly assumed that querying an input repeatedly would yield the same error behavior each time: that ε does not drift from query to query, that the channel is stationary across uses.

This is not the only conceivable behavior. A Markov-correlated channel, for instance, lets the error rate on query $t+1$ depend on what happened at query t. If the channel just made a false positive, it might be more likely

to make another. The rates are not stable; they are history-dependent. We did not model this, and for good reason: the stationary case is the tractable and practically important one. But we should say we are assuming it.

We have been assuming Axiom 2 since Section 3.1; let us make it explicit. Axiom 2 says: the error rates within each partition block of the input domain are the same for every query, unconditionally. There is one channel matrix Q per input-block-pair, and it does not depend on which query in a sequence we are currently processing. Under Axiom 2, the channel is genuinely memoryless within each block: its statistical behavior is the same on the hundredth query as on the first.

This is a meaningful restriction, not a triviality. It rules out fatigue, warm-up effects, adversarial adaptation, and history dependence. It also makes the math work cleanly. We formalize Axiom 2 in Section 4.3.

What the axioms buy

Once both axioms are in place, something striking follows. The joint confusion matrix describing m independent queries on the channel is not a free matrix with exponentially many parameters. Under Axioms 1 and 2, it *must* factor into a product of m small per-query matrices, one factor per query. The exponential-to-linear collapse in parameter count, made precise in Section 4.4, is the formal payoff of this chapter.

This is not merely an algebraic convenience. It is the theorem that makes the Bernoulli framework useful in practice. Without it, a fifty-query analysis would require specifying an astronomically large matrix. With it, you need fifty pairs of numbers. The saving is exponential.

We prove this Kronecker factorization theorem, derive the parameter count corollary, and work through explicit examples in Section 4.4. The theorem is the chapter's climax and the formal vindication of every computation in chapters 1–3.

The formal definition

Once the axioms and the Kronecker theorem are in hand, we can at last write down the formal definition of the type Bernoulli[bool] without fear of circularity. Chapter 1 named the noisy bit informally. Chapter 2 gave it a channel matrix. Chapter 3 showed how to compose these objects. Chapter 4 closes by showing that all of those things are instances of a single precise definition: a triple (X, Y, Q) where X and Y are the input and output domains $\{\top, \bot\}$, Q is a 2×2 stochastic matrix, and the whole structure

satisfies Axioms 1 and 2. We work this out in Section 4.5, including the biased-coin example, introduced in Section 1.1 and deliberately left without a full PMF in Section 1.3 until we had the formal machinery to handle it.

The trajectory of this chapter, then, is: axioms first (Sections 4.2 and 4.3), then the theorem they make possible (Section 4.4), then the definition the theorem earns us (Section 4.5).

By the end, the reader who worked through chapters 1–3 will recognize every example, every formula, and every derivation as an instance of one coherent definition. That is the point.

4.2 Axiom 1: Element-Wise Independence

The previous section sketched two kinds of independence that chapters 1–3 quietly assumed. This section makes the first of them precise.

A brief note on framing before we begin. In Section 4.1 we used a chain-of-stages example as motivation: k noisy channels in series, with end-to-end error rate $1-\prod_{i=1}^{k}(1-\eta_i)$. That example invokes a kind of independence, but it is not the same statement as Axiom 1. Axiom 1 governs independence across distinct input elements of a *single* approximate predicate. Stage-wise independence in a composition chain is a separate assumption: closely related, often co-occurring, but operationally distinct. Both are needed for the chain formula; both fail in similar circumstances; and in well-behaved implementations both hold. We state Axiom 1 cleanly here, on a single predicate. The reader who wants to recover the chain formula from these axioms should track the two independences separately and combine them as the composition theorem of **??** prescribes.

The formal statement

Let U be the universal set, the domain of all possible inputs to the approximate membership predicate. For a fixed objective set $A \subseteq U$, the predicate either correctly reports the true answer or makes an error. For each $x \in U$, define the error indicator

$$E_x \;=\; \mathbf{1}\big[\tilde{A}(x) \neq \mathbf{1}_A(x)\big],$$

where $\tilde{A}(x)$ is the approximate predicate's output on input x and $\mathbf{1}_A(x)$ is the exact answer. $E_x = 1$ means an error occurred at x; $E_x = 0$ means the answer was correct.

Axiom 1 (Element-wise independence)**.** *The family $\{E_x : x \in U\}$ is mutually independent given the objective set A and the error rates. In particular, for any distinct $x, y \in U$, the pairwise factorization*

$$\mathrm{P}\left(E_x = 1,\, E_y = 1\right) \;=\; \mathrm{P}\left(E_x = 1\right) \cdot \mathrm{P}\left(E_y = 1\right)$$

holds. Furthermore, within each partition block B_i of U (defined formally in Section 4.3), the error indicators $\{E_x : x \in B_i\}$ are identically distributed.

The notation $\tilde{A}(x)$ is deliberately abstract: it stands for any approximate membership test, whether implemented as a Bloom filter, a probabilistic data structure, a noisy sensor, or a randomized algorithm. Axiom 1 is a property of the *process* that generates the answers, not of any particular implementation.

Consequence: factored joint distributions

Axiom 1 is not merely a philosophical assertion. It has an immediate and quantitative consequence for joint query distributions.

Proposition 4.2.1 (Factored joint distribution)**.** *Under Axiom 1, the joint distribution of the error indicators over any finite set of distinct inputs $\{x_1, \ldots, x_n\} \subseteq U$ factors as a product of n marginal distributions:*

$$\mathrm{P}\left(E_{x_1} = e_1,\, \ldots,\, E_{x_n} = e_n\right) \;=\; \prod_{i=1}^{n} \mathrm{P}\left(E_{x_i} = e_i\right)$$

for any choice of $e_1, \ldots, e_n \in \{0, 1\}$. Equivalently, the joint distribution of the observed responses $\tilde{A}(x_1), \ldots, \tilde{A}(x_n)$ factors as

$$\mathrm{P}\left(\tilde{A}(x_1) = s_1, \ldots, \tilde{A}(x_n) = s_n | A\right) \;=\; \prod_{i=1}^{n} \mathrm{P}\left(\tilde{A}(x_i) = s_i | A\right),$$

where each factor equals ε, η, τ, or ω depending on whether $x_i \in A$ and whether s_i is a correct response.

Proof. The first identity is the definition of mutual independence for the family $\{E_x : x \in U\}$, which Axiom 1 asserts directly. Mutual independence of the family $\{E_x : x \in U\}$ (the primary assertion of Axiom 1) implies, by definition, mutual independence of any finite subfamily; hence the joint distribution over $\{x_1, \ldots, x_n\}$ factors into the stated product of n marginals. The second identity follows because $\tilde{A}(x_i) = s_i$ is a deterministic function of

E_{x_i} and the exact answer $\mathbf{1}_A(x_i)$: knowing A determines $\mathbf{1}_A(x_i)$, and given that, $\tilde{A}(x_i) = s_i$ iff E_{x_i} takes the appropriate value. The product therefore inherits its factored form from the independence of $\{E_{x_i}\}$. □

This proposition is the engine behind every multi-query formula in the book. Whenever we write a joint probability over several distinct inputs as a product of marginals, we are using this proposition, which rests on Axiom 1.

The factored joint distribution also licenses a key observation about the joint confusion matrix of m simultaneous queries. Without any independence assumption, that matrix lives in a space of dimension $n(2^m - 1)$ for an n-th order model. Axiom 1 forces the rate-conditional joint matrix to factor as a Kronecker product of per-query 2×2 matrices, collapsing the parameter count to at most $2m$. That is the content of Section 4.4; we state it here as a forward reference so the reader knows why Axiom 1 matters at the level of the whole framework.

Example 1: The Bloom filter's independence assumption

A Bloom filter [Blo70] answers membership queries on a set A stored in a bit array of m bits, using k independent hash functions. When k hashes are applied to a new element x, each hash selects a bit position; x is declared a member if and only if all k selected positions are 1.

For elements $x \notin A$, the probability that all k positions happen to be set (a false positive) is approximately $(1 - e^{-kn/m})^k$, where $n = |A|$. This familiar formula is derived under an explicit assumption: for two distinct non-members x and y, the event "x is a false positive" is independent of the event "y is a false positive." Without this assumption, the formula gives neither an exact probability nor a useful bound; instead, one would need to account for correlations between all pairs of non-members, producing a quantity with no closed form.

Axiom 1 is exactly the assumption the Bloom filter's analysis needs. The filter's k hash functions are chosen to be independent, and the hash values for distinct elements are treated as unrelated. Whether or not this is precisely true in a given implementation, it is the assumption under which the false-positive rate formula is derived, and it is the assumption that the Bernoulli framework formalizes.

We will analyze the Bloom filter fully in Part II (Chapter 5), where Axiom 1 will be verified to hold under the standard random-oracle model for the hash functions. For now, the take-away is structural: the Bloom filter

is a Bernoulli[bool] implementation, and Axiom 1 is the assumption that makes that claim non-trivial. (For a Bloom filter the FNR is degenerate at zero, since a member element always matches all k of its hash positions; the rates-as-random-variables framing of Axiom 1 here governs the per-element independence of FPR error events across distinct non-member queries, not the FNR.)

Example 2: Miller-Rabin's k-round bound requires Axiom 1

In Section 3.4 we derived the Miller–Rabin bound: after k independent rounds of witness testing, the probability that a composite number passes all k rounds (the k-round false-positive rate) is at most $(1/4)^k$. The bound rests on two facts: each round's false-positive probability is at most $1/4$, and the rounds are independent.

There is a subtle reframing required to fit Miller-Rabin under Axiom 1. The predicate being tested takes a *single* fixed input n (the integer in question), and randomness enters via the witness draw rather than via the input. To apply Axiom 1 cleanly, recast the test as a predicate $\tilde{P}\colon \{2, \ldots, n-2\} \to \{\text{composite}, \text{probably prime}\}$ on the witness pool: now distinct witnesses *are* distinct input elements, and Axiom 1's claim is that error events at distinct witnesses (the events "w_i is a strong liar for n") are independent. Whether or not this independence holds for any particular composite n depends on the structure of n's strong-liar set, a separate fact about Miller-Rabin's mathematical content.

With that reframing in place, the connection to Axiom 1 is clean. Each round tests a different witness w_i drawn uniformly at random from $\{2, \ldots, n-2\}$. Whether w_i happens to be a strong liar for n (the error event at w_i) must be independent of whether w_j is a strong liar (the error event at w_j), for $i \neq j$. The error events are the events that the witness fails to detect compositeness.

Under Axiom 1, the joint probability that all k witnesses are strong liars is the product of the individual probabilities:

$$\mathrm{P}\,(\text{all } k \text{ witnesses are strong liars}) \;\leq\; \prod_{i=1}^{k} \frac{1}{4} \;=\; \left(\frac{1}{4}\right)^{k}.$$

Without Axiom 1, the k error events could be positively correlated. The events "w_1 is a liar" and "w_2 is a liar" may be positively correlated, because both witnesses are sampled from the same composite n, which may itself happen to have an unusually large strong-liar set. Conditioning on

n, the witnesses are not independent of each other when the liar fraction is unknown. In that case, the product bound $(1/4)^k$ would underestimate the true false-positive probability. The correct expression would involve the joint distribution over all k witnesses, which does not simplify to a product without an independence assumption.

The Miller–Rabin analysis works because the witnesses are drawn uniformly and independently from a large set. In the worst case, at most $1/4$ of all candidates are strong liars; with independent uniform sampling, the fraction is driven down exponentially. This is Axiom 1 at work.

Classifier-verdict callback: a related independence failure

The classifier-verdict thread from Section 4.1 returns here, but with a deliberate caveat. Two correlated classifiers evaluating the *same* input is not literally an Axiom 1 failure: Axiom 1 governs distinct *inputs* to a single predicate, not distinct *predicates* on a single input. The example we work below is one step removed from Axiom 1, but it illustrates the same family of phenomena: when error events are correlated for any reason, the naive product formula underestimates the joint failure rate. The Axiom 1 case (correlated errors at distinct inputs to one predicate) is structurally identical.

Suppose two classifiers C_1 and C_2 are both trained on a shared dataset with overlapping features. Each classifier is a Bernoulli[bool] individually: on any given input x, C_1 has false-positive rate ε_1 and C_2 has false-positive rate ε_2.

Now consider their joint behavior on a single negative input $x \notin A$. If the errors were independent, the probability that both make a false positive would be:

$$\mathrm{P}\left(C_1(x) = 1,\, C_2(x) = 1 | x \notin A\right) \stackrel{?}{=} \varepsilon_1 \cdot \varepsilon_2.$$

But the two classifiers share training data, so their errors on x tend to be correlated.

Let $\rho \in [0, 1]$ be the correlation between the error indicators $E_x^{(1)}$ and $E_x^{(2)}$. The joint probability is

$$\mathrm{P}\left(E_x^{(1)} = 1,\, E_x^{(2)} = 1\right) \;=\; \varepsilon_1\varepsilon_2 \;+\; \rho\sqrt{\varepsilon_1(1-\varepsilon_1)\cdot\varepsilon_2(1-\varepsilon_2)}.$$

For a concrete example, let $\varepsilon_1 = \varepsilon_2 = 0.10$ and $\rho = 0.60$. The product bound predicts a joint false-positive rate of $0.10 \times 0.10 = 0.01$. The actual joint rate is

$$0.01{+}0.60{\times}\sqrt{0.10 \times 0.90 \times 0.10 \times 0.90} \;=\; 0.01{+}0.60{\times}0.09 \;=\; 0.01{+}0.054 \;=\; 0.064.$$

The inflation factor is $0.064/0.01 = 6.4$: the true joint false-positive rate is $6.4\times$ the naive product.

This is not a failure of the individual classifiers, which are each well-behaved Bernoulli[bool] objects; it is a failure of the independence assumption between them. The Axiom 1 violation has the same arithmetic; only the source of the correlation differs.

The twin lessons are these. First, Axiom 1 is a genuine modeling assumption, not an automatic consequence of each individual predicate being a Bernoulli[bool]: the individual marginals can be well calibrated while the joint distribution fails to factor. Second, correlated errors are worse than independent errors in a precise, quantifiable sense: the inflation factor grows with the correlation coefficient and with the individual error rates.

The Kronecker theorem proved in Section 4.4 makes this precise at the level of the joint confusion matrix: under Axiom 1, the matrix has a structured Kronecker-product form that encodes exactly the factored joint distribution in Proposition 4.2.1. Without Axiom 1, the matrix is unstructured, the parameter count explodes, and the elegant formulas of chapters 1–3 must be replaced by unwieldy joint distributions over all pairs and triples of query events.

4.3 Axiom 2: Conditional Independence of Block Error Rates

Section 4.2 gave the first independence axiom: error events at distinct inputs are mutually independent, given the objective set and the error rates. That axiom governs the *spatial* structure of errors, the relationship between different elements of the universal set queried simultaneously or in any order.

This section gives the second axiom. Axiom 2 governs how the random error rates at different blocks relate to each other: knowing the rate on one block tells you nothing about the rate on another. This is a *cross-block independence* claim, not a within-block or across-queries stability claim. Combined with Axiom 1, however, it implies a stability we will derive as a formal consequence in Proposition 4.3.1: per-block rates are constant across queries. We state the formal axiom first, then derive the stability consequence, then work through what fails when stability fails.

The two axioms are logically independent of each other. Axiom 1 alone does not imply Axiom 2, and Axiom 2 alone does not imply Axiom 1. Both are needed, and their conjunction is exactly what makes the Kronecker factorization theorem of Section 4.4 possible.

Partition blocks and model order

Before stating Axiom 2, we need the partition structure it ranges over. A full treatment appears in Section 2.4; we summarize here the minimum needed for the axiom statement.

Given an objective set $A \subseteq U$, the Bernoulli model partitions the universal set U into blocks $B_1, \ldots, B_n$ such that every element within a block has the same error rate. The *model order* is the number of blocks n. The simplest nontrivial case is order 2: the partition $\{B_1, B_2\} = \{A,\, A^c\}$ separates the positives from the negatives and assigns each group its own error rate.

For the Boolean universe $U = \{\top, \bot\}$ with objective set $A = \{\top\}$, model order 2 yields two blocks:

$$B_1 = \{\top\} = A,$$
$$B_2 = \{\bot\} = A^c.$$

Elements of B_1 have false-negative rate ω (they test positive correctly with probability τ, incorrectly with probability ω). Elements of B_2 have false-positive rate ε (they test negative correctly with probability η, incorrectly with probability ε). The channel matrix Q introduced in Section 2.1 captures exactly these two rates:

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix},$$

where row i describes the output distribution for an element in block B_i.

In general, an n-th order model has n blocks and a corresponding channel matrix with n distinct rows, one per block. Axiom 2 is a statement about those rows.

The formal statement

Axiom 2 (Conditional independence of block error rates)**.** *Let $\{\alpha_1, \ldots, \alpha_n\}$ be the random error rates for the respective partition blocks $B_1, \ldots, B_n$, conditional on the objective set A. The rates $\alpha_1, \ldots, \alpha_n$ are mutually independent:*

$$f(\alpha_1, \ldots, \alpha_n | A) \;=\; \prod_{i=1}^{n} f(\alpha_i | A).$$

In plain language: knowing the error rate on positives tells you nothing about the error rate on negatives, and vice versa. The block rates are independent of each other given the objective set. This applies to all n

blocks simultaneously: no block's rate gives information about any other block's rate.

A few clarifications help prevent common misreadings.

Rates as random variables. The rates α_i are random in the formal sense: they are the *expected* error frequencies, which may themselves vary across different realizations of the data structure or algorithm. A Bloom filter, for instance, has a false-positive rate that depends on the specific set of inserted elements and the hash function choices. Axiom 2 says that the resulting FPR and FNR are independent across blocks.

Stability, not constancy. Axiom 2 does not say the rates are the same for every element within a block (that is the identically-distributed clause of Axiom 1). It says the rates are jointly independent across blocks. The distinction matters: a system could have rates that vary from element to element within a block while still satisfying Axiom 2. The axioms play complementary roles; neither subsumes the other.

The order-2 specialization. In the second-order model, Axiom 2 reduces to a single statement: the random false-positive rate ε and the random false-negative rate ω are conditionally independent given A.[1] The joint density of the two rates factors:

$$f(\varepsilon, \omega | A) \;=\; f(\varepsilon | A) \cdot f(\omega | A).$$

This is the instance that arises in virtually every example in chapters 1–3.

Consequence: one channel matrix per block, stable across queries

Axiom 2 has a structural consequence for how we model repeated queries. Suppose we query an input $x \in B_i$ multiple times: first at step t, then again at step t', and so on. Axiom 2 says the error rate on query at step t does not influence the error rate on query at step t'. Combined with Axiom 1 (which says errors at distinct elements are independent), this means:

Proposition 4.3.1 (Stable channel matrices)**.** *Under Axiom 2, the channel matrix row corresponding to block B_i is the same for every query involving*

[1] The companion paper [Tow26a] specializes Axiom 2 in terms of (FPR, TPR) rather than (FPR, FNR). The two formulations are equivalent: TPR $= 1 -$ FNR is a deterministic transformation, so independence of (FPR, FNR) is the same as independence of (FPR, TPR).

an element of B_i, regardless of the order or history of prior queries. Formally, for any sequence of queries $(x_1, x_2, \ldots)$ with $x_t \in B_{i(t)}$, the conditional distribution

$$\mathrm{P}\left(\tilde{A}(x_t) = s | A,\ x_t \in B_{i(t)}\right)$$

is the same function of $i(t)$ and s for all t, independent of what happened at steps $1, \ldots, t-1$.

Proof. Fix block B_i and a sequence of queries. The distribution of the output at step t for $x_t \in B_i$ is determined by the rate α_i assigned to block B_i. By Axiom 2, α_i is independent of α_j for $j \neq i$. By Axiom 1, the error events at distinct elements are independent. Together, the conditional distribution of the output at step t is a function only of α_i and the block membership of x_t, not of any prior query outcomes.

By Axiom 1, every element of B_i uses the same rate α_i, drawn once at construction time for the data structure. Once that value is realized, the channel row for B_i is a fixed function of α_i at every subsequent query, regardless of the query index t. Marginalizing over the distribution of α_i then produces the same marginal channel row at every query. Because Axiom 1 also makes per-element error events conditionally independent given α_i, every query of an element from block B_i contributes the same row of Q. The Q matrix is therefore time-invariant across the query sequence. Since this holds for all t, the channel matrix row for block B_i is stable across the entire query sequence. □

This proposition is the precise sense in which "there is one channel matrix Q per input-block-pair." The matrix entry Q_{ij} gives the probability that an element of block B_i produces output j. That probability does not drift, does not warm up, and does not depend on how many prior queries have been processed. The channel is genuinely stationary within each block.

The practical consequence is large. Without this stability, we would need a separate channel description for each time step, and the joint distribution over a sequence of queries would require specifying an exponentially growing family of time-varying matrices. With Axiom 2, a single matrix Q suffices for the entire query sequence. This is what makes the Kronecker factorization of Section 4.4 possible: the m-query joint confusion matrix is the Kronecker product of m copies of the same Q, precisely because each query sees the same stable channel.

Worked illustration: the model-order-2 Boolean channel

Let us make Axiom 2's content concrete with the simplest nontrivial case, which will recur throughout the book: the Boolean universe with model order 2.

Set $U = \{\top, \bot\}$ and $A = \{\top\}$, giving two blocks: $B_1 = \{\top\}$ (positives) and $B_2 = \{\bot\}$ (negatives). The channel matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Now consider a sequence of three queries: query 1 asks for $\top$, query 2 asks for $\bot$, query 3 asks for $\top$ again. Axiom 2 says the following.

- Query 1 involves $\top \in B_1$. It produces a positive response with probability τ and a negative response with probability ω. These probabilities come from row 1 of Q.
- Query 2 involves $\bot \in B_2$. It produces a positive response with probability ε and a negative response with probability η. These probabilities come from row 2 of Q. *The outcome of query 1 is irrelevant.*
- Query 3 involves $\top \in B_1$ again. It again uses row 1 of Q, with the *same* probabilities τ and ω as query 1. The fact that query 1 may have made an error (false negative) does not change the probability of error on query 3.

This is the channel's stationarity within blocks: the same row of Q applies whenever the same block is queried, no matter what came before.

To write the joint distribution over the three-query sequence $(x_1, x_2, x_3) = (\top, \bot, \top)$ with outcomes (s_1, s_2, s_3), Axioms 1 and 2 together give:

$$\begin{aligned} &\mathrm{P}\left(\tilde{A}(\top) = s_1,\ \tilde{A}(\bot) = s_2,\ \tilde{A}(\top) = s_3 | A\right) \\ &\quad = \; Q_{1,s_1} \cdot Q_{2,s_2} \cdot Q_{1,s_3}, \end{aligned}$$

where $Q_{i,s}$ denotes the (i,s)-entry of Q. The distribution factors as a product of independent channel evaluations, each determined by the block of the queried element. No interaction terms appear; no history from query 1 flows into query 3.

For a concrete numerical example, take $\varepsilon = 0.05$ and $\tau = 0.90$ (so $\omega = 0.10$ and $\eta = 0.95$). The probability that the three-query sequence produces outcomes $(+, -, +)$, that is, two true positives and one true negative, is:

$$Q_{1,+} \cdot Q_{2,-} \cdot Q_{1,+} \;=\; 0.90 \times 0.95 \times 0.90 \;=\; 0.7695.$$

The probability that query 2 produces a false positive while queries 1 and 3 produce true positives is:

$$Q_{1,+} \cdot Q_{2,+} \cdot Q_{1,+} \;=\; 0.90 \times 0.05 \times 0.90 \;=\; 0.0405.$$

Both numbers follow from reading the appropriate rows of Q and multiplying. No additional modeling is needed. The stationarity guaranteed by Axiom 2 is what makes this reading-and-multiplying procedure valid.

Contrast: a Markov-correlated channel violates Axiom 2

To see what Axiom 2 rules out, consider a channel that violates it explicitly. A *Markov-correlated channel* lets the error rate on each query depend on the outcome of the previous query. Concretely:

1. If the previous query produced a correct response, the current query uses a "base" false-positive rate ε_0.
2. If the previous query produced an error, the current query uses an elevated false-positive rate $\varepsilon_1 > \varepsilon_0$ (errors cluster together).

This is a plausible model for hardware memory with correlated faults, or for a sensor that drifts after one bad reading. But it violates Axiom 2.

To see why, consider block $B_2 = \{\perp\}$ (negatives). On query t, the error rate for block B_2 is not a fixed value ε: it is ε_0 if the previous query was correct, and ε_1 if the previous query was an error. The rate α_2 at step t is not independent of the rate at step $t-1$; they are coupled through the Markov chain. The joint density of the rates at different steps does not factor, and Axiom 2's independence condition fails.

Let us put numbers to this. Suppose the base rate is $\varepsilon_0 = 0.05$ and the elevated rate is $\varepsilon_1 = 0.30$. The channel starts in the correct state. Over two successive queries to $\perp$, the joint distribution of outcomes is:

Outcome pair	Probability	Axiom-2 product	Discrepancy
$(-,-)$	$0.95 \times 0.95 = 0.9025$	0.9025	none
$(-,+)$	$0.95 \times 0.05 = 0.0475$	0.0475	none
$(+,+)$	$0.05 \times 0.30 = 0.0150$	0.0025	$6\times$ inflation
$(+,-)$	$0.05 \times 0.70 = 0.0350$	0.0475	deflation

Axiom 2 predicts the joint probability of two consecutive false positives as $\varepsilon_0^2 = 0.05^2 = 0.0025$, since under Axiom 2 the per-block false-positive rate is a fixed constant ε_0 across queries (one Q matrix per block, stable).

The Markov channel produces 0.0150 instead, a factor of 6× higher. The discrepancy is exactly what Axiom 2 forbids: the rate after a previous false positive ($\varepsilon_1 = 0.30$) is six times the rate after a previous correct response ($\varepsilon_0 = 0.05$). The per-block rate is no longer a constant; it depends on history.

The discrepancy is not small. The Markov channel produces error clustering that is invisible to the factored model. If a practitioner were to apply the Kronecker factorization of Section 4.4 to this channel, they would underestimate the probability of back-to-back errors by a factor of six. In applications where error clustering is costly, this is not a tolerable approximation error.

The Markov channel is not a pathological construction: it models a real failure mode. What Axiom 2 asserts is that the Bernoulli framework, in its standard form, is designed for channels where this clustering does not occur. If the channel is known to be Markov-correlated, the model must be extended, for example by conditioning on the hidden state, or by treating the full trajectory as the object to be analyzed. The Bernoulli axioms are then either violated, requiring a richer model, or satisfied at the level of the extended state space, in which case the standard machinery applies after the appropriate state augmentation.

What Axiom 2 earns us

Axiom 1 and Axiom 2 are the full axiomatic foundation of the Bernoulli framework. They are minimal in the sense that neither alone is sufficient for the Kronecker factorization theorem: Axiom 1 gives spatial independence across elements, Axiom 2 gives temporal stability of rates across queries. Together they imply that the joint confusion matrix over any sequence of queries is a Kronecker product of single-query matrices, one factor per query (Section 4.4).

The parametric savings are substantial. Consider the order-2 Boolean predicate that runs throughout this chapter: a general channel on m joint queries with two blocks, and no independence assumption, requires $2(2^m - 1)$ free parameters: one for each joint outcome of m Boolean tests in each of the two rows, save normalization. Under Axioms 1 and 2, the Kronecker structure forces the parameter count down to $2m$: two parameters per query (one per block), with no cross-query parameters at all. For $m = 50$ queries on the order-2 Boolean channel, that is $2m = 100$ parameters under the axioms versus $2(2^{50} - 1) \approx 2.25 \times 10^{15}$ in the general case. The axioms are not merely philosophically satisfying; they are what makes the model

tractable.

One refinement is worth flagging now and pursuing in Section 4.4. The Kronecker structure forces the parameter count down to nm for the rate-conditional channel; the unconditional matrix achieves the same count when no block is queried twice (Corollary 4.4.1.1) or when rates are deterministic (the regime of all worked examples in this book). When two queries hit the same block under genuinely random rates, the unconditional joint involves higher moments of those rates and the simple per-query count no longer applies; Section 4.4 treats this distinction explicitly.

The Kronecker factorization theorem in Section 4.4 makes this precise and derives the parameter reduction as a corollary. For an n-th order model on m joint queries, the general case has $n(2^m - 1)$ free parameters; under the axioms, the count falls to nm.

We prove the Kronecker factorization theorem and its corollary on parameter reduction in Section 4.4. That section is where the full benefit of both axioms becomes visible.

4.4 The Kronecker Factorization Theorem

Two axioms, one payoff

We have named two axioms and shown what each rules out. Section 4.2 gave Axiom 1: error events at distinct inputs are mutually independent. Section 4.3 gave Axiom 2: per-block error rates are conditionally independent of each other, which implies that each block's channel row is stable across queries. Both sections foreshadowed a theorem that their conjunction enables. We now prove it.

The question is this. Given m queries on a Bernoulli predicate, each on a distinct element of the universal set—what does the joint confusion matrix look like? The joint matrix $Q^{(m)}$ is an $n^m \times 2^m$ object: rows are indexed by source-block tuples $(j_1, \ldots, j_m) \in \{1, \ldots, n\}^m$ (one row per ordered selection of source blocks across the m queries), and columns are indexed by binary output patterns $(k_1, \ldots, k_m) \in \{0, 1\}^m$. Without any structural assumption, specifying this matrix requires $n^m(2^m - 1)$ real parameters (one distribution over 2^m output patterns per source-block tuple). For $m = 50$ and $n = 2$, that is over 10^{30} free parameters.

A different, natural parameterization treats only the n distinct source blocks, not all n^m block-tuples, as the basic unit. Under this per-source-block convention the joint conditional distribution over 2^m output patterns is the same for every query from block B_j, giving n distributions of size

2^m and hence $n(2^m - 1)$ free parameters. The parameter counts in this section's corollary and in Section 4.3 are stated under this convention—and it is exactly the convention the axioms enforce, as the corollary explains.

Under Axioms 1 and 2, the per-source-block parameter count drops to nm: n free parameters per query (one per block row), with no cross-query interaction, summed over m queries. In the order-2 Boolean case this is $2m$. This is not an approximation; it is the exact structure forced by independence.

Notation. *The* Kronecker product *of two matrices P and Q, written $P \otimes Q$, is the block matrix formed by replacing each entry P_{ij} with the scaled submatrix $P_{ij}Q$. If P is $r \times s$ and Q is $p \times q$, then $P \otimes Q$ is $(rp) \times (sq)$. The Kronecker product is associative, so $P \otimes Q \otimes R$ is unambiguous without parentheses. We use $\otimes$ for the Kronecker product throughout this section and the rest of the book.*

Theorem 4.4.1 (Kronecker Factorization, conditional form)**.** *Let $\tilde{A}$ be an n-th order Bernoulli predicate satisfying Axiom 1 (Axiom 1) and Axiom 2 (Axiom 2). For any realization of the per-block rates $\alpha = (\alpha_1, \ldots, \alpha_n)$, let $Q_1^*(\alpha), \ldots, Q_m^*(\alpha)$ be the resulting per-query channel matrices, where $Q_i^*(\alpha)$ is the $n \times 2$ stochastic matrix whose (j,k)-entry is the probability, under the realized rate α_j, that a query of an element in block B_j produces output label k on the i-th query. Then the* rate-conditional *joint confusion matrix $Q^{(m)*}(\alpha)$ of m queries on m distinct input elements, whose $(j_1 \cdots j_m,\, k_1 \cdots k_m)$-entry is*

$$\mathrm{P}\left(\tilde{A}(x_1) = k_1, \ldots, \tilde{A}(x_m) = k_m | \alpha,\ x_\ell \in B_{j_\ell} \text{ for each } \ell\right),$$

factors as the Kronecker product

$$Q^{(m)*}(\alpha) \;=\; Q_1^*(\alpha) \otimes Q_2^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha).$$

Corollary 4.4.1.1 (Unconditional factorization, distinct-block queries)**.** *Under the hypotheses of Theorem 4.4.1, if the queried blocks $j_1, \ldots, j_m$ are pairwise distinct (each query hits a different block), the unconditional joint confusion matrix factors as*

$$Q^{(m)} \;=\; Q_1 \otimes Q_2 \otimes \cdots \otimes Q_m,$$

where $Q_\ell \coloneqq \mathrm{E}\left[Q_\ell^*(\alpha)\right]$ is the rate-marginalized per-query channel.

Before the proof, two notes on scope. **First**, the theorem applies to m queries on m *distinct* input elements. Repeated queries of the same

element introduce correlations that Axioms 1 and 2 do not address: the two error events share a single realization of the element's underlying error draw, making them perfectly correlated (in a deterministic implementation) rather than independent. For repeated-query behavior, including the effects of memoization and caching, see Chapter 16 (negative space).

Second, the theorem's primary claim is at the rate-conditional level: given any realization of the per-block rates α, the joint confusion matrix factors. The unconditional matrix factors only when the queried blocks are pairwise distinct (Corollary 4.4.1.1). When two queries hit the same block B_j, both factors involve the same random rate α_j, and the unconditional joint $\mathrm{E}\left[\alpha_j^2\right]$ exceeds the Kronecker prediction $(\mathrm{E}\left[\alpha_j\right])^2$ whenever α_j is non-degenerate (a Jensen gap). The conditional factorization holds always; the unconditional factorization holds when either the rates are realized as constants (the deterministic-implementation case used throughout chapters 1–3) or when no block is queried twice. In the running BSC thread below and in subsequent chapters, we work in the deterministic-rates regime, where $Q_\ell^* = Q_\ell$ and the conditional and unconditional forms coincide.

Proof. We build up the joint distribution by hand and read off the matrix structure.

Step 1: reduce to a joint probability calculation. Index the m query positions as $\ell = 1, \ldots, m$. Position ℓ queries element $x_\ell \in B_{j_\ell}$ and observes output k_ℓ. Fix a realization of the per-block rates $\alpha = (\alpha_1, \ldots, \alpha_n)$. The $(j_1 \cdots j_m, k_1 \cdots k_m)$-entry of the rate-conditional matrix $Q^{(m)*}(\alpha)$ is

$$Q^{(m)*}(\alpha)_{j_1 \cdots j_m,\, k_1 \cdots k_m} \;=\; \mathrm{P}\left(\tilde{A}(x_1) = k_1, \ldots, \tilde{A}(x_m) = k_m | \alpha,\; x_\ell \in B_{j_\ell},\; \ell = 1, \ldots, m\right). \tag{a}$$

We want to evaluate this probability and show it equals $\prod_{\ell=1}^{m} (Q_\ell^*(\alpha))_{j_\ell, k_\ell}$, which is precisely the $(j_1 \cdots j_m, k_1 \cdots k_m)$-entry of $Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$.

Step 2: apply Axiom 2 to fix the per-block rates. By Axiom 2 (Axiom 2), the random rate α_i on block B_i is independent of the random rate $\alpha_{i'}$ on block $B_{i'}$ for $i \neq i'$. By Proposition 4.3.1, once the rates are realized, the rate-conditional channel matrix $Q_\ell^*(\alpha)$ for query ℓ has its row j determined by α_j alone: row j gives the conditional output distribution for an element in block B_j under the realized rate α_j. Write $(Q_\ell^*(\alpha))_{j_\ell, k_\ell}$ for the (j_ℓ, k_ℓ)-entry of this random matrix, the rate-conditional probability that query ℓ from block B_{j_ℓ} returns label k_ℓ given the rate α_{j_ℓ}. Distinct queries on the same block share the same realization of α_{j_ℓ} and hence the same row of $Q_\ell^*(\alpha)$; this row does not depend on what happened at other query positions. When all

m queries share a single realization of the channel family (the common case for a stationary data structure), $Q_1^*(\alpha) = Q_2^*(\alpha) = \cdots = Q_m^*(\alpha) = Q^*(\alpha)$; the theorem's subscript ℓ flags per-query generality, not necessarily distinct matrices.

Step 3: apply Axiom 1 to factor the joint distribution. Conditional on the realized rates $\alpha = (\alpha_1, \ldots, \alpha_n)$, the outputs at positions $\ell = 1, \ldots, m$ are mutually independent, because each output is a deterministic function of the (conditionally independent) error event at x_ℓ. By Axiom 1 (Axiom 1), these error events are mutually independent given the rates and the objective set A. Therefore, conditional on the rates, the joint probability in (a) factors:

$$\begin{aligned} &\mathrm{P}\left(\tilde{A}(x_1) = k_1, \ldots, \tilde{A}(x_m) = k_m | \alpha, x_\ell \in B_{j_\ell}\right) \\ &\quad = \prod_{\ell=1}^{m} \mathrm{P}\left(\tilde{A}(x_\ell) = k_\ell | \alpha_{j_\ell},\, x_\ell \in B_{j_\ell}\right) \\ &\quad = \prod_{\ell=1}^{m} (Q_\ell^*(\alpha))_{j_\ell, k_\ell}. \end{aligned}$$

Step 4: read off the conditional matrix. The expression $\prod_{\ell=1}^{m} (Q_\ell^*)_{j_\ell, k_\ell}$ derived in Step 3 is, by definition, the $(j_1 \cdots j_m,\, k_1 \cdots k_m)$-entry of the Kronecker product $Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$. Since this holds for every index tuple, the rate-conditional joint matrix factors:

$$Q^{(m)*}(\alpha) \;=\; Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha). \tag{b}$$

This proves Theorem 4.4.1.

To obtain the unconditional version (Corollary 4.4.1.1) under the additional hypothesis that the queried blocks are pairwise distinct, take expectation over $\alpha = (\alpha_1, \ldots, \alpha_n)$:

$$\mathrm{E}\left[\prod_{\ell=1}^{m} (Q_\ell^*)_{j_\ell, k_\ell}\right] \;=\; \prod_{\ell=1}^{m} \mathrm{E}\left[(Q_\ell^*)_{j_\ell, k_\ell}\right] \;=\; \prod_{\ell=1}^{m} (Q_\ell)_{j_\ell, k_\ell}.$$

The first equality uses Axiom 2: when $j_1, \ldots, j_m$ are pairwise distinct, each factor depends on a distinct rate α_{j_ℓ}, and the cross-block independence guaranteed by Axiom 2 makes the factors mutually independent random variables. (When two queries hit the same block, this equality fails; see the second scope note above.) □

Honesty note. The parametric parsimony observation underlying this theorem appears as Remark 3.4 in the companion paper [Tow26a], which notes that Axiom 1 forces the joint confusion matrix to factor as a Kronecker product. The companion paper states this remark in a regime where rates are either deterministic or implicitly conditioned upon; under that reading the remark is correct, and the conditional form of Theorem 4.4.1 above makes the conditioning explicit. Axiom 2's role in this textbook is not to extend the conditional factorization to the unconditional case in general. That is impossible without further hypotheses, since same-block queries on random rates produce a Jensen gap $\mathrm{E}\left[\alpha_j^2\right] > (\mathrm{E}\left[\alpha_j\right])^2$. Axiom 2 instead extends the unconditional factorization to the special case of distinct-block queries (Corollary 4.4.1.1) and, more importantly, certifies the rate-stability that lets us speak of "the" channel matrix Q_ℓ for each query position. The constructive proof above and the explicit conditional/unconditional decomposition are new contributions of this textbook.

Corollary: the parameter count drop

Corollary 4.4.1.2 (Parameter reduction, conditional form)**.** *Under Axioms 1 and 2, the rate-conditional joint confusion matrix $Q^{(m)*}(\alpha)$ of an n-th order Bernoulli predicate queried m times requires at most nm parameters to specify, rather than the $n(2^m - 1)$ parameters required by a general (unconstrained) m-query channel. For the order-2 Boolean predicate this reduces to $2m$ parameters (one per query per block) rather than $2(2^m - 1)$. The same count applies to the unconditional matrix $Q^{(m)}$ when rates are realized as constants (the deterministic-rates regime of all worked examples in this book) or when no block is queried twice (Corollary 4.4.1.1).*

Proof. We count the free parameters in $Q^{(m)*}(\alpha) = Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$.

Constrained case. Under the axioms, $Q^{(m)*}(\alpha)$ factors as $Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$ (Theorem 4.4.1), so each per-query matrix $Q_\ell^*(\alpha)$ contributes its own parameters independently. Each $Q_\ell^*(\alpha)$ is an $n \times 2$ stochastic matrix: rows are indexed by the n source blocks, and columns by the two binary output labels. Each row sums to 1, leaving 1 free parameter per row. With n rows, each $Q_\ell^*(\alpha)$ has n free parameters. By Axiom 2 the per-block rates are independent across queries, so the m matrices share no parameters. Across m queries: nm free parameters in total.

In the order-2 Boolean case ($n = 2$), the two blocks are positives and negatives, parametrized by (τ, ε) per query, or equivalently by (ω, ε). That is 2 parameters per query, for $2m$ parameters in total across m queries.

General case. The full block-tuple matrix has n^m rows and 2^m columns; specifying it freely would require $n^m(2^m - 1)$ parameters. The more compact count uses the per-source-block convention introduced above: without the Kronecker structure but still working source-block by source-block, each of the n source blocks contributes one (unconstrained) conditional distribution over 2^m joint output patterns, requiring $2^m - 1$ free parameters after normalization. Total under this convention: $n(2^m - 1)$. This is the comparison point used here and in Section 4.3, because it reflects the reduction attributable to the axioms alone rather than conflating it with the block-tuple explosion.

Transfer to the unconditional matrix. When per-block rates are deterministic constants (so $Q_\ell^*(\alpha) = Q_\ell$ identically), the conditional and unconditional matrices coincide and the nm count applies to $Q^{(m)}$ directly. When rates are random but the queried blocks are pairwise distinct, Corollary 4.4.1.1 gives $Q^{(m)} = Q_1 \otimes \cdots \otimes Q_m$ with each Q_ℓ contributing n free parameters, again totalling nm. Both regimes cover every worked example in Part I.

Ratio.

$$\frac{n(2^m - 1)}{nm} \;=\; \frac{2^m - 1}{m},$$

which grows exponentially in m. At $m = 50$ and $n = 2$: general case $2(2^{50} - 1) \approx 2.25 \times 10^{15}$; constrained case $2 \times 50 = 100$. □

The corollary matches the closing summary of Section 4.3: the order-2 case requires $2m$ parameters under the axioms versus $2(2^m - 1)$ in the general case; the n-th order case requires nm versus $n(2^m - 1)$. The Kronecker structure, forced by the two axioms, is what produces this reduction.

Running thread: the BSC composed m times

The binary symmetric channel (BSC) introduced in Section 2.1 is the simplest nontrivial instance of the Kronecker theorem. A BSC with crossover probability ε has channel matrix

$$Q \;=\; \begin{pmatrix} 1 - \varepsilon & \varepsilon \\ \varepsilon & 1 - \varepsilon \end{pmatrix}.$$

Since the BSC is symmetric, ε is both the false-positive rate (an element outside A is reported as inside) and the false-negative rate (an element inside A is reported as outside). This is the symmetric special case of the order-2 Bernoulli model: a single crossover parameter ε suffices because the

channel obeys FPR = FNR = ε (the symmetry constraint), and the two rows $[1-\varepsilon,\ \varepsilon]$ and $[\varepsilon,\ 1-\varepsilon]$ are reversals of each other. The model order is 2 because there are two partition blocks (B_1 = positives, B_2 = negatives); the symmetry constraint reduces the two free parameters of the general order-2 model to one.

Now suppose we run m independent queries, each through the same BSC. We work in the deterministic-rates regime announced in the second scope note above, where $Q_\ell^* = Q$ for every query and the conditional matrix of Theorem 4.4.1 coincides with the unconditional one. The joint confusion matrix is then

$$Q^{(m)} \;=\; \underbrace{Q \otimes Q \otimes \cdots \otimes Q}_{m \text{ copies}} \;=\; Q^{\otimes m}.$$

Explicit computation for $m = 2$. Two queries, each through a BSC with crossover ε. The joint input lives in $\{B_1, B_1\} \cup \{B_1, B_2\} \cup \{B_2, B_1\} \cup \{B_2, B_2\}$, where we write B_1 for "in A" and B_2 for "not in A". The joint output lives in $\{(+,+), (+,-), (-,+), (-,-)\}$, where $+$ means "reported in A" and $-$ means "reported not in A".

By the Kronecker product definition, $Q \otimes Q$ is the 4×4 matrix formed by scaling each entry of Q by the full matrix Q:

$$Q^{\otimes 2} \;=\; Q \otimes Q \;=\; \begin{pmatrix} (1-\varepsilon)Q & \varepsilon Q \\ \varepsilon Q & (1-\varepsilon)Q \end{pmatrix}.$$

Expanding each block gives

$$Q^{\otimes 2} \;=\; \begin{pmatrix} (1-\varepsilon)^2 & (1-\varepsilon)\varepsilon & \varepsilon(1-\varepsilon) & \varepsilon^2 \\ (1-\varepsilon)\varepsilon & (1-\varepsilon)^2 & \varepsilon^2 & \varepsilon(1-\varepsilon) \\ \varepsilon(1-\varepsilon) & \varepsilon^2 & (1-\varepsilon)^2 & (1-\varepsilon)\varepsilon \\ \varepsilon^2 & \varepsilon(1-\varepsilon) & (1-\varepsilon)\varepsilon & (1-\varepsilon)^2 \end{pmatrix}.$$

Reading the matrix: the $(1,1)$-entry is the probability that two elements from block B_1 (both positives) are both reported positive, namely $(1-\varepsilon)^2$. The $(1,4)$-entry is the probability that two positives are both reported negative, namely ε^2. The $(2,2)$-entry—one positive, one negative; both reported positive—is $(1-\varepsilon)\varepsilon$: the positive is correctly reported positive $(1-\varepsilon)$ and the negative is falsely reported positive (ε).

For a numerical example, take $\varepsilon = 0.05$.

Joint input blocks	Joint output probabilities			
	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$
(B_1, B_1)	0.9025	0.0475	0.0475	0.0025
(B_1, B_2)	0.0475	0.9025	0.0025	0.0475
(B_2, B_1)	0.0475	0.0025	0.9025	0.0475
(B_2, B_2)	0.0025	0.0475	0.0475	0.9025

The BSC thread that opened in Section 2.1 reaches its formal resolution here: m independent queries through the same BSC produce a joint channel whose confusion matrix is the m-fold Kronecker power $Q^{\otimes m}$. Each entry of $Q^{\otimes m}$ is a product of m individual BSC transition probabilities, each equal to ε or $1-\varepsilon$ according to whether that query position produced an error. The Kronecker structure is not a coincidence: it is exactly what Axioms 1 and 2 force.

A 2-query worked verification

We now verify Theorem 4.4.1 with a general order-2 channel to show that the BSC example is not special.

Let the predicate $\tilde{A}$ have false-positive rate ε and false-negative rate ω (so true-positive rate $\tau = 1-\omega$ and true-negative rate $\eta = 1-\varepsilon$). The per-query channel matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where row 1 corresponds to positives ($x \in A$, block B_1) and row 2 corresponds to negatives ($x \notin A$, block B_2).

Suppose we query two elements $x_1 \in B_{j_1}$ and $x_2 \in B_{j_2}$ and observe outputs k_1 and k_2 respectively. For concreteness, take both queries to the same channel (both $Q_1 = Q_2 = Q$, since the rates are stationary by Proposition 4.3.1, which relies on both Axioms 1 and 2). As in the BSC thread, we work in the deterministic-rates regime, where the conditional matrix of Theorem 4.4.1 coincides with the unconditional one.

Theorem 4.4.1 predicts:

$$Q^{(2)} \;=\; Q \otimes Q.$$

Let us compute $Q \otimes Q$ directly and then verify each entry against its direct probability calculation.

The Kronecker product $Q \otimes Q$.

$$Q \otimes Q = \begin{pmatrix} \tau \cdot Q & \omega \cdot Q \\ \varepsilon \cdot Q & \eta \cdot Q \end{pmatrix}$$

$$= \begin{pmatrix} \tau^2 & \tau\omega & \omega\tau & \omega^2 \\ \tau\varepsilon & \tau\eta & \omega\varepsilon & \omega\eta \\ \varepsilon\tau & \varepsilon\omega & \eta\tau & \eta\omega \\ \varepsilon^2 & \varepsilon\eta & \eta\varepsilon & \eta^2 \end{pmatrix}.$$

Row order is $(B_1B_1, B_1B_2, B_2B_1, B_2B_2)$ and column order is $(++, +-, -+, --)$.

Direct probability calculation. We verify two entries by direct calculation.

Entry $(B_1B_1, ++)$: two positives, both reported positive. By Axiom 1, the events are independent:

$$\mathrm{P}\left(\tilde{A}(x_1) = +, \tilde{A}(x_2) = + | x_1 \in B_1, x_2 \in B_1\right) \;=\; \tau \cdot \tau \;=\; \tau^2.$$

This matches $(Q \otimes Q)_{11} = \tau^2$. Axiom 1 licenses the product; Axiom 2 ensures each τ factor is the same stable value.

Entry $(B_2B_1, -+)$: negative then positive, first reported negative (correct), second reported positive (correct).

$$\mathrm{P}\left(\tilde{A}(x_1) = -, \tilde{A}(x_2) = + | x_1 \in B_2, x_2 \in B_1\right) \;=\; \eta \cdot \tau.$$

This matches $(Q \otimes Q)_{33} = \eta\tau$ (row 3, column 3; note column 3 is the $(-, +)$ output pattern).

Entry $(B_1B_2, +-)$: positive then negative, both correctly reported. Block $B_1 = \{\top\}$ (positive), output $+$ is a true positive; block $B_2 = \{\bot\}$ (negative), output $-$ is a true negative.

$$\mathrm{P}\left(\tilde{A}(x_1) = +, \tilde{A}(x_2) = - | x_1 \in B_1, x_2 \in B_2\right) \;=\; \tau \cdot \eta.$$

This matches $(Q \otimes Q)_{22} = \tau\eta$. For the mixed true-positive / false-positive entry—output $(+, +)$ when blocks are (B_1, B_2), meaning the positive is correctly reported positive and the negative is falsely reported positive—we need column 1: $(Q \otimes Q)_{21} = \tau\varepsilon$, which equals $\mathrm{P}(+, +|B_1, B_2) = \tau \cdot \varepsilon$. Confirmed.

All entries are products of individual channel probabilities. This is exactly what Theorem 4.4.1 predicts: each row of $Q^{(2)}$ is the tensor product of the corresponding per-query rows, and the resulting matrix has 4 parameters (ε and ω per query, with both queries sharing the same rates by Axiom 2), not the $2(2^2 - 1) = 6$ that a general 2-query channel would need.

What the Kronecker structure buys for §3.1

The AND derivation in Section 3.1 computed the joint error rate of two noisy bits by treating the two error events as independent and multiplying probabilities. We can now see exactly which theorem that derivation was tacitly invoking.

The two-query joint channel for an AND gate is a special case of $Q^{(2)} = Q_1 \otimes Q_2$ where Q_1 and Q_2 are the per-input channel matrices for the two operands. The independence assumed in Section 3.1 was Axiom 1 (the two error events are independent) plus the stability of each operand's rates (Axiom 2). The Kronecker factorization theorem makes this precise: the joint distribution of the two AND operands' channel outputs factors, and the AND gate's output distribution is a deterministic function of that joint distribution. Nothing more was needed than what the two axioms provide.

The formal type is next

The Kronecker theorem is the formal payoff for chapters 1–4's accumulated independence assumptions. We now have a complete characterization of the joint confusion matrix under the Bernoulli framework's two axioms: it is a Kronecker product of per-query channel matrices, one factor per query, with nm parameters total for an n-th order predicate on m queries.

In Section 4.5, we turn this machinery into a formal definition of the Bernoulli[bool] type—the object that chapters 1–3 have been pointing at. The Kronecker theorem is what makes that definition tractable: it tells us that the formal type is parametrized by finitely many numbers (two, for the Boolean case), not by an exponentially large family of joint distributions.

4.5 The Bernoulli[bool] Type, Formally

We have arrived at the formal definition. Four sections of scaffolding have built up to this moment: Section 4.1 explained why axioms matter, Section 4.2 stated that error events at distinct inputs are mutually independent, Section 4.3 stated that per-block error rates are mutually independent across blocks, and Section 4.4 proved that those two axioms together force the joint confusion matrix to factor as a Kronecker product. The formal definition is not an assertion; it is a synthesis of what the preceding chapters have earned.

The definition

Definition 4.5.1 (Bernoulli[bool])**.** *A* Bernoulli[bool] instance *is a triple* (X, Y, Q) *where:*

1. $X = \{\top, \bot\}$ *is the* input domain *(the Boolean universe), with* $B_1 = \{\top\}$ *and* $B_2 = \{\bot\}$ *as the two partition blocks.*
2. $Y = \{\top, \bot\}$ *is the* output domain *(the observed report).*
3. Q *is a* 2×2 *row-stochastic matrix*

$$Q = \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where row 1 governs elements in $B_1 = \{\top\}$ *(positives) and row 2 governs elements in* $B_2 = \{\bot\}$ *(negatives), with*

- $\tau \in [0, 1]$*: the true-positive rate (*$\mathrm{P}(\textit{reported } \top | \textit{true value } \top)$*),*
- $\omega = 1 - \tau$*: the false-negative rate (*$\mathrm{P}(\textit{reported } \bot | \textit{true value } \top)$*),*
- $\varepsilon \in [0, 1]$*: the false-positive rate (*$\mathrm{P}(\textit{reported } \top | \textit{true value } \bot)$*),*
- $\eta = 1 - \varepsilon$*: the true-negative rate (*$\mathrm{P}(\textit{reported } \bot | \textit{true value } \bot)$*).*

The triple satisfies Axiom 1 (Axiom 1: error events at distinct inputs are mutually independent) and Axiom 2 (Axiom 2: per-block error rates ε *and* ω *are conditionally independent of each other given the objective set).*

Notation. *We write* Bernoulli[bool] *for the* type*: the collection of all triples* (X, Y, Q) *satisfying the definition above. An individual predicate or data structure is an* instance *of* Bernoulli[bool] *when its behavior matches Definition 4.5.1. The two parameters* (ε, ω)*—or equivalently* (ε, τ)*— uniquely identify the channel matrix* Q *within the type.*

Several remarks on what the definition does and does not say.

Scope. This definition is for the Boolean special case: the input and output domains are both $\{\top, \bot\}$, giving a 2×2 channel matrix with two free parameters. The Bernoulli framework extends to arbitrary algebraic types—Bernoulli[T] for general T—but that extension is Part III material and does not appear here. For now, all instances are Boolean.

The role of the two axioms. The definition invokes both axioms explicitly because both are load-bearing. Axiom 1 (Axiom 1) is what licenses the

factored-product form of joint distributions: queries on distinct Boolean inputs produce independent error events. Without it, the channel matrix Q could not be read as a per-element description; correlation structure across inputs would require a far richer representation. Axiom 2 (Axiom 2) is what makes Q time-invariant: the false-positive rate ε and false-negative rate ω are jointly independent, so knowing one tells you nothing about the other, and neither varies with query history. Both axioms together are what makes Q a compact, two-parameter object sufficient to characterize an entire predicate's behavior—the content of Corollary 4.4.1.2.

The Q convention. Row 1 of Q corresponds to $B_1 = \{\top\}$ (positives); row 2 corresponds to $B_2 = \{\bot\}$ (negatives). Column 1 is the "report $\top$" output; column 2 is the "report $\bot$" output. This convention was introduced in Section 4.3 and used throughout Section 4.4; it is consistent across the entire book.

Biased-coin worked example

The biased-coin example has threaded through this book since Section 1.1. We introduced it there as one of the three faces of a noisy bit—a coin with bias p for heads, where the observation itself may be noisy. In Section 1.3 we looked at the Boolean universe qualitatively. We deferred the full joint-probability treatment to this section. Here it lands.

Setup. Let the coin have bias p: heads ($\top$) with probability p, tails ($\bot$) with probability $1-p$. The observation is noisy: it reports correctly for positives with probability $\tau = 1 - \omega$ and correctly for negatives with probability $\eta = 1 - \varepsilon$.

Example 1 (Biased coin as a Bernoulli[bool] **instance)** *Let $U = \{\top, \bot\}$, with $A = \{\top\}$ the objective set (the event "coin lands heads") and $A^c = \{\bot\}$ the complement (the event "coin lands tails"). Define:*

- *Coin bias: $p = \mathrm{P}(\textit{true value} = \top)$.*
- *Channel: false-positive rate ε (probability of reporting heads when the coin actually landed tails), false-negative rate ω (probability of reporting tails when the coin actually landed heads).*

The joint PMF over all four (true value, observed report) pairs is:[2]

[2]This formulation differs from the companion paper's Example 3.5, which gives the PMF over the realized approximate-set value rather than the joint distribution over true-vs-observed pairs; the two are equivalent but suit different purposes.

Outcome	*True value*	*Observed report*	*Probability*
True positive (TP)	$\top$	$\top$	$p\,(1-\omega) = p\,\tau$
False negative (FN)	$\top$	$\bot$	$p\,\omega$
False positive (FP)	$\bot$	$\top$	$(1-p)\,\varepsilon$
True negative (TN)	$\bot$	$\bot$	$(1-p)\,(1-\varepsilon) = (1-p)\,\eta$

Sanity check:

$$\begin{aligned} p\,\tau + p\,\omega + (1-p)\,\varepsilon + (1-p)\,\eta \;&=\; p(\tau+\omega) + (1-p)(\varepsilon+\eta) \\ &=\; p\cdot 1 + (1-p)\cdot 1 \;=\; 1. \quad \checkmark \end{aligned}$$

The channel matrix is:

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The triple $(\{\top,\bot\},\,\{\top,\bot\},\,Q)$ *is a* Bernoulli[bool] *instance whenever* ε *and* ω *are conditionally independent given the coin's objective set* $A = \{\top\}$*, which holds when the observation mechanism operates independently on each side of the partition. The prior* p *over the input is not part of the* Bernoulli[bool] *type itself—the type characterizes the* channel*, not the* source*—but* p *enters when we compute the joint or marginal distribution over outcomes.*

For a concrete instance, take $p = 0.6$*,* $\varepsilon = 0.05$*,* $\omega = 0.10$ *(so* $\tau = 0.90$*,* $\eta = 0.95$*). The four outcome probabilities are:*

Outcome	*Formula*	*Value*
TP	0.6×0.90	0.540
FN	0.6×0.10	0.060
FP	0.4×0.05	0.020
TN	0.4×0.95	0.380
Total		1.000

The channel matrix for this instance is:

$$Q \;=\; \begin{pmatrix} 0.90 & 0.10 \\ 0.05 & 0.95 \end{pmatrix}.$$

Axiom 1 is vacuously satisfied for this single-query example: there is only one input element, so "independence across distinct inputs" has no content. Axiom 1's operational role appears only with multi-query joint distributions (the setup of Theorem 4.4.1), where it justifies the factored joint matrix.

Here we are checking that the per-instance type structure does not violate Axiom 1, not that we exercise it.

Axiom 2 governs the joint distribution over random per-block rates; this concrete instance fixes a single realization $(\varepsilon, \omega) = (0.05, 0.10)$*. The choice of these two values without coupling between them is consistent with Axiom 2's cross-block independence. The full content of Axiom 2 emerges when we consider rates as random variables drawn from a joint distribution; see Proposition 4.3.1 and the closing summary of Section 4.3.*

This triple is a well-formed Bernoulli[bool] *instance.*

The biased-coin example is the simplest general (asymmetric) instance of the type. It has one free parameter in the source (p) and two in the channel (ε, ω). The source prior p controls how often each block is queried; the channel parameters control how reliably each query is reported. The formal Bernoulli[bool] type isolates the channel: it says exactly what can go wrong (and how often) for each kind of input, independently of how often that input arises.

Chapters 1 through 3 as instances

The formal definition gives us a vocabulary for what we have been doing all along. We can now look back at every example in chapters 1–3 and say, precisely, what type each object was.

Chapter 1: the noisy bit (Section 1.1). The three faces introduced in Section 1.1—biased coin, binary symmetric channel, classifier verdict—are all Bernoulli[bool] instances. The biased coin appears above as Example 1. The binary symmetric channel has $\varepsilon = \omega$ (symmetric noise), so its channel matrix is $Q = \left[\begin{smallmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{smallmatrix}\right]$, a special case of the general Q above. The classifier verdict has ε and ω determined by the classifier's decision boundary and the data distribution—arbitrary in general, but still a Bernoulli[bool] instance whenever Axioms 1 and 2 hold.

In Section 1.3 we walked through the Boolean universe qualitatively, deferring the full PMF treatment to here. That deferral has now paid off: Example 1 gives the PMF table, and Definition 4.5.1 gives the formal type.

Chapter 2: binary channels (Section 2.1, Section 2.2). The binary symmetric channel of Section 2.1 is exactly Bernoulli[bool] with Q symmetric: $\varepsilon = \omega$ reduces the two free parameters of the general Q to one. The formal Bernoulli[bool] type is strictly more general; the BSC lives as a one-parameter subfamily.

The asymmetric binary channel of Section 2.2 is Bernoulli[bool] with Q general: ε and ω may differ freely. The Z-channel (one-sided error) is the extreme case $\omega = 0$ or $\varepsilon = 0$. The Bloom filter, which reports false positives but never false negatives, is the $\omega = 0$ instance: a row-1 channel that always reports $\top$ correctly and a row-2 channel that reports $\top$ with probability $\varepsilon > 0$.

Chapter 3: AND, OR, NOT under noise (Section 3.1, Section 3.2, Section 3.3). The gate-level operations of chapter 3 are operations on Bernoulli[bool] instances that produce new Bernoulli[bool] instances.

AND: if $\tilde{b}_1$ has rates $(\varepsilon_1, \omega_1)$ and $\tilde{b}_2$ has rates $(\varepsilon_2, \omega_2)$, the AND of two noisy bits derived in Section 3.1 produces a new Bernoulli[bool] instance with closed-form ε_{AND} and ω_{AND}. The derivation in Section 3.1 implicitly invoked Axiom 1 (the two error events are independent) and Axiom 2 (the rates are stable across the joint computation). Now that both axioms are named, we can say it precisely: AND is a binary operation Bernoulli[bool] $\times$ Bernoulli[bool] $\to$ Bernoulli[bool].

OR: by De Morgan's duality, the OR of two Bernoulli[bool] instances is also a Bernoulli[bool] instance. Its rates can be derived from AND via De Morgan or computed directly from the truth table of the noisy OR (Section 3.2); either way, the operation has type signature Bernoulli[bool] $\times$ Bernoulli[bool] $\to$ Bernoulli[bool]. AND and OR together generate the rest of Boolean algebra under noise.

NOT: complement swaps the block labels, exchanging ε and ω. Let $\varepsilon' = \omega$ and $\omega' = \varepsilon$ denote the effective rates of the complemented instance. The result is a Bernoulli[bool] instance with

$$Q_{\text{NOT}} \;=\; \begin{pmatrix} 1-\omega' & \omega' \\ \varepsilon' & 1-\varepsilon' \end{pmatrix} \;=\; \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \omega & 1-\omega \end{pmatrix}.$$

Here the new instance's effective false-positive rate is the original ω and its effective false-negative rate is the original ε. NOT is a unary operation Bernoulli[bool] $\to$ Bernoulli[bool].

The composition theorem of Section 3.3—that error rates compose multiplicatively in series—is the special case of Theorem 4.4.1 (the Kronecker factorization) applied to chained Bernoulli[bool] instances. Each stage of the chain is one factor in the Kronecker product; the total joint distribution is the m-fold tensor $Q_1 \otimes \cdots \otimes Q_m$. This is not a coincidence: both rest on the independence structure that the two axioms supply. The composition theorem (matrix product, serial composition) and the Kronecker theorem (tensor product, parallel composition) are sister consequences of the same axiomatic foundation operating in different geometries.

What the formal definition earned

A pedagogy that opens with a formal definition and then works out examples is a different kind of book. This book chose the other order: examples first, patterns second, formal definition last. The benefit is that the definition arrives already supported by intuition—the reader knows what Bernoulli[bool] means before reading the words.

The cost of that ordering is the distance the reader must carry intuition before it is formalized. In Section 1.3, we described the Boolean universe qualitatively and said the full treatment would come later. That deferred a cognitive debt. This section retires it.

Chapters 1–3 built the intuition. Section 4.2 and Section 4.3 named the independence assumptions those chapters were silently invoking. Section 4.4 showed what those assumptions buy in terms of factorization and parameter reduction. Definition 4.5.1 collects all of that into a single, precise statement.

This is the atom-outward trajectory: intuition-first, formalism at the end of Part I rather than the beginning. The formal definition was not asserted; it was discovered, step by step, over four chapters. The reader who has followed those chapters does not need to be persuaded that the definition is the right one. They have seen it emerge.

In Section 4.6, we look ahead at where this definition leads.

4.6 What Comes Next

We have been building toward a single formal object—Bernoulli[bool] as defined in Section 4.5—and we have arrived. This is a good moment to take stock of where we stand and where Part II will take us.

What Part I delivered

Chapter 4 has completed Part I's formal work. The two axioms are now stated with precision (Sections 4.2 and 4.3), the Kronecker factorization theorem shows what those axioms license (Theorem 4.4.1), and Definition 4.5.1 gives Bernoulli[bool] a rigorous identity as the triple (X, Y, Q) under those axioms. Every example in chapters 1 through 3—the biased coin, the binary symmetric channel, the classifier verdict, AND and OR and NOT under noise, the k-round Miller-Rabin bound—is now recognizable as an instance or an operation on instances of this type. The definition was not asserted; it was earned.

Five Core Principles: status after Part I

The Bernoulli framework rests on five organizing principles. At the close of Part I, three are fully in view and two remain ahead.

Two Axioms **[Part I, complete**] Axiom 1 (element-wise independence: Section 4.2) and Axiom 2 (conditional independence of block error rates: Section 4.3) are both stated formally and proved to jointly imply the Kronecker factorization. They are the load-bearing assumptions throughout the rest of the book.

Composition Closure **[Part I, complete**] The composition theorem of Section 3.3 showed that chaining Bernoulli[bool] instances produces a new Bernoulli[bool] instance. AND, OR, and NOT are all closure-preserving operations. The Kronecker theorem of Theorem 4.4.1 is the tensor-product face of the same closure property: whether we compose instances serially (matrix product) or view them in parallel (tensor product), the type is closed. The Boolean universe is self-contained under the operations we care about.

Boolean Thread **[Part I, complete**] Part I chose $\mathsf{Bool} = \{\top, \bot\}$ as the input and output domain and stayed there. Two elements, two partition blocks, two error rates, a 2×2 channel matrix, two free parameters after Corollary 4.4.1.2 applies—the whole Part I story has Boolean structure at every level. This is not a limitation; it is the atom's defining feature. The Boolean thread runs from the biased coin in chapter 1 to the formal definition in chapter 4 without deviation. Part II lifts it to sets; Part III lifts it to maps.

Parametric Parsimony **[visible in Part I, completed in Part II**] The corollary to the Kronecker theorem (Corollary 4.4.1.2) showed the parameter count dropping from $n(2^m - 1)$ to $2m$ under the two axioms. For the Boolean case, that reduction is already dramatic. The full principle—why $2m$ parameters are not merely convenient but are, in an information-theoretic sense, the right number—requires the entropy-based lower bounds developed in Part II, chapter 8. The seed has been planted; the fruit arrives later.

Space-Accuracy Duality **[Part II, chapter 8**] This principle—that there is a fundamental trade-off between the space a Bernoulli structure consumes and the error rates it can achieve—has not yet appeared. Part I did not need it because we were analyzing a fixed Bernoulli[bool]

instance, not asking how small we can make one while meeting an accuracy target. Space-Accuracy Duality is the central organizing principle of Part II. It arrives in chapter 8.

From bits to sets

Part I gave us the atom. Part II asks: what happens when you build a set membership predicate from atoms?

The question is less abstract than it sounds. A set S over a universe U induces, for each element $u \in U$, a Boolean query: is $u \in S$? An *approximate* set replaces exact membership with a noisy channel: the predicate may report $u \in S$ when it should report $u \notin S$ (a false positive) or the reverse (a false negative). The per-element error events are—under Axiom 1—mutually independent. Each element's query is a Bernoulli[bool] instance. The approximate set as a whole is a structured collection of those instances, and its behavior is determined by how those instances are built and indexed.

The canonical first construction is the Bloom filter. A Bloom filter maintains a bit array of m bits and a family of k hash functions; it answers set-membership queries with zero false negatives and a tunable false-positive rate. Understanding the Bloom filter from the Bernoulli perspective—as a mechanism for manufacturing Bernoulli[bool] instances with $\omega = 0$ and a controlled ε—is the project of Chapter 5. The Bloom filter is not a special case we study because it is famous; it is the cleanest first construction because it makes the $\omega = 0$ boundary and the Space-Accuracy Duality both visible in one concrete object.

By the end of Chapter 5, the reader will see not just how the Bloom filter works but why its analysis is a consequence of the formal machinery we developed in Part I.

Part II then extends: chapter 6 studies how set operations (union, intersection, complement) propagate error rates; chapter 7 develops the classification measures that quantify approximate-set quality; and chapter 8 derives the information-theoretic lower bounds that make Space-Accuracy Duality precise. The Boolean thread does not disappear in Part II—it becomes the foundation on which more complex structures are built, one Bernoulli[bool] instance at a time.

Bibliographic Notes

The material in this chapter draws on three streams of literature.

The Kronecker product. The product that carries Leopold Kronecker's name appears in his 1866 work on bilinear forms, though its use in matrix algebra spread gradually through the following century. Kronecker himself was interested in the algebraic structure of determinants and quadratic forms; the representation-theoretic and probabilistic uses of the product came later. For a modern treatment of the Kronecker product's algebraic properties—including the mixed-product property, vectorization identities, and spectral structure—see Horn and Johnson [HJ91], whose Chapter 4 is the standard graduate reference. The formal theorem statement in Section 4.4 of this textbook—that under Axioms 1 and 2 the joint confusion matrix factors as a Kronecker product of per-query matrices—is a new contribution; see the honesty note at the end of Section 4.4 for a precise account of what is new and what descends from the companion papers.

Information-theoretic channels. Chapter 4's proof of the Kronecker factorization uses the language of channels and conditional distributions but does not develop information theory from scratch. Readers who want the theoretical background—binary symmetric channels, channel capacity, and the source-channel coding theorem—should consult Cover and Thomas [CT06]. Chapters 7 and 8 of that work cover binary channels at the depth Part I uses; Chapter 10 covers channel capacity. We borrow the channel-matrix vocabulary and the notion of a memoryless channel (which is, in the Bernoulli setting, precisely what Axioms 1 and 2 enforce), but we do not reproduce Cover and Thomas's development here.

The Bernoulli papers. The two-axiom framework that organizes this chapter comes from Towell [Tow26a]. Axioms 1 and 2, as formally stated in Sections 4.2 and 4.3, are reformulations of the axioms in that paper; the textbook's contribution is to separate them into named sections, give each its own worked examples, and prove the Kronecker factorization theorem as an explicit consequence rather than treating it as a remark. The companion paper presents the factorization in a regime where rates are deterministic or implicitly conditioned upon; the textbook makes the conditional and unconditional forms explicit (Theorem 4.4.1 and Corollary 4.4.1.1), noting that the unconditional form requires either deterministic rates or pairwise-distinct queried blocks. The model-order analysis—the idea that the partition of the input domain into blocks is what governs the parameter count—appears in Towell [Tow26d], which extends the Bernoulli framework from Boolean sets to approximate values over arbitrary algebraic types. The order-2 Boolean

case treated in this chapter is the simplest instance of that general theory.

This textbook reformulates and extends both papers. The formal theorem statement in Section 4.4, the full constructive proof, and the explicit connection between model order and the Kronecker product's parameter count are contributions of this text.

Exercises

1. ⋆**Axiom 1 in measure-theoretic notation.** Let $(\Omega, \mathcal{F}, \mathbb{P})$ be the underlying probability space, and for each $x \in U$ define the error event $E_x = \{\omega \in \Omega : \tilde{A}(x)(\omega) \neq \mathbf{1}_A(x)\} \in \mathcal{F}$. Reformulate Axiom 1 (Axiom 1) using this notation: write out the independence condition both pairwise and for an arbitrary finite family $\{x_1, \ldots, x_k\} \subseteq U$. Does the condition depend on the specific probability measure $\mathbb{P}$, or only on the sigma-algebra $\mathcal{F}$?

2. ⋆**Axiom 2 as a constraint on Q.** Restate Axiom 2 (Axiom 2) as a property of the channel matrix Q. Specifically: in the order-2 Boolean case, Q has two rows. Axiom 2 declares the random quantities governing those rows to be conditionally independent. Express this as a statement about the joint density $f(\varepsilon, \omega | A)$. What behavior does Axiom 2 *forbid* Q from exhibiting across a sequence of queries? (See Proposition 4.3.1 for the stability consequence derived in §4.3.)

3. ⋆**Computing a Kronecker product.** Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

   Compute $A \otimes B$ by hand. Verify that $A \otimes B$ is a 4×4 matrix. Then compute $B \otimes A$ and check whether $A \otimes B = B \otimes A$. (*Hint:* the Kronecker product is not in general commutative; the two products will differ, but both will be 4×4.)

4. ⋆⋆**Proving the parameter-count corollary.** Prove Corollary 4.4.1.2 by direct counting.

   (a) Without any independence assumption, an n-th order model on m joint queries has $n(2^m - 1)$ free parameters. Explain where this count comes from.

(b) Under Axioms 1 and 2, the Kronecker factorization (Theorem 4.4.1) forces the parameter count down to nm. Show why: how many free parameters does each per-query matrix Q_i contribute, and why do the m per-query matrices together total nm?

(c) For the order-2 Boolean channel ($n = 2$), compute the ratio $n(2^m - 1)/(nm)$ for $m = 10$ and $m = 50$. Comment on the magnitude of the reduction.

5. ⋆⋆**Verifying the Kronecker theorem by hand for $m = 2$.** Use the asymmetric channel from the biased-coin worked example (Example 1) with $\varepsilon = 0.05$ and $\omega = 0.10$, so $Q = \left(\begin{smallmatrix} 0.90 & 0.10 \\ 0.05 & 0.95 \end{smallmatrix}\right)$.

   (a) Compute $Q \otimes Q$ explicitly. Write out all sixteen entries of the resulting 4×4 matrix.

   (b) For the two-query input pair $(\top, \bot)$, use direct probability to compute $\mathrm{P}\left(\tilde{A}(\top) = s_1,\ \tilde{A}(\bot) = s_2 | A\right)$ for all four outcome pairs $(s_1, s_2) \in \{\top, \bot\}^2$.

   (c) Verify that the entries from (b) match the corresponding row of $Q \otimes Q$ from (a), confirming Theorem 4.4.1 for $m = 2$.

6. ⋆⋆**A Markov channel violates Axiom 1.** The Markov-correlated channel in §4.3 couples the error rate on each query to the outcome of the previous query: if the previous query produced an error, the current query uses elevated false-positive rate $\varepsilon_1 > \varepsilon_0$. Consider two consecutive queries on distinct elements $x, y \in B_2 = \{\bot\}$, with base rate $\varepsilon_0 = 0.05$ and elevated rate $\varepsilon_1 = 0.30$.

   (a) Compute $\mathrm{P}(E_x = 1,\ E_y = 1)$ under the Markov channel, where the channel starts in the correct state.

   (b) Compute $\mathrm{P}(E_x = 1) \cdot \mathrm{P}(E_y = 1)$ under the same parameters.

   (c) Show that these two quantities differ, thereby exhibiting a violation of Axiom 1 (Axiom 1).

   (*Note:* §4.3 frames this example as an Axiom 2 failure; here we recast it explicitly as an Axiom 1 failure. Both framings are valid; the Markov coupling violates both axioms simultaneously.)

7. ⋆⋆**The implicit Axiom-2 invocation in §3.1's AND derivation.** Re-read the derivation of AND under noise in Section 3.1. The derivation writes the joint probability that both inputs to an AND gate produce errors as a product of two individual error probabilities.

(a) Identify the step where Axiom 2 (Axiom 2) is invoked implicitly. Quote or describe the exact expression where the factoring occurs.

(b) Suppose Axiom 2 failed: the false-positive rate on the first input told you something about the false-positive rate on the second. Write an expression for the joint error probability in that case (in terms of the conditional density $f(\varepsilon_2|\varepsilon_1)$).

(c) Would the closed-form AND output rate from §3.1 survive if Axiom 2 failed? Explain in one or two sentences.

8. ⋆⋆**Constructing a** Bernoulli[bool] **from a biased coin.** Following Definition 4.5.1 and Example 1, construct a Bernoulli[bool] instance for a biased coin with bias $p = 0.7$, false-positive rate $\varepsilon = 0.10$, and false-negative rate $\omega = 0.05$.

(a) Write down the channel matrix Q and the joint PMF over all four (true value, observed report) pairs.

(b) Verify Axiom 1 (Axiom 1) in the following sense: the Boolean universe has only two elements ($\top$ and $\bot$), so state what elementwise independence means in this degenerate case and confirm it holds by construction.

(c) Verify Axiom 2 (Axiom 2) in the following sense: state the joint density condition $f(\varepsilon, \omega|A) = f(\varepsilon|A)\, f(\omega|A)$ and explain why it is satisfied when the observation mechanism acts independently on each partition block.

9. ⋆⋆⋆**Counterexample: Axiom 1 does not imply Axiom 2.** Construct a probability model that satisfies Axiom 1 (Axiom 1: error events at distinct elements are mutually independent) but violates Axiom 2 (Axiom 2: per-block error rates are conditionally independent across blocks).

(a) Describe the model explicitly: specify U, the partition blocks, and the joint distribution of error rates across blocks.

(b) Verify that Axiom 1 holds: for any fixed realization of the block rates, the per-element error events are independent.

(c) Show that Axiom 2 fails: the joint density of the block rates does not factor.

(*Hint:* consider a two-block model where the rates ε and ω are drawn from a bivariate distribution with nonzero correlation, such as a Dirich-

let or a copula, while the per-element draws given those rates remain independent.)

10. ⋆⋆⋆**Without Axiom 2, the channel-matrix view collapses.** Show that without Axiom 2 (Axiom 2), the joint distribution over m queries to an element in a fixed block B_i cannot in general be written as a product of m identical channel-matrix rows.

    (a) Give a concrete two-query example where the marginal per-query distribution is consistent with a channel matrix Q, yet the *joint* two-query distribution over outcomes for a single block B_i does not equal $(Q_{i,\cdot})^{\otimes 2}$. (You may use the Markov-correlated channel from §4.3 as your example.)

    (b) What mathematical object would replace the single matrix Q in a model without Axiom 2? Name or describe an appropriate generalization (e.g., a time-varying sequence of matrices, a hidden Markov model, or a conditional channel).

    (c) Explain in one paragraph why the Kronecker factorization theorem (Theorem 4.4.1) depends on Axiom 2 and not only on Axiom 1.

11. ⋆⋆⋆**Tightness of the parameter-count bound.** Corollary 4.4.1.2 states that under Axioms 1 and 2, an n-th order Bernoulli predicate on m queries requires nm free parameters. Argue that this bound is tight: there exist Bernoulli[bool] instances satisfying both axioms that *genuinely* require $2m$ parameters to specify (in the order-2 Boolean case).

    (a) Construct a family of Bernoulli[bool] instances parameterized by m independently chosen channel matrices $Q_1, Q_2, \ldots, Q_m$, each with its own $(\varepsilon_i, \omega_i)$. Verify that this family satisfies both axioms.

    (b) Show that no two distinct members of the family yield the same joint distribution over m queries, so distinct parameter tuples $(\varepsilon_1, \omega_1, \ldots, \varepsilon_m, \omega_m)$ produce distinct joint confusion matrices.

    (c) Conclude that the $2m$ parameters are not merely sufficient but necessary: no reparameterization can reduce the family to fewer than $2m$ free parameters without collapsing distinct members.

Part II

From Bits to Sets

Chapter 5

Approximate Sets and Predicates

After this chapter you will be able to:

- Define an approximate set as a function from a universe U to the noisy-bit atom of Part I, and recognize the membership predicate as a Bernoulli[bool] instance per element.
- Construct a Bloom filter from scratch and compute its empirical and theoretical false-positive rates.
- State the one-sided vs two-sided distinction for approximate sets and identify when each is appropriate.

5.1 From Atom to Set

Part I closed with a promise. Section 1.5 described the noisy bit as the framework's atom and previewed what Part II would do with it: take many of them, arrange them into a structure, and study the algebra of that structure. The single-Boolean object of chapters 1 through 4 would become a building block, and the natural building was a noisy bit per universe element. That object is the approximate set, and Part II opens with it.

Why is the construction inevitable? A classical set $A \subseteq U$ is characterized entirely by its membership predicate: a function $\mathbb{1}_A : U \to \{\top, \bot\}$ that returns $\top$ on the elements of A and $\bot$ everywhere else. Two sets are equal if and only if their predicates agree on every input. The predicate is the set; the set-builder notation $\{x : \mathbb{1}_A(x) = \top\}$ is just the predicate read backwards. Now replace the exact predicate with a noisy one. Each

query $x \mapsto \tilde{A}(x)$ returns a Boolean that is usually but not always the correct membership verdict. Each evaluation is a noisy bit, an instance of the formal type Bernoulli[bool] that Section 4.5 defined. An approximate set is the object whose membership predicate has been made noisy in this way, one query at a time.

The predicate framing

Let U be a fixed universe (finite or countably infinite) and let $A \subseteq U$ be the underlying *objective set* we intend to represent. An *approximate set* of A is a function

$$\mathsf{A}^{\pm} : U \longrightarrow \text{Bernoulli[bool]}$$

such that each evaluation $\mathsf{A}^{\pm}(x)$ is a Bernoulli[bool] instance. Per Definition 4.5.1, a Bernoulli[bool] instance is a triple (X, Y, Q) that produces a Boolean output when sampled; we write $\mathsf{A}^{\pm}(x) \in \{\top, \bot\}$ when referring to the sampled output of $\mathsf{A}^{\pm}$ at x. The notation is the bookkeeping: A is the truth, $\mathsf{A}^{\pm}$ is its noisy representation, and $\mathsf{A}^{\pm}(x) = \top$ is read as "the structure reports that x is a member of A." The structure may be a Bloom filter, a perfect hash filter, a learned classifier with a threshold, or any other implementation whose membership queries return Booleans. What makes it an approximate set, in the sense Part II develops, is that each per-element query is governed by a single Q-matrix from chapter 4: two rates of error, one rate for the positives and one rate for the negatives, and nothing else.

Two paragraphs are worth pausing on. First, the predicate framing is not a definitional choice; it is a recognition. The classical correspondence between a set and its indicator function is exact, and the approximate analogue is forced once we ask what a set looks like when each query may be wrong. The predicate is where the noise lives and there is nowhere else for it to live, because there is no other behavioral handle on a set besides the membership test. Second, the distinction between A and $\mathsf{A}^{\pm}$ is the distinction between truth and report. We never lose access to A at the level of the model: A is the objective set, and the rates of $\mathsf{A}^{\pm}$ are defined against it. What we lose, when we query the structure, is the ability to read A off directly. We see only the report, $\mathsf{A}^{\pm}(x)$, and the rates tell us how that report relates to the underlying truth.

The two error rates

Every query $\mathsf{A}^{\pm}(x)$ is one of two kinds, determined by whether x is in A. If $x \notin A$, then the correct verdict is $\bot$ and the structure errs only by

reporting $\top$. The false-positive rate of the approximate set is the conditional probability of that error:

$$\varepsilon \;=\; \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top | x \notin A\right).$$

If $x \in A$, then the correct verdict is $\top$ and the structure errs only by reporting $\bot$. The false-negative rate is the conditional probability of that error:

$$\omega \;=\; \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \bot | x \in A\right).$$

These are the two rates that chapter 1 introduced for a single noisy bit (Section 1.1) and that chapter 4 collected into the channel matrix Q (Definition 4.5.1). The change in Part II is one of grouping rather than of substance. A noisy bit had one positive input and one negative input; the approximate set has $|A|$ positive inputs (the members) and $|U \setminus A|$ negative inputs (the non-members). The two-rate description does not grow: the same ε governs every query on a non-member, and the same ω governs every query on a member.

That collapse from many per-element rates to two per-set rates is not automatic. In principle every element $x \in U$ could carry its own rate $\varepsilon(x)$ or $\omega(x)$, and the predicate would need a function-valued description rather than a two-number one. What collapses the description is Axiom 2 of chapter 4 (Axiom 2, Section 4.3): the per-block rates are stable across the block. Membership splits the universe into two blocks, and Axiom 2 forces a single rate per block. Section 5.3 makes this precise. For now, the operational takeaway is that an approximate set is described, at the rate level, by the pair (ε, ω) alone, no matter how large $|U|$ or $|A|$ is.

The block structure

The universe U does not enter the rate description as a flat list of elements. It enters as a two-block partition: the in-set block $B_1 = A$ and the out-of-set block $B_2 = U \setminus A$. Membership induces the partition; the partition determines which rate applies to which query. This is exactly the model-order-two block structure of chapter 4: two blocks, two rate-rows in the channel matrix Q, one row per block. An approximate set is a Bernoulli[bool] instance whose input domain is U rather than $\{\top, \bot\}$, with the membership partition standing in for the Boolean partition of the atomic case. The matrix Q that governs each query is the same:

$$Q \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

with row 1 for the in-set block and row 2 for the out-of-set block. Section 5.3 writes this out formally and treats the case where the rates depend on x before Axiom 2 collapses them.

The payoff of the block structure is that the per-query description generalizes cleanly to many queries. Chapter 4 (Section 4.4, Theorem 4.4.1) showed that n independent queries on a Bernoulli[bool] instance produce a joint distribution that factors as the Kronecker product of the n per-query channels. The same theorem applies here, with the membership partition in place of the Boolean partition. The Kronecker factorization of n-query behavior also relies on Axiom 1 (Axiom 1), which we silently invoked above: error events at distinct elements of U are mutually independent. Axiom 1 lets us treat the queries as independent draws; Axiom 2 lets us assign the same per-block rate to every draw within a block. Together they give the Kronecker product structure of Theorem 4.4.1. A workload of n queries on $\mathsf{A}^{\pm}$, drawn from the in-set and out-of-set blocks in some specified ratio, has a joint behavior that factors exactly as the tensor product of the per-query matrices. The price of the full joint description is the two rates, not 2^n probabilities. The chapter 4 machinery does not need to be rebuilt for Part II; it transfers over wholesale, and Section 5.3 states the transfer precisely.

Where the chapter goes from here

The construction is so far an abstraction. We have said what an approximate set is, namely a function from a universe to the noisy-bit atom of Part I, and we have named the rates that describe it. We have not yet said how anyone would build one. The most widely deployed construction is the Bloom filter, and Section 5.2 develops it from scratch as the canonical instance of the predicate framing. The construction will have a structural false-negative rate of zero, a tunable false-positive rate, and a closed-form formula for the false-positive rate as a function of three integers (the bit-array size m, the hash count k, and the number of stored items n). The Bloom filter is the example chapters 6 through 8 keep returning to, so getting it on the table first matters.

Section 5.3 then develops the predicate algebra in general. The Bloom filter is one approximate set; the predicate framing covers every approximate set, and the chapter 4 axioms and Kronecker theorem apply uniformly across the family. Section 5.4 draws the line between approximate sets with one structural zero rate and those with two positive rates: a distinction that organizes chapters 6 and 15.

5.2 The Bloom Filter From Scratch

Section 5.1 introduced the approximate set as an abstraction: a function $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$ whose per-element rates collapse to two per-set numbers. It said nothing about how such a function might be built in finite space. This section answers that. We will construct the Bloom filter from first principles as a bit array plus k hash functions, prove that its false-negative rate is exactly zero by inspection of the construction, derive a closed-form expression for its false-positive rate as a function of three integers, and find the choice of one of those integers that minimizes the rate. The Bloom filter is one specific realization of an approximate set, but it is the realization that chapters 6 through 8 keep returning to, and getting the construction fully on the table first repays itself many times over.

The construction

Fix a universe U, a positive integer m (the number of bits), and a positive integer k (the number of hash functions). The data structure consists of a bit array $b[1\,..\,m]$ of length m, initialized to all zeros, together with k independent hash functions

$$h_1, h_2, \ldots, h_k \;:\; U \longrightarrow \{1, 2, \ldots, m\}.$$

The independence assumed of the h_i is the strong assumption that makes the analysis below tractable: each h_i is treated as a uniform random function from U to $\{1, \ldots, m\}$, and the k functions are mutually independent. Real implementations approximate this with families like double hashing over SHA-256 output, and the divergence between the theoretical and empirical behaviour is small enough that we ignore it through this section.

Definition 5.2.1 (Bloom filter)**.** *The* Bloom filter *of parameters (m, k) is the data structure $(b, h_1, \ldots, h_k)$ together with the two operations:*

- INSERT(x)*: for $i = 1, \ldots, k$, set $b[h_i(x)] \leftarrow 1$.*
- QUERY(x)*: return $\top$ if $b[h_i(x)] = 1$ for every $i \in \{1, \ldots, k\}$; return $\bot$ otherwise.*

After inserting the n elements of a set $A \subseteq U$, the predicate $x \mapsto$ QUERY(x) defines the membership report of the approximate set $\mathsf{A}^{\pm}$.

Algorithm 2 states the two operations in pseudocode. Insert sets k bits. Query reads k bits and returns the conjunction.

Algorithm 2: Bloom filter insert and query

```
1 procedure INSERT(x)
2 |   for i ← 1 to k do
3 |   |   b[h_i(x)] ← 1;
4 |   end
5 function QUERY(x)
6 |   for i ← 1 to k do
7 |   |   if b[h_i(x)] = 0 then return ⊥;
8 |   end
9 |   return ⊤;
```

Both operations run in $O(k)$ time, independent of $|A|$ or $|U|$. The data structure uses exactly m bits of storage, also independent of $|A|$ or $|U|$. The bit array is the only state, and it is never read except via QUERY.

A notation note on n

The Bloom-filter literature universally writes (m, k, n) for the parameter triple, where m is the bit count, k is the hash count, and n is the number of items inserted (the cardinality of A). We adopt that convention here. The symbol n in this section is the set size, not the workload size or the number of independent queries. Chapter 1's n for sample size and chapter 4's n for query count are different objects. Where the context could be ambiguous (especially in chapter 6 once we start composing Bloom filters and querying many items against the composition), we write n_{items} for the set size and n_{queries} for the workload size. Inside the present section the meaning is unambiguous and we write n alone.

Why the false-negative rate is exactly zero

Suppose $x \in A$ has been inserted into the filter. By definition of INSERT, the bits $b[h_1(x)], b[h_2(x)], \ldots, b[h_k(x)]$ were all set to 1 at the time of insertion. Later insertions of other elements can only set additional bits to 1; nothing in the construction ever resets a bit to 0. Therefore at every subsequent time, the bits $b[h_1(x)], \ldots, b[h_k(x)]$ all remain 1, and QUERY(x) reads k ones and returns $\top$.

The argument is structural rather than probabilistic. It does not rely on the choice of hash family, on the cardinality of A, on the bit count m,

or on the workload distribution. It relies only on the fact that INSERT sets bits to 1 and no operation resets them. For every inserted item and every parameter setting,

$$\omega \;=\; \mathrm{P}\left(\textsc{Query}(x) = \bot \mid x \in A\right) \;=\; 0.$$

A Bloom filter is therefore *one-sided*: its only failure mode is the false positive. Section 5.4 will give the one-sided property a name and contrast it with two-sided approximate sets. For now, the point to register is that one of the two error rates of an approximate set, in this construction, has been driven to zero by construction rather than by parameter tuning. The remaining question is the size of the other rate.

The false-positive rate

Consider a query for an element $x \notin A$. The query returns $\top$ if and only if the bits $b[h_1(x)], \ldots, b[h_k(x)]$ are all 1. These k bits are positions in the same bit array that the n insertions populated. Whether the bits are 1 depends on whether the kn hash values produced during insertion ever landed on the same k positions that the query now reads.

The natural quantity to compute first is the probability that a *specific* bit, say $b[j]$ for some fixed j, is still 0 after the n insertions. Each insertion contributes k hash values, each uniformly distributed on $\{1, \ldots, m\}$. The probability that one hash value misses position j is $1 - 1/m$. The probability that all kn hash values miss position j, assuming they are independent, is

$$\mathrm{P}\left(b[j] = 0 \text{ after } n \text{ inserts}\right) \;=\; \left(1 - \frac{1}{m}\right)^{kn} \;\approx\; e^{-kn/m}.$$

The approximation uses $(1 - 1/m)^{kn} = \exp(kn \ln(1 - 1/m))$ and $\ln(1 - 1/m) \approx -1/m$ for large m. The error in the exponent is on the order of $1/m^2$ per insertion, so the substitution is excellent in any regime where m is at all large. The complementary probability is

$$\mathrm{P}\left(b[j] = 1 \text{ after } n \text{ inserts}\right) \;\approx\; 1 - e^{-kn/m}.$$

Call this probability p. Each bit of the array is, at the end of the insertion phase, a Bernoulli random variable with mean p. The bits are identically distributed and the mean p controls how full the array is. The product kn/m is the load: it counts the average number of hash hits per bit.

From bit occupancy to false-positive rate. A query for $x \notin A$ reads k bits at positions $h_1(x), \ldots, h_k(x)$. Each position is uniformly distributed in $\{1, \ldots, m\}$, and each bit at that position has marginal probability p of being 1. The query returns $\top$ if and only if all k read bits are 1. If the k bit-read events were mutually independent, the joint probability would be p^k, and we would conclude

$$\varepsilon \;\approx\; \left(1 - e^{-kn/m}\right)^k. \tag{5.1}$$

On the independence assumption. The k bits read during the query are positions in the same shared bit array, and the same n insertions populated the array. The events "$b[h_i(x)] = 1$" for $i = 1, \ldots, k$ are therefore not strictly independent. Knowing that one queried bit is 1 shifts the posterior on which positions were hit by the kn insertion-time hashes, and that in turn shifts the marginals on the remaining $k - 1$ queried bits. The dependence is positive (one bit being 1 raises the probability that the others are too) and small. Bloom [Blo70] already noted the approximation, and subsequent analyses have quantified the deviation rigorously: the approximation (5.1) is an under-estimate, and the true FPR is bounded by a slightly larger expression involving binomial coefficients. For sparse filters (small load kn/m) the deviation is negligible; for the practical regime $kn/m \approx \ln 2$ that we are about to derive as optimal, the deviation is below a percent of the rate itself, and we do not lose any practical accuracy by adopting (5.1) as the working formula. We flag the assumption explicitly so that the reader is aware of the gap, and we proceed with (5.1) for the optimization that follows. The exercises revisit the issue with a small worked counter-example (Item 7).

Optimal k for fixed m and n

The bit count m is the space budget. The set size n is given by the application. The hash count k is the one design knob the implementer controls. The natural question is: what choice of k minimizes the FPR (5.1) for fixed m and n?

Let $f(k) = (1 - e^{-kn/m})^k$. We minimize over $k > 0$, treating k as a continuous variable; the optimal integer k is then read off as the nearest integer to the continuous optimum. Taking logarithms,

$$\ln f(k) \;=\; k \ln(1 - e^{-kn/m}).$$

Differentiating with respect to k,

$$\frac{d}{dk}\ln f(k) \;=\; \ln(1-e^{-kn/m}) \;+\; k\cdot\frac{(n/m)e^{-kn/m}}{1-e^{-kn/m}}.$$

Setting the derivative to zero and writing $u = e^{-kn/m}$ for brevity,

$$\ln(1-u) \;+\; \frac{kn}{m}\cdot\frac{u}{1-u} \;=\; 0.$$

The quantity kn/m is exactly $-\ln u$, so the equation becomes

$$\ln(1-u) \;-\; (\ln u)\cdot\frac{u}{1-u} \;=\; 0.$$

Multiply through by $(1-u)$:

$$(1-u)\ln(1-u) \;-\; u\ln u \;=\; 0,$$

that is, $(1-u)\ln(1-u) = u\ln u$. By inspection, $u = 1/2$ solves this equation: both sides equal $(1/2)\ln(1/2) = -(\ln 2)/2$. This is a minimum of f, not a maximum or saddle, by the following boundary argument. The base $1-e^{-kn/m}$ lies in $(0,1)$ for every $k>0$, so $f(k)\in(0,1)$ everywhere in the interior. At the boundaries, $f\to 1$: as $k\to 0^+$, $\ln f(k) = k\ln(1-e^{-kn/m}) \approx k\ln(kn/m)\to 0$; as $k\to\infty$, $1-e^{-kn/m}\to 1$ fast enough that $k\ln(1-e^{-kn/m})\approx -ke^{-kn/m}\to 0$. So f is a smooth function on $(0,\infty)$ that sits strictly below 1 in the interior and tends to 1 at both endpoints; it therefore attains a global minimum at some interior critical point. Differentiating the FOC and substituting $u = 1/2$ gives

$$\left.\frac{d^2\ln f}{dk^2}\right|_{k=k^*} \;=\; 2\,(n/m)\,(1-\ln 2) \;>\; 0,$$

so $k^* = (m/n)\ln 2$ is a local minimum, and by direct check the FOC has $u = 1/2$ as its only solution in $(0,1)$ (see Item 5). The local minimum is therefore the global one.

Substituting back, $u = e^{-kn/m} = 1/2$ gives $kn/m = \ln 2$, hence

$$k^* \;=\; \frac{m}{n}\ln 2. \tag{5.2}$$

At this k^* each bit has probability exactly $1/2$ of being 1 (half the array is occupied) and the FPR formula evaluates to

$$\varepsilon^* \;\approx\; \left(\frac{1}{2}\right)^{k^*} \;=\; 2^{-(m/n)\ln 2} \;=\; (0.6185\ldots)^{m/n}. \tag{5.3}$$

The optimal Bloom filter spends m/n bits per stored item and pays roughly 0.62 for every additional bit per item by reducing the FPR by that factor. The half-full bit array at the optimum is a striking fact: the filter is most accurate not when the bits are sparse but when exactly half of them are set.

Theorem 5.2.1 (Optimal k for the Bloom filter)**.** *For fixed bit count m and item count n, the false-positive rate of a Bloom filter under the approximation* (5.1) *is minimized at*

$$k^* \;=\; \frac{m}{n}\ln 2,$$

at which the bit array is half-occupied in expectation and the false-positive rate satisfies

$$\varepsilon^* \;\approx\; \left(\frac{1}{2}\right)^{k^*} \;=\; 2^{-(m/n)\ln 2}.$$

The bit-per-item ratio m/n is the right way to think about a Bloom filter's space budget, and Theorem 5.2.1 says the FPR decays exponentially in that ratio with base $1/2$ (after the optimal k is chosen). Chapter 8 will compare this $0.6185^{m/n}$ scaling to the information-theoretic lower bound and will find a factor of about 1.44 between the two: the standard Bloom filter is within a constant of optimal but does not reach it. Chapter 14 closes the gap.

A worked example

Take $m = 10000$ bits and $n = 1000$ items, a comfortable 10 bits per item. Then $m/n = 10$ and

$$k^* \;=\; 10\ln 2 \;\approx\; 6.93.$$

The nearest integer is $k = 7$, which we adopt. The optimum FPR is

$$\varepsilon^* \;\approx\; (1/2)^7 \;=\; 1/128 \;\approx\; 0.0078,$$

or about one false positive in every 128 negative queries.

To see that $k = 7$ is genuinely near the minimum and not just our rounded approximation, evaluate (5.1) at $k = 5, 6, 7, 8, 9$ with $m = 10000, n = 1000$:

k	$1 - e^{-kn/m}$	$\varepsilon \approx (1 - e^{-kn/m})^k$
5	0.3935	0.00943
6	0.4512	0.00844
7	0.5034	0.00819
8	0.5507	0.00846
9	0.5934	0.00913

The FPR curve is shallow near the minimum: the difference between $k = 6, 7, 8$ is in the third significant digit. Practical implementations often pick $k = 7$ for $m/n = 10$ and rarely bother to fine-tune. The corresponding occupancy at $k = 7$ is $1 - e^{-7/10} = 1 - 0.4966 = 0.5034$, just over half the bits set, which matches the half-full prediction of Theorem 5.2.1 up to the rounding from $k^* = 6.93$ to $k = 7$.

Two sanity checks anchor the formula. First, the FPR depends only on m and n through the ratio m/n once k is optimized. Doubling both m and n leaves the optimal FPR unchanged and doubles the storage; the bit-per-item budget is what matters. Second, the FPR is indeed sensitive to k away from the optimum: a Bloom filter built with $k = 1$ and $m/n = 10$ has FPR $1-e^{-0.1} \approx 0.0952$, about 12 times the optimal rate; a Bloom filter built with $k = 20$ has FPR $(1 - e^{-2})^{20} \approx 0.0546$, about 7 times the optimal rate. Both off-optimum choices are markedly worse than $k = 7$, and the curve is shallow only in a small neighborhood of the optimum. The optimum sits where two opposing pressures balance. Increasing k widens the conjunction the query must satisfy, which on its own would drive the FPR down; but it also fills the array faster, so each individual bit-check is more likely $\top$, driving the FPR up. Decreasing k has the mirror effect: each bit-check is less likely $\top$, but the conjunction is narrower. The minimum at $k^* = (m/n) \ln 2$ is where these two forces cancel: half the array is occupied at the optimum, and the per-bit and per-query terms exactly trade off.

Looking ahead

We have constructed one approximate set in closed form. The Bloom filter has $\omega = 0$ by construction, $\varepsilon \approx (1 - e^{-kn/m})^k$ in the bit-independence approximation, and an optimal $k^* = (m/n) \ln 2$ at which the FPR decays as $0.6185^{m/n}$ in the bit-per-item budget. Every later chapter of Part II will use this filter as a running example.

The construction also stands as one instance of the abstraction Section 5.1 developed. The same predicate framing, the same per-block rate structure, the same chapter 4 axioms apply to the Bloom filter and to any other approximate set. Section 5.3 develops the predicate algebra in generality: the channel matrix per query, the block-stability under Axiom 2, and the Kronecker factorization over multiple queries. The Bloom filter is one row of that general theory. Section 5.4 then takes up the special structure of this row, the one-sided property $\omega = 0$, and contrasts it with the two-sided constructions Part II will also encounter in chapters to come.

5.3 The Predicate Algebra

Section 5.1 introduced the approximate set as a function $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$ and described its rates informally. Section 5.2 built one specific instance, the Bloom filter, and showed how the rates of that construction come out. Both sections leaned on the Bernoulli[bool] machinery of Chapter 4 without saying precisely how the lift to a set was structured at the matrix level. This section closes that gap. We restate the predicate $\mathsf{A}^{\pm}(x)$ as a Bernoulli[bool] instance per element, write down the channel matrix in its most general per-element form, show that Axiom 2 collapses the per-element rates to two per-block rates, and then invoke the Kronecker factorization of Theorem 4.4.1 to factor the joint distribution over many queries. The corollary is the chapter's first formal payoff: an approximate set's full probabilistic description, no matter how large the universe, requires exactly two parameters.

The predicate as a Bernoulli[bool] instance

Fix a universe U and an objective set $A \subseteq U$. For every $x \in U$, the query $\mathsf{A}^{\pm}(x)$ produces a Boolean output, and the rates of that output depend on whether x is inside or outside A. In the most general per-element setting, each x carries its own per-element rate. If $x \in A$, the false-negative rate at x is $\omega(x) = \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \bot | x \in A\right)$, and the structure correctly reports $\top$ with probability $1 - \omega(x)$. If $x \notin A$, the false-positive rate at x is $\varepsilon(x) = \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top | x \notin A\right)$, and the structure correctly reports $\bot$ with probability $1 - \varepsilon(x)$.

The four conditional probabilities assemble into a 2×2 channel matrix attached to the query at x:

$$Q_x \;=\; \begin{pmatrix} 1-\omega(x) & \omega(x) \\ \varepsilon(x) & 1-\varepsilon(x) \end{pmatrix}.$$

Row 1 corresponds to the in-set case (truth $\top$, that is, $x \in A$); row 2 corresponds to the out-of-set case (truth $\bot$, that is, $x \notin A$). Column 1 is the report $\top$; column 2 is the report $\bot$. The convention matches the Bernoulli[bool] definition of Definition 4.5.1: row 1 governs positives, row 2 governs negatives, and each row sums to 1. The matrix Q_x is the Bernoulli[bool] instance that describes the per-query behavior of $\mathsf{A}^{\pm}$ at x. There is one such instance per universe element, and in this most general formulation the family $\{Q_x : x \in U\}$ may carry $2|U|$ free parameters, one false-positive rate and one false-negative rate per element, clamped only by the obvious constraint that an in-set element has no false-positive rate and

an out-of-set element has no false-negative rate. Without further structure, this is the cost of describing $\mathsf{A}^{\pm}$ at the rate level: a parameter count that scales with the universe, not a constant. The next subsection shows that the cost collapses.

Axiom 2 collapses per-element rates to per-block rates

Axiom 2 of chapter 4 (Axiom 2, Section 4.3) asserts that the per-block error rates are conditionally independent of each other given the objective set. For an approximate set, the two blocks are the in-set block $B_1 = A$ and the out-of-set block $B_2 = U \setminus A$, and Axiom 2 forces a stronger condition along with the independence: the per-element rates inside each block agree. The reason is that Axiom 2's statement is at the block level, not the element level. The axiom assigns one random rate per block; once that rate is realized, every element of the block inherits it. Element x inside block B_i does not get its own draw separate from y inside the same B_i; both share the block's realized rate. Proposition 4.3.1 of chapter 4 already recorded this: once the rates are realized, the per-element channel matrices stabilize, with row i of every per-element matrix taking the realized rate of block B_i. The proposition below states the corresponding fact for the predicate algebra of an approximate set.

Proposition 5.3.1 (Block-stability of per-element rates)**.** *Let* $\mathsf{A}^{\pm} : U \to$ Bernoulli[bool] *be an approximate set satisfying Axiom 2 (Axiom 2) over the partition* $\{B_1, B_2\} = \{A, U \setminus A\}$*. Then there exist two scalar rates* $\varepsilon, \omega \in [0, 1]$ *such that*

$$\omega(x) \;=\; \omega \quad \textit{for all } x \in A, \qquad \varepsilon(x) \;=\; \varepsilon \quad \textit{for all } x \in U \setminus A,$$

and the per-element channel matrix is the same for every query:

$$Q_x \;=\; Q \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The rate description of $\mathsf{A}^{\pm}$ *collapses from* $2|U|$ *parameters to two:* (ε, ω)*.*

Proof. By Axiom 2, each block B_i carries a single random rate α_i. Proposition 4.3.1 of chapter 4 then says that once the rates are realized, the per-element channel matrix Q_x depends on x only through the block B_i to which x belongs, with row i of Q_x determined by α_i. For the two-block partition $\{A, U \setminus A\}$, this gives two scalars: ω for row 1 (governing every $x \in A$) and ε for row 2 (governing every $x \in U \setminus A$). The remaining row

entries follow from the stochasticity of Q_x. Since the row identified by an element's block determines its rate, the matrix Q_x is the same for every x in the same block; and since the rows correspond exactly to the two blocks, $Q_x = Q$ is the same matrix for every $x \in U$. □

Throughout Part II we work in the deterministic-rates regime of Chapter 4: the per-block rates ε and ω are fixed parameters of $\mathsf{A}^{\pm}$, not random variables. Under this regime $Q^*_\ell = Q_\ell$ and the conditional and unconditional Kronecker forms of Theorem 4.4.1 coincide, so we may apply the factorization directly to joint behavior over many queries without re-establishing the conditional structure each time. With this assumption in hand we suppress the per-element subscript on Q_x and write a single Q for the approximate set's channel matrix. What varies across queries is the row that the query reads, not the matrix itself.

The joint distribution over m queries factors via Kronecker

When the Bloom filter of Section 5.2 is queried on a workload of m items, the joint behavior over those m queries is what determines the workload-level error rate; the single-query analysis of Section 5.2 was only half the story. Suppose a workload issues m independent queries on $\mathsf{A}^{\pm}$, one on each of m distinct elements $x_1, \ldots, x_m \in U$. Each element x_ℓ belongs to one of the two blocks; call its block index $j_\ell \in \{1, 2\}$. Each query produces a Boolean output $k_\ell \in \{\top, \bot\}$, with conditional probability $(Q)_{j_\ell, k_\ell}$ as prescribed by the per-query matrix.

The joint distribution of $(k_1, \ldots, k_m)$ given the source-block tuple $(j_1, \ldots, j_m)$ is the object Theorem 4.4.1 of chapter 4 governs. Its hypotheses are exactly the two axioms of the Bernoulli framework: Axiom 1 (Axiom 1) gives the mutual independence of error events across distinct elements, and Axiom 2 (Axiom 2) gives the block-stability of the rates that Proposition 5.3.1 just established. Since every per-query matrix is the same Q, the theorem yields the m-fold Kronecker power:

$$Q^{(m)} \;=\; \underbrace{Q \otimes Q \otimes \cdots \otimes Q}_{m \text{ copies}} \;=\; Q^{\otimes m}.$$

Corollary 5.3.0.1 (Kronecker factorization for approximate sets)**.** *Let* $\mathsf{A}^{\pm} : U \to$ Bernoulli[bool] *be an approximate set satisfying Axioms 1 and 2 over the partition* $\{A, U \setminus A\}$*, and assume the deterministic-rates regime announced above. The joint confusion matrix* $Q^{(m)}$ *of* m *queries on* m *distinct*

elements of U factors as the m-fold Kronecker power $Q^{\otimes m}$ of the common per-query channel matrix

$$Q \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The full joint probabilistic description of $\mathsf{A}^{\pm}$ requires only the two parameters (ε, ω), independently of $|U|$ and $|A|$.

Proof. Proposition 5.3.1 makes the per-element matrix Q_x equal to a single matrix Q parameterized by (ε, ω). Theorem 4.4.1 then factors the joint matrix as $Q^{(m)} = Q_1 \otimes \cdots \otimes Q_m$; the deterministic-rates assumption makes the conditional and unconditional forms coincide, and every factor equals Q, so $Q^{(m)} = Q^{\otimes m}$. The two scalars (ε, ω) that fix Q fix the entire joint matrix. □

Corollary 5.3.0.1 is the chapter's first formal payoff. The informal claim of Section 5.1, that the rate description does not grow with $|U|$, has now been earned at the joint level: two scalars fix Q, Q fixes the Kronecker power, and the Kronecker power fixes the joint behavior of arbitrarily many queries. The parametric-parsimony observation of Chapter 4 carries over without dilution.

A worked example

Let us see the factorization concretely. Take $U = \{1, 2, \ldots, 10\}$, $A = \{1, 2, 3\}$, and an approximate set $\mathsf{A}^{\pm}$ with $\varepsilon = 0.1$ and $\omega = 0.05$. Block $B_1 = A$ has three in-set elements and $B_2 = U \setminus A$ has seven out-of-set elements. The per-query channel matrix is

$$Q \;=\; \begin{pmatrix} 1-0.05 & 0.05 \\ 0.1 & 1-0.1 \end{pmatrix} \;=\; \begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix}.$$

Now consider a workload of four queries: two on specific in-set elements (say $x_1 = 1$ and $x_2 = 2$, both in A) and two on specific out-of-set elements (say $x_3 = 4$ and $x_4 = 5$, both in $U \setminus A$). Suppose we want the probability that the structure reports $\top$ on all four queries. The two in-set queries are correct reports (true positives), and the two out-of-set queries are wrong reports (false positives).

By Corollary 5.3.0.1, the joint confusion matrix is $Q^{(4)} = Q^{\otimes 4}$, and the probability of the specific source-block tuple (B_1, B_1, B_2, B_2) producing the

specific output pattern $(\top, \top, \top, \top)$ is the product of per-query entries from Q:

$$\begin{aligned}
&\mathrm{P}\left(\mathsf{A}^{\pm}(x_1) = \top, \mathsf{A}^{\pm}(x_2) = \top, \mathsf{A}^{\pm}(x_3) = \top, \mathsf{A}^{\pm}(x_4) = \top | x_1, x_2 \in A,\ x_3, x_4 \notin A\right) \\
&\quad = Q_{1,1} \cdot Q_{1,1} \cdot Q_{2,1} \cdot Q_{2,1} \\
&\quad = (1-\omega)^2 \cdot \varepsilon^2 \\
&\quad = (0.95)^2 \cdot (0.10)^2 \\
&\quad = 0.9025 \cdot 0.01 \\
&\quad = 0.009025.
\end{aligned}$$

The two true-positive factors $(0.95)^2 = 0.9025$ come from the two in-set queries reading row 1 of Q, column 1; the two false-positive factors $(0.10)^2 = 0.01$ come from the two out-of-set queries reading row 2 of Q, column 1. Each factor is one cell of the per-query matrix, and the Kronecker product structure puts them together multiplicatively because the queries are mutually independent (Axiom 1) and share the same per-block rates across the workload (Axiom 2).

The same calculation would proceed identically for any other query pattern. The probability of reporting $\bot$ on the two in-set queries and $\top$ on the two out-of-set queries, for instance, would be $\omega^2 \cdot \varepsilon^2 = 0.0025 \cdot 0.01 = 0.000025$; the probability of reporting $\top$ on the two in-set queries and $\bot$ on the two out-of-set queries would be $(1-\omega)^2 \cdot (1-\varepsilon)^2 = 0.9025 \cdot 0.81 = 0.731025$, the most likely single outcome and the one in which every query is reported correctly. Across all $2^4 = 16$ output patterns, the probabilities sum to one, and each is a product of four per-query entries selected from Q according to the block of the query and the report of the output.

The point is the structure, not the numbers. Doubling the universe to 20 elements while keeping the same rates leaves this query pattern's probability unchanged; doubling the workload to 40 queries grows the Kronecker product to 40 copies of Q, but each entry is still a product of two-rate factors chosen one per query.

Looking ahead

The Bloom filter of Section 5.2 is one Bernoulli[bool] instance in this family: its Q has $\omega = 0$ structurally and ε tuned by the bit count, hash count, and item count, so the worked example above applies directly once the Bloom filter's ε is substituted for the generic 0.1. Any other approximate set construction yields a different Q but the same algebra: per-query matrix

in row-stochastic form, joint matrix as Kronecker product, two-parameter description of the full probabilistic behavior. The one-sided versus two-sided distinction of Section 5.4 is the next slice: what happens when one of the two parameters is structurally zero, as it is for the Bloom filter, and how that case contrasts with the two-positive-rate setting that arises elsewhere.

5.4 One-Sided vs Two-Sided Approximate Sets

The Bloom filter of Section 5.2 has a striking property: one of its two error rates is exactly zero, not by parameter choice but by construction. The bit array can only accumulate ones, so an inserted item is never reported missing; the other rate is positive and tunable. This is one shape an approximate set can take. A different construction might force the false-positive rate to zero on a designated set instead, leaving the false-negative rate positive; a third might allow both rates to be positive. The present section gives the three shapes their names and connects the distinction to the algebraic structure that chapter 6 will develop.

Definition

Definition 5.4.1 (One-sided and two-sided approximate sets)**.** *Let* $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$ *be an approximate set of* $A \subseteq U$ *with per-block rates* (ε, ω) *as in Section 5.3. We say* $\mathsf{A}^{\pm}$ *is*

- one-sided in FNR *(or* FNR-zero*) if* $\omega = 0$ *and* $\varepsilon > 0$*;*
- one-sided in FPR *(or* FPR-zero*) if* $\varepsilon = 0$ *and* $\omega > 0$*;*
- two-sided *if both* $\varepsilon > 0$ *and* $\omega > 0$*.*

The Bloom filter of Definition 5.2.1 is one-sided in FNR. The perfect hash filter of Chapter 15 is one-sided in FPR on its design domain.

The names track the rate that is zero, not the one that is positive. The Z-channel of Definition 2.2.2 is the per-query analog; the present section's one-sided property is its per-set lift. A two-sided approximate set sits in the interior of the rate square $[0, 1]^2$; a one-sided set sits on one of the two coordinate axes. The corner $\varepsilon = \omega = 0$ is the exact set, not an approximate set at all, and is excluded from Definition 5.4.1.

Why FNR-zero is the natural default for insertion-only structures

The Bloom filter's FNR-zero property was structural, not designed. The insert operation sets bits to 1, and no operation in the chosen insertion-only API ever resets them; a subsequent query for an inserted item reads bits that were set, so the query returns $\top$ regardless of what else has happened since. The argument depended only on the monotonicity of the bit array under that API. Insertions go up, never down. Any data structure built on the same monotone-insert primitive shares the property: counting sketches keyed by hash, summary structures over append-only logs, and any representation of an insertion-only bit-set query (read what was written) are one-sided in FNR for the same structural reason. (Counting Bloom filters, taken up below, restore deletions by storing small integer counters in place of bits, paying for that change of API with $\omega > 0$.)

The asymmetry runs the other way. Forcing $\varepsilon = 0$ on a designated set, while keeping $\omega > 0$ elsewhere, requires the construction to know which queries are guaranteed inside and which are not. A deterministic check against the underlying set would do this, but at the cost of storing the set itself, which destroys the space saving an approximate set was supposed to provide. The constructive path to an FPR-zero structure must be more clever. Chapter 15's perfect hash filter (Chapter 15) computes a perfect hash for a fixed A at construction time and stores only a fingerprint per element; a query for $x \in A$ lands on its fingerprint and is reported correctly with probability 1, while a query for $x \notin A$ may still collide. The set has to be known up front, and updates are expensive. Those costs buy the opposite asymmetry: certainty in the positive direction rather than the negative.

The trade-off: safety versus space

One-sided approximate sets are valuable because one direction of the answer is exact. If a Bloom filter reports $\bot$, the queried element is definitely not in the underlying set; downstream logic can act on the certain answer without further checks. An FPR-zero filter has the dual safety: a $\top$ report is certain, only $\bot$ needs follow-up. Many systems are built around exactly this asymmetric trust. Web caches use a Bloom filter to skip lookups for keys known to be absent; the certain $\bot$ avoids a slow disk read. Spell-checkers flag possibly misspelled words; the certain $\top$ on a known word avoids a distracting underline. In every such setting, the one-sided structure substitutes for an exact set precisely because the certain direction is the one the application

needs.

Two-sided sets give up the certain direction and recover space. Counting Bloom filters store small integer counters instead of bits to support deletions, paying for that with a nonzero ω. Cardinality estimators such as HyperLogLog and generic sublinear sketches are typically two-sided on derived predicates. The trade is honest: dropping one side of the asymmetry frees the construction to use space more aggressively.

A bit-budget comparison makes the size of the trade visible. By the optimal-k formula of Section 5.2, a Bloom filter at $\varepsilon = 0.01$ and $\omega = 0$ requires $m \approx 1.44\, n \log_2(1/\varepsilon) \approx 9.6\, n$ bits, since $\log_2(100)/\ln 2 \approx 9.58$. Chapter 8 will show that the information-theoretic floor for an FNR-zero approximate set at the same ε is $n \log_2(1/\varepsilon) = n \log_2 100 \approx 6.64\, n$ bits per element, so the Bloom filter pays a $9.6/6.64 \approx 1.44\times$ overhead above the floor that any FNR-zero construction must respect. That fixed overhead is the standard literature's $\log_2 e \approx 1.44$ constant, and chapter 14's Bernoulli Hash Function is the construction that closes the gap. A two-sided set is not bound by the same one-sided floor at ε, because it can split its error budget across both directions. Splitting the budget can purchase space below the FNR-zero floor, but it spends the certain $\perp$ that the Bloom filter buys with its extra bits. The qualitative shape is what matters here. One-sidedness is paid for in bits, and a construction that gives up the certain direction can in principle spend those bits elsewhere.

The perfect filter line

The two-dimensional rate plane $(\varepsilon, \omega) \in [0,1]^2$ parameterizes every approximate set. The one-sided sets occupy the two coordinate axes: the FNR-zero family on $\{\omega = 0,\, 0 < \varepsilon \leq 1\}$ and the FPR-zero family on $\{\varepsilon = 0,\, 0 < \omega \leq 1\}$. We call the union of these two segments the *perfect filter line*. The name is the convention of the underlying paper family; the word "line" is appropriate because each segment is a one-dimensional locus inside the two-dimensional plane. A one-sided approximate set has one degree of rate-freedom, not two. Two-sided sets fill the interior of the unit square; the exact set is the origin.

The perfect filter line is not a passive labeling. Chapter 6 (Chapter 6) will derive the union, intersection, and complement of two Bernoulli sets and show that

- the union of two FNR-zero sets is FNR-zero;
- the intersection of two FNR-zero sets is FNR-zero;

- the complement swaps the two axes, sending an FNR-zero set to an FPR-zero set and vice versa.

The FPR-zero side is in general not closed under the same operations: the union of two FPR-zero sets is FPR-zero only when both are exact on the same designated set, a fragile condition. The asymmetry parallels the construction-side asymmetry: FNR-zero is the natural default in many places, FPR-zero requires more careful design. Section 3.5 of chapter 3 already previewed the union-as-bit-OR identity for FNR-zero Bloom filters as a special case of the composition theorem; chapter 6 generalizes the calculation and makes the perfect filter line into an algebraic constraint.

Two further chapters take up the line as their concern. Chapter 14 (Chapter 14) develops the Bernoulli Hash Function and shows that within the FNR-zero family the bit-per-item cost is bounded below by an entropy formula, and the BHF achieves the bound. Chapter 15 (Chapter 15) takes the FPR-zero side seriously and constructs the perfect hash filter: an approximate set with $\varepsilon = 0$ on a designated set A and $\varepsilon > 0$ outside. Each chapter is one half of the same picture, sitting on one half of the perfect filter line.

Looking ahead

We have separated the approximate sets of Part II into three rate shapes: FNR-zero (the Bloom filter and its kin), FPR-zero (the perfect hash filter and its kin), and two-sided (counting filters and sketches). The shapes are not interchangeable. Each carries different safety guarantees, different space costs, and different behavior under algebraic operations on sets. Section 5.5 surveys the trajectory of the rest of the chapter and of Part II as a whole: the composition algebra of chapter 6, the classification measures of chapter 7, the entropy bound of chapter 8, and the space-optimal constructions of chapter 14 that close the gap the standard Bloom filter leaves on the table.

5.5 What Comes Next

The chapter has built one abstraction and worked one example. Section 5.1 lifted the noisy bit of Part I to a function $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$, the approximate set. Section 5.2 built the Bloom filter as the canonical realization of that abstraction and pinned down its rates. Section 5.3 placed the abstraction inside chapter 4's formal machinery: every query is a Bernoulli[bool]

instance, every n-query workload is a Kronecker product of per-block channels, and the per-set rates (ε, ω) are the only parameters that survive Axiom 2. Section 5.4 separated the shapes an approximate set can take and pointed at the perfect filter line. What remains to do in Part II is the algebra and the bounds that the abstraction underwrites. Four chapters follow, each picking up one strand from the present chapter and carrying it to its proper conclusion.

Chapter 6: the algebra of sets

Chapter 6 takes up set operations. Approximate sets have unions, intersections, complements, and differences; each operation has rates that follow from the rates of its inputs. The calculation is not new machinery; it is the composition theorem of Section 3.3 applied to the per-element predicates described in Section 5.3. The union of two Bloom filters by bit-OR gives an FPR of $1 - (1 - \varepsilon_1)(1 - \varepsilon_2)$, the intersection gives a cleaner $\varepsilon_1\varepsilon_2$, and the complement swaps the two coordinate axes. The chapter then asks which subfamilies of approximate sets are closed under these operations. The FNR-zero family of Section 5.4 is closed under union and intersection (one-sidedness is preserved in the same direction), but the complement maps it to the FPR-zero family on the opposite axis. The perfect filter line is a subalgebra under some operations and not others, and the asymmetry foreshadows chapter 15's constructive treatment of the FPR-zero side.

Chapter 7: classification measures

Chapter 7 measures quality. The two rates (ε, ω) are a complete intrinsic description, but practitioners often want derived quantities: positive predictive value, negative predictive value, recall, F1. Each is a function of the two rates and the workload's prevalence of in-set elements, and each depends on the workload in a way the rates themselves do not. A Bloom filter with $\varepsilon = 0.01$ achieves a PPV near 0.99 on a workload of mostly true positives and a PPV near 0.5 on a workload of mostly out-of-set queries. The chapter develops the closed forms, derives sampling distributions and confidence intervals for the estimators, and retires the deferral Section 1.4 flagged: the Wald interval was a placeholder, and the Wilson, Clopper-Pearson, and Jeffreys alternatives that handle the boundary properly are stated and applied. The chapter that began with a sawtooth in chapter 1's notebook ends here with the proper estimator vocabulary in place.

Chapter 8: information-theoretic bounds

Chapter 8 bounds the space. Any FNR-zero approximate set that achieves false-positive rate ε must consume at least $-\log_2 \varepsilon$ bits per stored element. The proof runs through Shannon entropy and the source-coding theorem; the statement is sharp. The standard Bloom filter of Section 5.2 achieves $\log_2 e \cdot \log_2(1/\varepsilon) \approx 1.44 \log_2(1/\varepsilon)$ bits per element under Theorem 5.2.1, which is roughly 44% above the floor. That 44% overhead is exactly the gap Section 5.4 named in its bit-budget comparison. The chapter's job is to make the gap precise. The construction that closes it is chapter 14's, not chapter 8's.

Chapter 14: the construction that closes the gap

Chapter 14 is the constructive payoff of the bound. The Bernoulli Hash Function (BHF) is an approximate set whose bit-per-item cost achieves the entropy lower bound up to lower-order terms. The 44% overhead the standard Bloom filter pays is the right kind of unfinished business: a gap that is real, that is calculable, and that admits a closed-form remedy. The BHF construction is one of the climactic results of the book; chapters 5 through 8 build the language in which the result can be both stated and appreciated, and chapter 14 then delivers it.

Chapter 15 is the FPR-zero counterpart on the other half of the perfect filter line. It takes the chapter 6 algebraic constraint seriously and constructs a representation that achieves $\varepsilon = 0$ on a designated set, paying the dual price the asymmetry of Section 5.4 predicts.

Closing

The bibliographic notes of Section 5.2's parent chapter, drafted next, name the antecedents of what we have done: Bloom's 1970 paper, the survey literature that grew up around it, and the Bernoulli-set papers that recast the data structure as an instance of a predicate algebra. The four chapters previewed above develop that algebra into a working theory of approximate-set operations, classification measures, information-theoretic bounds, and space-optimal constructions. Each picks up one of the strands the present chapter has set down; together they make the approximate set a first-class object of study.

Bibliographic Notes

Bloom's original paper. The data structure at the center of Section 5.2 is due to Bloom [Blo70], in a four-page *Communications of the ACM* note from 1970. Bloom's framing was operational rather than probabilistic: he was concerned with hyphenation dictionaries for a typesetting application, and the question was how much memory could be saved by accepting a small rate of mishyphenated words in exchange for not having to store the dictionary itself. The analysis in that paper already contains the optimal-k derivation that Theorem 5.2.1 reproduces in modern notation, including the $k = (m/n) \ln 2$ result and the resulting per-bit false-positive rate $(1/2)^k$ at the optimum. What this textbook adds is not the construction or its analysis but the framing: the Bloom filter as a particular construction of a function $U \to$ Bernoulli[bool], rather than as a clever bit-array trick for set membership. The two readings agree on every numerical prediction.

Survey and textbook context. For an entry into the broader literature on approximate-set data structures, two references are standard. Mitzenmacher and Upfal [MU17] devotes Chapter 5 to hashing, Bloom filters, and the balls-and-bins analysis that underlies both; the proof of the optimal-k result in that text follows the same calculus argument used in Section 5.2, though without the Bernoulli[bool] framing. Broder and Mitzenmacher [BM04] collects applications of Bloom filters across networking, distributed systems, and database query processing, organized by the kind of approximate-set operation each application requires. The survey is the natural follow-up reading for anyone who wants to see how widely the Bloom filter's $\omega = 0$ property gets used in practice; it also documents the variants (counting filters, scalable filters, compressed filters) that the present chapter mentions only in passing.

The set-as-predicate viewpoint. The framing of an approximate set as a function $U \to$ Bernoulli[bool], rather than as a data structure that happens to have an error rate, is the contribution of Towell [Tow26a]. Earlier literature, including Bloom [Blo70] and the broad sweep of Broder and Mitzenmacher [BM04], treats approximate sets as *data structures with error rates*: an array of bits plus a membership test, characterized by a single false-positive probability. The Bernoulli-sets reading inverts that emphasis. The object of study is the function itself, the per-query Boolean output is a Bernoulli[bool] instance in the sense of Towell [Tow26d], and the data structure is one of many possible realizations. Sections 5.1 and 5.3 develop

this framing as a continuation of Part I's atom-and-axiom construction; the algebra of Section 5.3 is the direct lift of the order-2 Bernoulli[bool] algebra of chapter 4 to the per-element setting.

One-sided and perfect-filter lineage. The one-sided / two-sided dichotomy of Section 5.4 has two ancestors in the literature. The FNR-zero shape is the Bloom filter's structural property, named as such in Bloom [Blo70] but not isolated as a category until the broader approximate-set literature accumulated multiple constructions sharing it. The FPR-zero shape—a *perfect filter* in the sense of Definition 5.4.1, on a designated set—has its constructive roots in the perfect-hashing literature opened by Carter and Wegman [CW79]. Carter and Wegman's universal hash families and the perfect hashing constructions that followed are what makes a fingerprint-per-element scheme realizable in the first place: without a perfect hash on A, the fingerprint table cannot guarantee $\varepsilon = 0$ on $x \in A$. Chapter 15's perfect hash filter (Chapter 15) is the modern descendant of that line. The chapter's exposition treats the Bernoulli-set characterization first, then the construction; the historical order is the opposite.

Forward bridge: chapters 14 and 15. Both chapter 14 (Chapter 14) and chapter 15 (Chapter 15) build on a single foundational manuscript: Towell [Tow26e], which derives the space-optimal Bernoulli hash filter (BHF) for both the unrestricted and FPR-zero-on-A regimes and proves the entropy lower bound that makes "optimal" precise. The construction in that paper realizes the FPR-zero shape of Definition 5.4.1 explicitly and shows that the Bloom filter is, on the relevant axis, off-optimal by a constant factor that is well understood. Readers who want the punch line of the chapters that follow—why *optimal* can be said precisely, and what the cost of optimality is in construction time and update support—will find it in that manuscript.

Exercises

1. ⋆ **Predicate-to-rates.** Let $U = \{1, 2, \ldots, 10\}$ and $A = \{2, 5, 7\}$. An approximate set $\mathsf{A}^{\pm}$ is queried once on every element of U and produces the following table of reports:

x	1	2	3	4	5	6	7	8	9	10
$\mathsf{A}^{\pm}(x)$	$\bot$	$\top$	$\top$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$

(a) Identify which queries hit in-set elements and which hit out-of-set elements; tabulate the four outcome counts (TP, FP, FN, TN) using A as ground truth.

(b) Compute the empirical false-positive rate $\hat{\varepsilon}$ as the fraction of out-of-set queries that report $\top$, and the empirical false-negative rate $\hat{\omega}$ as the fraction of in-set queries that report $\bot$. Be explicit about the denominators.

(c) State in one sentence why these two empirical rates are the natural sample-based estimators of the per-block parameters (ε, ω) that Section 5.3 attaches to $\mathsf{A}^{\pm}$.

2. $\star$ **Bloom filter parameters.** A Bloom filter is built with $m = 8000$ bits, $n = 1000$ inserted elements, and $k = 2$ hash functions.

(a) Compute ε from Equation (5.1), that is, $\varepsilon \approx (1 - e^{-kn/m})^k$ with the given (m, n, k). Report at least three significant figures.

(b) For the same (m, n), compute the optimal hash count $k^* = (m/n) \ln 2$ from Theorem 5.2.1. Round to the nearest integer if necessary and report both the real-valued optimum and the integer used.

(c) Evaluate (5.1) at $k = k^*$ and compare to the rate from (a). By what factor does the optimal k reduce the false-positive rate at this (m, n)?

3. $\star$ **One-sided check.** For each of the following data structures, state whether it is one-sided in ω ($\omega = 0$, $\varepsilon > 0$), one-sided in ε ($\varepsilon = 0$, $\omega > 0$), two-sided (both rates positive), or exact (both rates zero). Give a one-sentence justification per item drawn from the construction.

(a) A Bloom filter (Definition 5.2.1).

(b) An exact set-membership test on a hash table that stores every element of A verbatim.

(c) A cardinality estimator such as HyperLogLog used as a membership oracle by thresholding the estimated cardinality.

4. $\star\star$ **Notebook-tied Bloom filter empirics.** Open the chapter 5 companion notebook `python/notebooks/ch05-approximate-sets.ipynb`. Build a Bloom filter with $m = 10000$ bits and $n = 1000$ inserted random strings, and vary the hash count over $k \in \{1, 2, 3, \ldots, 15\}$.

(a) For each k, estimate the empirical false-positive rate $\hat{\varepsilon}(k)$ by querying a large pool of strings known to be absent from the inserted set and recording the fraction reported as $\top$.

(b) On the same axes, plot $\hat{\varepsilon}(k)$ and the theoretical curve $(1-e^{-kn/m})^k$ from Equation (5.1). Describe in one sentence how closely the two curves agree across the range of k.

(c) Read off the integer k that minimizes $\hat{\varepsilon}$ and compare to the optimum $k^* = (m/n)\ln 2$ predicted by Theorem 5.2.1. Do the empirical and theoretical optima agree to within the one-bin resolution of your sweep?

5. ⋆⋆ **Uniqueness of the optimal k.** The proof of Theorem 5.2.1 in Section 5.2 reduced the first-order optimality condition to

$$(1-u)\ln(1-u) \;-\; u\ln u \;=\; 0, \qquad u \in (0,1),$$

under the change of variable $u = e^{-kn/m}$, and observed by inspection that $u = 1/2$ solves it. The proof then claimed that $u = 1/2$ is the *only* solution in $(0,1)$, leaving the uniqueness to this exercise.

(a) Define $\phi(u) = (1-u)\ln(1-u) - u\ln u$ on $(0,1)$. Verify the symmetry $\phi(1-u) = -\phi(u)$, and confirm that $\phi(u) \to 0$ as $u \to 0^+$ and as $u \to 1^-$. Conclude that $\phi(1/2) = 0$ without further calculation.

(b) Compute $\phi'(u)$ and verify that $\phi'(u) = -\ln(1-u) - \ln u - 2$. Show that ϕ' attains its minimum at $u = 1/2$, where $\phi'(1/2) = 2\ln 2 - 2 < 0$, and tends to $+\infty$ at both endpoints. Conclude that ϕ' changes sign exactly twice on $(0,1)$, once on $(0,1/2)$ and once on $(1/2,1)$.

(c) Use the sign pattern of (b) and the boundary behavior of (a) to conclude that ϕ is positive throughout $(0,1/2)$ and negative throughout $(1/2,1)$, with the unique zero located at $u = 1/2$.

(d) Transfer the conclusion back to k via the change of variable $u = e^{-kn/m}$. Confirm that $k^* = (m/n)\ln 2$ is the unique interior critical point of $f(k) = (1-e^{-kn/m})^k$ on $(0,\infty)$, and combine with the second-derivative check in Section 5.2 to conclude that it is the global minimum.

6. ⋆⋆ **Predicate algebra: joint probability via Kronecker.** Let U have $|U| = 6$ elements and $A \subseteq U$ with $|A| = 2$. An approximate set

$\mathsf{A}^{\pm}$ has per-block rates $\varepsilon = 0.1$ and $\omega = 0.05$ in the deterministic-rates regime of Section 5.3.

(a) Fix three specific out-of-set elements $x_1, x_2, x_3 \in U \setminus A$. Compute the probability that $\mathsf{A}^{\pm}$ reports $\top$ on each of x_1, x_2, x_3 (three simultaneous false positives), assuming Axioms 1 and 2 are in force.

(b) Write the same probability as a single entry of the Kronecker power $Q^{\otimes 3}$, identifying which row and column of the per-query matrix $Q = \left(\begin{smallmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{smallmatrix}\right)$ each of the three factors reads.

(c) Justify the factorization in one or two sentences by naming the role of Theorem 4.4.1 (which gives the joint matrix as a Kronecker product) and Corollary 5.3.0.1 (which specializes the product to the two-parameter approximate-set setting).

7. ⋆⋆ **Bit-check independence counter-example.** The derivation of $\varepsilon \approx (1 - e^{-kn/m})^k$ in Section 5.2 treats the k bit-check events as independent. Equation (5.1) is therefore an approximation; the exact false-positive rate is in general different. This exercise constructs the smallest worked counter-example.

(a) Take $m = 4$ bits, $n = 1$ inserted element, $k = 2$ independent hash functions, each uniform over $\{1, 2, 3, 4\}$. Enumerate all $4 \times 4 = 16$ assignments of $(h_1(\text{inserted}), h_2(\text{inserted}))$ with equal probability $1/16$; for each assignment, write out which bits are set.

(b) Now consider a query on a fresh out-of-set element y. Its two hashes $(h_1(y), h_2(y))$ are also uniform over $\{1, \ldots, 4\}^2$, independent of $h_i(\text{inserted})$. Compute the exact probability that both $b[h_1(y)] = 1$ and $b[h_2(y)] = 1$, that is, the exact false-positive rate, by averaging over all $16 \times 16 = 256$ joint hash assignments.

(c) Compare the exact rate from (b) to the approximation (5.1), $\varepsilon \approx (1 - e^{-2 \cdot 1/4})^2 = (1 - e^{-1/2})^2 \approx 0.155$. State in one or two sentences why the two values differ and identify the source of the correlation: if $h_1(y) = h_2(y)$, the two "independent" bit-check events collapse into one.

8. ⋆⋆ **Two-sided vs one-sided space trade-off via the entropy floor.** Section 5.4 previews chapter 8's entropy floor: any FNR-zero approximate set at false-positive rate ε requires at least $-\log_2 \varepsilon$ bits per stored element. The standard Bloom filter, at its optimal k^* (Theorem 5.2.1), pays $\approx 1.44 \cdot \log_2(1/\varepsilon)$ bits per element.

(a) Compute the entropy floor $-\log_2 \varepsilon$ in bits per element for the three target rates $\varepsilon \in \{0.1, 0.01, 0.001\}$. Tabulate to one decimal place.

(b) Compute the Bloom filter's actual bit-per-element cost $1.44 \cdot \log_2(1/\varepsilon)$ for the same three targets.

(c) Report the absolute overhead in bits per element (Bloom cost minus entropy floor) at each target. Interpret in one sentence: as ε shrinks, does the absolute overhead grow, shrink, or stay constant, and what does that mean for the practical cost of one-sidedness at very low false-positive rates?

9. $\star\star\star$ **Counting Bloom filter sketch.** The standard Bloom filter (Definition 5.2.1) supports no deletions: clearing a bit on DELETE(x) would clear bits shared by other inserted elements and break their queries. A *counting Bloom filter* resolves this by replacing each bit with a small unsigned integer counter (typically 4 bits); INSERT increments the k chosen counters and DELETE decrements them; QUERY returns $\top$ if all k counters are positive.

   (a) Argue that DELETE is correct *provided* no counter ever overflows during INSERT and no counter is ever decremented for an element that was not previously inserted. Both conditions can fail in practice; identify the failure mode of each.

   (b) Sketch why a counting Bloom filter is in general two-sided rather than one-sided: counter overflow (saturation at the maximum value) and the use of small counters together introduce a positive false-negative rate. Counter overflow caps the count, so subsequent decrements may zero a counter that should still be positive, causing a query on an inserted element to return $\bot$.

   (c) Compare the space cost. A b-bit counter Bloom filter occupies bm bits, compared with m bits for the standard filter. For the typical choice $b = 4$, this is a $4\times$ overhead. Comment in one or two sentences on the trade-off: the counting filter buys an additional API (DELETE) at the price of $4\times$ space, structural two-sidedness, and a small overflow-induced loss.

10. $\star\star\star$ **Block-stability under non-uniform priors.** Proposition 5.3.1 of Section 5.3 invokes Axiom 2 to collapse the per-element rates $\varepsilon(x), \omega(x)$ to per-block rates ε, ω. The collapse depends on the assumption that

the random rate attached to each block is shared by every element in that block. Construct a scenario where this assumption fails.

(a) Take a learned classifier (a noisy approximate-membership oracle, in our framework) whose accuracy varies across the out-of-set block: queries on out-of-set elements with very low prior probability of confusion are answered correctly with rate $1 - \varepsilon_{\text{easy}} = 0.99$, while queries on near-the-boundary out-of-set elements achieve only $1 - \varepsilon_{\text{hard}} = 0.70$. Describe how this construction gives a $\varepsilon(x)$ that genuinely depends on x within the out-of-set block $B_2 = U \setminus A$.

(b) Argue informally that the construction violates the spirit of Axiom 2 even though the classifier itself is deterministic. (*Hint:* Axiom 2 says the per-block rate is a single random variable shared by every block member; here the per-element rates are distinct, so no single block rate fits.)

(c) State the immediate consequence: the Kronecker factorization of Corollary 5.3.0.1 no longer applies in its two-parameter form; the joint distribution over m queries must instead carry per-element rates and is no longer universe-size-independent. Briefly identify which workloads make the gap visible and which mask it (*hint:* a workload that samples uniformly from B_2 averages over the heterogeneity and looks like a single-rate workload; a workload that adversarially selects near-boundary out-of-set elements exposes the gap).

11. $\star\star\star$ **Forward to chapter 6: union FPR via the composition theorem.** Chapter 6 will derive the false-positive rate of the union of two approximate sets $\mathsf{A}_1^{\pm} \cup \mathsf{A}_2^{\pm}$ from the rates of the constituents. For two FNR-zero approximate sets with FPRs $\varepsilon_1, \varepsilon_2$, both representing the same underlying set A on the same universe, and assuming independent error events, the union (implemented by bit-OR on Bloom filters with disjoint hash families) has FPR

$$\varepsilon_{\text{union}} \;=\; 1 - (1 - \varepsilon_1)(1 - \varepsilon_2).$$

(a) Derive this expression from Theorem 3.3.1 of chapter 3. (*Hint:* for a fixed out-of-set element x, the union reports $\top$ if and only if at least one of the two constituent queries reports $\top$. The complement is that both report $\bot$, which under independence factors as $(1 - \varepsilon_1)(1 - \varepsilon_2)$.)

(b) Verify the formula on a numerical example: take $\varepsilon_1 = 0.10$ and $\varepsilon_2 = 0.05$ and compute $\varepsilon_{\text{union}}$. Confirm that the result lies between $\max(\varepsilon_1, \varepsilon_2)$ and $\varepsilon_1 + \varepsilon_2$, with the second-order term $\varepsilon_1\varepsilon_2$ accounting for the gap.

(c) Describe a concrete situation where the independence assumption breaks and the formula no longer applies. For instance, the two filters share the same hash family over the same universe; then the bit-OR event on the same out-of-set element x is perfectly positively correlated across the two filters whenever both filters set the same bits, and the union FPR collapses to a single-filter FPR rather than the composition-theorem prediction. State the qualitative effect on the union rate (over-estimate or under-estimate) and identify which structural property of the construction the formula presupposes.

Chapter 6

Composition of Sets

After this chapter you will be able to:

- Derive the error rates of $\cup, \cap, \setminus, \complement$ for approximate sets by applying the composition theorem of Theorem 3.3.1 to the predicate framing of Chapter 5.
- Recognize the cancellation phenomena that let composed sets escape the perfect-filter line.
- State the closure properties of the perfect-filter line and name which set operations preserve one-sidedness.

6.1 Union of Approximate Sets

Chapter 5 framed an approximate set $\mathsf{A}^{\pm}$ as a noisy membership predicate over a universe U, with two per-block error rates (ε, ω). The Bloom filter of Definition 5.2.1 was the canonical instance: a structurally one-sided approximate set with $\omega = 0$ and a tunable ε. The present section takes the first compositional step. Given two approximate sets $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ over the same universe, we ask what their union is and what error rates the union inherits. The answer is the composition theorem of Theorem 3.3.1 applied to the OR gate of Section 3.2, lifted from the per-bit level to the per-set level.

Definition

A classical union $A \cup B$ is characterized by its membership predicate: $\mathbb{1}_{A \cup B}(x) = \mathbb{1}_A(x) \vee \mathbb{1}_B(x)$. The set-builder reading is the same: $x \in A \cup B$ if and only if

$x \in A$ or $x \in B$. The membership test is a Boolean OR of two simpler membership tests. The predicate framing of Section 5.1 lifts this characterization directly to approximate sets.

Definition 6.1.1 (Union of approximate sets)**.** *Let* $\mathsf{A}^{\pm}, \mathsf{B}^{\pm} : U \to$ Bernoulli[bool] *be two approximate sets over the same universe* U*. Their* union $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *is the approximate set whose membership predicate is the Boolean OR of the two operand predicates: for every* $x \in U$*,*

$$(\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm})(x) \;=\; \mathsf{A}^{\pm}(x) \;\vee\; \mathsf{B}^{\pm}(x).$$

The union approximates the objective set $A \cup B$*, where* A *and* B *are the underlying objective sets that* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *represent.*

The definition is structural and it makes no commitment to a particular implementation. To evaluate the union's predicate at x, query $\mathsf{A}^{\pm}(x)$ and $\mathsf{B}^{\pm}(x)$, then take the OR of the two Boolean outputs. The union is therefore well-defined as soon as the two operand predicates are, and its rates will follow from the rates of the operands together with the OR-gate analysis already done in Chapter 3.

Two realizations on Bloom filters: predicate-OR and bit-OR-Bloom

Definition 6.1.1 commits to a predicate-level operation: evaluate $\mathsf{A}^{\pm}(x)$, evaluate $\mathsf{B}^{\pm}(x)$, take the Boolean OR of the two outputs. We call this realization *predicate-OR*. It keeps the two filters as separate objects and asks two queries per lookup. The proposition derived below (Proposition 6.1.1) and its formula $1-(1-\varepsilon_A)(1-\varepsilon_B)$ describe this construction and only this one.

When both operands are Bloom filters with the same parameters, a tempting operational shortcut presents itself. Suppose $\mathsf{A}^{\pm}$ is the Bloom filter $(b_1, h_1, \ldots, h_k)$ for a set A and $\mathsf{B}^{\pm}$ is the Bloom filter $(b_2, h_1, \ldots, h_k)$ for a set B, using the same bit-array size m and the same hash family $h_1, \ldots, h_k$. The bit-wise OR $b_1 \vee b_2$ is itself a length-m bit array, and one can query the combined bitmap with a single k-position lookup. Call this realization *bit-OR-Bloom*: one bitmap, one query, one returned verdict. It is a different operation from predicate-OR, and it gives a different FPR.

Both realizations preserve FNR-zero on FNR-zero operands. For bit-OR-Bloom: every element of A has its k bits set in b_1, hence in $b_1 \vee b_2$, and symmetrically for B, so every member of $A \cup B$ is reported $\top$ with certainty.

For predicate-OR: each operand correctly reports $\top$ on its inserted items, and the OR of any two outputs containing a $\top$ is $\top$.

The two realizations diverge on false positives. Bit-OR-Bloom is exactly a Bloom filter with parameters (m, k) holding the union of inserted items, so its FPR is

$$\varepsilon_{\text{bit-OR}} \approx \left(1 - e^{-k(n_A+n_B)/m}\right)^k, \tag{6.1}$$

the chapter 5 Bloom-filter formula evaluated at the combined item count $n_A + n_B$. This is materially different from the predicate-OR rate $1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$. At $m = 5000$, $k = 7$, $n_A = n_B = 500$, the per-filter rate is $\varepsilon \approx 0.0082$; predicate-OR gives ≈ 0.0163 while (6.1) gives ≈ 0.137, an order of magnitude higher. The chapter notebook measures both and recovers each formula within sampling noise.

The gap is structural. Predicate-OR asks $(\forall i : b_1[h_i(x)] = 1) \vee (\forall i : b_2[h_i(x)] = 1)$: one filter must have *all* k bits set for the query to pass. Bit-OR-Bloom asks $\forall i : (b_1[h_i(x)] = 1 \vee b_2[h_i(x)] = 1)$: at every queried position, *either* filter's bit suffices, and the responsible filter may differ position by position. Queries that fail predicate-OR can pass bit-OR-Bloom by combining bits from different sources, so bit-OR-Bloom is the more permissive operation and carries the larger FPR.

Predicate-OR is the union of approximate sets in the sense of Definition 6.1.1: the lift of $\vee$ at the predicate level, inheriting its rates from the composition theorem of Chapter 3. Bit-OR-Bloom is a different data-structure choice, the choice of inserting both sets into a single Bloom filter to save one bitmap and one k-position lookup per query, paid for by the higher FPR of (6.1). Bit-OR-Bloom also requires the two operand filters to share parameters and hash family; bit-wise OR between filters with different hash families produces a bit array whose query behavior does not correspond to any clean set-level operation. Predicate-OR is well-defined regardless. Exercises in this chapter and the chapter notebook revisit these distinctions.

False-positive rate

Proposition 6.1.1 (Union rate formulas)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be approximate sets of* $A, B \subseteq U$ *with per-block rates* $(\varepsilon_A, \omega_A)$ *and* $(\varepsilon_B, \omega_B)$ *respectively. Assume the error events of* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *at every query point* $x \in U$ *are mutually independent. Then the union* $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *has false-positive rate*

$$\varepsilon_{\cup} = 1 - (1 - \varepsilon_A)(1 - \varepsilon_B). \tag{6.2}$$

The false-negative rate $\omega_\cup$ depends on the prior distribution over the three regions $A \setminus B$, $B \setminus A$, $A \cap B$ of the objective union $A \cup B$; for a query distribution that draws x from each region with respective weights w_1, w_2, w_3 summing to one,

$$\omega_\cup \;=\; w_1\omega_A(1-\varepsilon_B) \;+\; w_2\omega_B(1-\varepsilon_A) \;+\; w_3\omega_A\omega_B. \tag{6.3}$$

We prove the two parts in turn.

FPR derivation. Fix a query point $x \notin A \cup B$. Since x is outside both operand sets, the correct verdict for the union at x is $\bot$, and the only way for $(\mathsf{A}^\pm \cup \mathsf{B}^\pm)(x) = \top$ to occur is for at least one of the two operand predicates to falsely report $\top$ at x. The two events $\mathsf{A}^\pm(x) = \top$ (given $x \notin A$) and $\mathsf{B}^\pm(x) = \top$ (given $x \notin B$) are exactly the per-operand false-positive events at x, with conditional probabilities ε_A and ε_B respectively. By the cross-set independence assumption, the two events are independent. We compute the union false-positive event as the complement of the joint non-event:

$$\begin{aligned}
\mathrm{P}\left((\mathsf{A}^\pm \cup \mathsf{B}^\pm)(x) = \top | x \notin A \cup B\right) &= 1 - \mathrm{P}\left(\mathsf{A}^\pm(x) = \bot,\, \mathsf{B}^\pm(x) = \bot | x \notin A,\, x \notin B\right) \\
&= 1 - \mathrm{P}\left(\mathsf{A}^\pm(x) = \bot | x \notin A\right)\, \mathrm{P}\left(\mathsf{B}^\pm(x) = \bot | x \notin B\right) \\
&= 1 - (1-\varepsilon_A)(1-\varepsilon_B).
\end{aligned}$$

The result holds for every $x \notin A \cup B$ with the same probability, so the union's false-positive rate is $\varepsilon_\cup = 1 - (1-\varepsilon_A)(1-\varepsilon_B)$, as claimed.

FNR derivation. A false negative of the union at $x \in A \cup B$ requires both operand predicates to report $\bot$. Whether each report is itself a false negative or a true negative depends on which of the three regions x comes from. Partition the objective union into the disjoint blocks

$$A \cup B \;=\; (A \setminus B) \sqcup (B \setminus A) \sqcup (A \cap B).$$

For $x \in A \setminus B$ we have $x \in A$ and $x \notin B$, so $\mathsf{A}^\pm(x) = \bot$ is a false negative of $\mathsf{A}^\pm$ with probability ω_A, while $\mathsf{B}^\pm(x) = \bot$ is a true negative of $\mathsf{B}^\pm$ with probability $1 - \varepsilon_B$. By independence the joint probability that both predicates report $\bot$ on this region is $\omega_A(1-\varepsilon_B)$. The region $B \setminus A$ contributes symmetrically $\omega_B(1-\varepsilon_A)$. For $x \in A \cap B$ both operand predicates must miss their respective member, contributing $\omega_A\omega_B$ by independence. Weighting the three regions by the prior w_1, w_2, w_3 on $A \cup B$ and applying the law of total probability yields (6.3).

The role of the prior. The union false-positive rate $\varepsilon_\cup$ is a closed-form polynomial in the two operand FPRs alone; no prior is required because every negative of $A \cup B$ falls into one and the same region (the complement of the union). The union false-negative rate, in contrast, mixes three different per-region rates, and the closed form depends on how often a query lands in each region. The weights w_i are determined by the cardinalities $|A \setminus B|, |B \setminus A|, |A \cap B|$ together with the prior over $A \cup B$. When the operand sets are disjoint the third region vanishes and the formula simplifies; when the operands coincide the first two regions vanish and only the $\omega_A \omega_B$ term survives. The general expression is the *marginal* rate over the union; chapter 7's classification measures will revisit it for confidence-interval purposes.

Worked example: union of two Bloom filters

Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two Bloom filters over the same universe with the same parameters, with operand rates

$$\varepsilon_A = 0.01, \qquad \omega_A = 0, \qquad \varepsilon_B = 0.02, \qquad \omega_B = 0.$$

Both operands are FNR-zero, in the sense of Definition 5.4.1: they sit on the FNR-zero axis of the perfect filter line. The union has

$$\varepsilon_\cup \;=\; 1 - (1 - 0.01)(1 - 0.02) \;=\; 1 - 0.99 \times 0.98 \;=\; 1 - 0.9702 \;=\; 0.0298.$$

The union FNR is $\omega_\cup = 0$, by direct inspection of (6.3) with $\omega_A = \omega_B = 0$: every term in the weighted sum vanishes. Both realizations of the union preserve FNR-zero: predicate-OR because the OR of two correct $\top$'s is $\top$, and bit-OR-Bloom because every member of either operand set still has all k bits set in the OR'd array. The arithmetic above, however, is the predicate-OR FPR: it composes the two operand rates via $1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$, the rate of the Boolean OR of two independent operand verdicts. The bit-OR-Bloom realization would give a different (larger) number through (6.1), since merging two filters into one bitmap inflates the load.

The increase from 0.02 (the worse operand) to 0.0298 is the price of the OR composition; the multiplicative second-order term $\varepsilon_A \varepsilon_B = 0.0002$ is small relative to the additive part, so for small operand rates the union FPR is well-approximated by the sum $\varepsilon_A + \varepsilon_B$.

Connection to the composition theorem

(6.2) is the composition theorem of Theorem 3.3.1 applied to the OR gate of Section 3.2 with $k = 2$ stages and $\eta_i = \varepsilon_i$. Chapter 3 §3.2 already analyzed

the OR-under-noise event at the per-bit level: the false positive of OR is the failure to read a $\perp$ from at least one input, and the probability of that joint failure factors into the product of per-input correctness probabilities $1 - \varepsilon_i$. Section 3.5 previewed the lift to sets; the present section completes it. The Boolean operation OR is the same at both levels and the algebra is the same; what changes is the interpretation of the rates. At the bit level ε_i is the per-bit crossover of an asymmetric channel; at the set level it is the per-block rate of an approximate set. The composition theorem governs both.

The connection is the central pedagogical claim of chapters 6 and later: every set operation has an underlying Boolean gate, every gate has its noise propagation rule from Chapter 3, and the set-level rate algebra is the Boolean noise algebra reread at the set scale. The remaining sections of this chapter walk through that correspondence for intersection, complement, and difference, then identify the regimes in which the composed rates leave the perfect filter line.

What comes next

Section 6.2 takes up intersection, complement, and difference. Intersection is the AND gate at the set level, with the dual structure of the union formulas: multiplicative FPR and an additive-style FNR. Complement is the NOT gate and swaps the two rates without further work. Set difference is intersection with the complemented second operand, and its rates fall out by composing the two formulas. Section 6.3 then turns to the regimes in which composition escapes the perfect filter line, when a chain of operations forces both rates positive even though each operand sat on an axis, and Section 6.4 restates the line as the formal subalgebra it really is.

6.2 Intersection, Complement, Difference

The union of Section 6.1 was the first compositional step. The Boolean algebra of classical sets has three other operations to account for: intersection, complement, and difference. Each lifts to approximate sets in the same way the union did. The membership predicate of the composed set is a Boolean combination of the operand predicates, and the rates of the composed set follow from the corresponding gate analysis of Chapter 3 together with the cross-set independence assumption already invoked in §6.1. This section takes the three operations in turn, derives the rate formulas, collects them

in a worked numerical example, and signposts the cancellation phenomena that §6.3 will scrutinize.

Intersection

A classical intersection $A \cap B$ has the AND of the two operand membership predicates: $\mathbb{1}_{A\cap B}(x) = \mathbb{1}_A(x) \wedge \mathbb{1}_B(x)$. The predicate framing carries over.

Definition 6.2.1 (Intersection of approximate sets)**.** *Let* $\mathsf{A}^{\pm}, \mathsf{B}^{\pm} : U \to$ Bernoulli[bool] *be two approximate sets over the same universe* U*. Their* intersection $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *is the approximate set whose membership predicate is the Boolean AND of the two operand predicates: for every* $x \in U$*,*

$$(\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm})(x) \;=\; \mathsf{A}^{\pm}(x) \wedge \mathsf{B}^{\pm}(x).$$

The intersection approximates the objective set $A \cap B$*, where* A *and* B *are the objective sets that* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *represent.*

The operational realization for two Bloom filters with the same parameters and the same hash family is the bit-wise AND of the two bit arrays, in the same spirit as the bit-wise OR for union. The bit-AND construction has a subtlety we will flag at the end of this subsection; the predicate-level definition above is unambiguous and is the one the rate algebra operates on.

Proposition 6.2.1 (Intersection rate formulas)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be approximate sets of* $A, B \subseteq U$ *with per-block rates* $(\varepsilon_A, \omega_A)$ *and* $(\varepsilon_B, \omega_B)$ *respectively. Assume the error events of the two operands at every query point* x *are mutually independent (the same cross-set independence assumption as Proposition 6.1.1). Then the intersection* $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *has false-negative rate*

$$\omega_{\cap} \;=\; 1 - (1 - \omega_A)(1 - \omega_B), \tag{6.4}$$

and false-positive rate

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \varepsilon_B \tag{6.5}$$

on negatives at which the two operand sets agree (the bulk regime; see below).

FNR derivation. Fix a query point $x \in A \cap B$. Both operand predicates should report $\top$. A false negative of the intersection occurs when at least one of the two reports $\bot$, that is, when the AND fails to be $\top$. We compute the false-negative event as the complement of the joint correct-report event:

$$\begin{aligned}
\mathrm{P}\left((\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm})(x) = \bot | x \in A \cap B\right) &= 1 - \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top,\, \mathsf{B}^{\pm}(x) = \top | x \in A,\, x \in B\right) \\
&= 1 - \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top | x \in A\right)\, \mathrm{P}\left(\mathsf{B}^{\pm}(x) = \top | x \in B\right) \\
&= 1 - (1 - \omega_A)(1 - \omega_B).
\end{aligned}$$

The probability is the same for every $x \in A \cap B$, so the intersection's false-negative rate is (6.4). The formula is the dual of the union FPR (6.2) with ω in place of ε, exactly as De Morgan duality leads us to expect.

FPR derivation, simplified. A false positive of the intersection requires both operand predicates to report $\top$ at an $x \notin A \cap B$. For $x \notin A$ and $x \notin B$ simultaneously, the two reports are false positives of the operands with conditional probabilities ε_A and ε_B, so by cross-set independence the joint event has probability $\varepsilon_A \cdot \varepsilon_B$, giving (6.5). The full intersection FPR mixes this contribution with two boundary regions where x lies in exactly one operand set; the resulting weighted average is bounded above by $\varepsilon_A + \varepsilon_B$ and below by $\varepsilon_A \cdot \varepsilon_B$, and the simplified form is exact when the two operand sets coincide on the negatives of the intersection. We adopt (6.5) as the working formula for the intersection FPR; the full mixture appears in the Proposition 6.1.1-style derivation that the companion composition paper records.

The multiplicative shape matters. The intersection FPR is the *product* of the operand FPRs, not a sum. For small operand rates the contrast with the union is sharp: two filters at $\varepsilon \approx 0.01$ give a union FPR near 0.02 but an intersection FPR near 0.0001. Intersection drives false positives exponentially toward zero with the number of independent filters; union inflates them roughly linearly. The dual fact for false negatives is that intersection's FNR adds in the same inclusion-exclusion sense the union's FPR did, while union's FNR is multiplicative across the regions where both operands could miss.

Complement

Complement is the simplest of the three operations to analyze.

Definition 6.2.2 (Complement of an approximate set)**.** *Let* $\mathsf{A}^{\pm} : U \to$ Bernoulli[bool] *be an approximate set of* $A \subseteq U$*. Its* complement $\complement\mathsf{A}^{\pm}$ *is the approximate set whose membership predicate is the Boolean NOT of* $\mathsf{A}^{\pm}$*'s predicate:* $(\complement\mathsf{A}^{\pm})(x) = \neg\mathsf{A}^{\pm}(x)$*. The complement approximates the classical complement* $\complement A = U \setminus A$*.*

Proposition 6.2.2 (Complement-swap of rates)**.** *If* $\mathsf{A}^{\pm}$ *has per-block rates* $(\varepsilon_A, \omega_A)$*, then its complement* $\complement\mathsf{A}^{\pm}$ *has rates*

$$\varepsilon_{\complement} = \omega_A, \qquad \omega_{\complement} = \varepsilon_A. \tag{6.6}$$

The result is the set-level reading of the NOT-under-noise calculation already done in Section 3.2 at the per-bit level, lifted from the noisy bit to the noisy predicate. We give the two-line proof.

A false positive of $\complement\mathsf{A}^{\pm}$ is the event that the truth says negative and the report says positive on the complement side: $x \in \complement A$ is false (so $x \in A$) while $\complement\mathsf{A}^{\pm}(x) = \top$ (so $\mathsf{A}^{\pm}(x) = \bot$). The combined event is $\mathsf{A}^{\pm}(x) = \bot$ given $x \in A$, which is a false negative of $\mathsf{A}^{\pm}$ at rate ω_A. So $\varepsilon_{\complement} = \omega_A$. Dually, a false negative of $\complement\mathsf{A}^{\pm}$ is the event $x \in \complement A$ true (so $x \notin A$) while $\complement\mathsf{A}^{\pm}(x) = \bot$ (so $\mathsf{A}^{\pm}(x) = \top$). The combined event is $\mathsf{A}^{\pm}(x) = \top$ given $x \notin A$, which is a false positive of $\mathsf{A}^{\pm}$ at rate ε_A. So $\omega_{\complement} = \varepsilon_A$. The two rates swap, as (6.6) records.

The same result was proven at the noisy-bit level in Section 3.2. The lifting to the set level uses the predicate framing of Chapter 5 and nothing else. NOT is its own inverse: applying complement twice returns the original rates, $(\varepsilon_A, \omega_A) \mapsto (\omega_A, \varepsilon_A) \mapsto (\varepsilon_A, \omega_A)$. The complement operation is an involution on the rate plane $[0,1]^2$, exchanging the two coordinate axes.

Difference

Set difference is intersection with the complemented second operand.

Definition 6.2.3 (Difference of approximate sets)**.** *For approximate sets $\mathsf{A}^{\pm}, \mathsf{B}^{\pm}$ over U, the* difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ *is the approximate set whose membership predicate is $\mathsf{A}^{\pm}(x) \wedge \neg\mathsf{B}^{\pm}(x)$; equivalently, $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm} = \mathsf{A}^{\pm} \cap \complement\mathsf{B}^{\pm}$. It approximates the classical difference $A \setminus B = A \cap \complement B$.*

Proposition 6.2.3 (Difference rate formulas)**.** *Under the same cross-set independence as Proposition 6.2.1, the difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ has false-positive rate*

$$\varepsilon_{\setminus} \;=\; \varepsilon_A \cdot \omega_B \tag{6.7}$$

on the bulk negative region, and false-negative rate

$$\omega_{\setminus} \;=\; 1 - (1 - \omega_A)(1 - \varepsilon_B). \tag{6.8}$$

The proof is compositional. The complement of $\mathsf{B}^{\pm}$ has rates $(\varepsilon_B^{\complement}, \omega_B^{\complement}) = (\omega_B, \varepsilon_B)$ by Proposition 6.2.2. Substitute these into the intersection formulas of Proposition 6.2.1 applied to $\mathsf{A}^{\pm}$ and $\complement\mathsf{B}^{\pm}$:

$$\varepsilon_{\setminus} \;=\; \varepsilon_A \cdot \varepsilon_B^{\complement} \;=\; \varepsilon_A \cdot \omega_B,$$

which is (6.7), and

$$\omega_{\setminus} \;=\; 1 - (1 - \omega_A)(1 - \omega_B^{\complement}) \;=\; 1 - (1 - \omega_A)(1 - \varepsilon_B),$$

which is (6.8).

The formulas mix the two operands' rates in a characteristic way: the difference FPR pairs ε_A with ω_B (the swapped complement-side FPR), and the difference FNR pairs ω_A with ε_B (the swapped complement-side FNR). Both operands' *both* rates appear in the answer; even an FNR-zero $\mathsf{A}^{\pm}$ inherits false negatives from the difference when $\mathsf{B}^{\pm}$ has a nonzero FPR, because elements of $A \setminus B$ that are false positives of $\mathsf{B}^{\pm}$ are wrongly excluded.

Worked numerical example

Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two Bloom filters over the same universe with the same parameters and operand rates

$$\varepsilon_A = 0.01, \qquad \omega_A = 0, \qquad \varepsilon_B = 0.02, \qquad \omega_B = 0.$$

Both operands are FNR-zero (Definition 5.4.1), so they sit on the FNR-zero axis of the perfect filter line. The intersection has

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \varepsilon_B \;=\; 0.01 \times 0.02 \;=\; 0.0002, \qquad \omega_{\cap} \;=\; 1 - (1-0)(1-0) \;=\; 0.$$

The intersection of two FNR-zero Bloom filters is itself FNR-zero, and its false-positive rate is smaller than either operand's by an order of magnitude (an absolute drop of nearly a factor of 50 relative to the worse operand). The contrast with the union's $\varepsilon_{\cup} = 0.0298$ of Section 6.1 is striking: composing two filters multiplicatively reduces the false-positive rate, while composing them disjunctively inflates it. The two operations are exact duals of one another, and the choice of which to use is dictated by what one wants the composed filter to assert.

The complement of $\mathsf{A}^{\pm}$ has rates $(\varepsilon_{\complement}, \omega_{\complement}) = (\omega_A, \varepsilon_A) = (0, 0.01)$. The complement has moved $\mathsf{A}^{\pm}$ off the FNR-zero axis and onto the FPR-zero axis: what was a one-sided false-positive machine is now a one-sided false-negative machine. The next section returns to this geometric observation.

The difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ has

$$\varepsilon_{\setminus} \;=\; \varepsilon_A \cdot \omega_B \;=\; 0.01 \times 0 \;=\; 0, \qquad \omega_{\setminus} \;=\; 1 - (1-0)(1-0.02) \;=\; 0.02.$$

The difference of two FNR-zero filters has dropped its false-positive rate to zero (because $\omega_B = 0$ kills the FPR product) and picked up a false-negative rate equal to ε_B. The structural reason is that $\mathsf{B}^{\pm}$'s false positives become misclassified exclusions in $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$: an element of $A \setminus B$ that $\mathsf{B}^{\pm}$ erroneously reports as a member of B will be filtered out by the difference even though it should be kept. The difference exemplifies the cancellation phenomena §6.3 will scrutinize.

A caveat on bit-AND of Bloom filters

The intersection of Definition 6.2.1 is a predicate-level operation: query $\mathsf{A}^{\pm}$, query $\mathsf{B}^{\pm}$, AND the two Boolean outputs. For two Bloom filters with the same parameters and the same hash family, this predicate intersection is the canonical definition and the rate formulas of Proposition 6.2.1 apply to it. The operational shortcut of bit-wise AND of the two underlying bit arrays is a different construction. The bit-AND of two Bloom filters gives a bit array whose query behaviour reports $\top$ when the AND of the bit arrays has 1 at every queried position; this is the same as both filters individually reporting $\top$, so on items inserted into *both* sets the bit-AND query matches the predicate AND. On items inserted into one set only, however, the bit-AND filter can still report $\top$ when both bits happen to be set in the AND'd array (one from the legitimate insertion, one from a coincidental hash collision of an insertion of the other set), and the resulting filter is not in general a Bloom filter for $A \cap B$. The bit-AND is an operational shortcut that coincides with the predicate intersection in some regimes and not others; the chapter notebook explores the discrepancy empirically, and we will not rely on bit-AND in the rate analyses to come.

What comes next

The four operations of §6.1 and §6.2 (union, intersection, complement, difference) suffice to express every Boolean combination of approximate sets, with closed-form rate formulas derived from the composition theorem of Theorem 3.3.1 applied to the corresponding Boolean gate. Section 6.3 takes up the phenomena that the difference example already foreshadowed: composition can move an operand off the perfect filter line, can swap which side of the line a one-sided set sits on, and can introduce both rates as nonzero even when each input had one rate set to zero by construction. Section 6.4 then restates the line itself as a formal subalgebra: the FNR-zero axis is closed under union and intersection, the FPR-zero axis is closed dually, and the two axes together form a non-convex set whose closure properties this chapter's operations either preserve or break.

6.3 Cancellation and the Loss of One-Sidedness

The operand rates of Section 6.1 and Section 6.2 were drawn from one-sided approximate sets sitting on the perfect filter line of Definition 5.4.1. The union of two FNR-zero Bloom filters stayed FNR-zero; the intersection of

two FNR-zero Bloom filters stayed FNR-zero; the worked examples landed back on the same axis the operands came from. This section asks the natural follow-up question. Are the operations of chapter 6 always axis-preserving in this way, or can a chain of operations take an operand off the line entirely? The answer is that composition can both move a one-sided set to the other axis and force both rates positive at once. The phenomenon is unavoidable once the complement operator enters the picture and intersection involves a mixture of axes, and §6.4 will reread it as a closure question about the perfect filter line.

Complement swaps which axis

The simplest escape from a single axis is the complement operation. Proposition 6.2.2 stated the swap at the rate level: if $\mathsf{A}^{\pm}$ has rates $(\varepsilon_A, \omega_A)$, then $\complement\mathsf{A}^{\pm}$ has rates $(\varepsilon_{\complement}, \omega_{\complement}) = (\omega_A, \varepsilon_A)$. For a one-sided operand the consequences are immediate.

Proposition 6.3.1 (Complement swaps the perfect-filter axis)**.** *Let $\mathsf{A}^{\pm}$ be a one-sided approximate set in the sense of Definition 5.4.1. Then $\complement\mathsf{A}^{\pm}$ is also one-sided, but on the other axis: if $\mathsf{A}^{\pm}$ is FNR-zero then $\complement\mathsf{A}^{\pm}$ is FPR-zero, and if $\mathsf{A}^{\pm}$ is FPR-zero then $\complement\mathsf{A}^{\pm}$ is FNR-zero.*

Proof. By Proposition 6.2.2 the complement sends $(\varepsilon_A, \omega_A)$ to $(\omega_A, \varepsilon_A)$. If $\mathsf{A}^{\pm}$ is FNR-zero then $\omega_A = 0 < \varepsilon_A$, so the swap produces $(\varepsilon_{\complement}, \omega_{\complement}) = (0, \varepsilon_A)$ with $\varepsilon_{\complement} = 0 < \omega_{\complement}$: the complement is FPR-zero. The FPR-zero case is symmetric. □

Geometrically the rate plane has two coordinate axes and the complement operator exchanges them. A Bloom filter with $\varepsilon = 0.01$ and $\omega = 0$ sits at the point $(0.01, 0)$ on the FNR-zero axis. Its complement sits at the point $(0, 0.01)$ on the FPR-zero axis. The two structures encode the same information in dual senses. The Bloom filter is certain about absence and uncertain about presence, and its complement is certain about presence and uncertain about absence. A single application of complement walks across the rate plane from one axis to the other; nothing in between is visited because the operation is a discrete reflection rather than a continuous deformation.

Worked example: complement of a Bloom union

The first compound escape combines union and complement. Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two FNR-zero Bloom filters with operand rates

$$\varepsilon_A = 0.01, \qquad \omega_A = 0, \qquad \varepsilon_B = 0.02, \qquad \omega_B = 0,$$

the same numerical example as Section 6.1 and Section 6.2. The union $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ inherits both rates from Proposition 6.1.1: it is FNR-zero with

$$\varepsilon_{\cup} \;=\; 1 - (1 - 0.01)(1 - 0.02) \;=\; 0.0298, \qquad \omega_{\cup} \;=\; 0.$$

Now form the complement $\complement(\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm})$ and apply Proposition 6.2.2. The complement swaps the union's two rates:

$$\varepsilon' \;=\; \omega_{\cup} \;=\; 0, \qquad \omega' \;=\; \varepsilon_{\cup} \;=\; 0.0298.$$

The original two operands sat on the FNR-zero axis. The union stayed on that axis. The complement of the union moved to the FPR-zero axis. A two-step chain (union, then complement) has walked a one-sided approximate set from one half of the perfect filter line to the other. The composed structure is still one-sided, and still has only one degree of rate-freedom, but the certain direction has flipped. Where the union answered "is x in A or in B?" and reported $\bot$ with certainty, the complement of the union answers "is x outside both A and B?" and reports $\top$ with certainty. The rate algebra has done the bookkeeping for us; the qualitative content of the structure has changed.

Worked example: intersection with a complemented operand

The second compound escape composes intersection with a single complement and produces both rates positive at once. Let $\mathsf{A}^{\pm}$ again be FNR-zero with $\varepsilon_A = 0.01, \omega_A = 0$, and let $\mathsf{B}^{\pm}$ be FNR-zero with $\varepsilon_B = 0.02, \omega_B = 0$. The complement $\complement\mathsf{B}^{\pm}$ has rates $(0, 0.02)$ by Proposition 6.2.2; it sits on the FPR-zero axis. The mixed intersection $\mathsf{A}^{\pm} \cap \complement\mathsf{B}^{\pm}$ asks both operands to report $\top$ on points of $A \cap \complement B = A \setminus B$; this is the difference of Definition 6.2.3, which the intersection-with-complement decomposition makes explicit.

Apply Proposition 6.2.1 to the two operands $\mathsf{A}^{\pm}$ with rates $(\varepsilon_A, \omega_A) = (0.01, 0)$ and $\complement\mathsf{B}^{\pm}$ with rates $(\omega_B, \varepsilon_B) = (0, 0.02)$:

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \omega_B \;=\; 0.01 \times 0 \;=\; 0,$$

$$\omega_{\cap} \;=\; 1 - (1 - \omega_A)(1 - \varepsilon_B) \;=\; 1 - (1 - 0)(1 - 0.02) \;=\; 0.02.$$

The composition has landed at $(0, 0.02)$ on the FPR-zero axis in this specific arithmetic; both numbers came out as integer-times-zero products of the FNR-zero structure of the operands. The point is not that the result is generic; the point is that the moment one operand's FNR is genuinely

positive, both output rates also become positive. Take the same composition with $\varepsilon_A = 0.01$, $\omega_A = 0$ and $\complement\mathsf{B}^{\pm}$ having rates $(\varepsilon_B^{\complement}, \omega_B^{\complement}) = (\omega_B, \varepsilon_B) = (0.005, 0.02)$ (coming from a two-sided $\mathsf{B}^{\pm}$ with rates $(0.02, 0.005)$):

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \varepsilon_B^{\complement} \;=\; 0.01 \times 0.005 \;=\; 5 \times 10^{-5},$$

$$\omega_{\cap} \;=\; 1 - (1 - \omega_A)(1 - \omega_B^{\complement}) \;=\; 1 - 1 \times 0.98 \;=\; 0.02.$$

Both rates are now strictly positive. The composition has left both axes and landed in the two-sided interior of the rate plane. The operands did not need to be far from the line for this to happen; even an arbitrarily small leak in the FNR-zero structure of one operand produces a two-sided composed set.

Proposition 6.3.2 (Mixed-axis intersection escapes the line)**.** *Let $\mathsf{A}^{\pm}$ be FNR-zero with $\varepsilon_A > 0$ and let $\mathsf{B}^{\pm}$ be FPR-zero with $\omega_B > 0$. Then $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ generically has both rates strictly positive: $\varepsilon_{\cap} = 0$ and $\omega_{\cap} = \omega_B > 0$ in one boundary regime, while a two-sided $\mathsf{B}^{\pm}$ with any positive ε_B in place of the strict FPR-zero gives $\varepsilon_{\cap} = \varepsilon_A \varepsilon_B > 0$ and $\omega_{\cap} = 1 - (1 - \omega_B) > 0$. The line is not preserved by this composition.*

Proof. Substitute the operand rates into Proposition 6.2.1. The boundary case has $\varepsilon_B = 0$, giving $\varepsilon_{\cap} = \varepsilon_A \cdot 0 = 0$ and $\omega_{\cap} = 1 - (1 - 0)(1 - \omega_B) = \omega_B$. The non-boundary case has $\varepsilon_B > 0$, giving the two positive expressions above. In neither case does the result fall on the FNR-zero or FPR-zero axis of the operands. □

The escape, viewed geometrically

The rate plane $[0, 1]^2$ has two distinguished one-dimensional loci: the FNR-zero axis (the segment with $\omega = 0$ and $\varepsilon > 0$) and the FPR-zero axis (the segment with $\varepsilon = 0$ and $\omega > 0$). Their union is the perfect filter line. Three classes of compositional behavior arise.

Axis-preserving operations. Union of two FNR-zero sets stays FNR-zero. Intersection of two FNR-zero sets stays FNR-zero. Symmetric statements hold on the FPR-zero side. These are the compositions in which both operands sit on the same axis and the rate algebra returns a result on the same axis. The operations of Proposition 6.1.1 and Proposition 6.2.1 drive false positives and false negatives toward zero on the operand's axis without ever "leaking" the other rate.

Axis-swapping operations. Complement of a one-sided set is one-sided on the other axis (Proposition 6.3.1). The composition does not stay on

either axis individually but stays on the line as a set: it walks across the line, from one half to the other, in a single discrete step.

Axis-leaving operations. Intersection of an FNR-zero set with an FPR-zero set is generically two-sided (Proposition 6.3.2). Difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$, being intersection with the complemented second operand, is two-sided unless one of the operands has both rates zero or both operands sit on the same axis after the complement. These are the compositions that leave the line entirely; the composed rate lives in the interior of the rate square, with both directions of report now uncertain.

The three classes exhaust the qualitative behavior of the chapter 6 operations on the perfect filter line. The line is closed under axis- preserving operations, mapped to itself by axis-swapping operations, and exited by axis-leaving operations. Section 6.4 will state this classification as a closure proposition; for now it is enough that we have identified the qualitative regimes and connected each one to a specific composition.

Why this matters

The cancellation phenomena are not abstract. Three downstream chapters depend on whether a composed set sits on the perfect filter line.

Chapter 7 develops PPV and NPV from the FPR and FNR. A one-sided structure simplifies the formulas: an FNR-zero structure has NPV $= 1$ on its domain, and the confidence-interval treatment reduces to the one rate that is nonzero. A two-sided structure exercises the full machinery, with posterior weight on both directions of report and intervals on both rates.

Chapter 14 develops the Bernoulli Hash Function and the entropy-floor argument for FNR-zero approximate sets. The information-theoretic lower bound is $-\log_2 \varepsilon$ bits per element at false-positive rate ε, and the BHF achieves this floor up to a small additive constant. A compositional chain that leaves the axis puts the result outside the regime the bound measures; the resulting two-sided structure would need a separate analysis to assess space-optimality.

Chapter 15 develops the perfect hash filter and achieves FPR-zero on a designated input set A, paying with a construction-time perfect-hash computation. The chapter operates entirely on the FPR-zero axis. Composing a perfect hash filter with a Bloom filter via mixed-axis intersection produces a two-sided structure by Proposition 6.3.2, and the certain $\top$ answer is lost.

The pattern across chapters 7, 14, and 15 is the same. Each chapter is the constructive realization of one half of the perfect filter line at the lowest space cost, and each pays a real penalty when a compositional chain forces

the result off the line. Composition discipline is the price of staying near the floor.

What comes next

Section 6.4 states the perfect filter line as a formal definition and collects the closure results of this section into a coherent algebraic statement. The FNR-zero axis is closed under union and intersection but not under complement; the FPR-zero axis is closed dually; the union of the two axes is closed under no single operation but is mapped to itself by axis-swapping operations and exited by mixed-axis intersection. The section organizes the classification, and chapters 7, 14, and 15 then take up the consequences of staying on the line or leaving it.

6.4 The Perfect Filter Line

Section 5.4 named the perfect filter line as the union of the two coordinate axes of the rate plane and previewed its relevance to chapter 6. The composition formulas of Section 6.1 and Section 6.2, together with the cancellation classification of Section 6.3, have given us everything required to read the line as an algebraic object: a distinguished subset of $[0,1]^2$ that some operations preserve, some move within, and some leave. This section collects the closure results into a single statement, makes the geometric structure of the line explicit, and points forward to the two chapters that take each half of the line as their constructive target.

Definition

Definition 6.4.1 (The perfect filter line)**.** *The* perfect filter line *is the subset of the rate plane* $[0,1]^2$ *consisting of points with at least one coordinate equal to zero. Explicitly, the line is the union of two coordinate axes:*

$$\mathcal{L} \;=\; \mathcal{L}_\omega \cup \mathcal{L}_\varepsilon,$$

where the FNR-zero axis *is* $\mathcal{L}_\omega = \{(\varepsilon, 0) : 0 < \varepsilon \leq 1\}$ *and the* FPR-zero axis *is* $\mathcal{L}_\varepsilon = \{(0, \omega) : 0 < \omega \leq 1\}$*. An approximate set* lies on the line *when its rate point* (ε, ω) *belongs to* $\mathcal{L}$*.*

The two axes meet only at the origin $(0,0)$, which is the exact-set point and is excluded from $\mathcal{L}$ by the strict inequalities in the definition. An exact set is not an approximate set, in the sense that one of its two rates is not

merely zero but unmeasured: the construction does not face the trade-offs the rate plane parameterizes. The perfect filter line is the set of approximate sets that achieve certainty in exactly one direction of report; its origin neighbor achieves certainty in both, at the cost of the space savings that motivated approximate sets in the first place.

Closure properties of the FNR-zero axis

The composition results of the previous sections specialize cleanly when one or both operands sit on $\mathcal{L}_\omega$. We collect the four cases.

Proposition 6.4.1 (FNR-zero axis closure)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be FNR-zero approximate sets, so* $\omega_A = \omega_B = 0$ *and the operand rate points lie on* $\mathcal{L}_\omega$*. Then:*

1. $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *is FNR-zero, with* $\omega_\cup = 0$ *and* $\varepsilon_\cup = 1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$*.*
2. $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *is FNR-zero, with* $\omega_\cap = 0$ *and* $\varepsilon_\cap = \varepsilon_A \varepsilon_B$*.*
3. $\complement\mathsf{A}^{\pm}$ *is FPR-zero, with rates* $(0, \varepsilon_A)$*; the complement maps the FNR-zero axis to the FPR-zero axis pointwise.*
4. $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ *is FNR-zero when* $\mathsf{B}^{\pm}$ *is in addition FPR-zero (so* $\varepsilon_B = 0$ *also), giving rates* $(0, \omega_B \cdot 0$ *terms vanish); otherwise the difference generically leaves the line.*

In short, $\mathcal{L}_\omega$ *is closed under union and intersection, mapped to* $\mathcal{L}_\varepsilon$ *by complement, and only closed under difference under a strong condition on the second operand.*

Proof. Parts (1) and (2) substitute $\omega_A = \omega_B = 0$ into Proposition 6.1.1 and Proposition 6.2.1. The union FNR formula (6.3) of Section 6.1 becomes $w_1 \cdot 0 \cdot (1 - \varepsilon_B) + w_2 \cdot 0 \cdot (1 - \varepsilon_A) + w_3 \cdot 0 \cdot 0 = 0$, and the intersection FNR formula (6.4) becomes $1 - (1 - 0)(1 - 0) = 0$. The two FPR formulas follow without modification because they do not involve the FNRs of the operands.

Part (3) is the special case of Proposition 6.2.2 with $\omega_A = 0$. The swap sends $(\varepsilon_A, 0)$ to $(0, \varepsilon_A)$, which is a point on $\mathcal{L}_\varepsilon$.

Part (4) substitutes the operand rates into Proposition 6.2.3. With $\omega_A = \omega_B = 0$ the difference FNR formula becomes $\omega_\setminus = 1 - (1 - 0)(1 - \varepsilon_B) = \varepsilon_B$, which is positive whenever $\varepsilon_B > 0$, so the difference leaves $\mathcal{L}_\omega$. Imposing $\varepsilon_B = 0$ in addition makes $\mathsf{B}^{\pm}$ a perfect filter on $\mathcal{L}_\varepsilon$, the difference FNR collapses to 0, and $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ reduces to an exact-set test. □

Union and intersection are the two operations that close $\mathcal{L}_\omega$ on its own; complement closes the line $\mathcal{L}$ as a whole by exchanging the two axes; and difference is conditionally closed, requiring the second operand to be a perfect filter on the opposite axis. The cancellation analysis of Section 6.3 already recorded the geometric reason difference fails: it is intersection with a complemented operand, and the intermediate complement walks the second operand across the line.

Closure properties of the FPR-zero axis

The symmetric statement for $\mathcal{L}_\varepsilon$ follows from $\mathcal{L}_\omega$ by applying the complement-swap to both operands of any composition and unswapping the result. We record the companion proposition without re-deriving each case.

Proposition 6.4.2 (FPR-zero axis closure)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be FPR-zero approximate sets, so* $\varepsilon_A = \varepsilon_B = 0$*. Then* $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *has* $\varepsilon_\cup = 0$*, and* $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *has* $\varepsilon_\cap = 0$*; both compositions are FPR-zero. The complement of an FPR-zero set is FNR-zero, mapping* $\mathcal{L}_\varepsilon$ *to* $\mathcal{L}_\omega$ *pointwise. The difference* $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ *is FPR-zero only under a strong condition on the second operand and is otherwise two-sided.*

The proof is by complement-swap from Proposition 6.4.1: any FPR-zero set is the complement of an FNR-zero set, the rate-plane algebra is symmetric under coordinate exchange, and the conclusions transpose accordingly.

The algebraic shape of the line

The line $\mathcal{L}$ is a non-convex subset of $[0,1]^2$: the midpoint of any segment from a point $(\varepsilon, 0)$ on $\mathcal{L}_\omega$ to a point $(0, \omega)$ on $\mathcal{L}_\varepsilon$ is $(\varepsilon/2, \omega/2)$, which lies in the interior of the rate square and not on either axis. The two axes meet only at the excluded origin, so $\mathcal{L}$ is a wedge of two open segments rather than a connected curve; it has two "arms" meeting at the corner the definition excises. The shape matters because no convex combination of operations preserves the line: mixing one operand from each axis produces a point off both, which is exactly the conclusion Proposition 6.3.2 reached in Section 6.3.

This is the formal sense in which the perfect filter line is a *subalgebra* of the rate plane: closed under a specific subset of the chapter 6 operations, exited by the remaining operations, and internally structured into two halves that the complement operator swaps. Same-axis operations compose freely without leaving the line; cross-axis operations require care, because their

composed structure is no longer a perfect filter and the chapter 14 and chapter 15 space guarantees do not apply.

Forward to chapters 14 and 15

Two later chapters take the two halves of the line as their constructive concern. The forward references close the picture of why the line matters for the rest of Part II.

Chapter 14 develops the Bernoulli Hash Function, the constructive realization of an approximate set on $\mathcal{L}_\omega$ at the lowest possible space cost. The information-theoretic floor of chapter 8 gives the minimum number of bits per element required to represent an FNR-zero approximate set at false-positive rate ε; the BHF achieves the floor up to a small additive constant, closing the $\log_2 e \approx 1.44$ multiplicative overhead the standard Bloom filter of chapter 5 leaves above the floor on the same axis. The closure results of this section then govern composed BHFs: union and intersection of BHFs remain on $\mathcal{L}_\omega$ at rates the chapter 6 formulas predict, so the entropy floor extends to the composition.

Chapter 15 develops the perfect hash filter, the constructive realization on $\mathcal{L}_\varepsilon$ for a designated input set A. A perfect-hash computation at build time stores a fingerprint per element; a query for $x \in A$ lands on its fingerprint and is reported correctly with probability one, while a query for $x \notin A$ may still hash to a stored fingerprint. The construction sits on the FPR-zero axis and stays there under same-axis union and intersection by Proposition 6.4.2. Mixed-axis composition with a BHF, by Proposition 6.3.2, produces a two-sided structure off the line and the FPR-zero certainty is lost; the designer must respect the closure constraints to retain the constructive guarantees of either chapter.

6.5 What Comes Next

The chapter has fixed the algebra. Section 6.1 and Section 6.2 gave closed-form rates for union, intersection, complement, and difference of approximate sets. Section 6.3 classified the compositions that escape the perfect filter line. Section 6.4 collected the closure results into a single subalgebra statement and named the two halves of the line. What we have not yet done is connect those rates to the two questions that motivate the rest of Part II: how good is an approximate set when measured against a workload, and how small can one be made? The next two chapters take up each question in turn.

Chapter 7: rates feed into classification measures

Chapter 7 takes the per-block rates (ε, ω) this chapter computed and turns them into the workload-dependent quality measures practitioners actually report: positive predictive value, negative predictive value, F_1, and the recall-precision pair. Each is a function of (ε, ω) and the prevalence of in-set elements in the query stream. The composition formulas of this chapter feed those derivations directly. A union of two FNR-zero Bloom filters at output FPR $\varepsilon_\cup$, by Proposition 6.4.1, is itself FNR-zero; chapter 7's PPV expression in $\varepsilon_\cup$ and the workload prevalence then gives the PPV for the composed structure with no additional work.

Chapter 7 also retires the deferral Section 1.4 flagged in Part I. The Wald confidence interval for a binomial rate was a working placeholder, and the chapter introduces the Wilson, Clopper-Pearson, and Jeffreys alternatives that handle the boundary at $\varepsilon = 0$ or $\varepsilon = 1$ properly. The estimator vocabulary chapter 1 left incomplete is finished there.

Chapter 8: rates have a space-cost analog

Chapter 8 bounds how compact an approximate set can be. Every operation of this chapter has a space-cost analog: the entropy of an FNR-zero set with false-positive rate ε is $-\log_2 \varepsilon$ bits per stored element, the entropy of a composed structure is the entropy of its rate point, and the chapter 6 closure results on the FNR-zero axis then bound the bits per element a composition can possibly cost. The floor governs compression along the axis; Chapter 14 achieves it constructively with the Bernoulli Hash Function, closing the standard Bloom filter's $\approx 44\%$ overhead. The composition theorem of chapter 3 returns in chapter 8 dressed as the bit-budget arithmetic for chained approximate sets, and the chapter 6 formulas this section closes are the operations the bit budget applies to.

Closing

The bibliographic notes that follow name the antecedents of the set-operation framework: the Bernoulli composition manuscript that collected the closed-form rates, Mitzenmacher and Upfal for the survey context of Bloom-filter union and intersection, and the chapter 3 treatment of De Morgan under noise that the complement-swap result specializes. Chapter 7 then opens the second half of Part II.

Bibliographic Notes

The composition framework. The rate-propagation formulas of Sections 6.1 and 6.2 are the set-level instances of a more general framework developed in Towell [Tow26c]. That manuscript proves the rate-propagation laws for the full algebra of approximate sets—union, intersection, complement, difference, and the symmetric difference—under independence of the constituent Bernoulli[bool] outputs, and catalogues which compound operations preserve and which destroy one-sidedness. The chapter's contribution is to derive each rule constructively from the predicate algebra of Section 5.3 and the AND/OR/NOT rules of Section 3.2, rather than stating the formulas as a fait accompli. The composition manuscript treats the rules at the level of joint (ε, ω) pairs; chapter 6 unpacks the proofs and connects each rule back to its Boolean-algebra ancestor in chapter 3.

Bloom filter union and intersection. The well-known fact that two Bloom filters of the same length and the same hash family admit a bitwise union (logical OR of the bit arrays) and a bitwise intersection (logical AND) is folklore by now, but the analysis of what the resulting filter *computes*—and at what error rate—is treated systematically in Mitzenmacher and Upfal [MU17], Chapter 5, and surveyed across applications in Broder and Mitzenmacher [BM04]. The union case is clean: the bitwise OR of two filters for sets A and B is exactly a filter for $A \cup B$, with $\omega = 0$ preserved and a slightly elevated ε that chapter 6's Proposition 6.1.1 accounts for. The intersection case is famously subtle: the bitwise AND of two filters does *not* in general produce a filter for $A \cap B$, because the bit pattern of an AND filter admits ghost elements not in either input. Chapter 6 sidesteps the bitwise-AND trap by treating intersection at the predicate level instead, which gives the correct rate without committing to a specific representation. Readers who want the bitwise variant and its diagnostic issues should consult the survey directly.

De Morgan under noise. The complement-swap result of Proposition 6.2.2—that complementing an approximate set exchanges its FPR and FNR—is the set-level shadow of the Boolean-algebra rule worked out in Section 3.2: NOT swaps the two error rates of a noisy bit. Classical De Morgan reads $\complement(A \cup B) = \complement A \cap \complement B$, an identity between sets. Under noise the identity at the set level still holds, but the error rates of the two sides traverse the rate plane along different paths: one side touches the FNR-zero axis if both inputs are FNR-zero, the other side touches the FPR-zero axis after the

swap. The "different paths, same destination" phenomenon is what makes the perfect-filter-line classification of Section 6.4 informative: it tells you which compound operations stay on a one-sided axis and which mixed operations are forced into the interior. This chapter's treatment is the set-algebra rephrasing of the noisy-bit De Morgan worked through in chapter 3.

Perfect-filter literature and the bridge forward. The perfect-filter line of Section 6.4—the locus of one-sided sets in the rate plane (Section 5.4)—is where chapter 6 lands its main classification result. The line itself is a chapter 5 construct, but the question of *which* compound operations preserve membership on the line is the chapter 6 contribution. The constructive side of the story, how to actually build a set whose rates sit exactly on the FPR-zero axis, has its roots in the perfect-hashing literature opened by Carter and Wegman [CW79] and is taken up in full in chapter 15 (Chapter 15), which builds on Towell [Tow26e] for the space-optimal construction. The Bloom filter of Bloom [Blo70] occupies the opposite axis; it has been the canonical one-sided approximate set since 1970, and its algebraic behaviour under union and intersection is exactly what this chapter's closure analysis formalizes. Chapter 6 sits between the predicate-algebra reading of chapter 5 and the constructive perfect-filter machinery of chapter 15: the algebra of operations, divorced from any specific representation.

Exercises

1. ⋆ **Union rates for two FNR-zero Bloom filters.** Two FNR-zero Bloom filters on the same universe with the same parameters have operand rates $\varepsilon_1 = 0.05, \omega_1 = 0$ and $\varepsilon_2 = 0.10, \omega_2 = 0$.

   (a) Apply Equation (6.2) of Proposition 6.1.1 to compute the union false-positive rate $\varepsilon_\cup$. Report to four decimal places.

   (b) Apply Equation (6.3) to compute the union false-negative rate $\omega_\cup$, noting that every term in the weighted sum carries at least one operand ω as a factor.

   (c) Confirm that the inequality $\max(\varepsilon_1, \varepsilon_2) \le \varepsilon_\cup \le \varepsilon_1 + \varepsilon_2$ holds for the numbers you computed, and name the second-order term that accounts for the gap between $\varepsilon_\cup$ and $\varepsilon_1 + \varepsilon_2$.

2. ⋆ **Intersection rates for the same two filters.** Take the same operand rates as Item 1: $\varepsilon_1 = 0.05, \omega_1 = 0, \varepsilon_2 = 0.10, \omega_2 = 0$.

(a) Apply Proposition 6.2.1 to compute the intersection false-positive rate $\varepsilon_{\cap}$. Report to four decimal places.

(b) Compute the intersection false-negative rate $\omega_{\cap}$. State why $\omega_{\cap} = 0$ when both operands are FNR-zero.

(c) Compare the magnitudes of $\varepsilon_{\cap}$ and $\varepsilon_{\cup}$ from Item 1. By what factor does intersection reduce the false-positive rate relative to the smaller operand ε_1, and by what factor does union inflate it?

3. $\star$ **Complement-swap verification.** An approximate set $\mathsf{A}^{\pm}$ has per-block rates $\varepsilon = 0.03$ and $\omega = 0.07$.

(a) Apply Proposition 6.2.2 to write the rates $(\varepsilon_{\complement}, \omega_{\complement})$ of the complement $\complement\mathsf{A}^{\pm}$.

(b) Apply complement a second time. Verify that the twice-complemented rates return to $(0.03, 0.07)$, confirming that the complement operation is an involution on the rate plane.

(c) In one sentence, name the per-bit precedent for this swap that chapter 3 §3.2 already established at the noisy-bit level.

4. $\star\star$ **Notebook-tied set-op verification.** Open the chapter 6 companion notebook `python/notebooks/ch06-set-composition.ipynb`. Build two Bloom filters with $m = 5000$ bits, $n = 500$ inserted random strings each, and $k = 7$ hash functions, drawn from disjoint inserted sets S_1 and S_2 so that the operand rates approximate $\varepsilon_1 \approx \varepsilon_2 \approx 0.01$ with $\omega_1 = \omega_2 = 0$.

(a) Construct the union by bit-wise OR of the two underlying bit arrays. Query a pool of 10000 random strings known to be absent from both inserted sets, and record the empirical false-positive rate $\hat{\varepsilon}_{\cup}$.

(b) Compute the closed-form prediction $1 - (1 - \hat{\varepsilon}_1)(1 - \hat{\varepsilon}_2)$ from Equation (6.2), using empirical operand rates from the notebook in place of the targets, and compare to $\hat{\varepsilon}_{\cup}$. State in one sentence how close the empirical and closed-form values agree.

(c) Repeat the construction with the operand sets S_1 and S_2 chosen to overlap by 250 elements. Does the empirical union FPR still match the formula, and if so, why does the formula not depend on the cardinality of $S_1 \cap S_2$? Refer back to the FPR derivation in Proposition 6.1.1.

5. ⋆⋆ **Difference algebra.** Proposition 6.2.3 gave the difference rate formulas $\varepsilon_{\setminus} = \varepsilon_A \cdot \omega_B$ and $\omega_{\setminus} = 1 - (1 - \omega_A)(1 - \varepsilon_B)$ on the bulk region. Specialize these to two boundary cases.

   (a) Take $\mathsf{B}^{\pm}$ to be a perfect filter on the FPR-zero axis, so $\varepsilon_B = 0$ and $\omega_B \geq 0$. Substitute into the difference FPR formula and confirm that $\varepsilon_{\setminus} = \varepsilon_A \cdot \omega_B$ is the only surviving term. Substitute into the difference FNR formula and show that $\omega_{\setminus} = \omega_A$, that is, $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ inherits its FNR from the first operand alone.

   (b) Now swap: take $\mathsf{B}^{\pm}$ to be FNR-zero, so $\omega_B = 0$ and $\varepsilon_B \geq 0$. Substitute into the difference FPR formula and confirm $\varepsilon_{\setminus} = 0$. Then substitute into the difference FNR formula and derive $\omega_{\setminus} = 1 - (1 - \omega_A)(1 - \varepsilon_B)$. In one sentence, explain the mechanism: false positives of $\mathsf{B}^{\pm}$ become wrongful exclusions in $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$, and therefore false negatives of the difference.

   (c) Combine the two boundary readings to argue that the difference operation is FPR-preserving when the second operand is FNR-zero, and FNR-preserving when the second operand is a perfect filter on the FPR-zero axis.

6. ⋆⋆ **De Morgan duality under noise.** The classical De Morgan laws assert $\complement(A \cup B) = \complement A \cap \complement B$ and $\complement(A \cap B) = \complement A \cup \complement B$. This exercise verifies that the rate algebra of chapter 6 preserves the first of these identities under the cross-set independence assumption of Proposition 6.1.1.

   (a) Compute the rates of $\complement(\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm})$ by first applying Proposition 6.1.1 to get the union rates and then applying Proposition 6.2.2. Write the resulting (ε, ω) as a pair of expressions in the operand rates.

   (b) Compute the rates of $\complement\mathsf{A}^{\pm} \cap \complement\mathsf{B}^{\pm}$ by first applying Proposition 6.2.2 to each operand and then applying Proposition 6.2.1 to the complements.

   (c) Compare the two answers and confirm that the FPR expressions agree and the FNR expressions agree. The De Morgan duality holds for the rate algebra whenever the cross-set independence assumption is in force for both compositions.

7. ⋆⋆ **Independence sensitivity.** The FPR derivation in the proof of Proposition 6.1.1 assumed that the two operand error events at a query point x are mutually independent. This exercise constructs a small example where the assumption fails and shows what happens to the formula.

(a) Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two Bloom filters that share *exactly the same* hash family $h_1, \ldots, h_k$, that store the same set S, and that are derived from the same bit array. Argue that the two operand predicates are not merely correlated but identical: $\mathsf{A}^{\pm}(x) = \mathsf{B}^{\pm}(x)$ for every $x \in U$.

(b) Compute the actual union FPR for this construction. Since the two predicates are identical, the OR of their outputs agrees with the single-filter output, and the union FPR equals the common operand FPR ε. Compare to the formula prediction $1 - (1 - \varepsilon)(1 - \varepsilon) = 2\varepsilon - \varepsilon^2$ and describe the direction of the discrepancy.

(c) In one sentence, identify which structural property of the construction the formula presupposes, and which paragraph of Section 6.1 flags the requirement that the bit-wise OR construction needs distinct (or at least uncorrelated) hash families to match the rate prediction.

8. $\star\star$ **Intersection escape from the line.** Proposition 6.4.1 stated that intersection of two FNR-zero sets stays FNR-zero, but the same need not hold when one operand is FNR-zero and the other is FPR-zero.

(a) Let $\mathsf{A}^{\pm}$ be FNR-zero with $\varepsilon_A > 0$ and $\omega_A = 0$, and let $\mathsf{B}^{\pm}$ be FPR-zero with $\varepsilon_B = 0$ and $\omega_B > 0$. Substitute these operand rates into Proposition 6.2.1 and derive the intersection rates as expressions in ε_A and ω_B.

(b) Verify that $\varepsilon_\cap = 0$ in this regime, and that $\omega_\cap = \omega_B > 0$. The intersection has migrated to the FPR-zero axis: starting from one operand on each axis, the composition has landed on a single axis but on the FPR-zero one, not the FNR-zero one of $\mathsf{A}^{\pm}$.

(c) Now perturb the construction: let $\mathsf{B}^{\pm}$ instead be two-sided with $\varepsilon_B > 0$ and $\omega_B > 0$, leaving $\mathsf{A}^{\pm}$ FNR-zero. Substitute again and show that both rates of the intersection become strictly positive. In one sentence, name the geometric phenomenon (the composed point now lies in the interior of the rate square) and refer the reader to Definition 6.4.1 for the line whose escape this exercise concretely exhibits.

9. $\star\star\star$ **Three-way union via the composition theorem.** Extend Proposition 6.1.1 to three operands. Let $\mathsf{A}^{\pm}, \mathsf{B}^{\pm}, \mathsf{C}^{\pm}$ be three approximate sets with per-operand FPRs $\varepsilon_A, \varepsilon_B, \varepsilon_C$, all pairwise independent at every query point.

(a) Compute $\varepsilon_\cup$ for $\mathsf{A}^\pm \cup \mathsf{B}^\pm \cup \mathsf{C}^\pm$ by chained pairwise union: first apply Equation (6.2) to $\mathsf{A}^\pm \cup \mathsf{B}^\pm$, then apply it again to the result composed with $\mathsf{C}^\pm$. Simplify the nested expression to a single product form.

(b) Compute $\varepsilon_\cup$ directly from independence as in the proof of Proposition 6.1.1: a false positive of the three-way union at an x outside all three sets requires at least one of the three operand predicates to report $\top$, so the union FPR is 1 minus the joint probability that all three report $\bot$. Use independence to factor the joint into a product.

(c) Confirm that (a) and (b) agree. Then argue that the result is the composition theorem Theorem 3.3.1 of chapter 3 §3.3 instantiated at $k = 3$, that is, Equation (3.7) with $\eta_i = \varepsilon_i$. The same product formula governs every $k \geq 2$ by induction.

10. ⋆ ⋆ ⋆ **Perfect filter as algebraic inverse.** A perfect filter on a designated input set A is an approximate set with $\varepsilon = 0$ on A, paying with a positive ω on the complement domain. Proposition 6.4.1 part (4) suggested that pairing such a filter with an FNR-zero filter can give an exact-set test on a constrained domain. This exercise sketches the construction.

(a) Let $\mathsf{A}^\pm_+$ be a perfect filter for A with rates $(0, \omega_+)$ on its designed domain, and let $\mathsf{A}^\pm_-$ be an FNR-zero filter for A with rates $(\varepsilon_-, 0)$. Construct the composed predicate $\mathsf{A}^\pm_+(x) \wedge \mathsf{A}^\pm_-(x)$ and apply Proposition 6.2.1 to compute its rates. Confirm $\varepsilon = \varepsilon_- \cdot 0 = 0$ and $\omega = 1 - (1 - \omega_+)(1 - 0) = \omega_+$.

(b) State the condition under which $\omega_+ = 0$ also. The perfect filter has zero FNR on a subset of its designed domain (its "acceptance set"); if that subset coincides with A itself, the composition has both rates zero on A, that is, an exact-set test.

(c) Describe the architectural reading: the FNR-zero filter serves as a fast first-pass classifier (eliminating most non-members with one cheap query), and the perfect filter serves as a precise second-pass classifier (verifying the remaining candidates with certainty on the designed domain). Sketch why this two-pass composition is the algorithmic skeleton of the perfect hash filter that chapter 15 will develop in detail.

11. ⋆ ⋆ ⋆ **Entropy floor on the union operation.** Chapter 8 will prove

that any FNR-zero approximate set at false-positive rate ε requires at least $-\log_2 \varepsilon$ bits per stored element to represent, and chapter 14 will exhibit the Bernoulli Hash Function construction that achieves this floor. This exercise applies the floor to the union of two FNR-zero sets and shows that no clever implementation of union can beat it.

(a) Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two FNR-zero approximate sets for $A, B \subseteq U$, with operand FPRs $\varepsilon_A, \varepsilon_B$. By Proposition 6.4.1 part (1) the union $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ is FNR-zero with FPR $\varepsilon_\cup = 1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$. Apply the chapter 8 floor (anticipated) to the union: any representation of $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ as a single FNR-zero approximate set of the objective set $A \cup B$ requires at least $|A \cup B| \cdot (-\log_2 \varepsilon_\cup)$ bits.

(b) Argue that the chapter 6 composition formulas do not cheat the floor. The composed structure stores no more than the two operands together (its bit array, in the Bloom case, is the OR of the two operand bit arrays and so occupies m bits, the same as each operand). However, $\varepsilon_\cup > \max(\varepsilon_A, \varepsilon_B)$, so the floor for the composed structure permits a correspondingly looser bound; the bit budget allocated by each operand at its own floor is sufficient (and asymptotically tight, by chapter 14) for the composed FPR.

(c) In one or two sentences, draw the structural conclusion that motivates chapter 14. The union operation does not save bits beyond what the operands already paid for; constructive realizations that achieve the floor on each operand also achieve the floor on their union, modulo the small inflation $\varepsilon_\cup > \max(\varepsilon_A, \varepsilon_B)$ that the union FPR formula records. Chapter 14 develops the constructive realization (the BHF) that saturates this regime.

Chapter 7

Classification Measures

Placeholder. Content drafted in Plan 2.

Chapter 8

Information-Theoretic Bounds

Placeholder. Content drafted in Plan 2.

Part III

From Sets to Maps

Chapter 9

Approximate Functions and n-ary Channels

Placeholder. Content drafted in Plan 3.

Chapter 10

Sketches and Estimators

Placeholder. Content drafted in Plan 3.

Chapter 11

Randomized Algorithms

Placeholder. Content drafted in Plan 3.

Chapter 12

Approximate Relations

Placeholder. Content drafted in Plan 3.

Chapter 13

The Algebra of Approximate Types

Placeholder. Content drafted in Plan 3.

Part IV

Optimality and Construction

Chapter 14

Optimal Bernoulli Sets

Placeholder. Content drafted in Plan 4.

Chapter 15

Perfect Hash Filters

Placeholder. Content drafted in Plan 4.

Part V

The Boundary and Beyond

Chapter 16

Approximate but Not Bernoulli

Placeholder. Content drafted in Plan 4.

Chapter 17

Cipher Maps and Trapdoor Computing

Placeholder. Content drafted in Plan 4.

Chapter 18

Encrypted Search as Relaxed Trapdoor

Placeholder. Content drafted in Plan 4.

Appendix A

Probability Refresher

Placeholder. Content drafted in Plan 5.

Appendix B

The Five Core Principles

Placeholder. Cheat-sheet of the Two Axioms, Parametric Parsimony, Space-Accuracy Duality, Composition Closure, and the Boolean Thread. Content drafted in Plan 5.

Appendix C

Library Walkthrough

Placeholder. Tour of the `bernoulli` *Python package as a unified entry point. Content drafted in Plan 5.*

Appendix D

Selected Exercise Solutions

Placeholder. Roughly half of the end-of-chapter exercises are worked here; the rest are reader practice. Content drafted in Plan 5.